

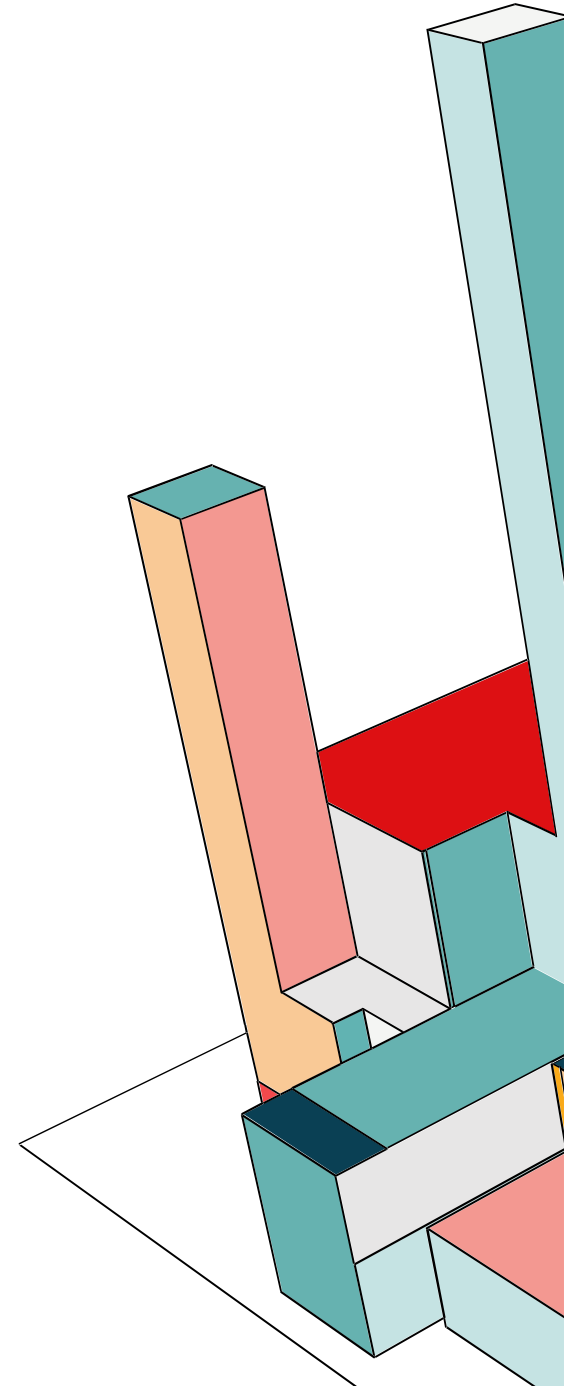
PROGETTO SYSTEM SECURITY

Francesca Beneventano M63001785
Andrea Giaquinto M63001800

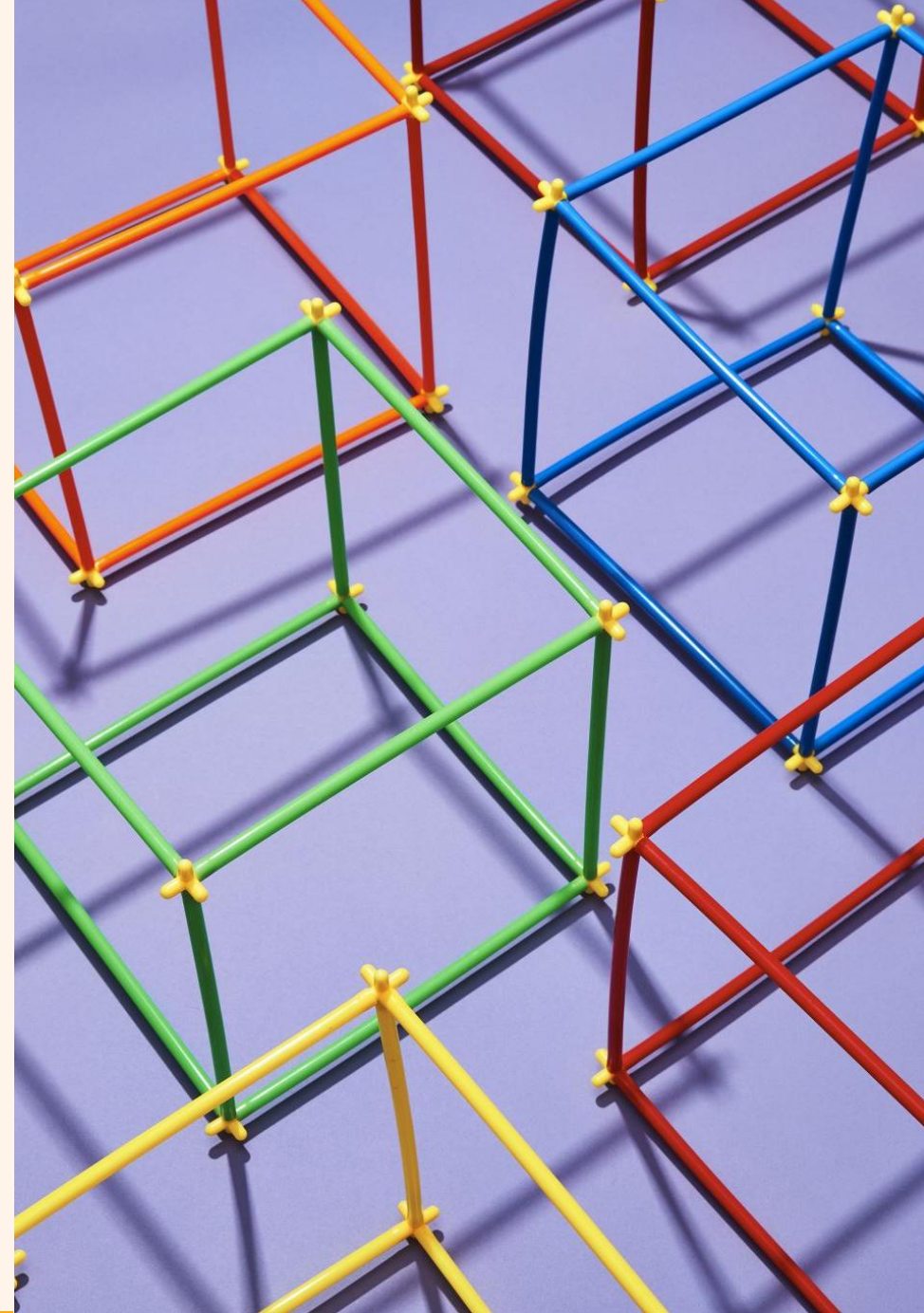


INDICE

- 1) OpenSSL
- 2) Java Cryptography Architecture
- 3) Configurazione Apache/Tomcat con https
- 4) Configurazione di un meccanismo di Identity Management e Access Control
- 5) Protezione delle credenziali dei vari asset con un meccanismo di credential management (Vault)



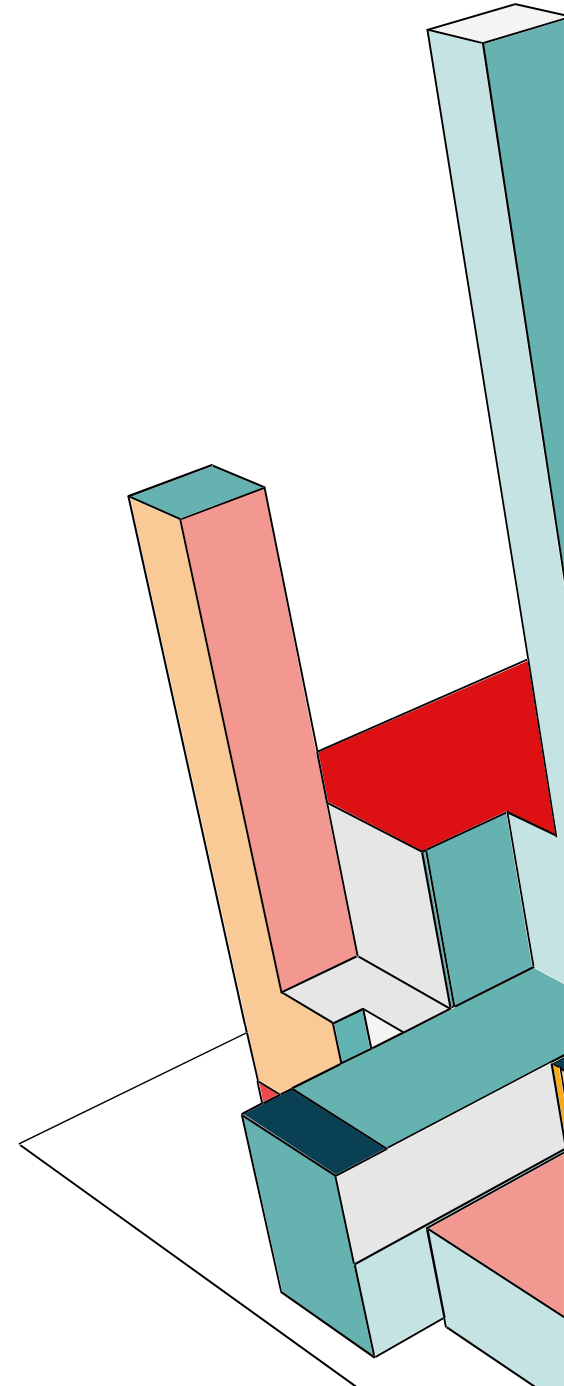
1) OPENSSL



CRITTOGRAFIA

L'obiettivo della crittografia è rendere non leggibile un file e la definiamo come una quintupla con due funzioni principali: cifratura e decifratura. La prima si occupa di associare ad ogni carattere del testo in chiaro, combinato con una chiave, un testo cifrato. La seconda funzione si occupa di prendere il testo cifrato, usa la chiave, e ritorna il testo in chiaro. Grazie alla crittografia soddisfiamo i seguenti requisiti:

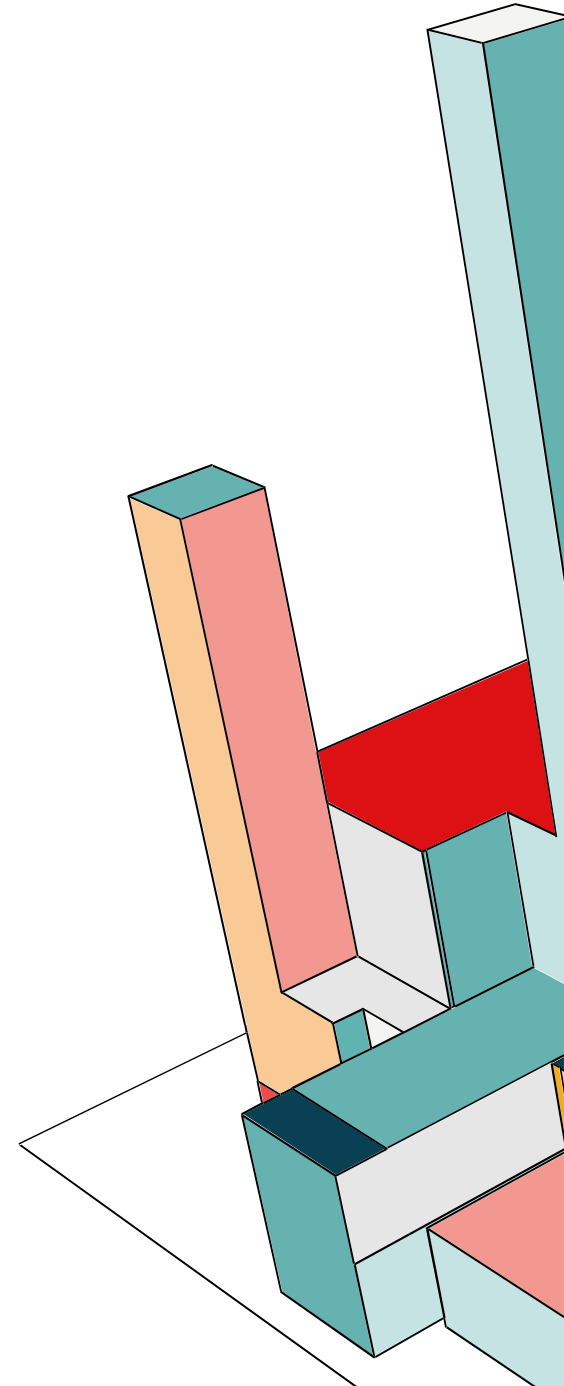
- **Confidenzialità:** solo il proprietario conosce la propria chiave privata; quindi, i testi criptati non possono essere letti da nessuno se non il proprietario della chiave privata
- **Autenticazione:** solo il proprietario conosce la propria chiave privata, quindi il testo cifrato con la propria chiave è sicuramente stata creata dal proprietario.
- **Integrità:** le informazioni criptate non possono essere cambiate in modo nascosto senza conoscerne la chiave privata.
- **Non ripudio:** un messaggio criptato con una chiave privata deriva sicuramente da qualcuno che la conosce.



COS'È OPENSSL?

OpenSSL è una libreria open source che implementa i principali protocolli e algoritmi crittografici. Serve per:

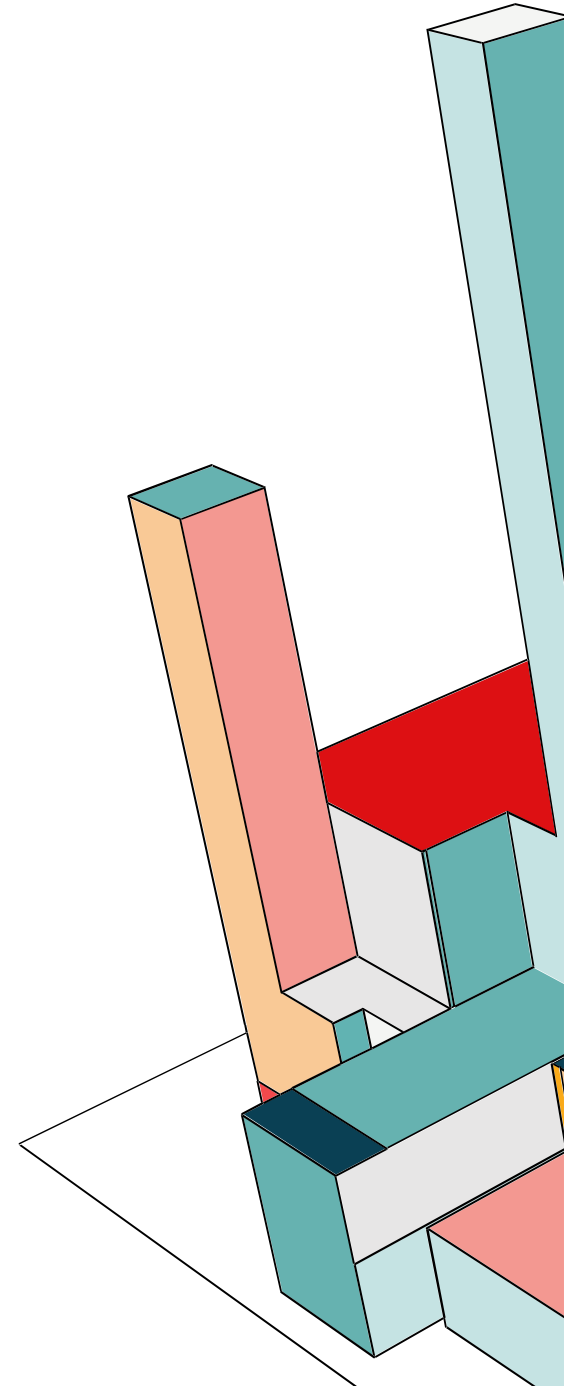
- generare chiavi (pubbliche, private o simmetriche),
- cifrare e decifrare file o messaggi,
- creare certificati digitali,
- verificare firme e autenticità.
- Oltre alla libreria, include anche uno strumento a riga di comando molto potente per eseguire operazioni di crittografia direttamente dal terminale.



CRITTOGRAFIA A CHIAVE SIMMETRICA

Nella crittografia simmetrica la stessa chiave viene usata sia per cifrare che per decifrare i dati. Quindi mittente e destinatario devono condividere in modo sicuro la chiave e la cifratura e decifratura sono molto veloci, quindi ideali per grandi quantità di dati.

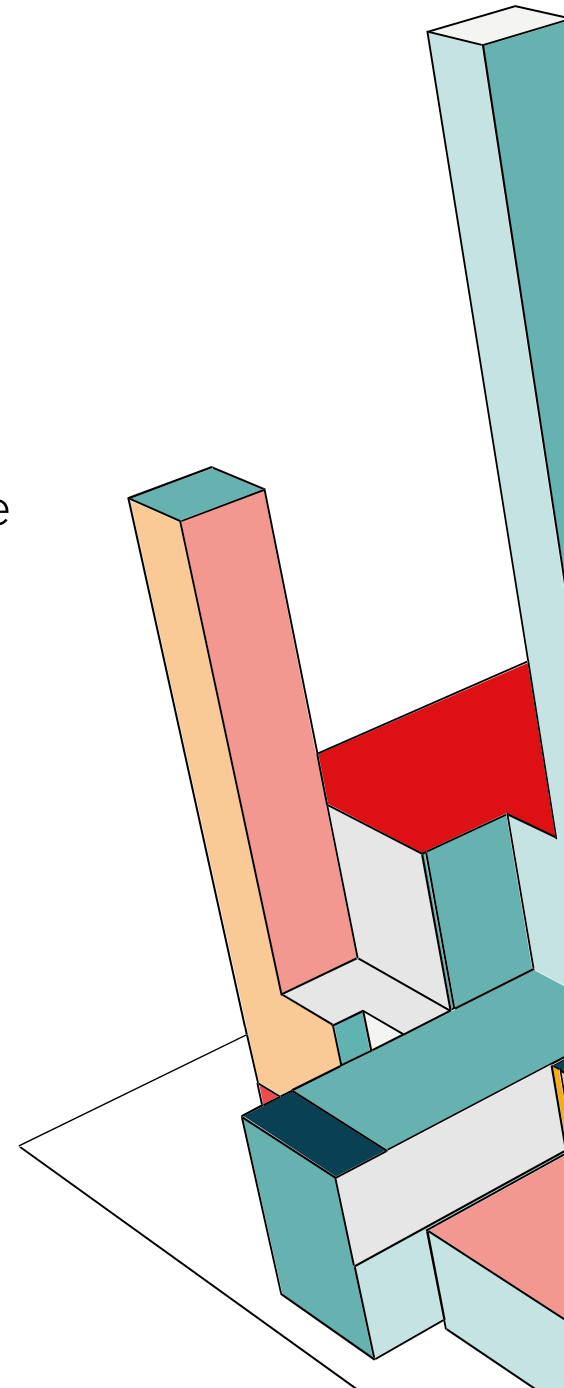
Proviamo a usare questi algoritmi crittografici a chiave simmetrica:
DES, 3DES, AES



DES

L'algoritmo DES è basato sul **cifrario di Feistel** che è una combinazione di funzioni di trasposizione e sostituzione. Il blocco prende 64 bit in ingresso e restituisce 2^{64} possibili combinazioni. Il cifrario di Feistel, invece di creare un unico grande blocco di 64 ingressi, è fatto con una composizione di diverse tecniche di cifratura e fatte in sequenza. Alla base del cifrario abbiamo le S-box e P-box, ovvero le due primitive crittografiche base che fanno permutazione e sostituzione, che in sequenza, permettono di implementare un cifrario più grande. Godono delle due proprietà principali:

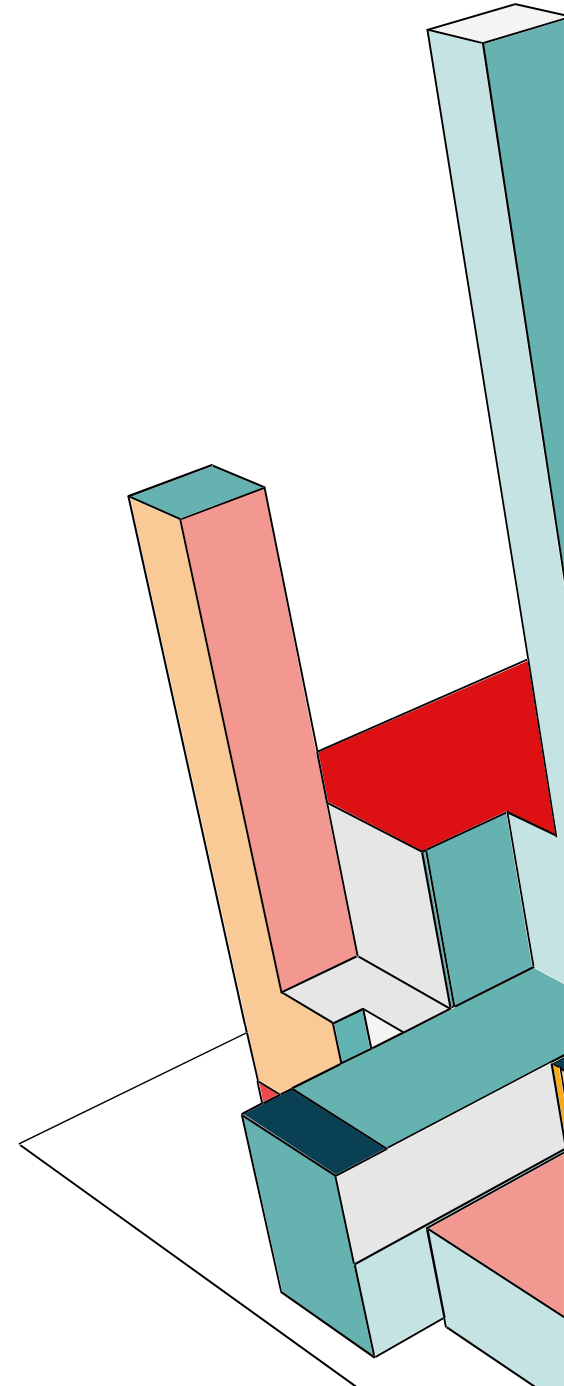
- **diffusione**: dissipare le proprietà statistiche (cioè quando prima avevo THEB avevo lo stesso testo cifrato) del testo in chiaro
- **confusione**: farlo rispetto alla chiave, quindi rendere la relazione tra testo in chiaro, cifrato e chiave quanto più complessa possibile. Perché se no se ci sono dei pattern subito me ne accorgo.



DES: BLOCK CHIPHER

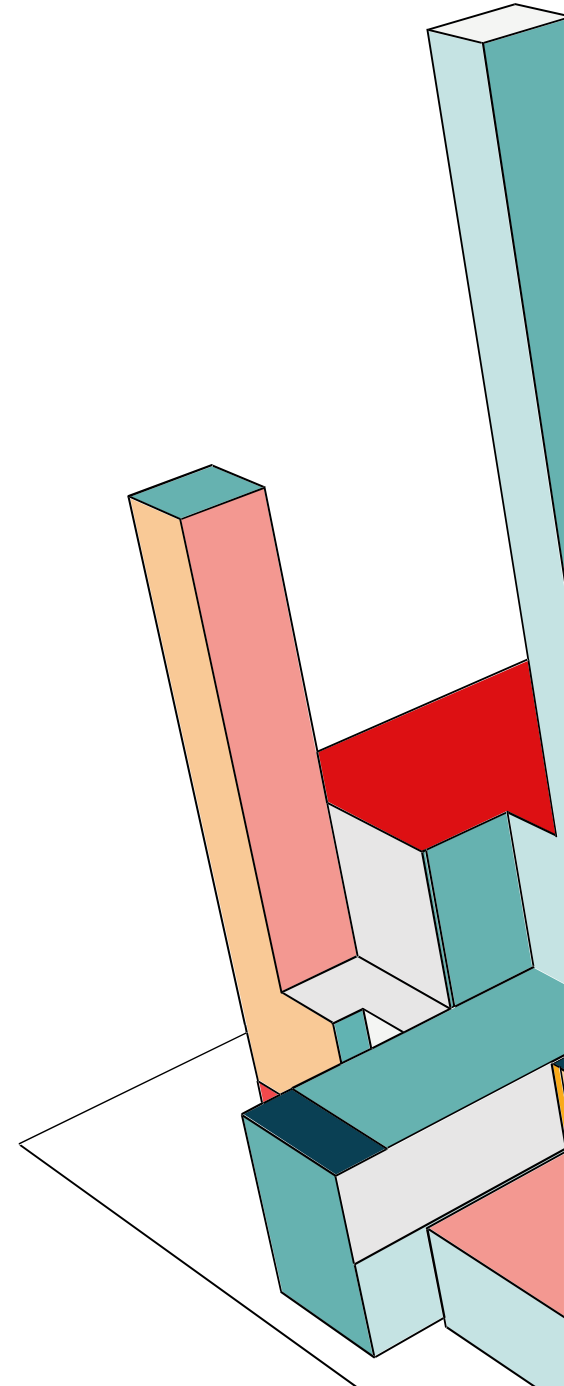
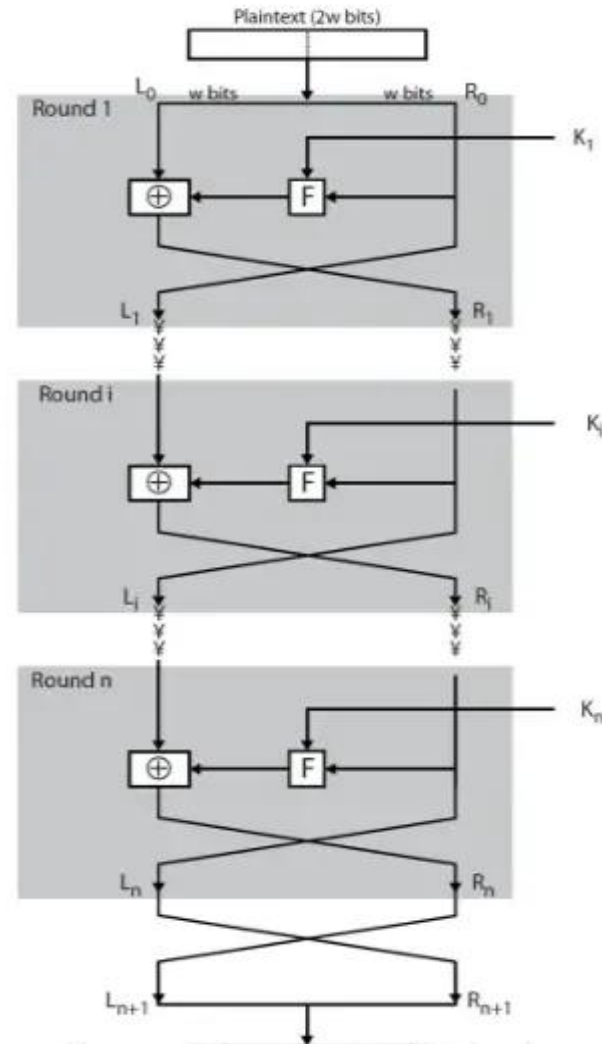
Sono cifrari **iterativi** che hanno due componenti che viaggiano in parallelo:

- **key schedule algorithm**: una chiave, ma in realtà ad ogni iterazione dell'algoritmo uso un pezzo della chiave diverso.
- **round function**: ad ogni iterazione applico questa funzione che mi permette di lavorare su parti del messaggio diverse.

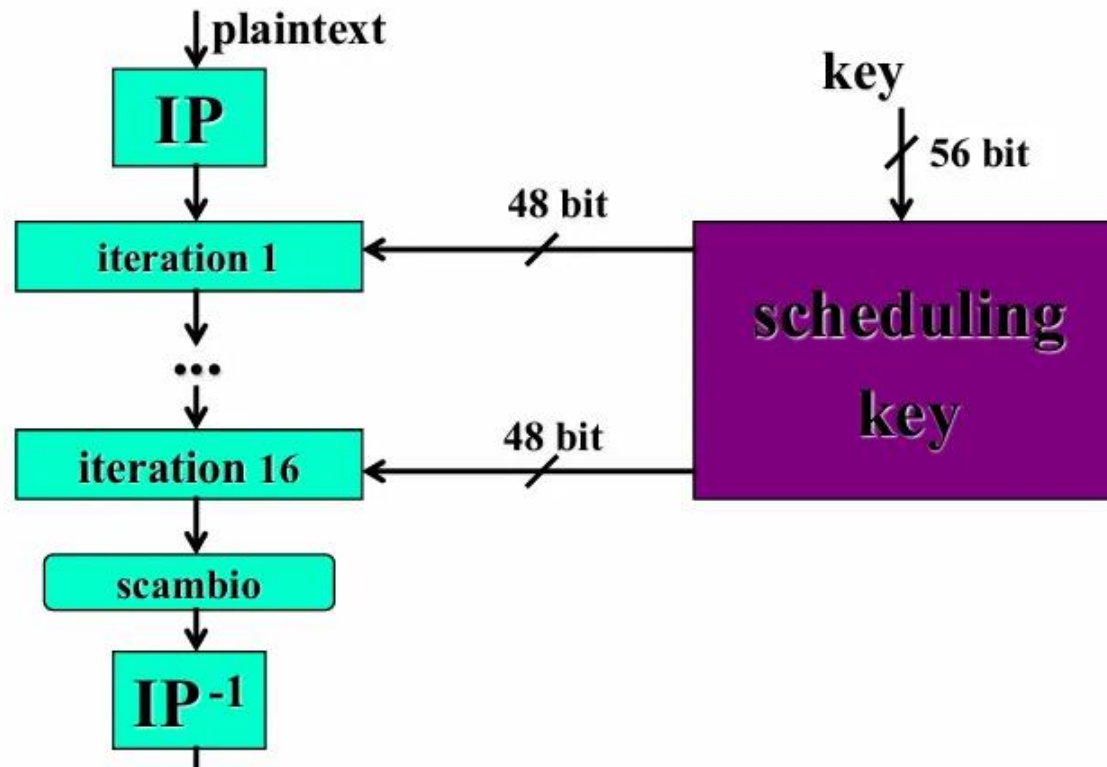


DES: STRUTTURA CIFRARIO DI FEISTEL

Ad ogni iterazione divido il blocco di 64 bit in due parti uguali, la parte destra viene inoltrata come parte sinistra al blocco che esegue l'iterazione successiva, e la parte sinistra viene messa in XOR con l'uscita della round function che prende in ingresso la parte destra del blocco coordinata con la sottochiave.



DES



Il singolo blocco ha lunghezza fissa di 64 bit così come la chiave, e si ottiene in uscita un testo cifrato di 64 bit. E' un algoritmo prodotto perché è un mix di permutazioni e sostituzioni. Inoltre è costituito da 16 iterazioni che usano tutte una sottochiave da 48 bit diversa della chiave iniziale.

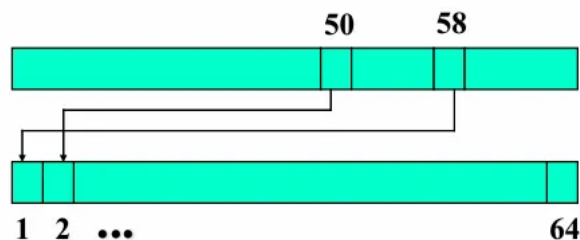


DES: INITIAL PERMUTATION IP E INVERSE PERMUTATION IP-1

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

bit before

bit after

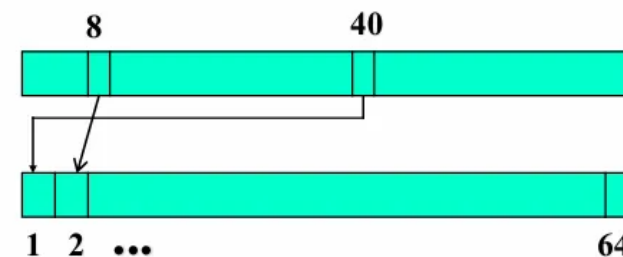


IP

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

bit before

bit after

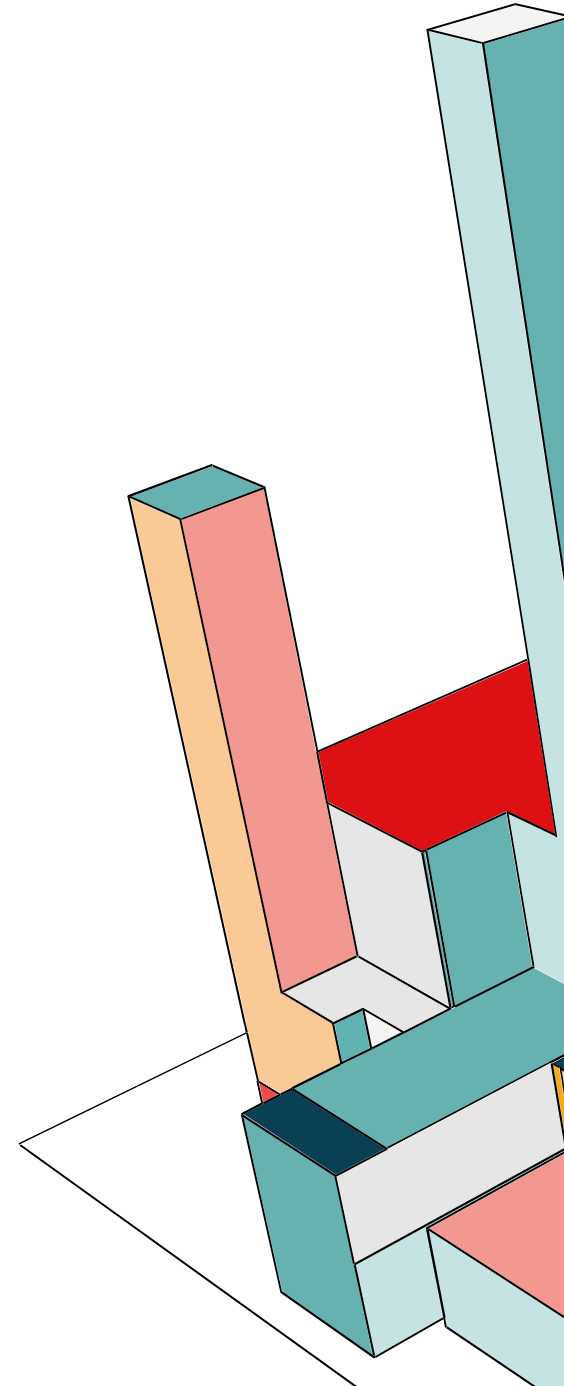


IP-1

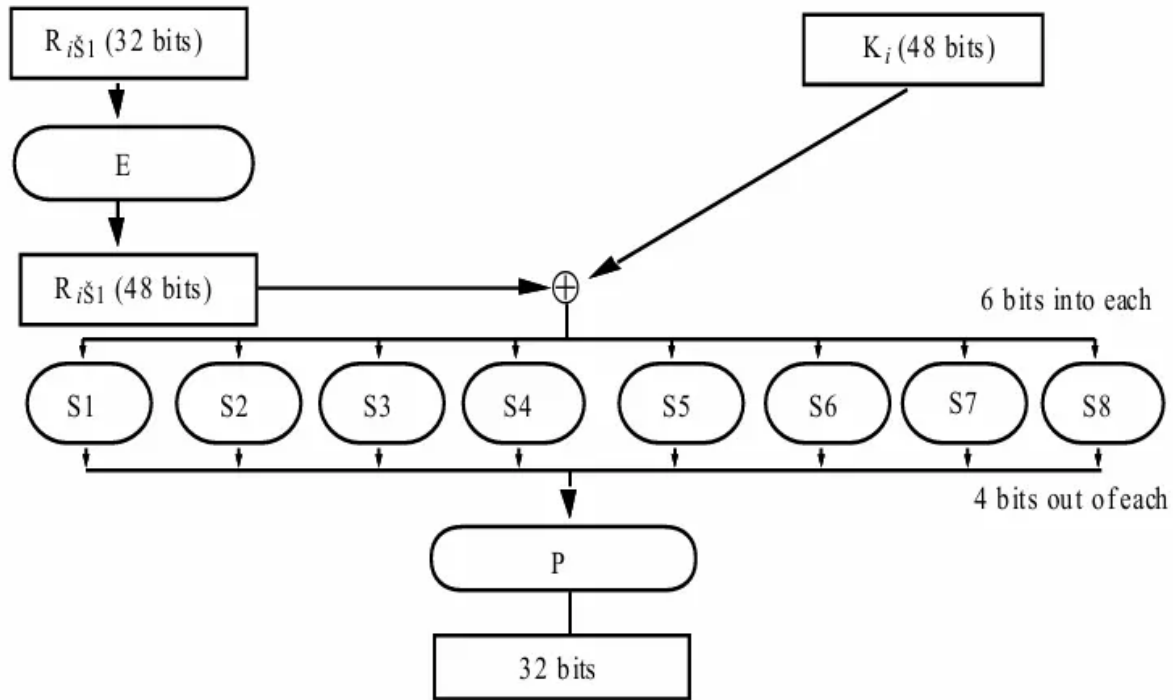


DES: SINGOL ITERATION

La parte destra diventa la parte sinistra dell'iterazione successiva. E la parte sinistra coincide con questa funzione che prende in ingresso i 32 bit della parte destra, la sottochiave dell'interazioni i-esima, la elabora secondo questa funzione F e tutto viene messo in XOR. La funzione F è una funzione di expansion a 48 bit che vengono aggiunti alla sottochiave con una XOR, che poi passano per delle S-box che sono delle tabella che mi dicono in funzione del valore della chiave, quali degli 8 bit devo prendere (devo prendere 4 bit) per costruire un nuovo blocco da 32.



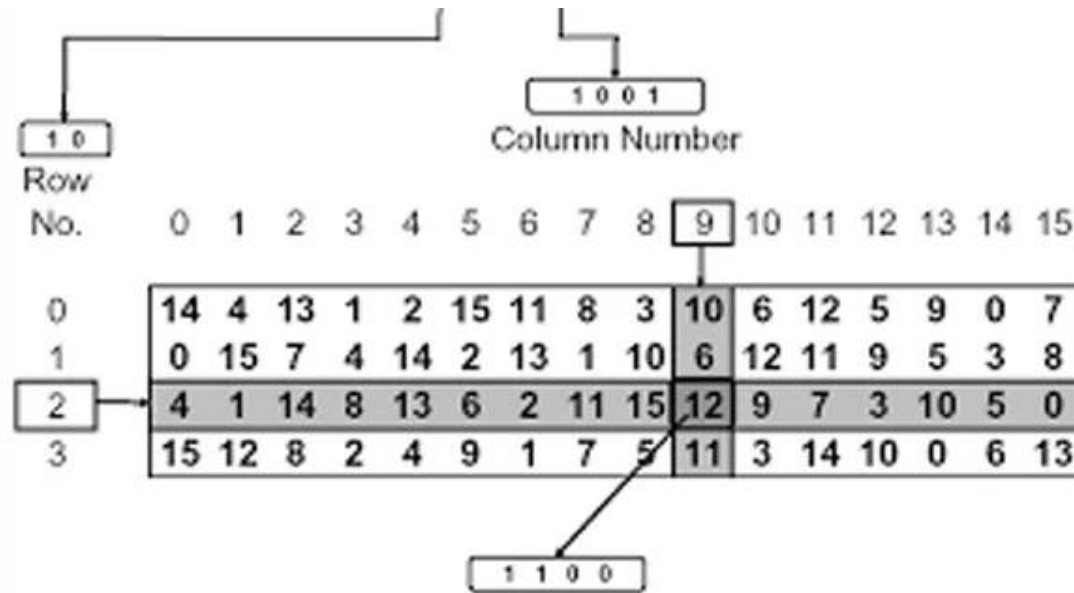
DES: F FUNCTION



La F function mi garantisce che i 48 bit in ingresso saranno spalmati in uscita in maniera quanto più dissipata possibile rispetto al valore della chiave.



DES: S-BOXES

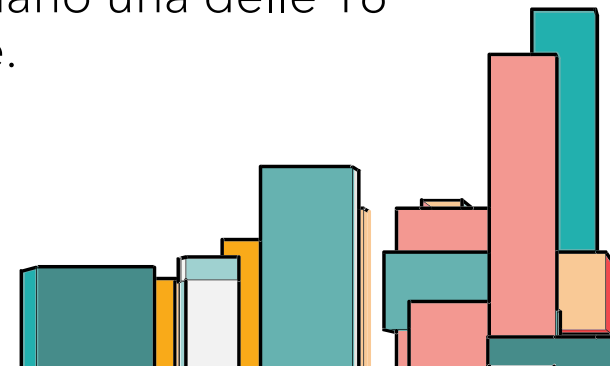


Input: 110010

Row: 10 (2)

Column: 1001 (9)

Le S-boxes sono delle matrici o comunque delle lookup table di 4x16 tutte diverse fra loro. Esse mappano i 6 bit in ingresso con 4 in uscita, in base a delle tabelle. In particolare, i due bit esterni selezionano una riga delle 4 della tabella S e i restanti bit da 1 a 4 selezionano una delle 16 colonne.



DES

Creiamo un messaggio in chiaro da cifrare (messaggio.txt).

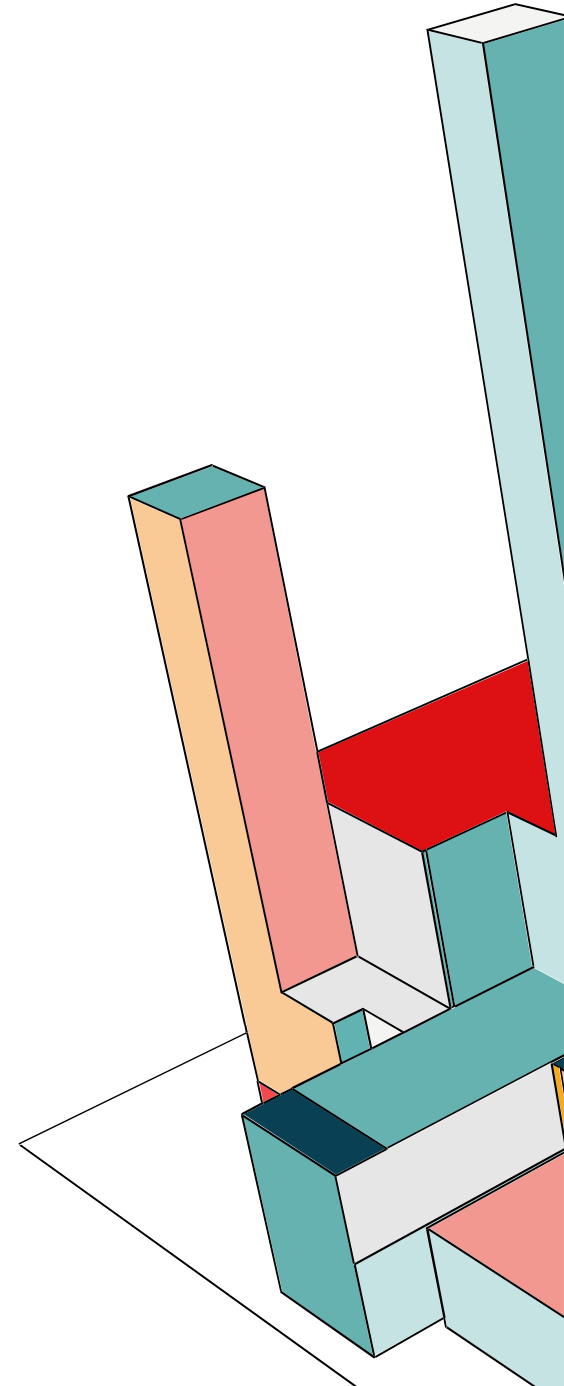
```
C:\Program Files\WSL\wsl.exe x + v
GNU nano 4.8 messaggio.txt
Questo messaggio è cifrato con crittografia a chiave simmetrica con DES.
```

Eseguiamo il comando `openssl rand -out chiave_des.key 8` che crea un file `chiave_des.key` con 8 byte casuali (64 bit totali, di cui 56 effettivi + 8 bit di parità).

```
C:\Program Files\WSL\wsl.exe x + v
francesca@LAPTOP-5V7UQQGH:~$ openssl rand -out chiave_des.key 8
francesca@LAPTOP-5V7UQQGH:~$ xxd chiave_des.key
00000000: 4164 cdee d9e4 2449                Ad....$I
francesca@LAPTOP-5V7UQQGH:~$ openssl enc -des-cbc -salt -in messaggio.txt -out messaggio_cifrato_des.bin -pass file:./chiave_des.key
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

DES: WARNING

Il WARNING che abbiamo ricevuto deriva dal fatto che il metodo di derivazione della chiave usato di default da OpenSSL è obsoleto. Quando usiamo l'opzione `-pass`, OpenSSL non prende direttamente la chiave dal file, ma usa una password per derivare la vera chiave crittografica e l'IV. Un metodo più moderno e robusto, invece, è PBKDF2 (Password-Based Key Derivation Function 2), che aggiunge iterazioni e salt per aumentare la sicurezza. In questo modo la chiave derivata è più sicura contro attacchi di forza bruta o dizionario.



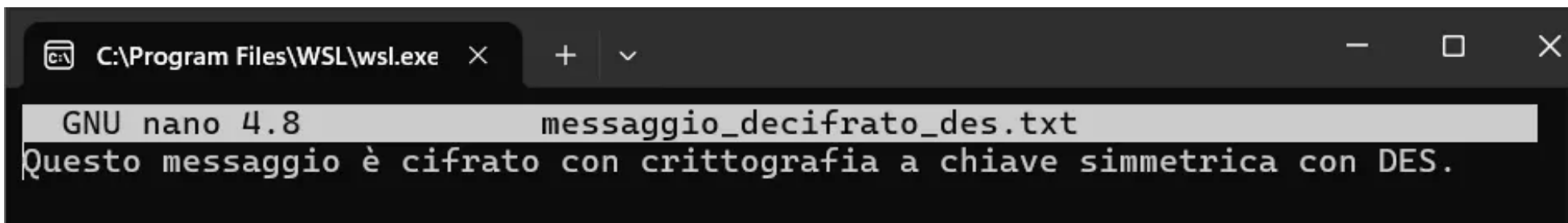
DES: DECRITTAZIONE

Eseguiamo il comando per decrittare il messaggio.txt:

```
openssl enc -d -des-cbc -pbkdf2 -in messaggio_cifrato_des.bin -out  
messaggio_decifrato_des.txt -pass file:./chiave_des.key
```

```
francesca@LAPTOP-5V7UQQGH:~$ openssl enc -des-cbc -salt -pbkdf2 -in messaggio.txt -out messaggio_cifrato_des.bin -pass file:./chiave_des.key  
francesca@LAPTOP-5V7UQQGH:~$ ls  
chiave_des.key  messaggio.txt  messaggio_cifrato_des.bin  
francesca@LAPTOP-5V7UQQGH:~$ openssl enc -d -des-cbc -pbkdf2 -in messaggio_cifrato_des.bin -out messaggio_decifrato_des.txt -pass file:./chiave_des.key  
francesca@LAPTOP-5V7UQQGH:~$ ls  
chiave_des.key  messaggio_cifrato_des.bin  
messaggio.txt  messaggio_decifrato_des.txt
```

Visualizziamo il messaggio decrittato:

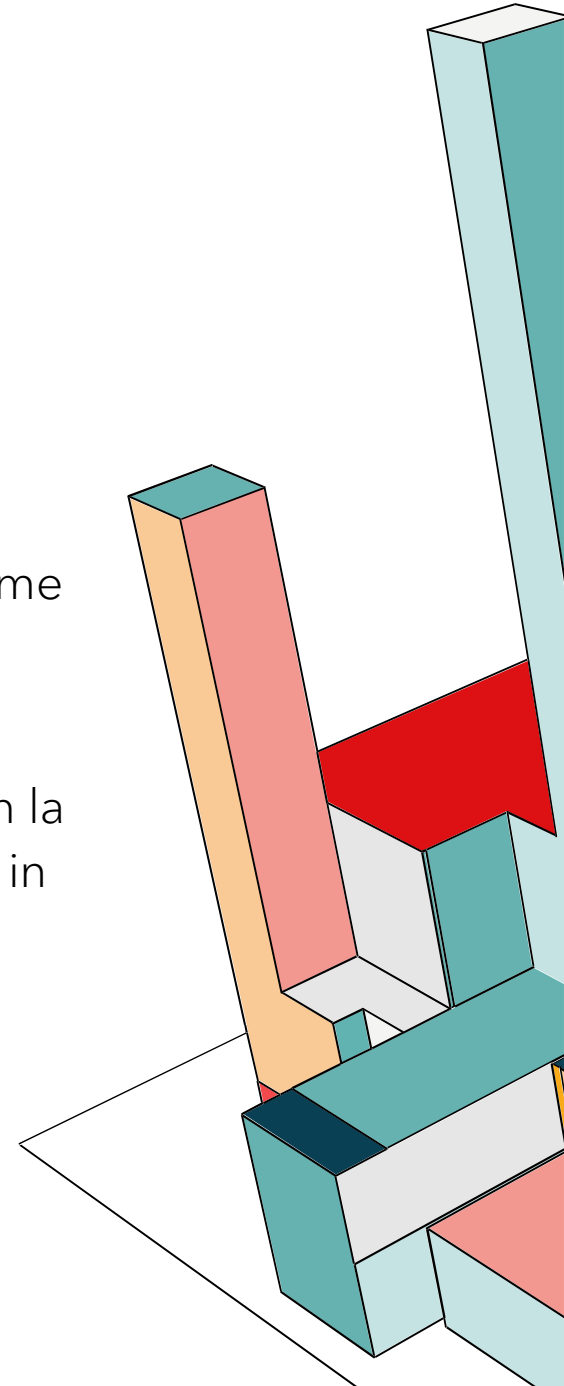


The screenshot shows a terminal window titled "C:\Program Files\WSL\wsl.exe". Inside, the GNU nano 4.8 editor is open, editing the file "messaggio_decifrato_des.txt". The text displayed in the editor is "Questo messaggio è cifrato con crittografia a chiave simmetrica con DES." The terminal window has standard Windows-style window controls (minimize, maximize, close) in the top right corner.

PUNTI DI DEBOLEZZA DEL DES

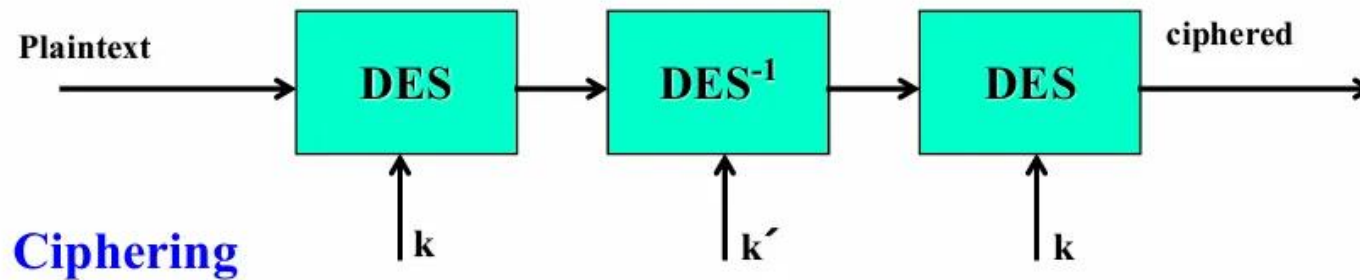
- **Gli attacchi sono ormai noti:** brute force e altri attacchi che sfruttano le strutture interne dell'algoritmo.
- **Blocchi troppo piccoli:** blocchi da 64 bit rendono DES vulnerabile a fenomeni come il birthday attack.
- **Chiave troppo corta:** già nel 1998, la macchina "Deep Crack" della Electronic Frontier Foundation riuscì a trovare una chiave DES in meno di 3 giorni. Oggi, con la potenza di calcolo moderna, 56 bit si possono forzare con un attacco brute force in poche ore o minuti.

DES è deprecato ufficialmente dal NIST dal 2005 ed è stato sostituito da AES (Advanced Encryption Standard), oppure 3DES (Triple DES) per compatibilità con sistemi vecchi, ma anche 3DES è considerato obsoleto.

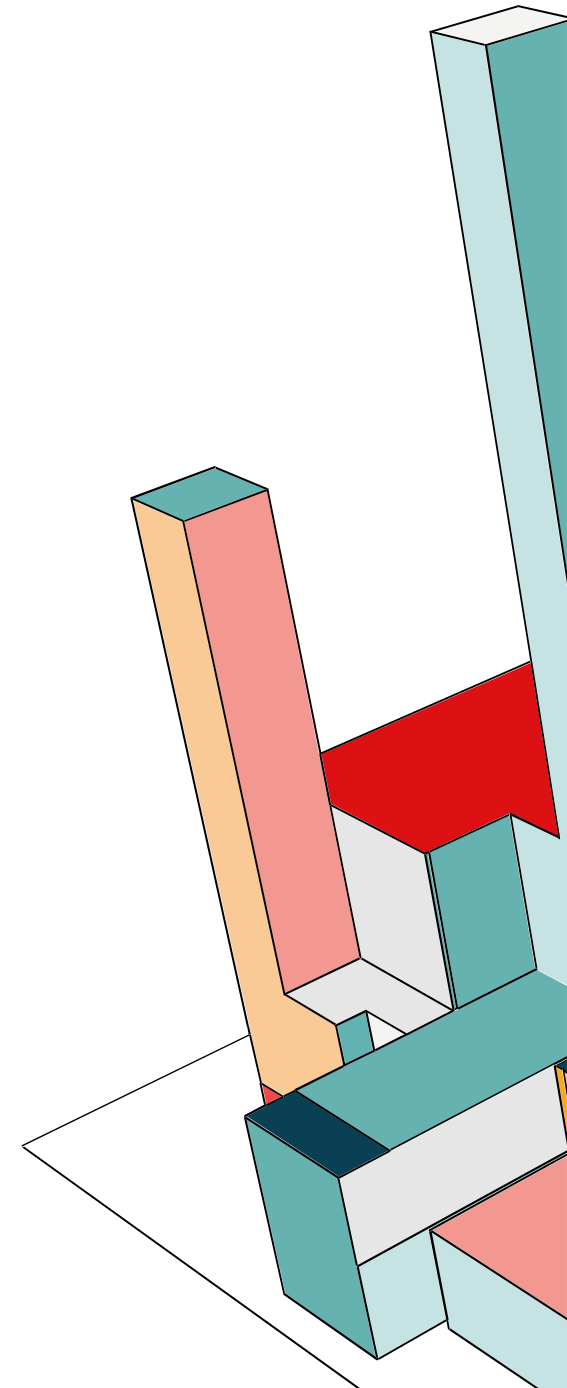


TRIPLE-DES

Consiste nel fare una sequenza di critto-decritto-critto con due chiavi, ma c'è anche una variante a tre chiavi che è quella che useremo noi.



- lunghezza blocco = 64 bit
- chiave (k, k') lunga $56+56 = 112$ bit (more usable than 3 keys, same security)
- spesso chiamato $EDE_{k,k'}$ (acronimo per Encrypt Decrypt Encrypt)
- adottato negli standard X9.17 e ISO 8732



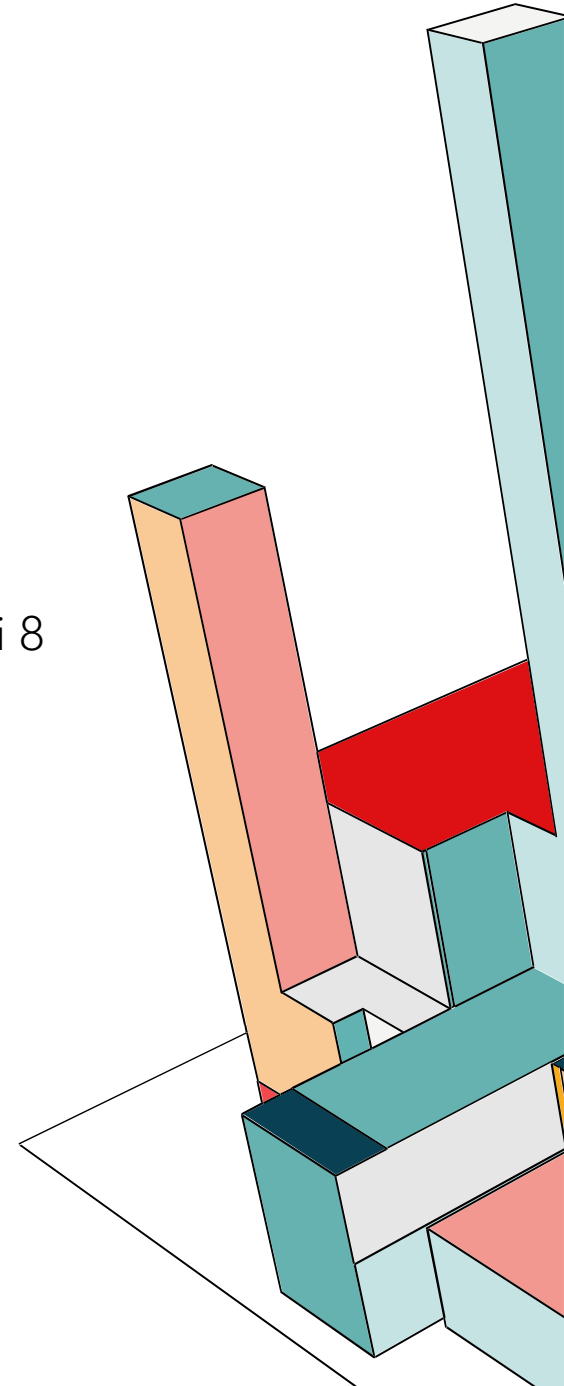
TRIPLE-DES

Messaggio da cifrare:

```
francesca@LAPTOP-5V7UQQGH:~$ cat messaggio.txt
Ciao! Questo è un messaggio di prova da crittare e decrittare.
francesca@LAPTOP-5V7UQQGH:~$ |
```

Generiamo una sequenza casuale di 24 byte in esadecimale per la chiave e un IV di 8 byte in esadecimale:

```
francesca@LAPTOP-5V7UQQGH:~$ key_hex=$(openssl rand -hex 24)
francesca@LAPTOP-5V7UQQGH:~$ iv_hex=$(openssl rand -hex 8)
francesca@LAPTOP-5V7UQQGH:~$ echo "$key_hex" > key.hex
francesca@LAPTOP-5V7UQQGH:~$ echo "$iv_hex" > iv.hex
francesca@LAPTOP-5V7UQQGH:~$ chmod 600 key.hex iv.hex
francesca@LAPTOP-5V7UQQGH:~$ ls
iv.hex  key.hex  vault_1.14.3_linux_amd64.zip
```



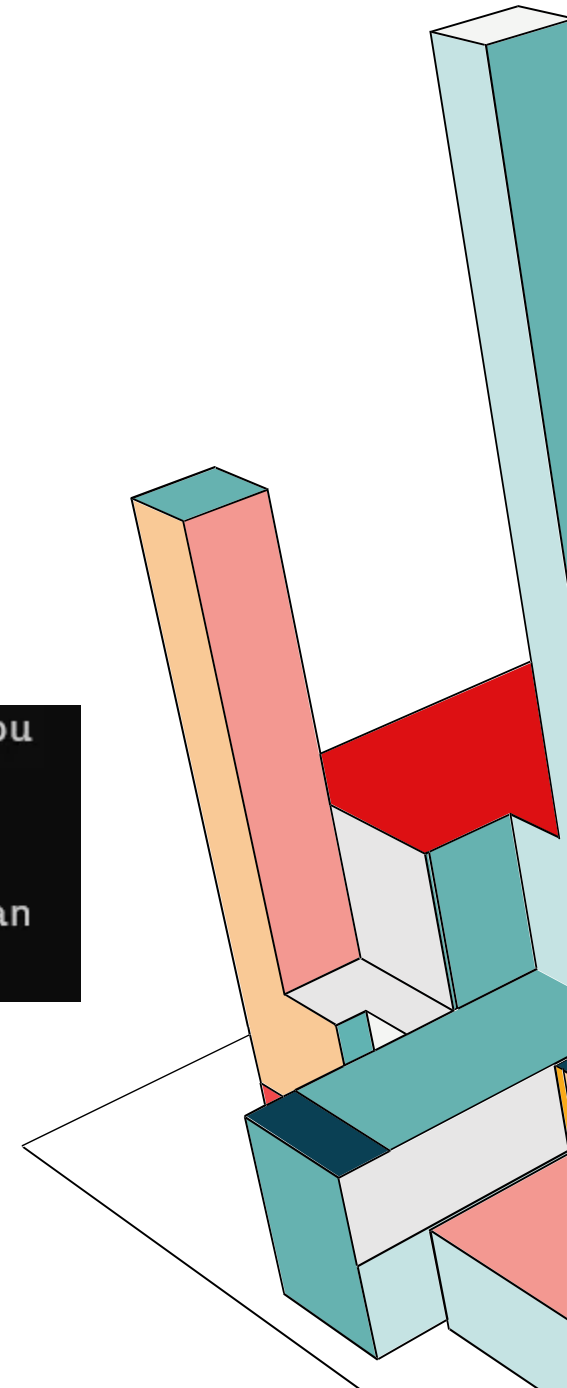
TRIPLE-DES

Visualizziamo la chiave e l'IV:

```
francesca@LAPTOP-5V7UQQGH:~$ echo "KEY (hex): $key_hex"
KEY (hex): f893dd4a035dce4b193e37c0085980fdd682224cf1e07e28
francesca@LAPTOP-5V7UQQGH:~$ echo "IV (hex): $iv_hex"
IV (hex): bd1459a8c1e0fed4
```

Cifriamo il file generando un file .enc codificato:

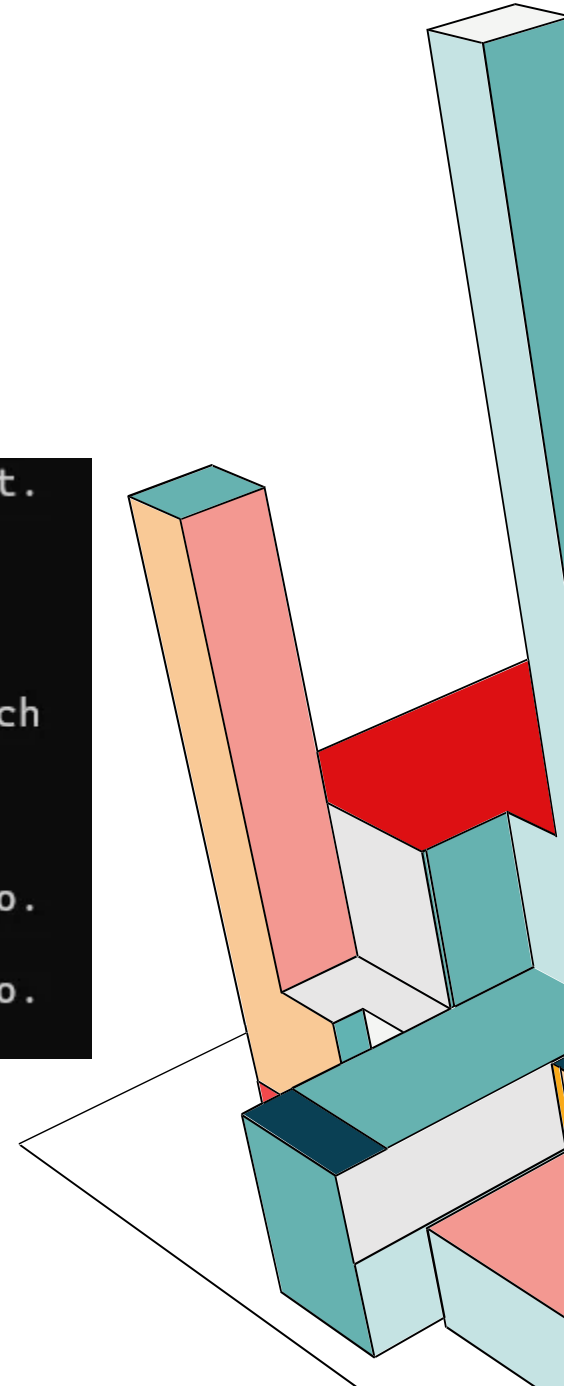
```
francesca@LAPTOP-5V7UQQGH:~$ openssl enc -des-ede3-cbc -in messaggio.txt -out messaggio.txt.enc -K "$key_hex" -iv "$iv_hex"
francesca@LAPTOP-5V7UQQGH:~$ cat messaggio.txt.enc
Ta?vB>7"5D000
          900p0000D0040m0^0lT|R0&0hA0l0Ud00x 0j000X00b40?w00R8fran
cesca@LAPTOP-5V7UQQGH:~$ |
```



TRIPLE-DES

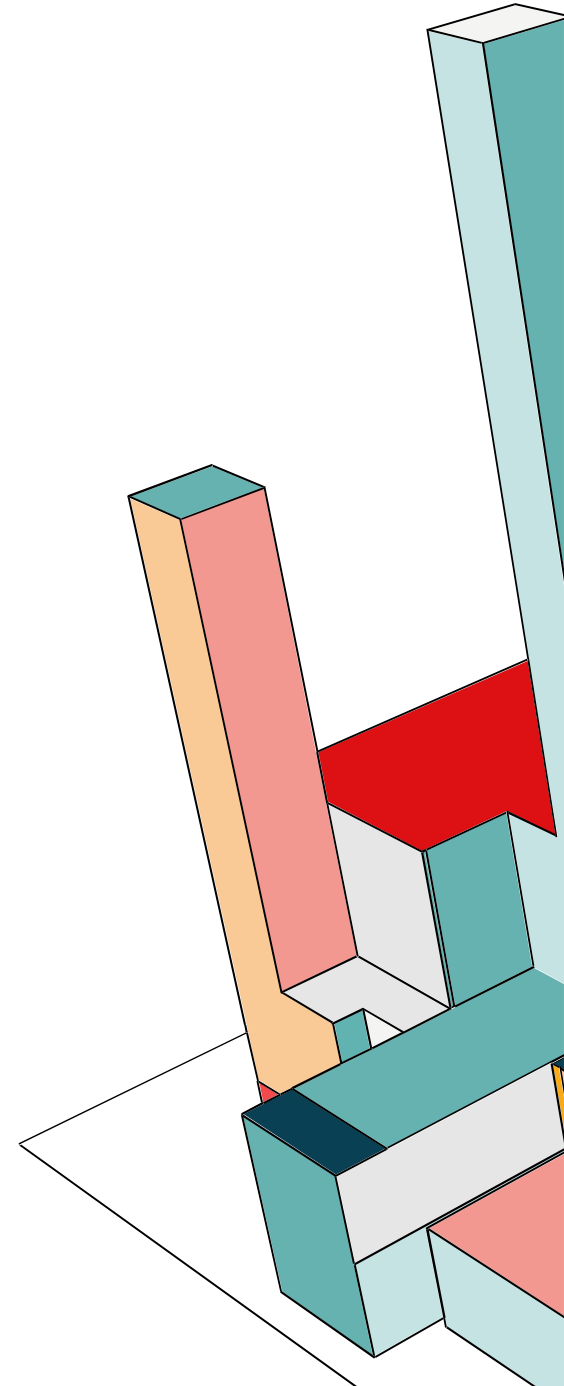
Adesso decrittiamo il file e lo confrontiamo con l'originale sia con un confronto byte-a-byte che con un controllo hash.

```
francesca@LAPTOP-5V7UQQGH:~$ openssl enc -d -des-ede3-cbc -in messaggio.txt.  
enc -out messaggio.dec -K "$key_hex" -iv "$iv_hex"  
francesca@LAPTOP-5V7UQQGH:~$ ls  
iv.hex    messaggio.dec  messaggio.txt.enc  
key.hex   messaggio.txt  vault_1.14.3_linux_amd64.zip  
francesca@LAPTOP-5V7UQQGH:~$ cmp --silent messaggio.txt messaggio.dec && ech  
o "OK: file identici" || echo "DIFFERENZE TROVATE"  
OK: file identici  
francesca@LAPTOP-5V7UQQGH:~$ sha256sum messaggio.txt messaggio.dec  
7e86752962eedbcd5326f84a699304d7e056311de15425355c4489f387a5c0ac  messagg  
io.txt  
7e86752962eedbcd5326f84a699304d7e056311de15425355c4489f387a5c0ac  messagg  
io.  
dec
```



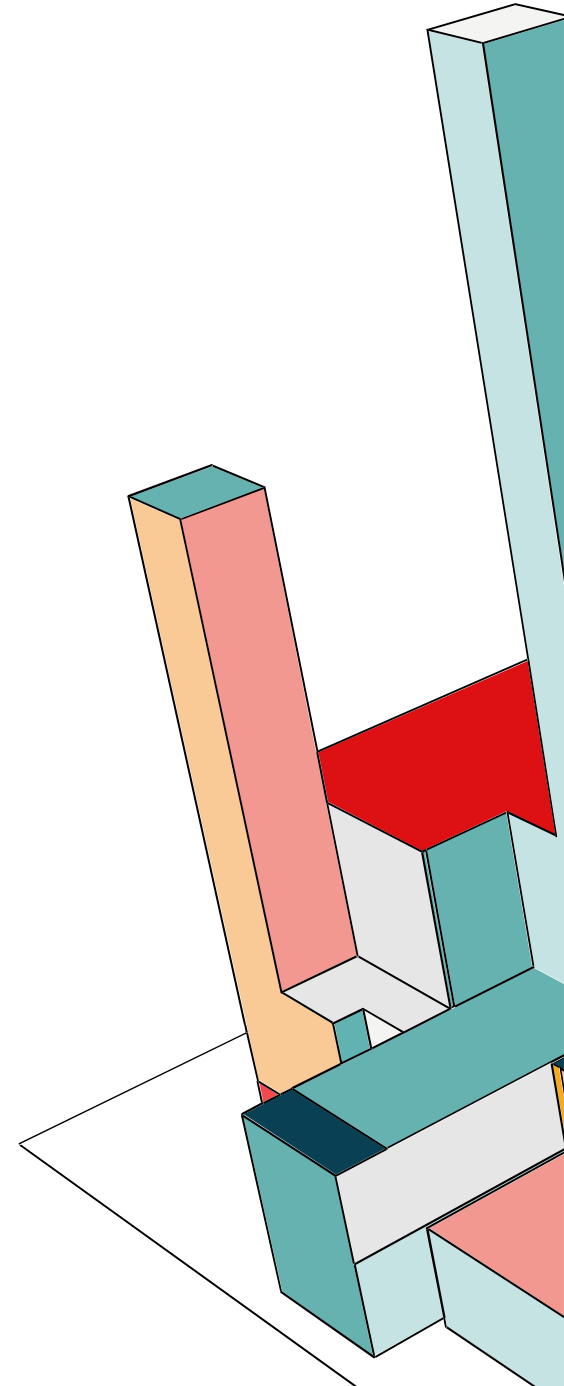
AES

AES (Advanced Encryption Standard) è un algoritmo di cifratura simmetrica a blocchi adottato come standard dal NIST nel 2001, in sostituzione di DES e 3DES. L'approccio matematico alla base è diverso dal DES perché AES non si basa sui blocchi di Feistel, ma usa una rete SPN (Substitution-Permutation Network). AES non divide il blocco, ma lavora su tutto il blocco di 128 bit contemporaneamente in ogni passaggio. Inoltre non lavora bit a bit come in DES, ma lavora a byte. Nel DES, la struttura è rigida, ovvero la chiave deve essere di 64 bit e i round sono sempre 16. Invece nell'AES, lo standard definisce tre livelli di sicurezza tra cui si può scegliere in fase di implementazione.



AES: PUNTI DI FORZA

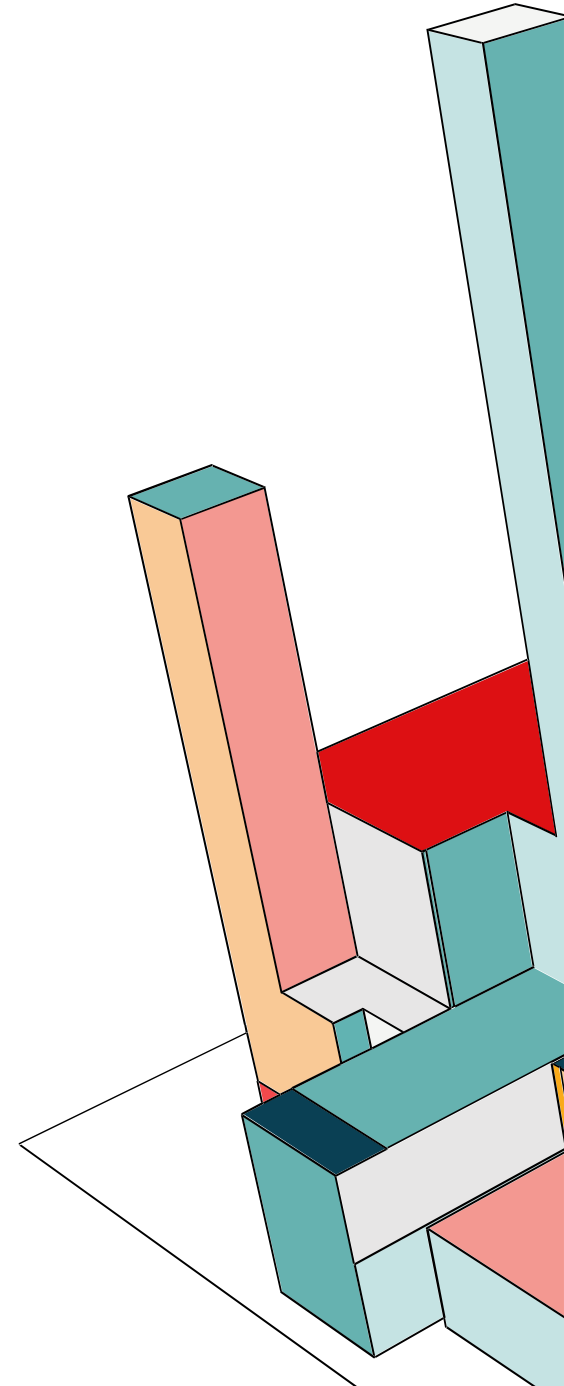
- **Sicuro:** nessun attacco efficace conosciuto.
- **Veloce:** ottimizzato per hardware e software.
- **Blocchi grandi (128 bit):** evita problemi che avevamo in DES e 3DES.
- **Standard mondiale:** usato in HTTPS, VPN, Wi-Fi, dischi cifrati, ecc.



AES CON OPENSSL

Useremo AES-256 con una chiave da 32 byte e un IV da 16 byte perché lo usiamo con DES in modalità CBC e cambiamo i permessi ai file che custodiscono la chiave e l'IV.

```
francesca@LAPTOP-5V7UQQGH:~$ key_hex=$(openssl rand -hex 32)
francesca@LAPTOP-5V7UQQGH:~$ iv_hex=$(openssl rand -hex 16)
francesca@LAPTOP-5V7UQQGH:~$ echo "$key_hex" > key.hex
francesca@LAPTOP-5V7UQQGH:~$ echo "$iv_hex" > iv.hex
francesca@LAPTOP-5V7UQQGH:~$ chmod 600 key.hex iv.hex
francesca@LAPTOP-5V7UQQGH:~$ cat key.hex
5fba05409302fab38006cb7523d280ee22888188d24602d8833222038168c814
francesca@LAPTOP-5V7UQQGH:~$ cat iv.hex
b4da5d4b48791a19b348e0a24f3996cc
```



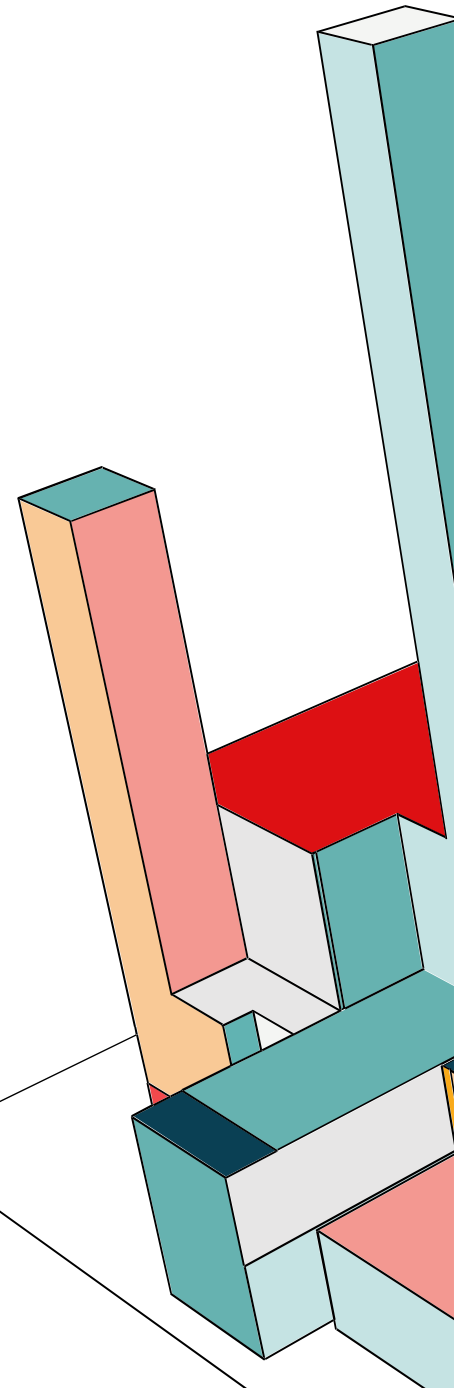
AES CON OPENSSEL

Adesso cifriamo il messaggio:

```
francesca@LAPTOP-5V7UQQGH:~$ openssl enc -aes-256-cbc -in messaggio.txt -out  
messaggio.txt.enc -K "$key_hex" -iv "$iv_hex"  
francesca@LAPTOP-5V7UQQGH:~$ cat messaggio.txt.enc  
GKn- S'7Nm SLE(  >8b&Vt{GA3D:  
l1Lfrancesca@LAPTOP-5V7UQQGH:~$ |
```

Infine, decifriamo:

```
francesca@LAPTOP-5V7UQQGH:~$ openssl enc -d -aes-256-cbc -in messaggio.txt.e  
nc -out messaggio.dec -K "$key_hex" -iv "$iv_hex"  
francesca@LAPTOP-5V7UQQGH:~$ cat messaggio.dec  
Ciao! Questo è un messaggio di prova da crittare e decrittare.  
Buon weekend.
```

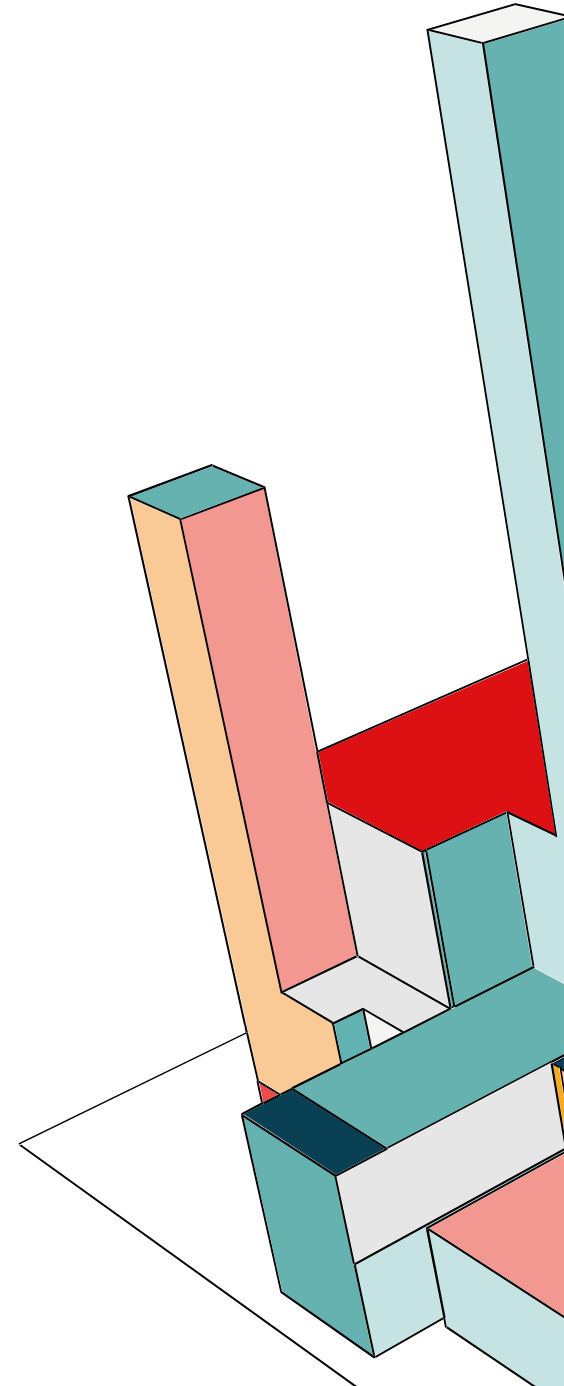


CRITTOGRAFIA A CHIAVE ASIMMETRICA

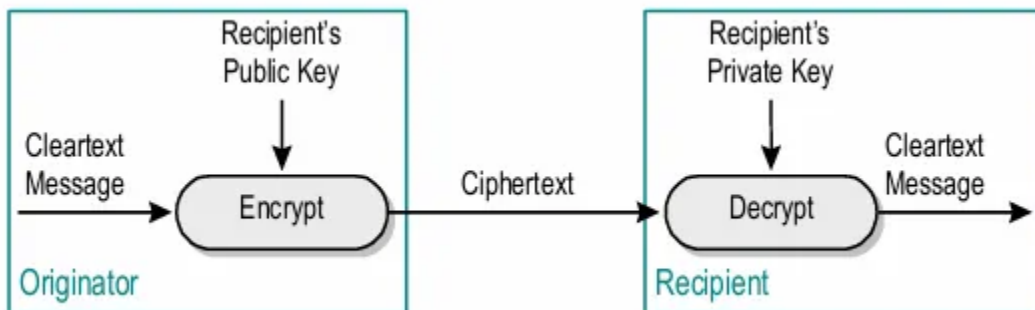
Mentre la crittografia simmetrica si basa un'unica chiave condivisa, la crittografia asimmetrica utilizza la coppia chiave pubblica e privata. La crittografia a chiave pubblica è usata per:

- crittare/decrittare
- firma digitale
- chiavi di sessione

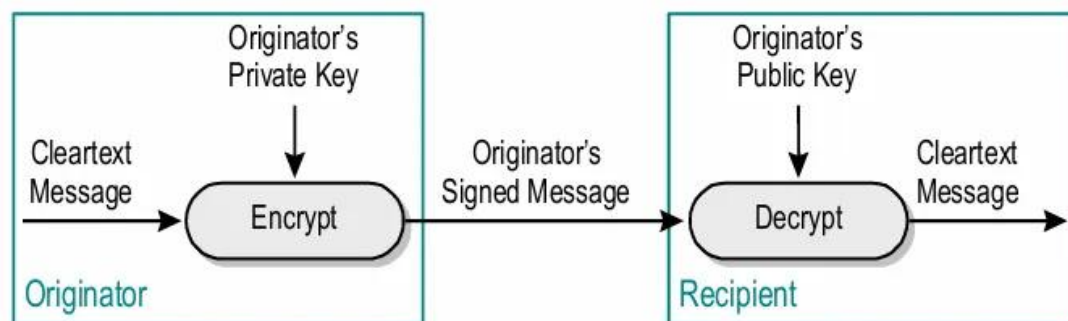
Lo svantaggio di questo tipo di crittografia è che il calcolo delle chiavi è computazionalmente oneroso, ma è preferibile dal punto di vista dello scambio sicuro delle chiavi in ambienti grandi e della robustezza rispetto agli attacchi a forza bruta.



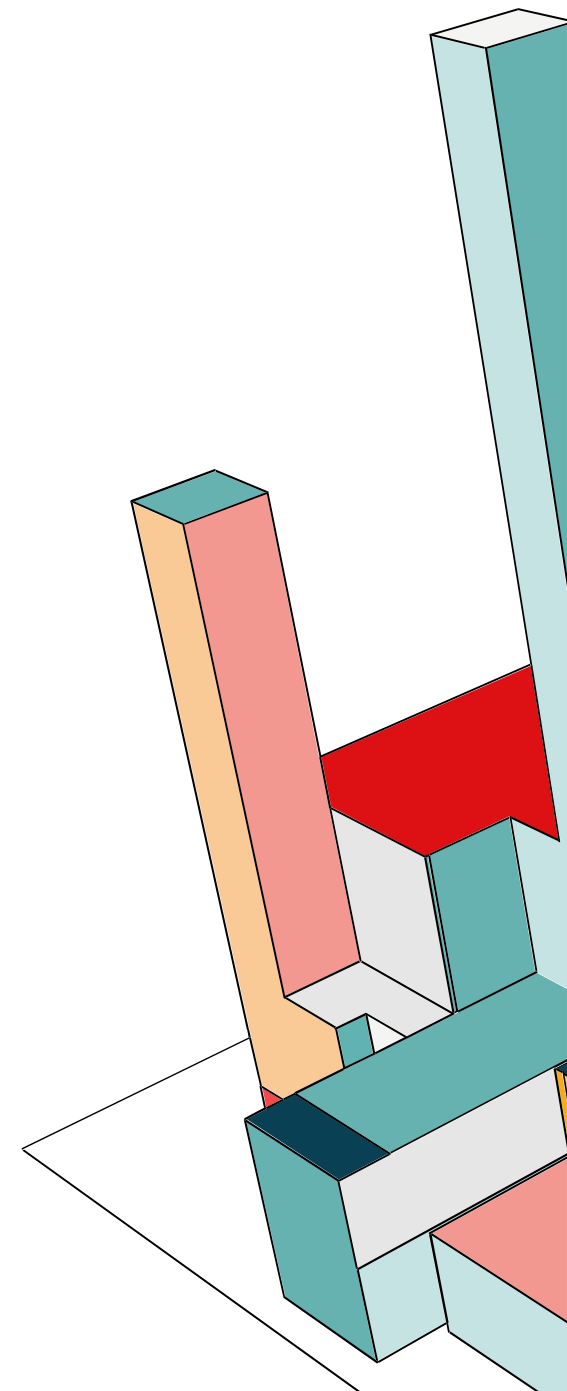
CRITTOGRAFIA A CHIAVE ASIMMETRICA



Se critto con la chiave pubblica del destinatario ho garantito integrità e confidenzialità.

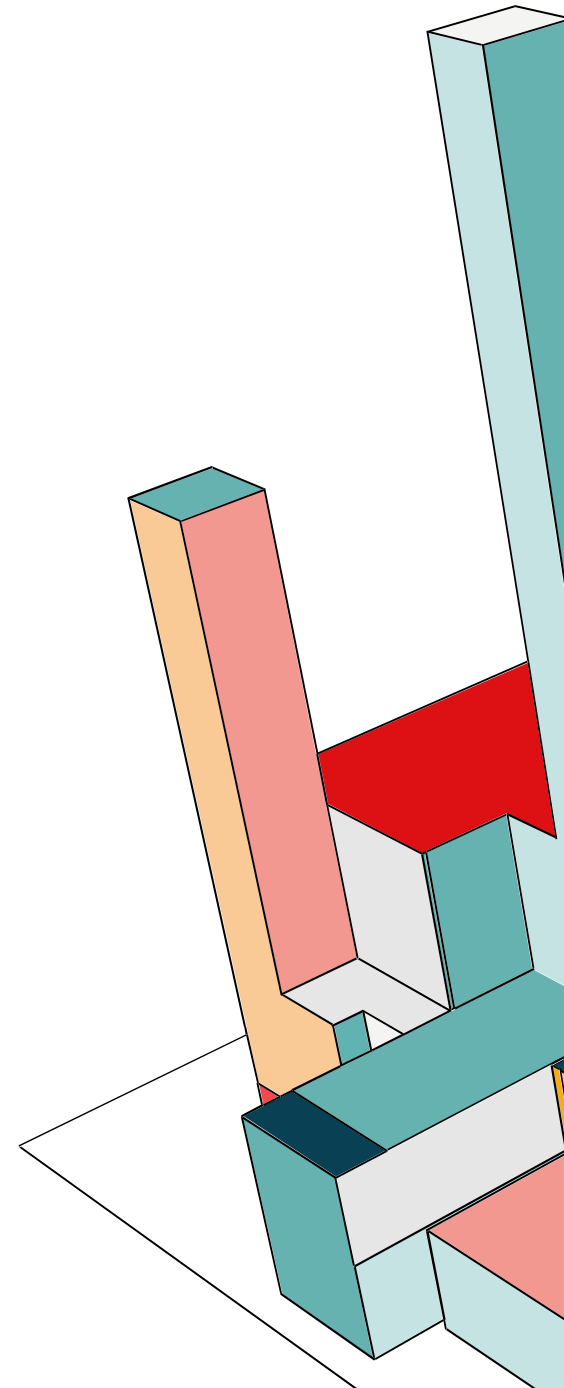
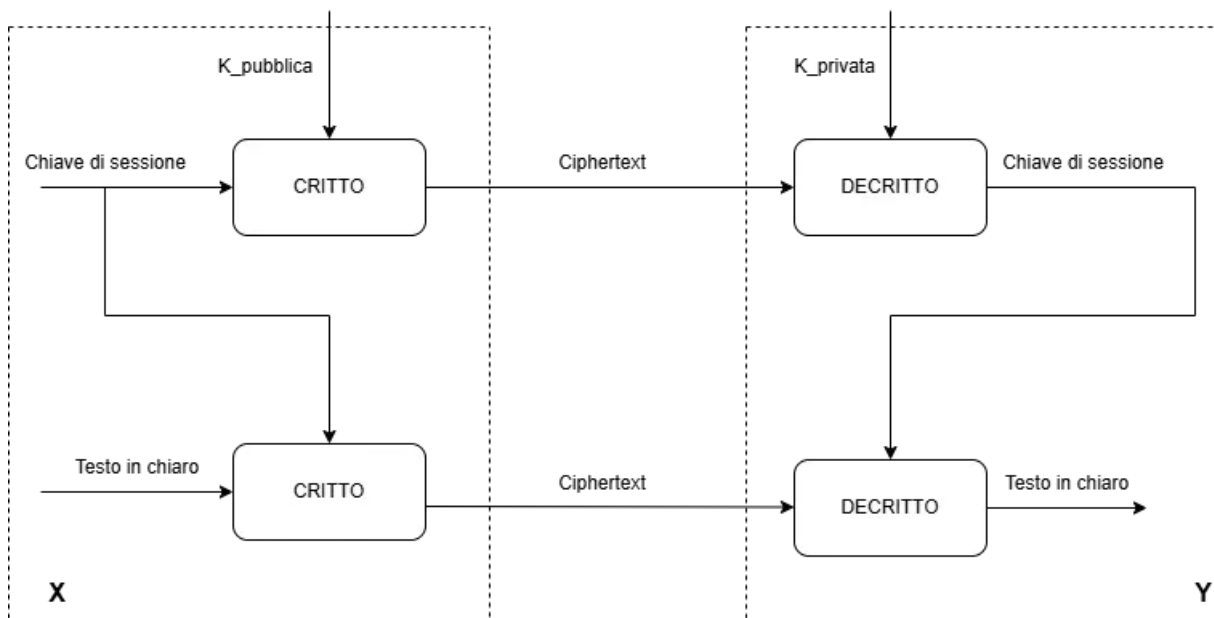


Per garantire autenticazione devo firmare con la chiave pubblica. Quindi critto con la chiave privata.



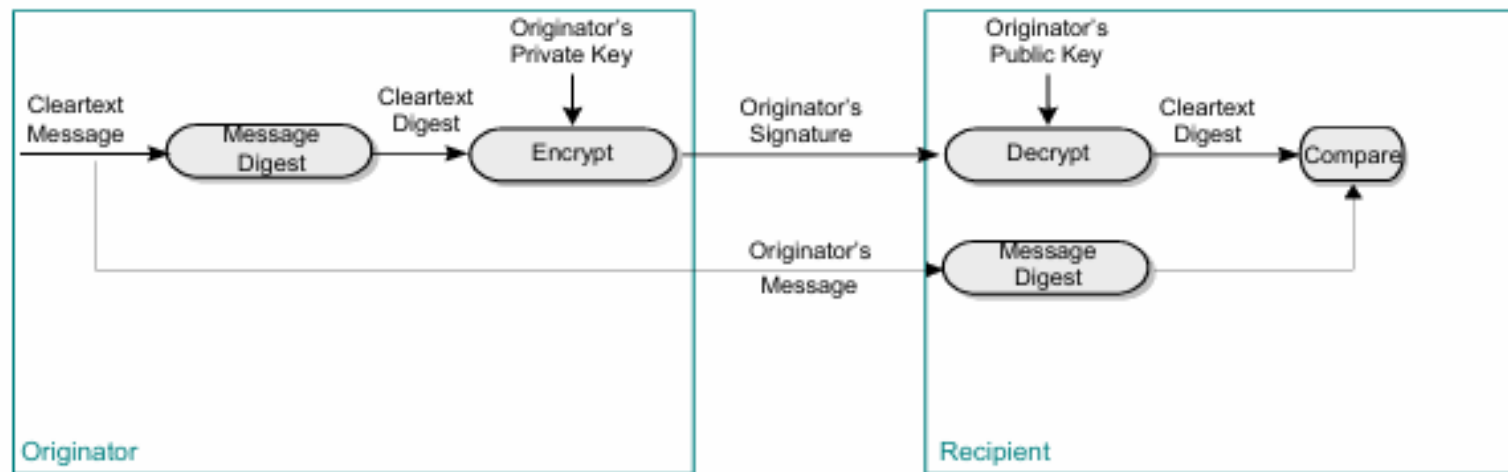
CRITTOGRAFIA A CHIAVE ASIMMETRICA

E' possibile distribuire chiavi di sessione a breve termine, cifrandole e decifrandole mediante la coppia di chiavi pubbliche e private.



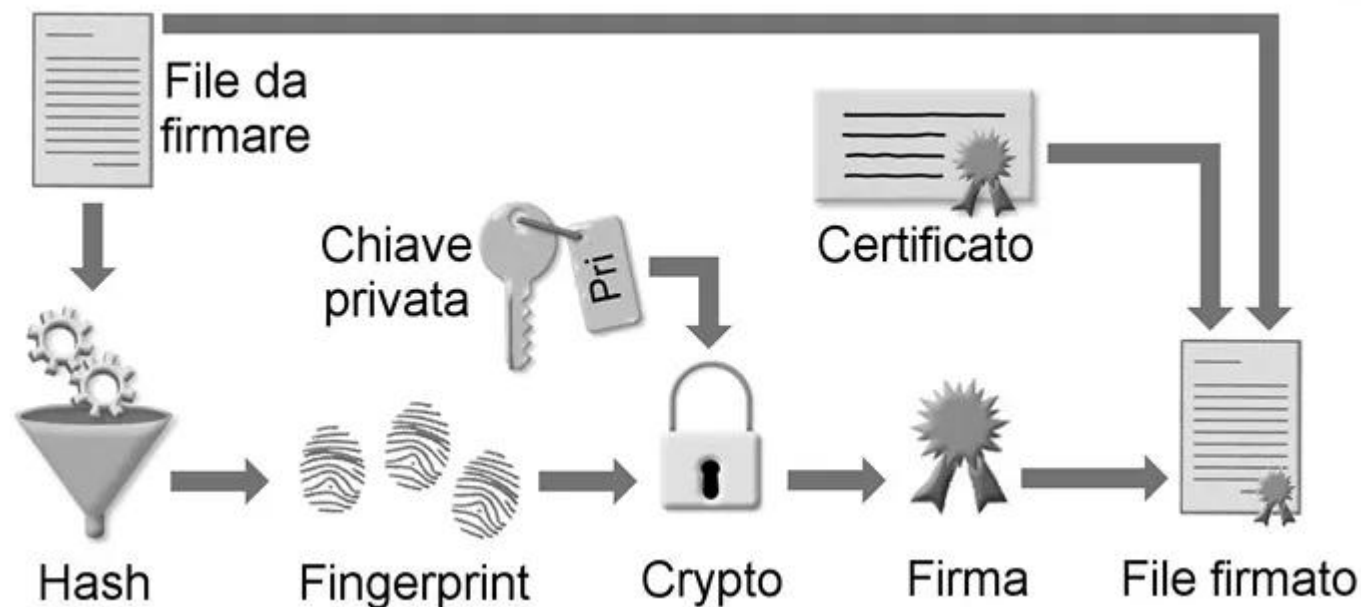
CRITTOGRAFIA A CHIAVE ASIMMETRICA

Se voglio aggiungere integrità e non ripudio devo basarmi sull'utilizzo di funzioni di hash, anche chiamate impronte digitali. Si applicano a un messaggio da crittare, generando delle impronte associate al messaggio stesso. La funzione di hash è iniettiva, cioè non invertibile, quindi dall'impronta non è possibile risalire al messaggio che l'ha generata.

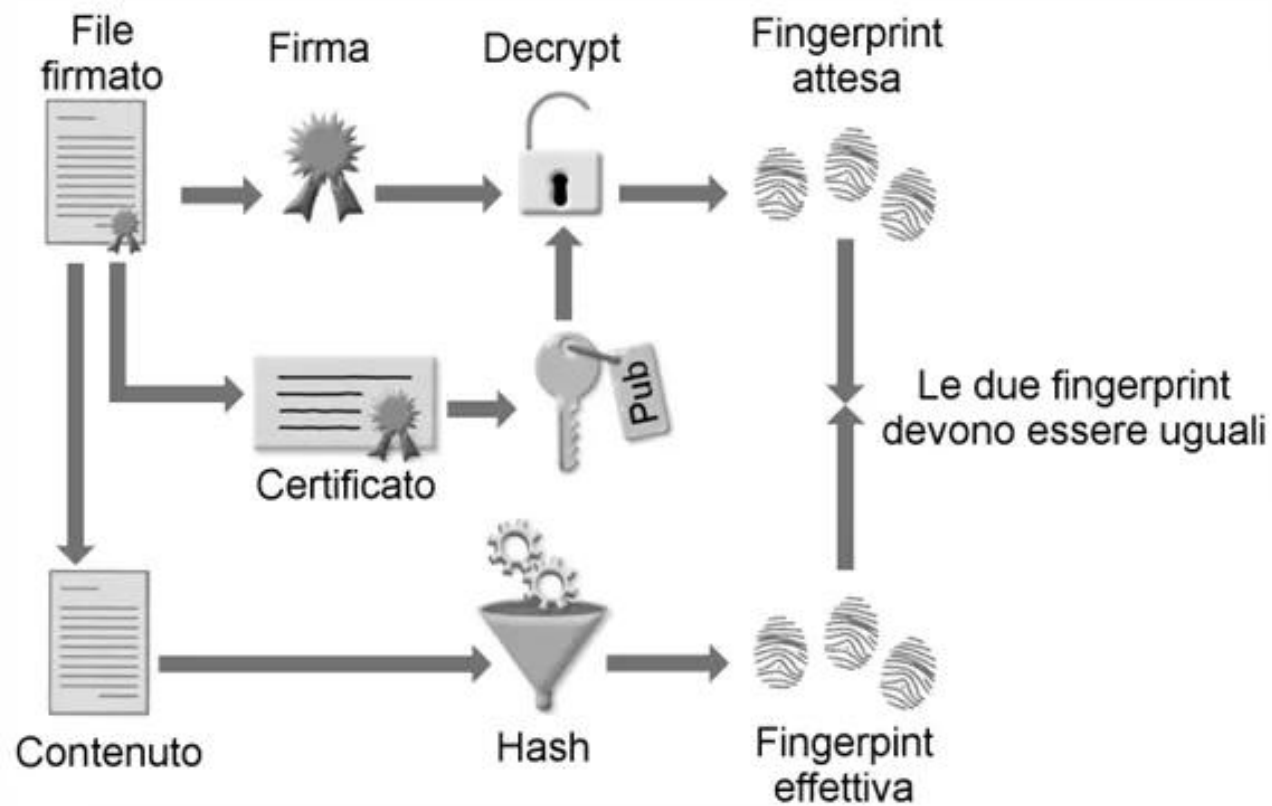


CRITTOGRAFIA A CHIAVE ASIMMETRICA

Una volta generata l'impronta, invece di applicare la funzione crittografica direttamente al messaggio, la applico all'impronta usando la chiave privata. Il tutto viene poi incapsulato in un pacchetto con formato standard di tipo PKCS7.



CRITTOGRAFIA A CHIAVE ASIMMETRICA



GENERARE CERTIFICATI SECONDO LO STANDARD X.509 V3

```
francesca@LAPTOP-5V7UQQGH:~$ openssl req -x509 -newkey rsa:4096 -nodes -out
certificate.pem -keyout chiave_privata.pem -days 365
Generating a RSA private key
.....++++
.....++++
writing new private key to 'chiave_privata.pem'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:IT
State or Province Name (full name) [Some-State]:Napoli
Locality Name (eg, city) []:Fuorigrotta
Organization Name (eg, company) [Internet Widgits Pty Ltd]:BeneventanoGiaqui
nto
Organizational Unit Name (eg, section) []:uni
Common Name (e.g. server FQDN or YOUR name) []:Francesca
Email Address []:francescabeneventano.fb@gmail.com
francesca@LAPTOP-5V7UQQGH:~$
```

L'RSA è un cifrario esponenziale e la sua sicurezza risiede nella difficoltà computazionale di fattorizzare grandi numeri interi, rendendo impraticabile risalire alla chiave privata partendo da quella pubblica.

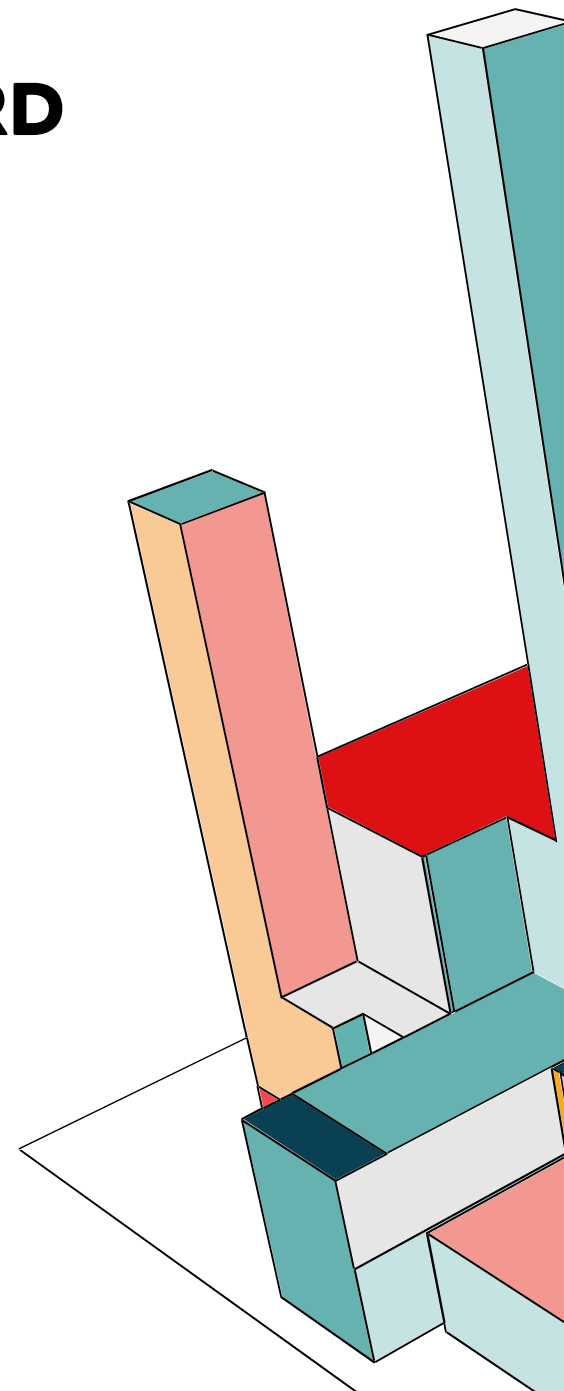
Nel nostro esempio, generiamo un certificato secondo lo standard X.509, utilizzando l'algoritmo asimmetrico RSA con una chiave privata a 4096 bit, che useremo per firmare il certificato.



GENERARE CERTIFICATI SECONDO LO STANDARD X.509 V3

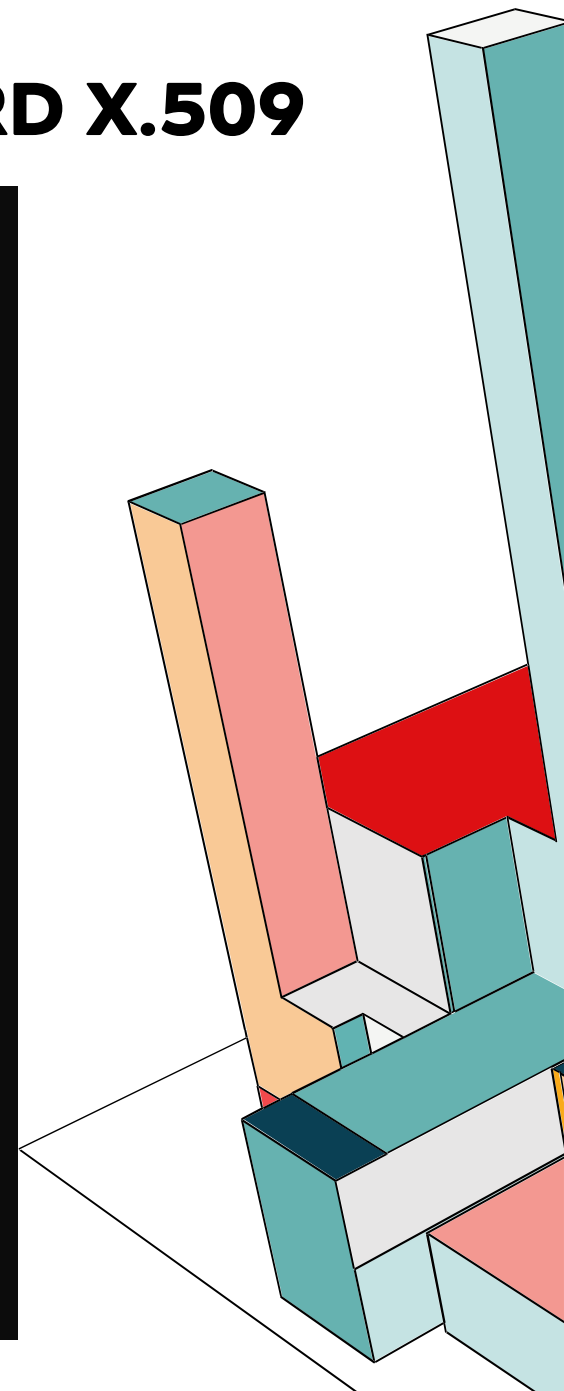
Visualizziamo la chiave privata che abbiamo generato:

```
francesca@LAPTOP-5V7UQQGH:~$ cat chiave_privata.pem
-----BEGIN PRIVATE KEY-----
MIIBJQIBADANBgkqhkiG9w0BAQEFAASCCSwwggknAgEAAoICAQC76Y0ge+d1uesN
gXJJJhU+s2Jo+e1zk/u3U5g95pKMYAVQrKejwbDMLEQGqWlowVesnbjZMuDkpuu/
gkbY0CeAcJBw3+7QfX19nkV258iR5FUHuqQ/2JdYdLM07Mvs2L5gBCFWJ2qdHoHB
LMLxyWldJuVCqM7YgvnnexRavXFz0/ZY72IVCoogabHo3zh1HGv1Z+w5yEXkDS5x
OnoFbX6XrMvQ5AX38WYzBrEehDmHk8dU6ZD1X16ZACcmJj8u/1bGrY5877GrwV/n
GDSiMzIzJ4AUblbgyXmQ6ci7Sk+vpjQz3Wu1HdIvep+S33ga1IEIDjtW0KHzaBPs
bCF0K7y35ZIIhpduNpUtQFHxo0pZrSj8T2Yn8UvG1rbn1nwBz8pyIp+2jvlnrIEg
QAADH3KVJxUsV/fF/CXtgG9GIu/Px5RWOPh3GqShE5iaEqItbap65WgoBosSXUZe
8z09dxsgtCgePZIH0Xr5NO1f0p9z06tiw/4i41z/Gv+05T5CMNjHajfMbbidT0bs
IgeRJDmVNO3OMwmGFpGwZPnc21/ys48LH/1vq7XrWfYsNZX4Qedji+4D0Io1p1rx
uTPoOQ+ZADvL0Cw+VL4QbXPn0k3sX3NIgbA7SMVP0Ktvfd8mHELFY+/8DSmggcFNF
PiAA29nBq418SQL6gtu7wG2w8IptTwIDAQABAoICAEBu7P1efxdXEMoowM9kR5Sg
XfYw/MLM01shRuqyzT1lEagUC8es+tSrYkgHDCh7d6Ic0uInDMZaliousC5P8buQ
3XZW0sSawXQC7NRU1eLwqddkoD1kP0ENgvhzKdmlZwb1Y6HYwXntCoEeSvou/fn7
BFG/IG4NOz0seGZE30nsHaSRMMiWPJawp1h6chl2DW6wm8eUbKkuAmbA7mY+DVJk
5d5S0dka5ThJ6yHTpQ2GhdF35US91uoPq1zhcHjLBRRLHrioV/vU6tWk70F03L+9
o6Vd0FfxzuUN/ZL0SNhC8pNnD6tLHzvKI0X7DMA0EyJxChQYkI8kavvoqea4j8S1m
6bJt0ymIB3R0C9G+HeLG+J+FPj/wvaLC5Ir3d6BiYBFWVTCc7vCPAC32SNHhus3q
oxK+Ei9Gp/LQbeK5ly291FmSMZM4nIbyvIB28bkUniUwcbq22xioCJwvnyIkH7eK
7W2TA597ppAGTBd2dFD2H1Vq8ESf6E9eAVQMcVyr+m353G/05M01eFY92tZ7+XII
Y6rvqQ+Jhi6UsW8/Qzcewwf8Sc5I8kiyTQ4d2pIXRAP7bkd/Q06Rn8Ush/984p/
zRoec6UeuBqmJvzJX40Mqk3N/LTtwcuor/rhGML6MSAVesPvJcEv7SBn36CnQ09Y
PEWP4sblGWZRkhpT8oBAoIBAQM44iNdgm4pU2I2Yx1UpEjuilB3NMxQnHc6XBI
psQLoCY4pv/iQgaAX1dupHeKZx8D4XyQJm8gywUyJaxzfETANwSBEN9IRJKY6W8M
hpHiH5zL0S/vtQJxNg1ML5KPEEtH0kqkBC9QLeb2cWkx+QS8k15woI8A+Hi08hmd
ZXcKPrpTgWR7b3t2SgRWI+2I9MyaL2hGL3KKHrPgaH2RDcty381c0hBGdwsnBxhy
JI4Yj7m0dr2Hbz/VZE7Zg6innTAVMARXbeBJLodzE0XXs2uRtkzeVvqf5/LsTX1r
rYT1Yxp3C5Lgxvi7HUTsXsc4y0gJ20hw5hTM9Ezp7qFtc+9hAoIBAQQDQWXSALeDD
vFTJPRdT4L+ViES3/EfSBI2Qo9eod0BFTJzE/LxNB2zceJSGAJutG0E3g40kzlyu
EcZHTsJWey5B5TWS9amhh4ZDPFLPLXSmKbJ5c5hj3uhsu5sa6+FP3sX+UynmdXvr
Z7ttdpwNzUmwjtDbimfruTLnXON+3+P+H7qK+V66IDmJMotp16IpcQR8A/LAhK1b
K53+KCGLM14vWds/00cfQ/XwJnbT/QDPrCPsEerg4aUf0Jpco+RELE6iDRWAg//4
ldXKgzsvKohSAB8te03YoLti/B3Nz0UyQur2yyZwt8wx6AUwVnx4Y1tFJuX3eg1D
ziJ0m7LTCAqvAoIBAFWcEEnDdp9MS49uerIVx1Mj96BALN6a8HUK6ULG7ts0qp1e
7ooKSTbYIycXPGVJCKR7tJ9QI0vz2x1Y8rRvZceNntHsSDHv+zVPeKmATB838YkV
I10SFLG9hPi2uvTo6cCjUIG404LTQkwmIqBKeoMhcWb5YYz0LIUMJ4Gc4aKe15+W
3pcWtUdcAmwKkZafhnhAfoKtaGY+MFAfS6HmuQhL5f+v4IhKbViy48GSL1vHTJB
vMy250hMRSBJvQNJOubh5WmZjnI6EAePIdp04HHK0n2uaodYEBekM9j1Id0Kx4LU
dUV6/t4EYqVypvPcC/37nMSfyx6jhtFGdfxk7IECggEADW2Xhd7P6XRu78cxK0dc
vpIE3QImj9P9xZPW33rInAhL33nIO2ba0ZxXkkoLj403QztIq6iNZVlPTTeh7kI
o4wq7b5ADkKbavovBX532hWDJpE6fms5iDmZwD0x2Fyya2kayvzMAjsRaJw8NCct
ZMrWvjXiS/hcFELS0PteJRsnnCi6NlpjN3TMA84/khZvRQKIfkFZwo/xL065b86h
TGxkg3IF9nVTjJFK3pAWAesfAUxeGvdSLcniJTVWJbpNYBnwXv0qfKxdEO+gUiwr
gzktB300J6W7UK+ee6MFIDeDgGc3FMfCwgxQa6ij/5K2Im3oCZRQ5wgclg9c0WqI
UwKCAQAvwy3D6Yb3VhdfCcx74Nqic5eiz4S0Y6twwIHW45J0BaVfuJ2NUnXpg/WL
RWFtWUtudEqbDfJA2Cvvgv3wVgDk08IeuqL94k9i3PILKZaUrA3HHP5Ze/I31A2y
rPLK0zi+kECNIhpgbreLChKwcvruisEQ0JWudGFmR32YGCWwgy0vJx/Qp77H1Psk
XTGTRPGbt3YI4W8A0P4Hh4l+/j7DsilsY0m6GnpbEvrrn4Wov78KEZG12ILXSG2ND
YGX9JHY/V/mtLunOTI/EvgdXw8FP7eU4MbU0yK6ZvoX7abeUuh/Bo45auEZ7L6S
xxHi+4Ft5JmSLtIsFS/Iz0iXzvAM
-----END PRIVATE KEY-----
francesca@LAPTOP-5V7UQQGH:~$ |
```



Visualizziamo anche il certificato:

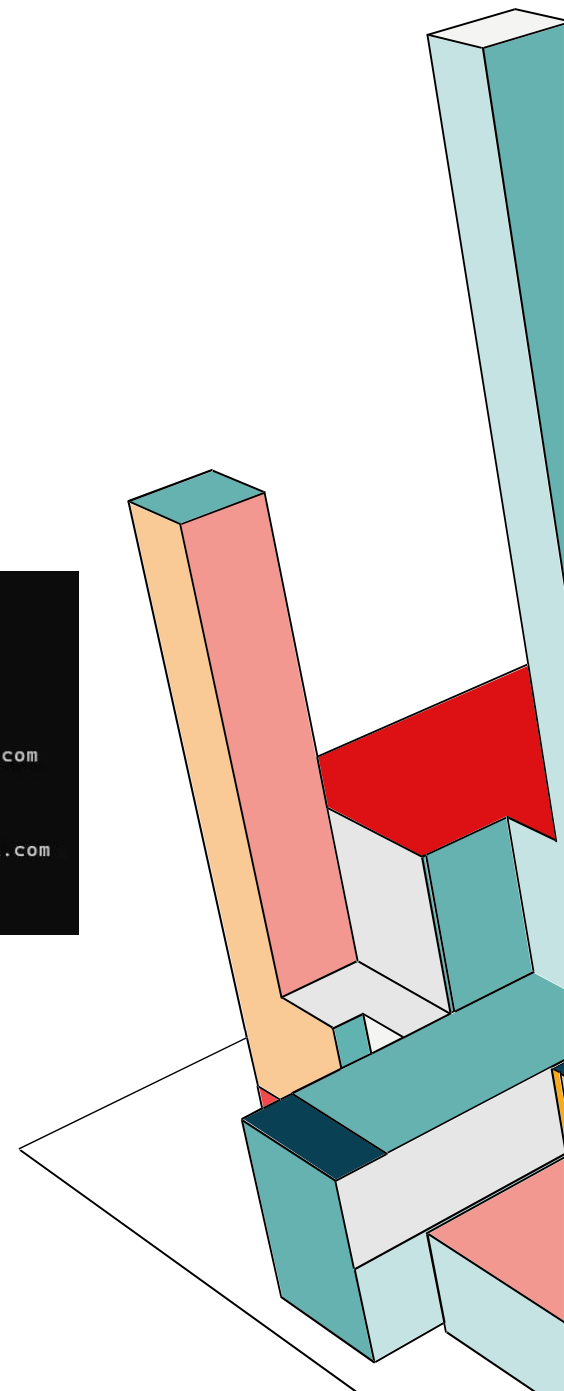
```
francesca@LAPTOP-5V7UQQGH:~$ ls
certificate.pem  chiave_privata.pem
francesca@LAPTOP-5V7UQQGH:~$ cat certificate.pem
-----BEGIN CERTIFICATE-----
MIIGMTCCBBmgAwIBAgIUNriOkdsb4ORdew/WpR6pQhLLpwMwDQYJKoZIhvcNAQEL
BQAwgagcCzAJBgNVBAYTAklUMQswDQYDVQQLIDAOYXBvbGxkFDASBgNVBAsCMC0Z1
b3JpZ3JvdHhrrMR0wGwYDVQKQDBRCZlW5ldmVudGFub0dpYXFlaw50bzEMMAoGA1UE
CwwDdW5pMRIwEAYDVQQDDALGcmFuY2VzY2ExMDAuBgkqhkiG9w0BCQEWIwZyYW51
ZXNjYwJlbnV2ZW50YW5vLmZiQGdtYwlsLmNvbTAeFw0yNTEeXmJEWOTU5NThaFw0y
NjExMjEwOTU5NThaMIGnMQswCQYDVQQGEWJJVDEPMA0GA1UECAwGTmFwY2xpMRQw
EgYDVQQHDAtGdW9yaWdyb3R0YTEdMBsGA1UECgwUQmVuZCZlbnRhbW9haWwFxdWlW
dG8xDDAKBgNVBAsMA3VuaTESMBAGA1UEAwwJRnJhbmNlc2NhMTAwLgYJKoZIhvcN
AQkBFIFmcmFuY2VzY2FiZW5ldmVudGFuby5mYkNbnWfPbC5jb20wggEiIiMA0GCSqG
S1B3DQEAQUAUAICDwAwggIKAOICAQC76Y0ge+d1uesNgXJJJhU+s2Jo+e1zk/u3
U5g95pKMYAvQrKejwDbMDLEQGqwlOwVesnjbZMeDdkpuu/gkbY0CeACJBw3+7QfXi9
nKv258iR5FUHuqQ2JdYdLM07Mvs2L5gBCFwJZqdHoHBLMLxyWldJuVCqM7Ygvrn
exRavXFz0/ZY72IVCoogabHo3zh1HGv1Z+w5yEXkDS5x0noFbX6XrMvQ5AX38WYz
BrEehDmHk8dU6ZD1X16ZACcmJj8u/1bGrY5877GrwV/nGDSiMzIzJ4AUblbgYxmQ
6ci75k+vpjQz3Wu1HdIvep+S33ga1EIDjtWOkHzaBPsbCF0K7y35ZIIhpduNpUt
QFHxo0pZrSj8T2Yn8UvG1rbn1nwBz8pyIp+2jvlnrIEgQAAHd3KVJxUsV/fF/CXt
gG9GIu/Px5RW0Ph3GqShE5iaEqItbap65WgoBosSXUZe8z09dxsxtCgePZIH0Xr5
N01f0p9z06tiw/4i41z/Gv+05T5CMNJhaffMbdiDt0bSiG6RJDmVn030MwmGFpGW
zPnc21/s48lH/1vq7XrWfYSNZX4Qedji+4D0Io1p1rxuTPoOQ+ZADvL0Cw+VL4Q
bXPn0k3sX3NIgBA7SMVP0Ktvfd8mHELFY+/8DSmggcFNpIAA29nBq418SQL6gtu7
wG2w8IptTwIDAQAB01MwUTADBgNVHQ4EFgQUBwSwLVtDON+iygCt/ypy8MoD07Lsw
HwYDVR0jBBgwFoAUBwSwLVtDON+iygCt/ypy8MoD07LswDwYDVR0TAQH/BAUwAwEB
/zANBgkqhkiG9w0BAQsFAA0CAgEAI76bzylQNbhaLkbjLJ8/ZM6krqjXK9sP/Cre
rHTf7vquedP05yHa/1Adaks9G+FbhJxsgANmWllxOkImBSwuAFv169V8vQFLA2e+
nEYsnfDKMNSy1TUAJmt+5L8jHEpbr/AAPMP51/s0GN5DPvvr0NdqW3/miaXkCtJz
zkjrkf4hKX9c2H07WLfD/dHMuReSaulshMVswSboIngrnk/ALseAVk4aTMkQ48C
s0VpC2D2LeRJSMJSyohuLdI0MECCRk2ebwc08ey4TypNzIB12uoycDYUp5Cf5AIm
Ezs72fYK10F+ck+YHY1VamFSJ6lycI37oPAZPNzBzFFHdnGptEsOXFxc9/hsqG+hn
uSBaBXVipcIMyPScojlnEzvhQ6K2MXIrXz1uDggttXFQp3YKXceNwSpMyJmxoNNL
p8UscjvX2Tll+rK+4NtbIuNRaLA++faED9rEc5EwU407JHAjXleJ4RL4pzCcJNzm
nj/ejwvW3YGCqBUX8E/PXgvQcpYeHCEn60rB3LUDeqHt+yEFMK41PMwM601AKgN7
KksU3+5GVXm+wVKS3+IzaXpSDyDZoFKK7wAtWsljTUhFpSbEcU8Lcp4W+UkwUZ7c
Dhx3z4B4sVLPICBreIZQx13L6Fbx9Y19kgyRMTAiBTCHHOqtOI6Ppgc49MdDnXtW
0Hszg7E=
-----END CERTIFICATE-----
francesca@LAPTOP-5V7UQQGH:~$ |
```



GENERARE CERTIFICATI SECONDO LO STANDARD X.509 V3

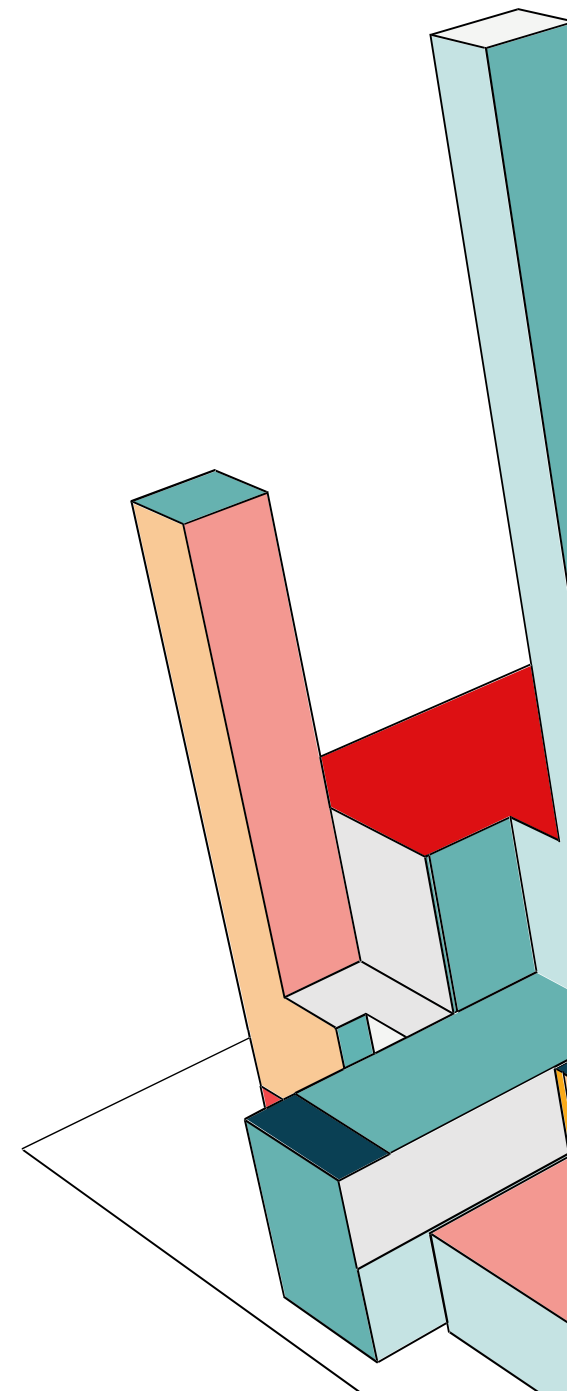
Adesso visualizziamo sempre in certificato, ma in modo che sia leggibile:

```
francesca@LAPTOP-5V7UQQGH:~$ openssl x509 -in certificate.pem -text -noout
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      36:b8:8e:29:db:1b:e0:e4:5d:7b:0f:d6:a5:1e:a9:42:19:4b:a7:03
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C = IT, ST = Napoli, L = Fuorigrotta, O = BeneventanoGiaquinto, OU = uni, CN = Francesca, emailAddress = francescabeneventano.fb@gmail.com
    Validity
      Not Before: Nov 21 09:59:58 2025 GMT
      Not After : Nov 21 09:59:58 2026 GMT
    Subject: C = IT, ST = Napoli, L = Fuorigrotta, O = BeneventanoGiaquinto, OU = uni, CN = Francesca, emailAddress = francescabeneventano.fb@gmail.com
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      RSA Public-Key: (4096 bit)
```



GENERARE CERTIFICATI SECONDO LO STANDARD X.509 V3

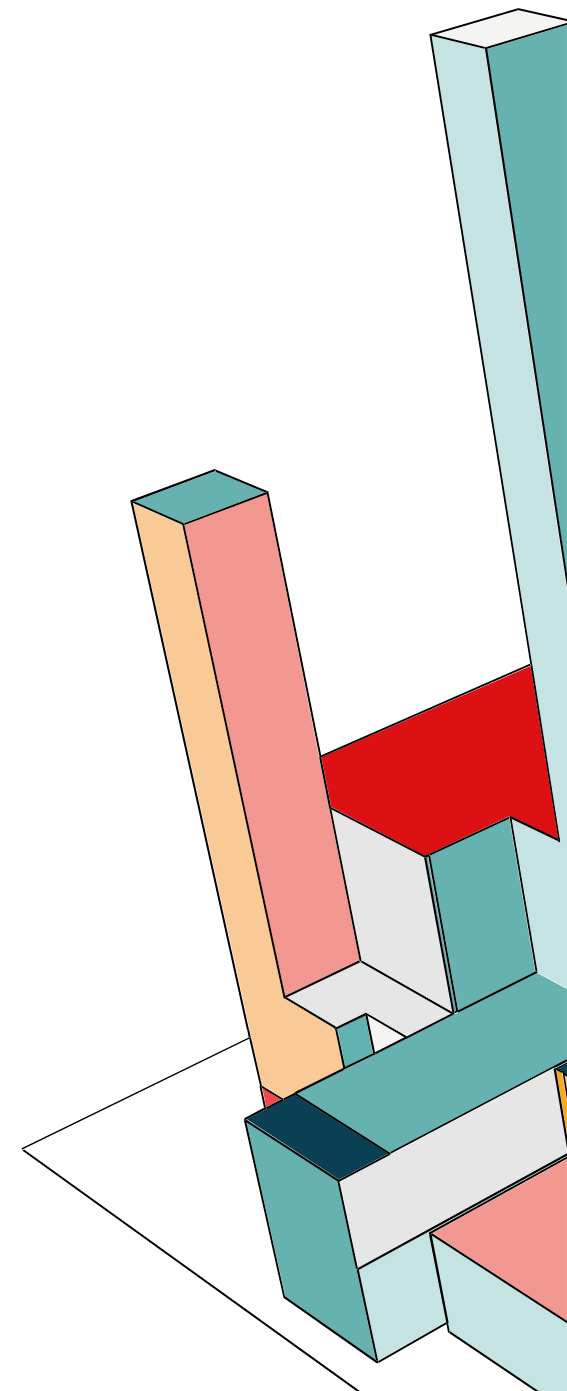
```
Modulus:
00:bb:e9:8d:20:7b:e7:75:b9:eb:0d:81:72:49:26:
15:3e:b3:62:68:f9:ed:73:93:fb:b7:53:98:3d:e6:
92:8c:60:0b:d0:ac:a7:a3:c1:b0:cc:94:44:06:ab:
09:4e:c1:51:2c:9d:b8:d9:32:e0:e4:a6:eb:bf:82:
46:d8:d0:27:80:70:90:70:df:ee:d0:7d:78:bd:9c:
ab:f6:e7:c8:91:e4:55:07:ba:a4:3f:d8:97:58:74:
b3:0e:ec:cb:ec:da:5e:60:04:21:70:25:9a:9d:1e:
81:c1:94:c2:f1:c9:69:5d:26:e5:42:a8:ce:d8:82:
f9:e7:7b:14:5a:bd:71:73:3b:f6:58:ef:62:15:0a:
8a:20:69:b1:e8:df:38:75:1c:6b:f5:67:ec:39:c8:
45:e4:0d:2e:71:3a:7a:05:6d:7e:97:ac:cb:d0:e4:
05:f7:f1:66:33:06:b1:1e:84:39:87:93:c7:54:e9:
90:f5:5f:5e:99:00:27:26:26:3f:2e:ff:56:c6:ad:
8e:7c:ef:b1:ab:c1:5f:e7:18:34:a2:33:32:33:27:
80:14:6e:56:e0:c9:79:90:e9:c8:bb:4a:4f:af:a6:
34:33:dd:6b:b5:1d:d2:2f:7a:9f:92:df:78:1a:23:
51:08:0e:3b:56:3a:41:f3:68:13:ec:6c:21:74:2b:
bc:b7:e5:92:08:86:97:6e:36:95:2d:40:51:f1:a3:
4a:59:ad:28:fc:4f:66:27:f1:4b:c6:d6:b6:e7:d6:
7c:01:cf:ca:72:22:9f:b6:8e:f9:67:ac:81:20:40:
00:07:77:72:95:27:15:2c:57:f7:c5:fc:25:ed:80:
6f:46:22:ef:cf:c7:94:56:38:f8:77:1a:a4:a1:13:
98:9a:12:a2:2d:6d:aa:7a:e5:68:28:06:8b:12:5d:
46:5e:f3:33:bd:77:1b:20:b4:28:1e:3d:92:07:d1:
7a:f9:34:ed:5f:3a:9f:73:3b:ab:62:5b:fe:22:e3:
5c:ff:1a:ff:b4:e5:3e:42:30:d2:61:6a:37:cc:6d:
d8:9d:4f:46:d2:22:0e:91:24:39:af:34:ed:ce:33:
09:86:16:91:96:cc:f9:dc:db:5f:f2:b3:8f:25:1f:
fd:6f:ab:b5:eb:58:5c:92:35:95:f8:41:e7:63:23:
ee:03:38:8a:25:a7:5a:f1:b9:33:e8:39:0f:99:00:
3b:e5:d0:2c:3e:56:5e:10:6d:73:e7:d2:4d:ec:5f:
73:48:81:b0:3b:48:c5:4f:d0:ab:6f:7d:df:26:1c:
49:45:63:ef:fc:0d:29:a0:81:c7:cd:3e:20:00:db:
d9:c1:ab:8d:7c:49:02:fa:82:db:bb:c0:6d:b0:f0:
8a:6d:4f
Exponent: 65537 (0x10001)
X509v3 extensions:
X509v3 Subject Key Identifier:
6D:2C:0B:56:D0:CE:37:E8:B2:80:2B:7F:CA:9C:BC:32:80:F4:EC:BB
X509v3 Authority Key Identifier:
keyid:6D:2C:0B:56:D0:CE:37:E8:B2:80:2B:7F:CA:9C:BC:32:80:F4:EC:BB
```



GENERARE CERTIFICATI SECONDO LO STANDARD X.509 V3

```
X509v3 Basic Constraints: critical
CA:TRUE
Signature Algorithm: sha256WithRSAEncryption
23:be:9b:cf:29:50:35:b8:5a:2c:a6:e3:94:9f:3f:64:ce:a4:
ae:a8:d7:2b:db:0f:fc:2a:de:ac:74:df:ee:fa:ae:79:d3:f4:
e7:21:da:ff:50:1d:6a:4b:3d:1b:e1:5b:84:9c:6c:80:09:cc:
c2:59:71:3a:42:26:05:2c:2e:01:f5:75:eb:d5:7c:bd:01:4b:
03:67:9f:9c:46:2c:9d:f0:ca:30:d4:b2:d5:35:00:26:6b:7e:
e6:5f:23:1c:4a:5b:af:f0:00:a4:c3:f9:d7:fb:34:18:de:43:
3e:fb:d1:d0:d7:6a:5b:7f:e6:89:a5:ca:71:38:f3:ce:48:eb:
29:fe:21:29:7f:5c:d8:73:bb:58:b7:c3:fd:d1:cc:b9:17:92:
6a:ec:e5:b2:13:15:b1:64:9b:a3:59:e0:9e:b9:3f:00:bb:1e:
01:59:38:69:33:24:43:8f:02:b0:e5:69:09:90:f6:2d:e4:49:
48:c2:52:ca:88:6e:2e:20:74:30:40:a4:46:4d:9e:6f:07:34:
f1:ec:b8:23:2a:4d:cc:80:75:da:ea:32:70:36:14:a7:90:9f:
e4:02:26:13:3b:3b:d9:f6:0a:d7:41:7e:72:4f:98:1d:8d:55:
6a:61:52:27:a9:72:70:8d:fb:a0:f0:19:3c:dc:c1:7c:51:dd:
9c:6a:6d:12:c3:97:17:17:3d:fe:1b:2a:1b:e8:67:b9:20:5a:
05:75:62:a5:c2:0c:c8:f4:9c:a2:39:67:13:3b:e1:43:a2:b6:
31:72:2b:5f:3d:6e:0c:68:2d:b5:71:50:a7:76:0a:5d:c7:8d:
59:2a:4c:c8:99:b1:a0:d3:65:a7:c5:2c:72:3b:d7:d9:39:65:
fa:b2:be:e0:db:5b:22:e3:51:68:b0:3e:f9:f6:84:0f:da:c4:
73:91:30:53:83:bb:24:70:23:5e:57:a3:e1:12:f8:a7:30:9c:
8c:dc:e6:9e:3f:de:8f:0b:d6:dd:81:82:a8:15:17:f0:4f:cf:
5e:0b:d0:72:96:1e:1c:21:27:eb:4a:c1:dc:b5:03:7a:a1:ed:
fb:21:05:30:ae:25:3c:cc:0c:e8:ed:40:2a:03:7b:2a:4b:14:
df:ee:46:55:79:be:c1:52:92:df:e2:33:69:7a:52:0f:20:d9:
a1:f2:8a:ef:00:2d:5a:c9:63:4d:48:45:a5:26:c4:71:4f:25:
72:9e:16:f9:49:30:51:9e:dc:0e:1c:77:cf:80:78:b1:59:4f:
20:20:6b:78:86:50:c7:5d:cb:e8:56:f1:f5:8d:7d:92:0c:91:
31:30:08:6d:30:87:1c:ea:ad:38:8e:8f:a6:07:38:f4:c7:43:
9d:7b:56:d0:7b:33:83:b1
```

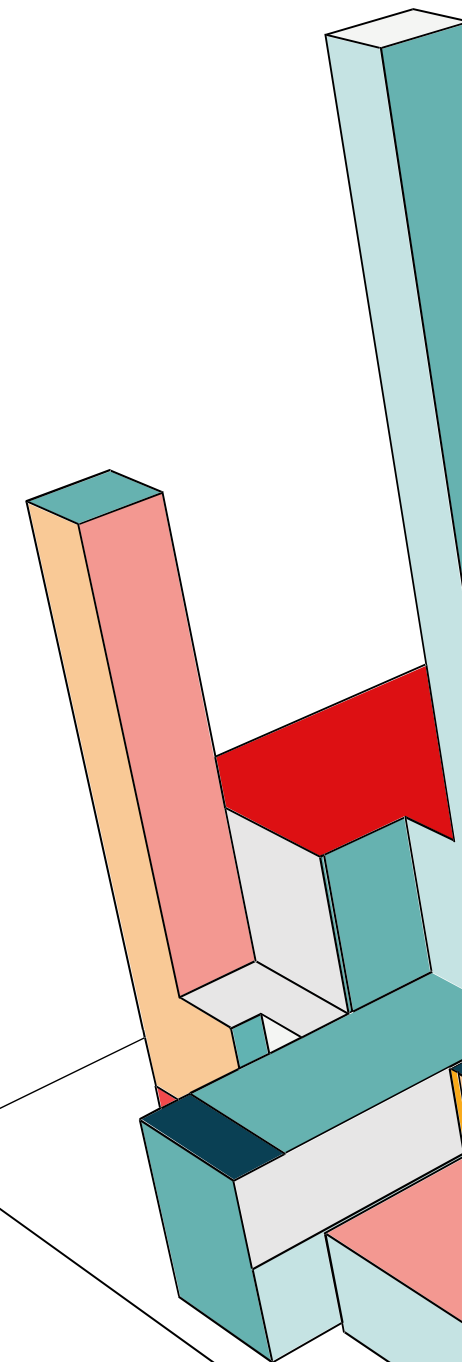
```
francesca@LAPTOP-5V7UQQGH:~$
```



GENERARE CERTIFICATI SECONDO LO STANDARD X.509 V3

Abbiamo estratto la chiave pubblica dal certificato appena creato e l'abbiamo salvata nel file `chiave_pubblica.pem`, che poi abbiamo usato per crittare il `messaggio.txt`.

```
francesca@LAPTOP-5V7UQQGH:~$ openssl x509 -in certificate.pem -pubkey -noout > chiave_pubblica.pem
francesca@LAPTOP-5V7UQQGH:~$ openssl pkeyutl -encrypt -inkey chiave_pubblica.pem -p
ubin -in messaggio.txt -out messaggio.enc
francesca@LAPTOP-5V7UQQGH:~$ ls
certificate.pem      chiave_pubblica.pem  messaggio.txt
chiave_privata.pem  messaggio.enc
francesca@LAPTOP-5V7UQQGH:~$ cat messaggio.enc
}D5=/Wtwe%De=S?5jpe lzz>I)T%-eV<Vf$~K9;L.7K
8n2Q;_ $y8Xe'd'4]  𐤃 \b\
;B}gg?c{8FtXh@$~i< }tWPaaP,R, w -w@CW0, b
00'4000O&ÈZ';UIIu$~<G, vÚ[vaRWcrr>N M} \[HvA(?[
3
/<B5| |r a[ | Y
yyç0WypjQpI.Der Oh9
_ye7e>Xnn%G^_K;&W?"xju]frances
```



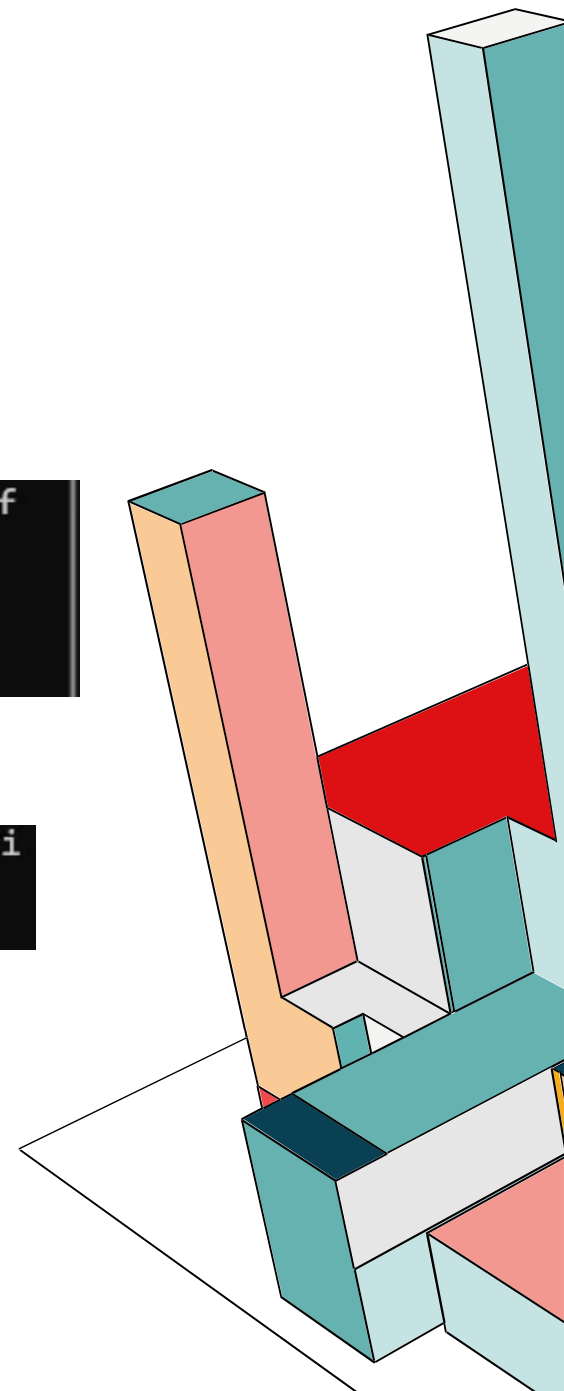
GENERARE CERTIFICATI SECONDO LO STANDARD X.509 V3

Ora firmiamo il `messaggio.txt` con la chiave privata:

```
francesca@LAPTOP-5V7UQQGH:~$ openssl dgst -sha256 -sign chiave_privata.pem -out firma.bin messaggio.txt
francesca@LAPTOP-5V7UQQGH:~$ ls
certificate.pem      chiave_pubblica.pem  messaggio.enc
chiave_privata.pem  firma.bin            messaggio.txt
```

Verifichiamo se la **firma** è valida:

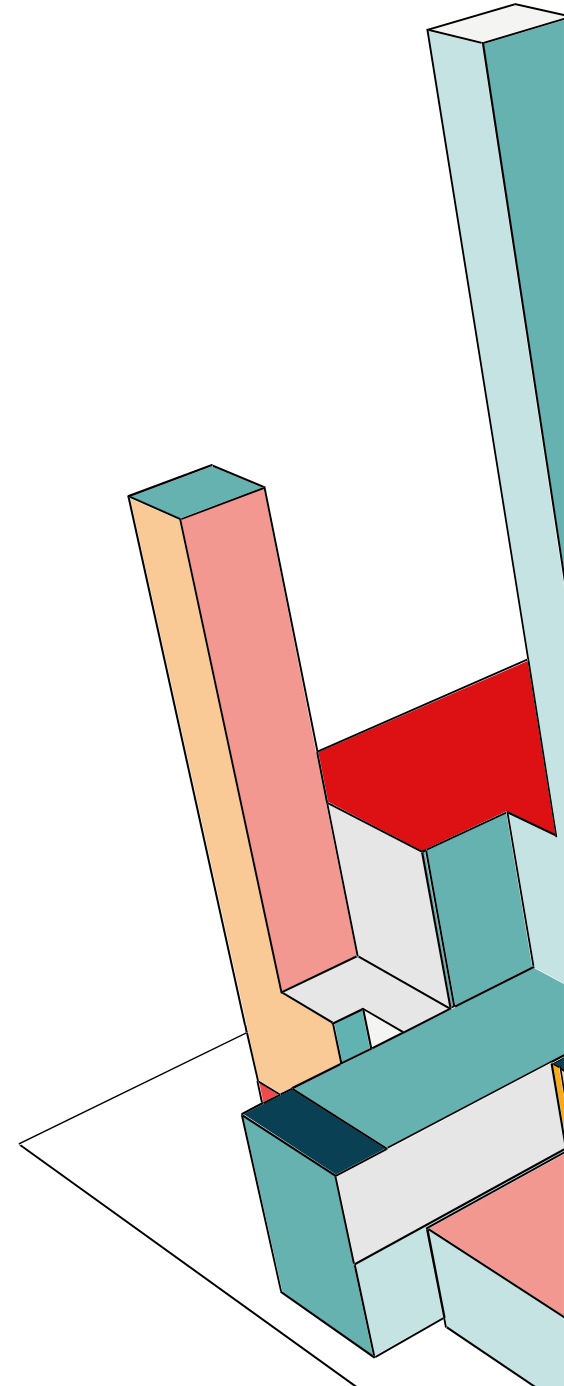
```
francesca@LAPTOP-5V7UQQGH:~$ openssl dgst -sha256 -verify chiave_pubblica.pem -signature firma.bin messaggio.txt
Verified OK
```



CHE COS'È .P12?

Il file .p12 è un formato di file binario che segue lo standard PKCS#12 (Public-Key Cryptography Standards). Serve per archiviare e trasferire in modo sicuro sia la chiave privata che il certificato pubblico in un unico file.

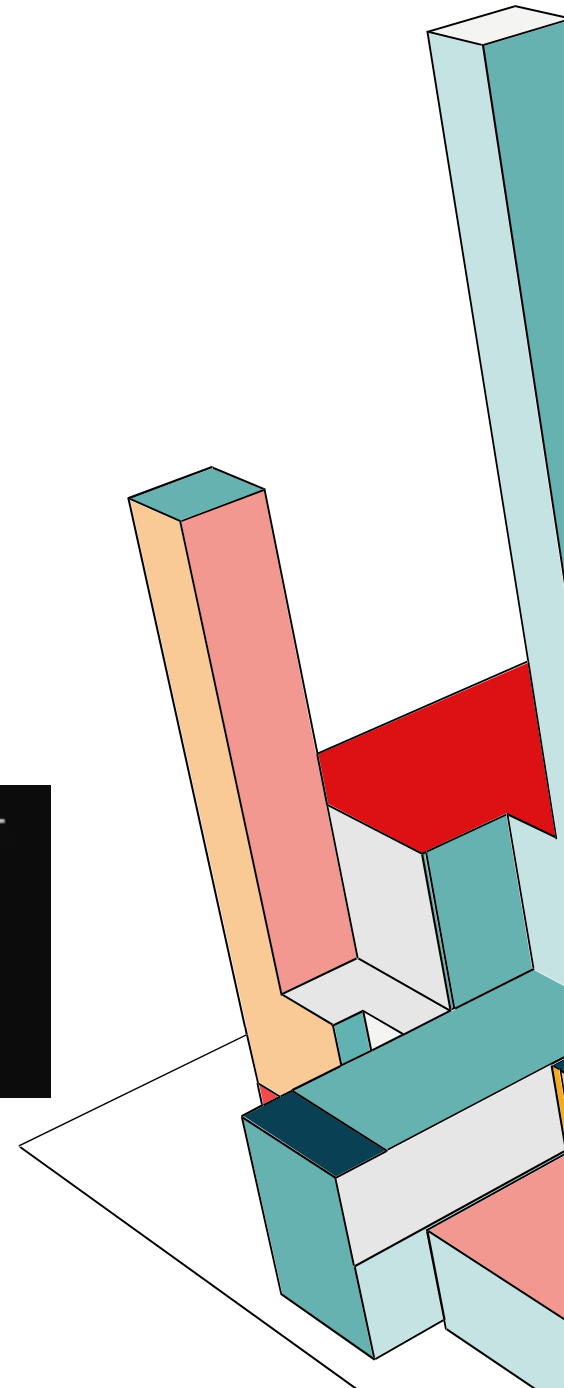
A differenza dei file .pem, che sono spesso file contenenti solo la chiave o solo il certificato, il .p12 li unisce e li protegge con una password. Questo è fondamentale perché per dimostrare l'identità hai bisogno di entrambi.



CONSERVARE LA CHIAVE PRIVATA IN P12

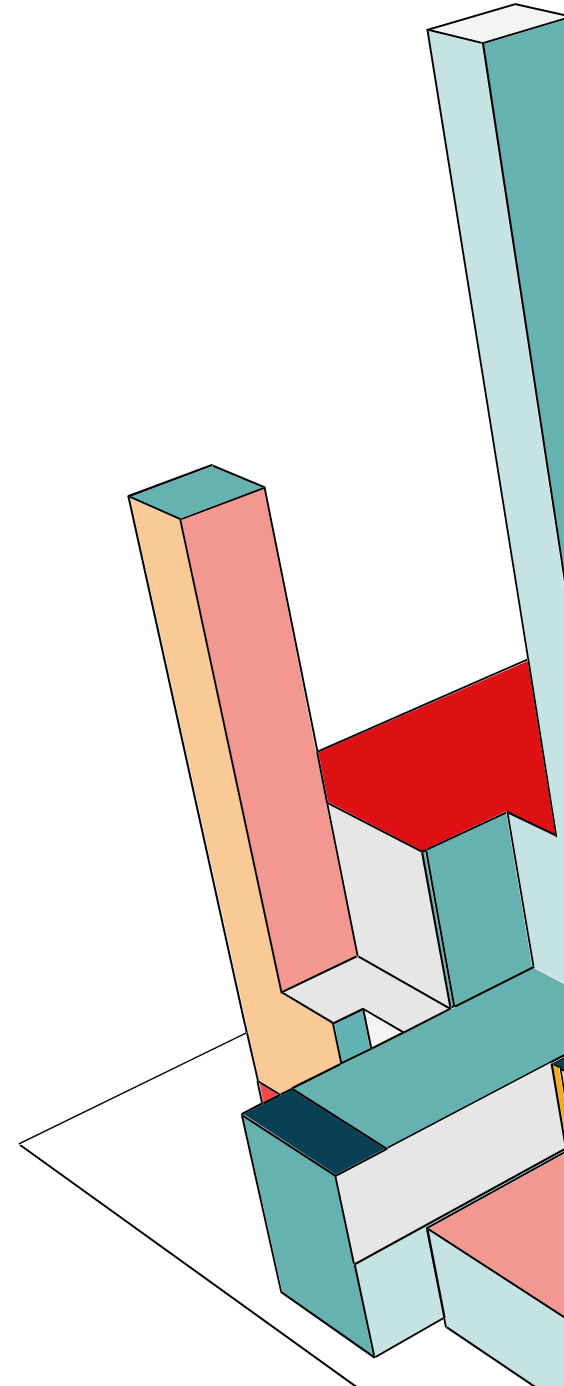
Abbiamo utilizzato il comando `pkcs12 -export` per generare il file `.p12` in cui salviamo la `chiave_privata.pem`, il `certificato.pem` e lo abbiamo chiamato con l'alias generico «myalias». Il tutto viene poi protetto scegliendo una password di esportazione.

```
francesca@LAPTOP-5V7UQQGH:~$ openssl pkcs12 -export -inkey chiave_privata.pem -  
in certificate.pem -name "myalias" -out archivio.p12  
Enter Export Password:  
Verifying - Enter Export Password:  
francesca@LAPTOP-5V7UQQGH:~$ ls  
archivio.p12      chiave_privata.pem  firma.bin  
certificate.pem  chiave_pubblica.pem  messaggio.txt
```



CHE COS'È KEYTOOL?

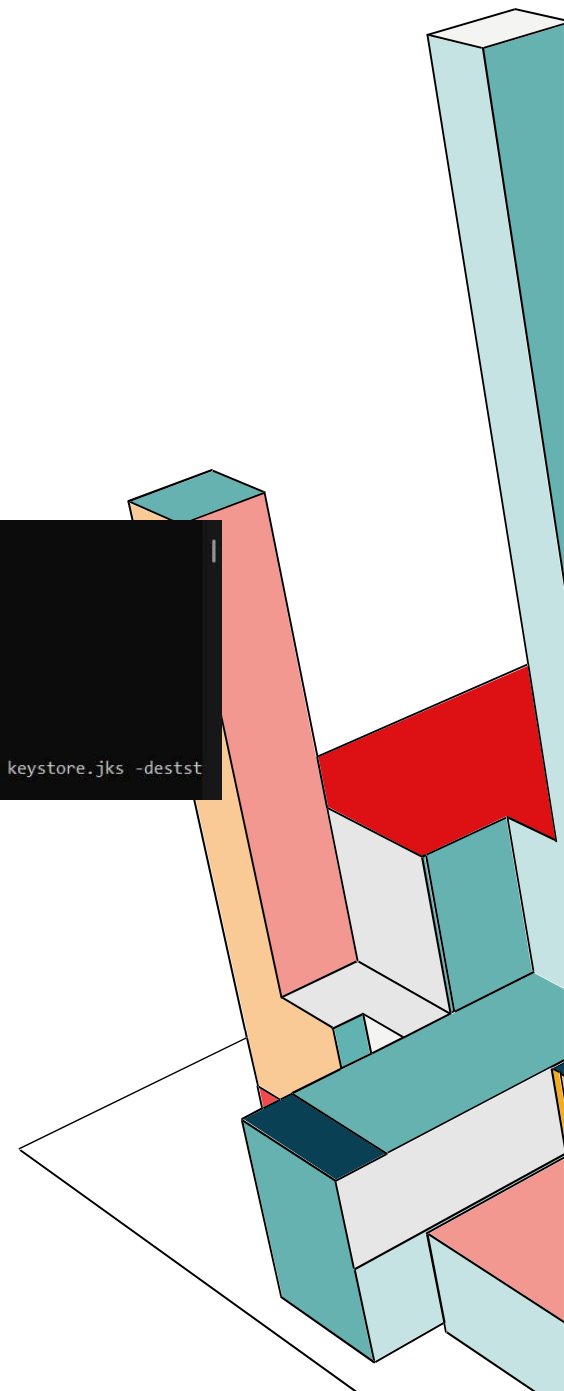
Keytool è un'utility a riga di comando inclusa nel JDK (Java Development Kit). È lo strumento standard di Java per gestire chiavi e certificati. Nello specifico, permette di gestire i **Keystore**, ovvero dei database crittografati che contengono le chiavi private e certificati pubblici.



IMPORTAZIONE P.12 IN KEYTOOL

Nel nostro caso abbiamo usato **Keytool** per prendere il file standard **.p12** e importarlo dentro un **keystore.jks**.

```
francesca@LAPTOP-5V7UQQGH:~$ keytool -importkeystore \  
> -srckeystore archivio.p12 -srcstoretype PKCS12 \  
> -destkeystore keystore.jks -deststoretype JKS \  
Importing keystore archivio.p12 to keystore.jks... \  
Enter destination keystore password: \  
Re-enter new password: \  
Enter source keystore password: \  
Entry for alias myalias successfully imported. \  
Import command completed: 1 entries successfully imported, 0 entries failed or cancelled \  
  
Warning: \  
The JKS keystore uses a proprietary format. It is recommended to migrate to PKCS12 which is an industry standard format using "keytool -importkeystore -srckeystore keystore.jks -destkeystore keystore.jks -deststoretype pkcs12".
```



IMPORTAZIONE .P12 IN KEYTOOL

```
francesca@LAPTOP-5V7UQ0GH:~$ keytool -list -v -keystore keystore.jks
Enter keystore password:
Keystore type: JKS
Keystore provider: SUN

Your keystore contains 1 entry

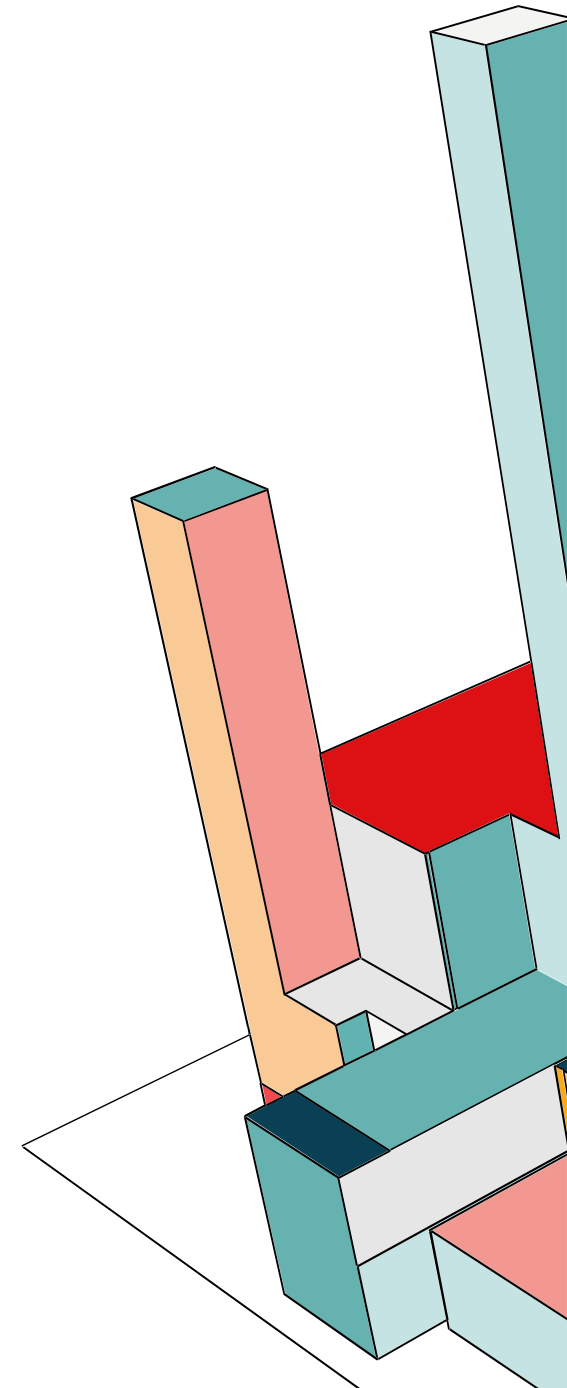
Alias name: myalias
Creation date: Nov 21, 2025
Entry type: PrivateKeyEntry
Certificate chain length: 1
Certificate[1]:
Owner: EMAILADDRESS=francescabeneventano.fb@gmail.com, CN=Francesca, OU=uni, O=BeneventanoGiaquinto, L=Fuorigrotta, ST=Napoli, C=IT
Issuer: EMAILADDRESS=francescabeneventano.fb@gmail.com, CN=Francesca, OU=uni, O=BeneventanoGiaquinto, L=Fuorigrotta, ST=Napoli, C=IT
Serial number: 36b88e29db1be0e45d7b0fd6a51ea942194ba703
Valid from: Fri Nov 21 10:59:58 CET 2025 until: Sat Nov 21 10:59:58 CET 2026
Certificate fingerprints:
    SHA1: 1B:E1:75:AD:B0:F2:C5:3A:38:21:BD:69:B3:C5:17:5C:9B:06:9F:B2
    SHA256: 5C:F4:EA:6E:70:6D:BF:A3:27:57:EB:D8:B8:C3:17:9A:C1:1F:13:2E:43:04:19:0E:11:CA:F0:B4:C2:FD:EE:F1
Signature algorithm name: SHA256withRSA
Subject Public Key Algorithm: 4096-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.35 Criticality=false
AuthorityKeyIdentifier [
KeyIdentifier [
0000: 6D 2C 0B 56 D0 CE 37 E8   B2 80 2B 7F CA 9C BC 32   m,.V..7...+....2
0010: 80 F4 EC BB               ....
]
]

#2: ObjectId: 2.5.29.19 Criticality=true
BasicConstraints:[
CA:true
PathLen: no limit
]

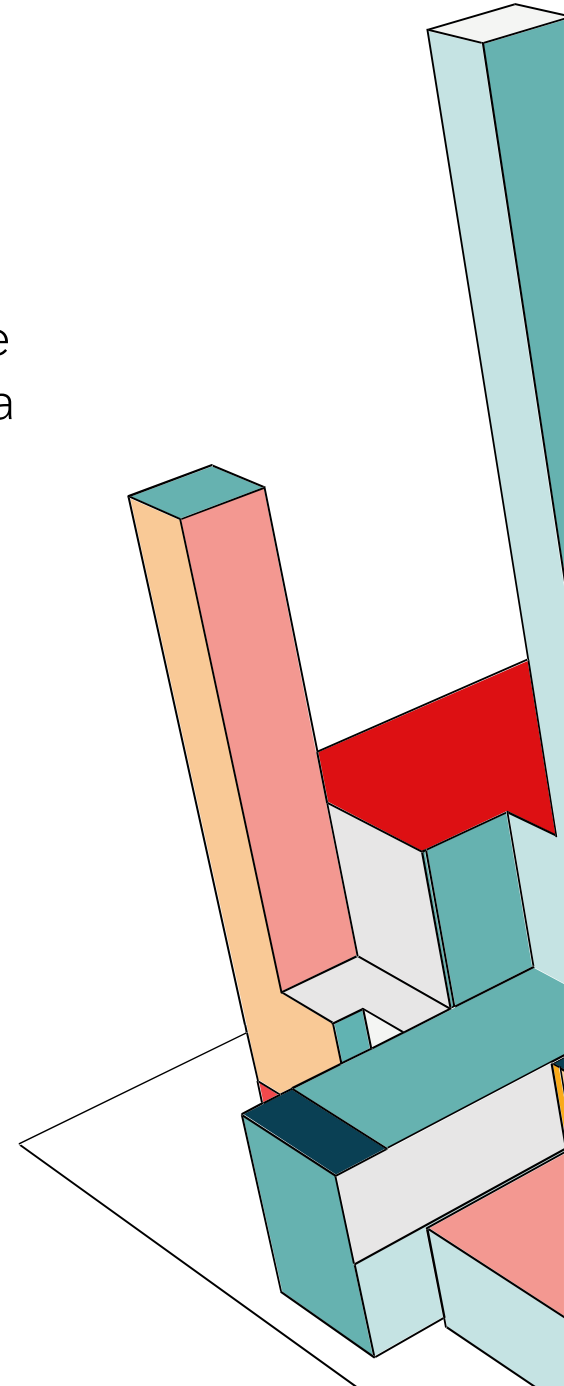
#3: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 6D 2C 0B 56 D0 CE 37 E8   B2 80 2B 7F CA 9C BC 32   m,.V..7...+....2
0010: 80 F4 EC BB               ....
]
]
```



CHE COS'È VAULT?

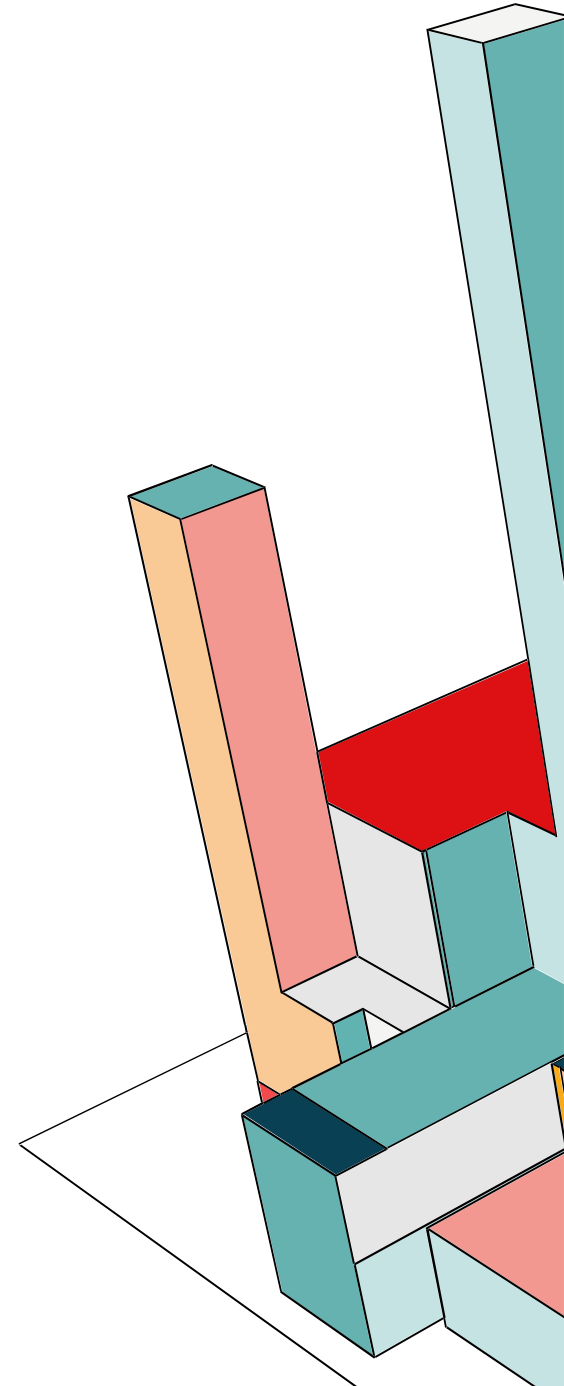
Vault è uno strumento per la gestione sicura dei segreti, delle chiavi e delle credenziali all'interno di sistemi, server e applicazioni. Si tratta di un sistema **centralizzato** che conserva, protegge e controlla l'accesso a informazioni sensibili come:

- password
- chiavi API
- chiavi AES, RSA, certificati
- token di accesso
- credenziali per database
- file o dati crittografati
- certificati TLS automatici



VAULT

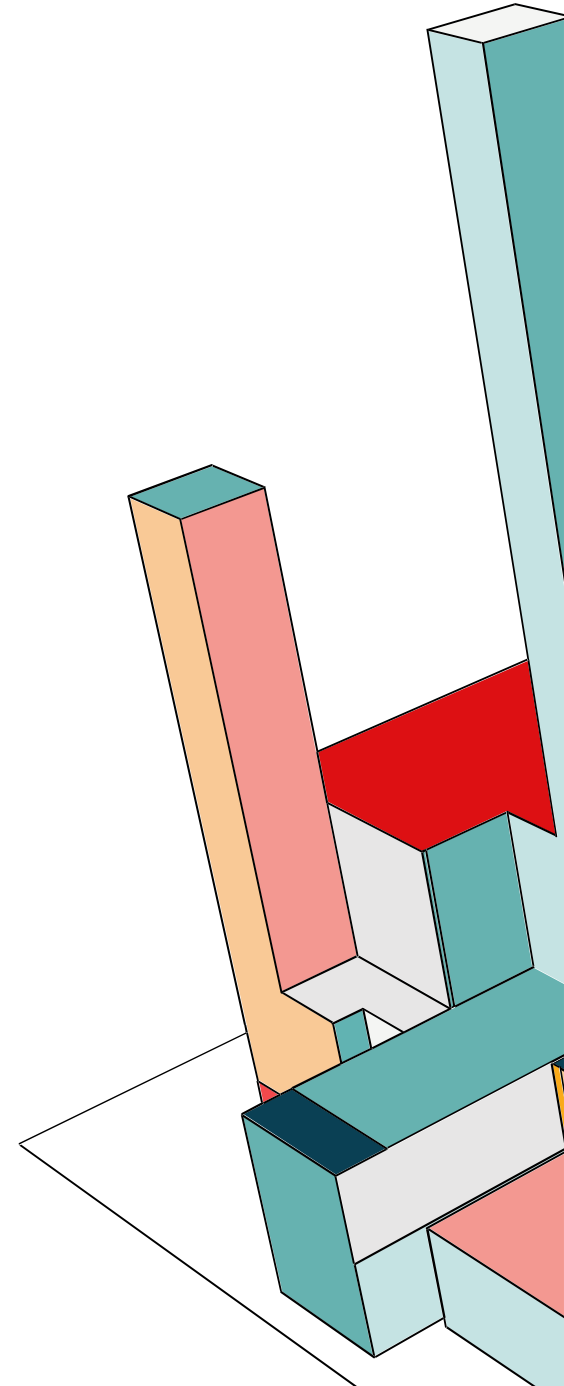
In questo modo risolviamo il problema di dove salvare password e chiavi per garantirne la sicurezza. Lo strumento infatti permette di memorizzarle in un unico punto, cifrate e accessibili solo tramite policy, API e token temporanei.



SALVARE SEGRETI IN VAULT

Come prima cosa proteggiamo la chiave privata con crittografia, gestendone anche l'accesso. Usiamo sempre la chiave privata generata con RSA e la proteggiamo con una password che riteniamo sicura. Il risultato sarà un file `chiave_privata_enc.pem` cifrato.

```
francesca@LAPTOP-5V7UQQGH:~$ openssl rsa -aes256 -in chiave_privata.pem -out chiave_privata_enc.pem
writing RSA key
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
francesca@LAPTOP-5V7UQQGH:~$ ls
archivio.p12      chiave_privata.pem      chiave_pubblica.pem  keystore.jks
certificate.pem   chiave_privata_enc.pem  firma.bin             messaggio.txt
```



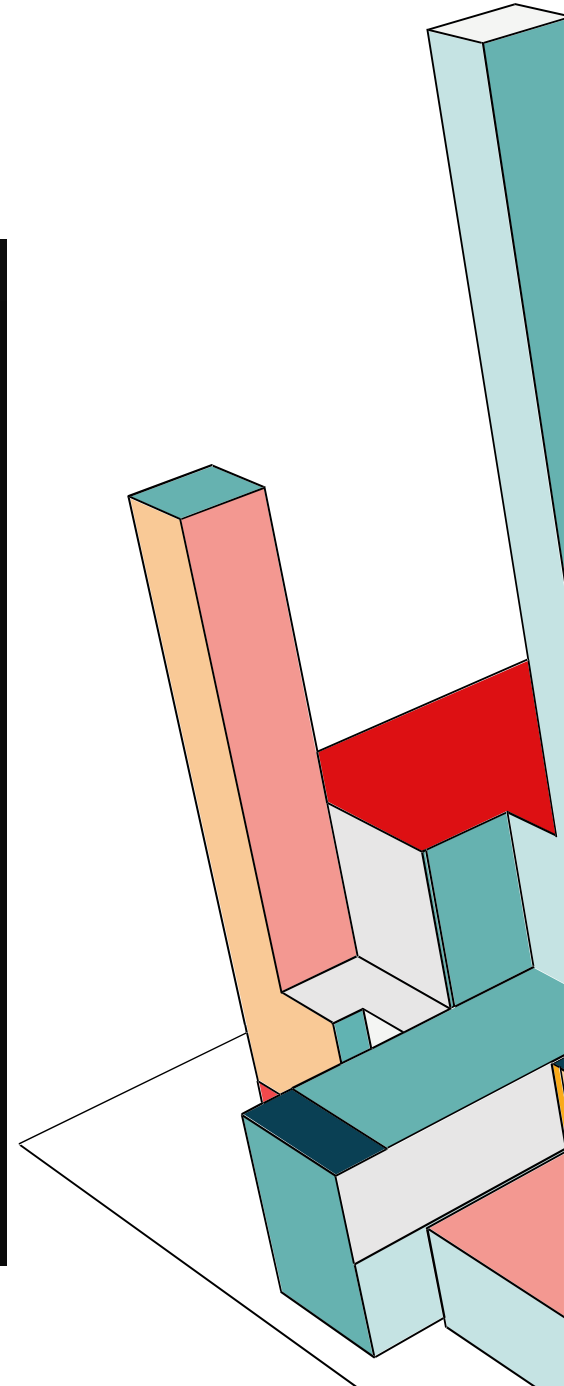
SALVARE SEGRETI IN VAULT

Adesso avviamo Vault in modalità development, ci verrà fornito un Root Token che possiamo utilizzare per autenticarci in Vault e caricare le chiavi e infatti lo facciamo da terminale.

```
francesca@LAPTOP-5V7UQQGH:~$ export VAULT_ADDR='http://127.0.0.1:8200'
loginfrancesca@LAPTOP-5V7UQQGH:~$ vault login hvs.x1NPxl3cGJvrDjj6p501sx9u
Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key          Value
---          -
token        hvs.x1NPxl3cGJvrDjj6p501sx9u
token_accessor syzc73ueHyF1hjUXTUfZqGFs
token_duration ∞
token_renewable false
token_policies ["root"]
identity_policies []
policies      ["root"]
francesca@LAPTOP-5V7UQQGH:~$ vault kv put secret/chiavi/chiave_privata value
=@chiave_privata.pem
===== Secret Path =====
secret/data/chiavi/chiave_privata

===== Metadata =====
Key          Value
---          -
created_time  2025-11-21T11:03:46.208986817Z
custom_metadata <nil>
deletion_time n/a
destroyed     false
version       1
francesca@LAPTOP-5V7UQQGH:~$ ls
archivio.p12      chiave_privata_enc.pem  keystore.jks
certificate.pem   chiave_pubblica.pem    messaggio.txt
chiave_privata.pem firma.bin               vault_1.14.3_linux_amd64.zip
francesca@LAPTOP-5V7UQQGH:~$ vault kv get -field=value secret/chiavi/chiave_
privata > chiave_privata_recuperata.pem
```



SALVARE SEGRETI IN VAULT

< secret < chiavi < chiave_privata

chiavi/chiave_privata

Secret Metadata

JSON

Delete

Copy

Version 1

Create new version +

Key

Value

Version created Nov 21, 2025 12:03 PM

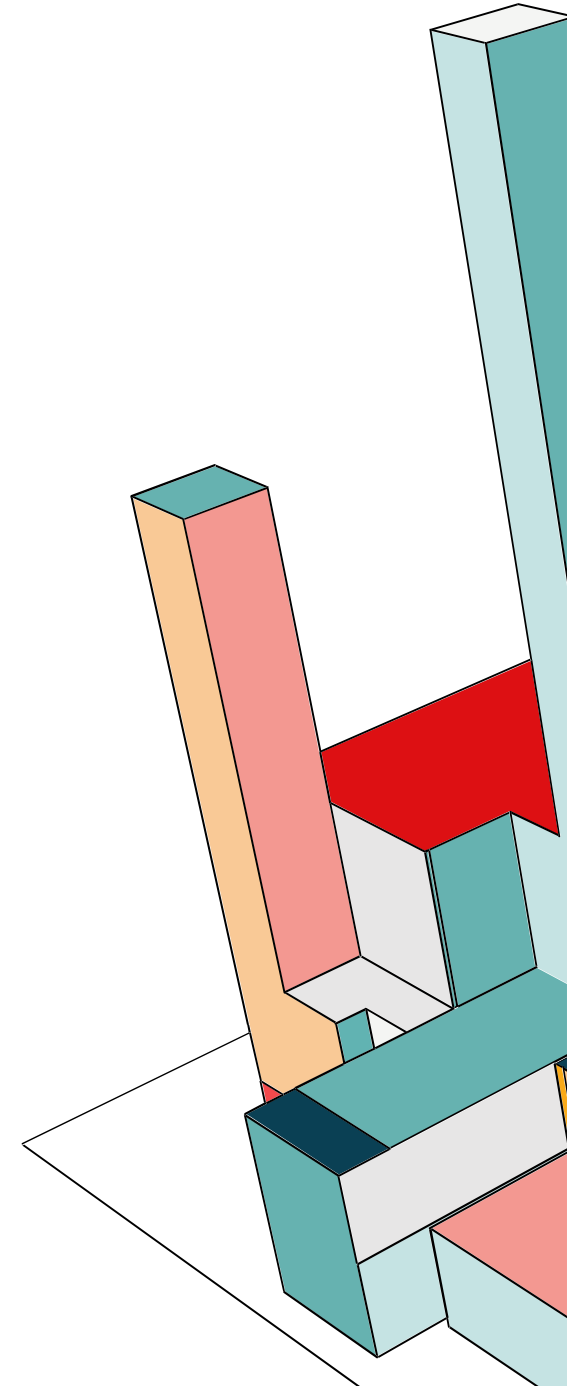
-----BEGIN PRIVATE KEY-----

```
MIIEQgIBADANBgkqhkiG9w0BAQEFAASCCSswggknAgEAAoICAQC76Y0ge+d1uesN
gXJJJhU+s2Jo+e1zk/u3U5g95pKMYAvQrKejwbDM1EQGqwl0wVEsnbjZMuDkpuu/
gkbY0CeAcJBw3+7QfXi9nKv258iR5FUHuqQ/2JdYdLMO7Mvs215gBCFWJZqdHoHB
lMLxyWldJuVCqM7YgvnnexRavXFzO/ZY72IVCoogabHo3zh1HGv1Z+w5YEXkDS5x
OnoFbX6XrMvQ5AX38WYzBrEehDmHk8dU6ZD1X16ZACmJj8u/1bGrY5877GrwV/n
GDSiMzIzJ4AublbgyXmQ6ci7Sk+vpjQz3Wu1HdIvep+S33gaI1EIDjtwOkHzaBP
bCF0K7y35ZIIhpduNpUtQFHxo0pZrSj8T2Yn8UvG1rbn1nwBz8pyIp+2jvlnrIEg
QAAHd3KVJxUsV/ff/CXtgG9GIu/Px5RWOPh3GqShE5iaEqItbap65WgoBosSXUZe
8z09dxsgtCgePZIH0Xr5N0iF0p9z06tiW/4i41z/Gv+05T5CMNHajfMbdidT0bS
Igr6RJDmvNO30MwmGFpGwzPnc21/ys48lH/1vq7XrWfY5NZX4Qedji+4D0Io1p1rx
uTPoOQ+ZADv10Cw+V14QbXPn0k3sX3NIgbA7SMVP0Ktvfd8mHElFY+/8DSmggcFN
PiAA29nBq418SQL6gtu7wG2w8IptTwIDAQABAOICAEBu7P1efxdXEMooWm9KR5Sg
XfYw/MLM01shRuqyzT11EagUC8eS+tSrYkgHdCH7d6Ic0u1nDMZaliuSc5P8buQ
3XZW0sSawXQC7NRU1elwqddkoD1kP0ENGvvhzKdmLZwb1Y6HYwxNtCoEeSVou/fn7
BFG/IG4NOz0seGZE30nsHaSRMMiWPJawp1h6chl2DW6wm8eUbKkuAmbA7mY+DVJk
5d5S0dka5ThJ6yHTpQZgHdF35US91uoPq1zhcHj1BRRLHrioV/vU6tWK70F03L+9
o6Vd0FxxzuUN/Z10SNhC8pNnD6tLHvzKI0X7DMaOEyJxChQYkI8kavvoqea4j8S1m
6bJtOymIB3R0C9G+HelG+J+Ppj/wvaLC5Ir3d6B1YBFWVTc7vCPAC32SNHHus3q
oxK+Ei9Gp/LQbEK5ly291FmSMZM4nIbYvIB28bkUnUwcbq22xioCJwvnyIKH7eK
7W2TA597ppAGTBd2dFD2H1Vq8ESf6E9eAVqMcVyR+m353G/05M01eFY92tZ7+XII
Y6rvqQ+Jh16UsW8/QzcewWf8Sc5I8KiYgTQ4d2pIXRAP7bkd/Q06Rn8Ush/984p/
zRoec6UeuBqmJvzJX40Mqk3N/LTtwcuor/rhGM16MSAVesPvJcEv75Bn36CNq09Y
PEwP4sblGWZrkhpTs8oBAoIBAQM44iNdgmpU2I2Yx1UpEjuilB3NMxQnHc6XBI
psqLoCY4pv/iQgaAX1dupHeKZx8D4XyO3m8gywUyJaxzfEtANwSBKE9IRJKY6W8M
hpHiH5z10S/vtQJxNg1ML5KPEEThOkqkBC9QLeb2cWkx+Q58k15woI8A+Hi08hmd
ZXcPrpTgWR7b3t2SqRWI+2I9Myal2hg13KKHrPgah2RDcty381cOhBGdwsnBxhy
J14Yj7mOdr2Hbz/VZE7Zg6innTAVMARXbeBJ10dzE0XXs2uRtkzeVvqf5/1sTX1r
rYT1Yxp3C5LgxvI7HutSxsc4y0gJ20hw5hTM9Ezp7qFtC+9hAoIBAQQWXSAlEdd
vFTJPRdT41+ViES3/EfSB12Qo9eOD0bFTJzE/LxNB2zceJSGAJutG0E3g40kzlyu
EcZHTsJwey5B5TW5amhh4ZDPFLP1XSmKbJ5c5hj3uhsu5sa6+FP3sX+UynmdXvr
Z7ttDpwNzUMWjtdbIMfruTLnXON+3+P+H7qK+V66IDMjMotp16IpcQR8A/1AhK1b
```

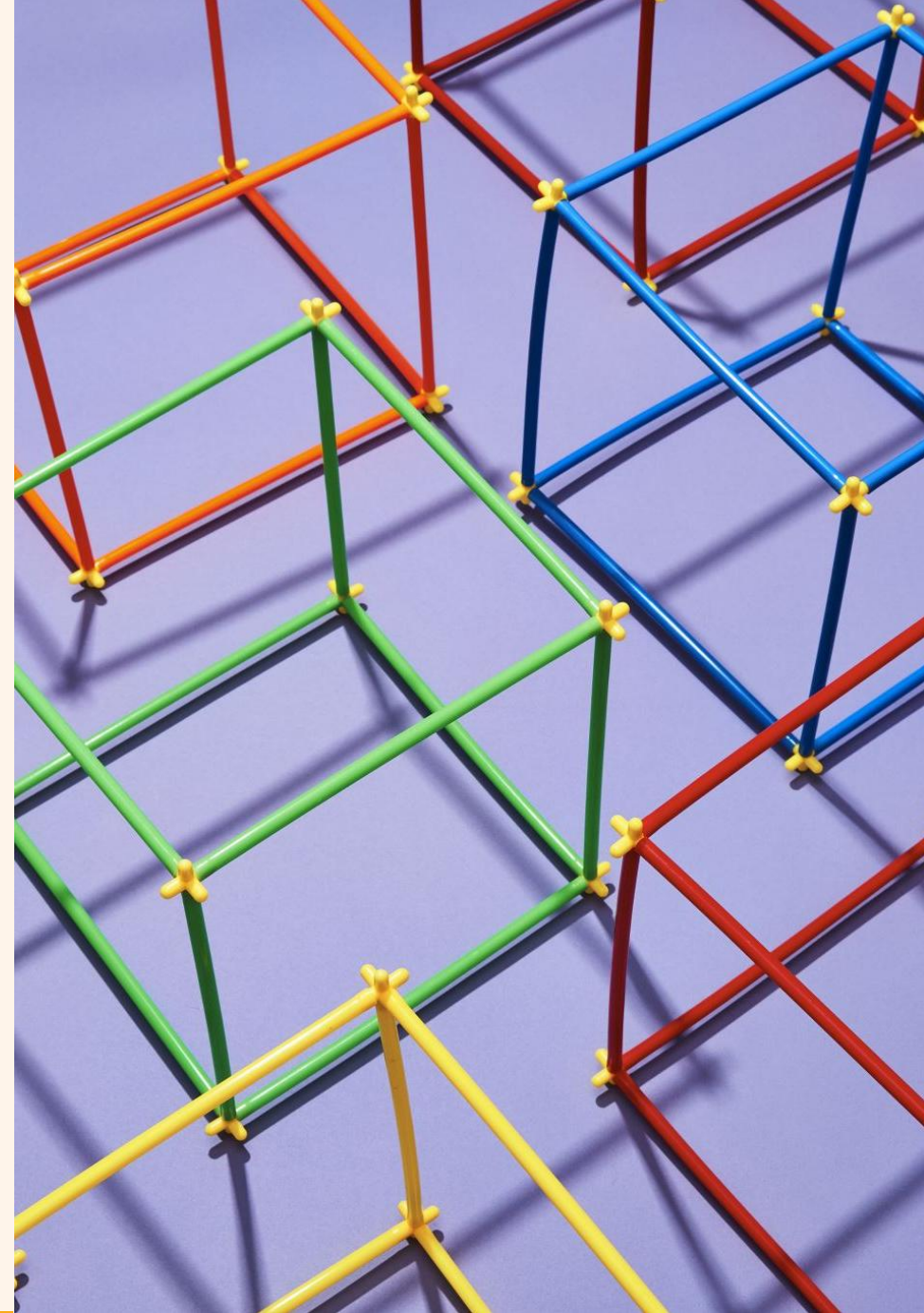
value



Collegandoci all'interfaccia web di Vault possiamo visualizzare i segreti che vi abbiamo appena salvato:

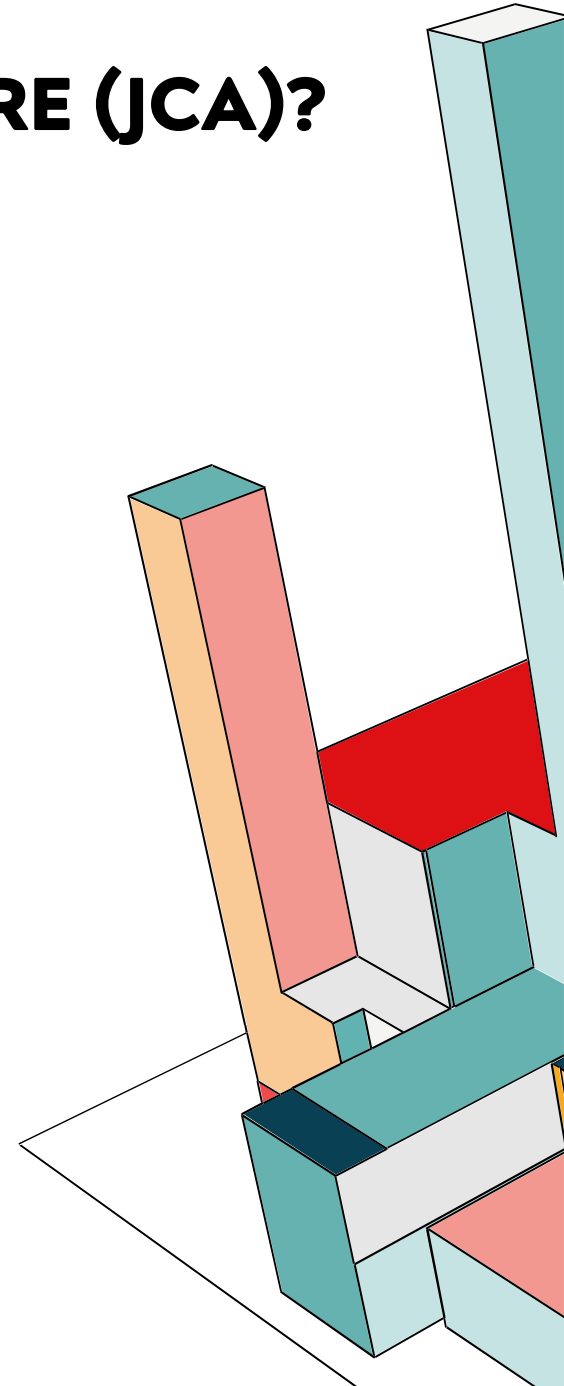


2) JAVA CRYPTHOGRAPHY ARCHITECTURE



CHE COS'È JAVA CRYPTOGRAPHY ARCHITECTURE (JCA)?

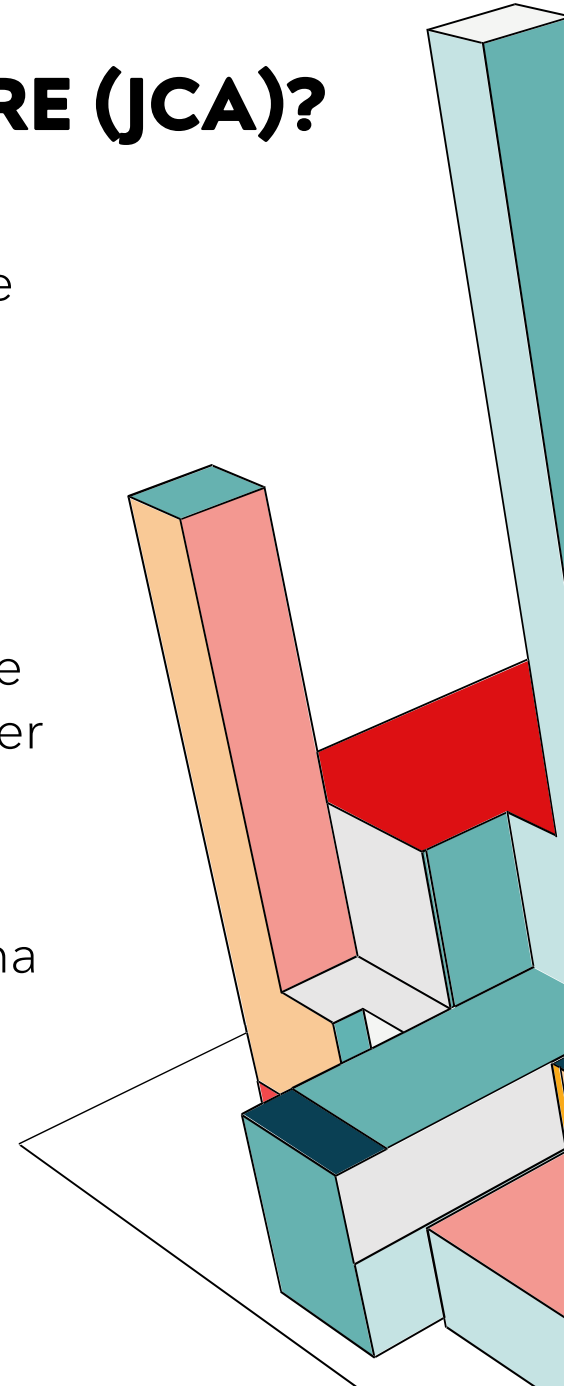
Java Cryptography Architecture è un framework contenente strumenti crittografici come algoritmi per crittografia e firme digitali, nonché classi per la gestione di chiavi e certificati. Si basa su un insieme di API e supporta un'ampia gamma di algoritmi standard come RSA, AES e SHA.



CHE COS'È JAVA CRYPTOGRAPHY ARCHITECTURE (JCA)?

Queste API consentono di integrare facilmente la sicurezza nel codice delle applicazioni. L'architettura è stata progettata attorno ai seguenti principi:

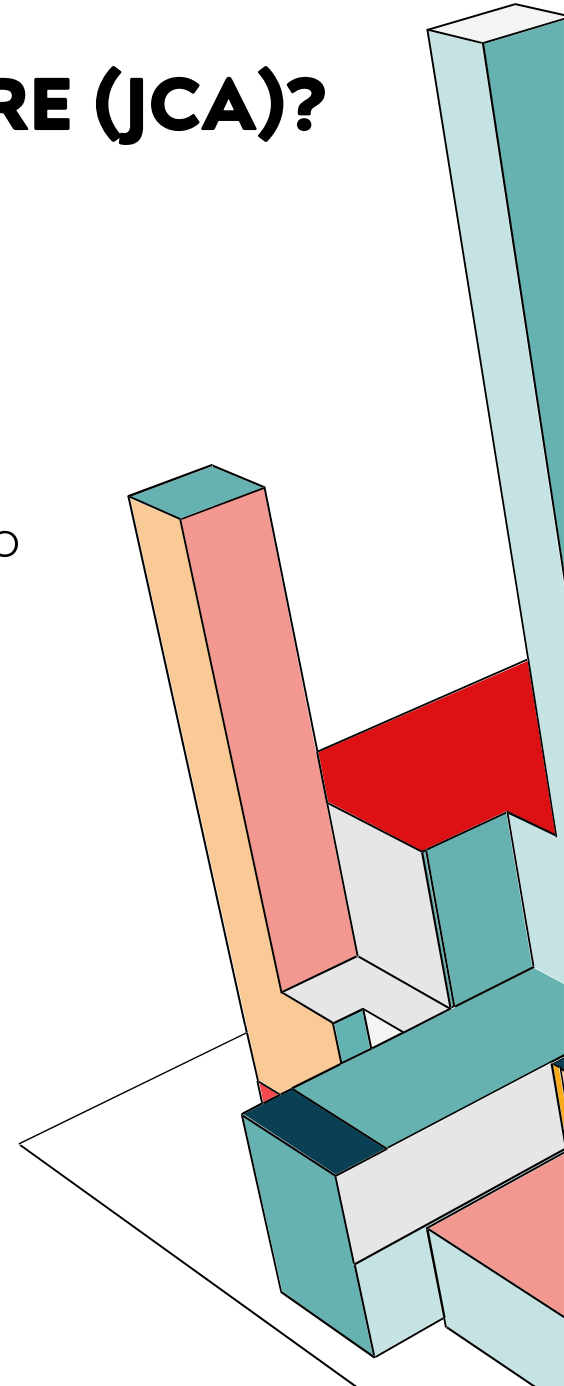
- **Indipendenza dell'implementazione:** Le applicazioni non devono implementare gli algoritmi di sicurezza. I servizi di sicurezza sono implementati nei provider, che applicano gli algoritmi.
- **Interoperabilità dell'implementazione:** I provider sono interoperabili tra le applicazioni. Nello specifico, un'applicazione non è vincolata a un provider specifico e un provider non è vincolato a un'applicazione specifica.
- **Estensibilità degli algoritmi:** La piattaforma Java include una serie di provider integrati che implementano un set base di servizi di sicurezza, ma se un domani dovesse essere inventato un nuovo tipo di crittografia, non servirebbe aspettare una nuova versione di Java, ma basta installare un plug-in.



CHE COS'È JAVA CRYPTOGRAPHY ARCHITECTURE (JCA)?

La JCA è composta da due elementi principali:

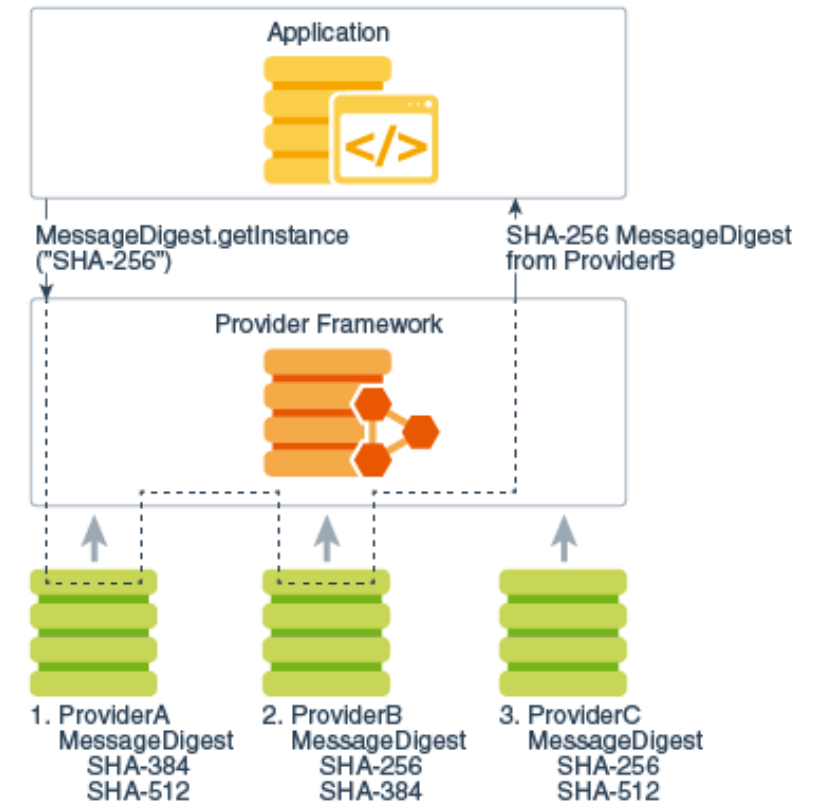
- **Il Framework:** Le API (come `java.security`, `javax.crypto`) che definiscono quali operazioni è possibile fare, per esempio creare un firma.
- **I Provider:** Le implementazioni reali che contengono il codice matematico che esegue l'operazione.



CHE COS'È JAVA CRYPTOGRAPHY ARCHITECTURE (JCA)?

Un **Provider** è un pacchetto software che implementa uno o più algoritmi di sicurezza. Java ha una lista di provider installati e un ordine di preferenza, per cui quando chiediamo un servizio alla JCA, possiamo farlo in due modi:

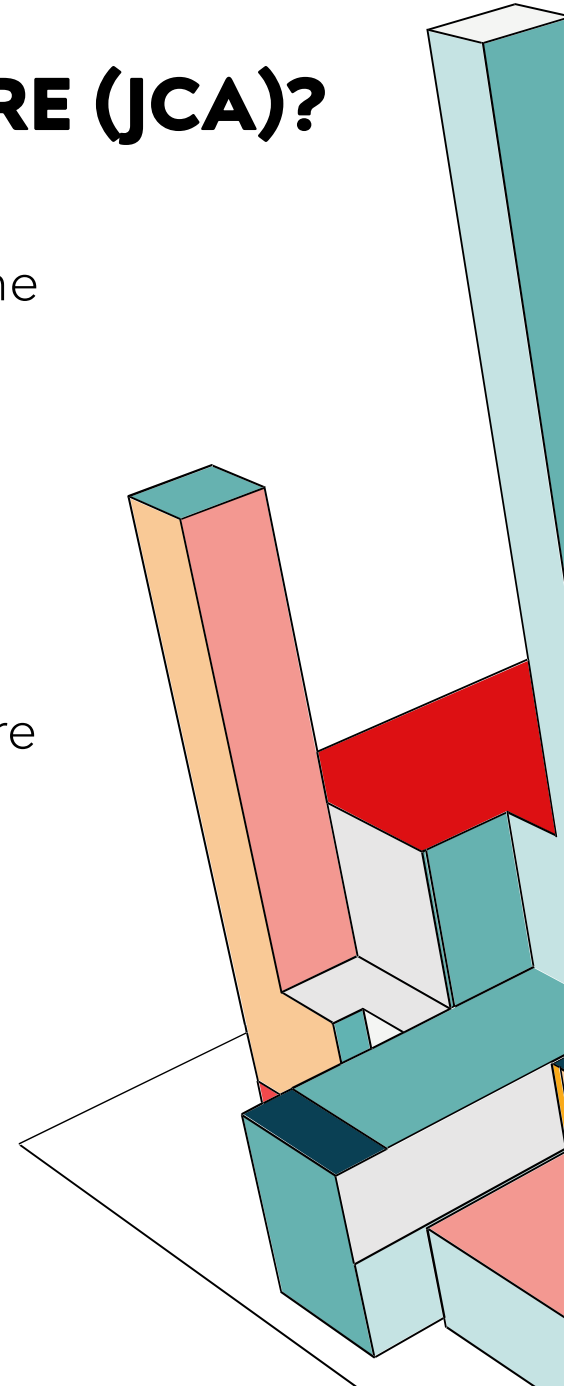
- **Richiesta Automatica**, ovvero si specifica solo l'algoritmo e Java cercherà tra i provider installati, partendo dal numero 1 nella lista di preferenza, finché non ne trova uno che supporta SHA-256.
- **Richiesta Specifica**, si può usare un provider specifico, per esempio se abbiamo esigenza di una certificazione specifica.



CHE COS'È JAVA CRYPTOGRAPHY ARCHITECTURE (JCA)?

Per ottenere l'indipendenza dall'implementazione, Java usa una separazione tra l'interfaccia pubblica e il codice privato. Infatti abbiamo:

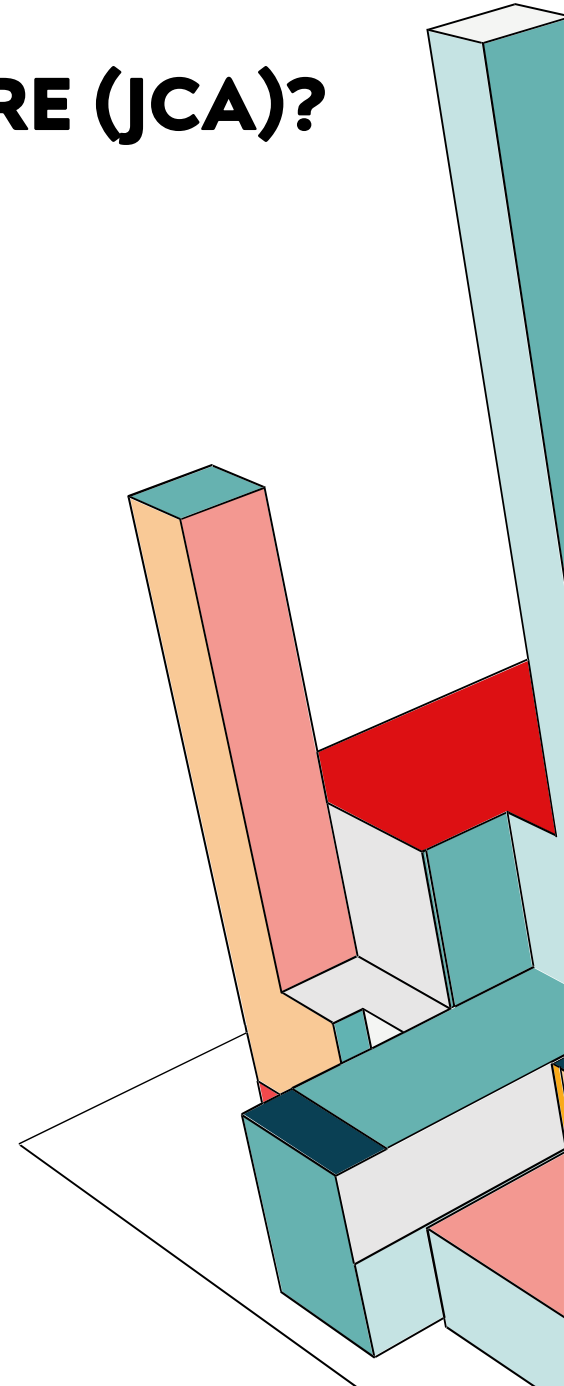
- **Engine Classes:** Sono le classi che usiamo nel codice, come Cipher, Signature o MessageDigest. Definiscono metodi generici come sign() o encrypt().
- **SPI (Service Provider Interface):** Per ogni classe Engine, esiste una classe corrispondente. I provider devono estendere queste classi SPI per scrivere la loro logica reale.



CHE COS'È JAVA CRYPTOGRAPHY ARCHITECTURE (JCA)?

Inoltre altri strumenti essenziali sono:

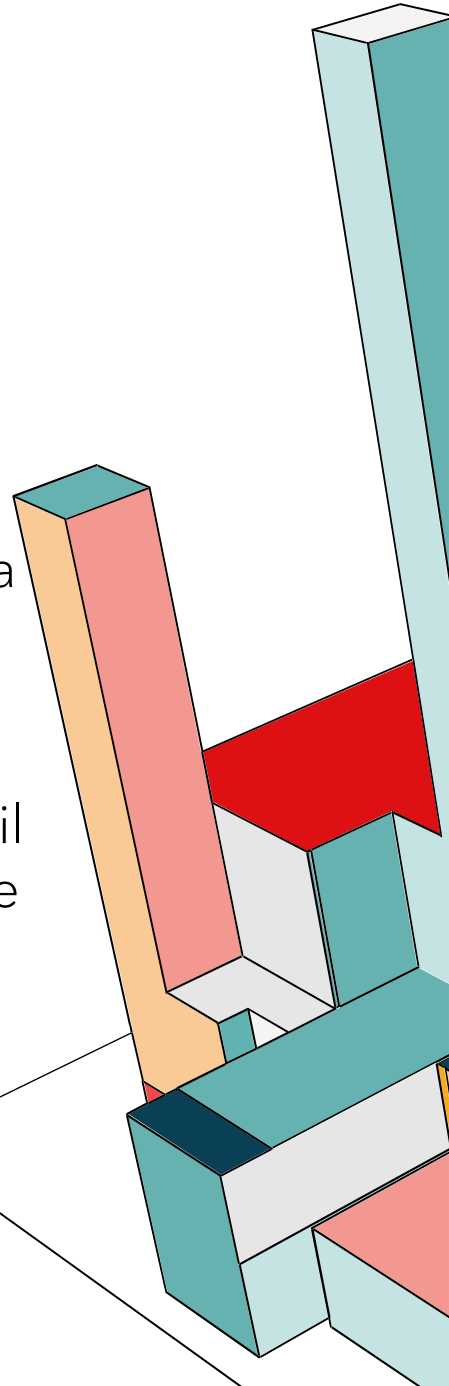
- **Keystores:** Sono database protetti per salvare chiavi crittografiche e certificati. Il formato standard raccomandato da JDK 9 in poi è **pkcs12**.
- **Classi Principali:**
 - **SecureRandom:** Per generare numeri casuali sicuri.
 - **MessageDigest:** Per calcolare hash.
 - **Signature:** Per firmare digitalmente i dati e verificare le firme.
 - **Cipher:** Per criptare e decriptare dati (AES, RSA, ecc.).
- **Gestione della Sicurezza:** La classe Security gestisce quali provider sono installati. Per aggiungere o rimuovere provider dinamicamente,



JCA: CRITTOGRAFIA SIMMETRICA DES

```
public static void main(String[] args) throws Exception {  
  
    // Leggiamo il file "testo.txt" e mostriamo il testo in chiaro  
    String plaintext = new String(Files.readAllBytes(Paths.get("testo.txt")));  
    System.out.println("Il testo in chiaro è:\n" + plaintext);  
  
    // Genera chiave DES  
    KeyGenerator keyGen = KeyGenerator.getInstance("DES");  
    SecretKey desKey = keyGen.generateKey();  
  
    for (String mode : modes) {  
        System.out.println("\n=== DES/" + mode + " ===");  
  
        // IV necessario  
        IvParameterSpec iv = null;  
        if (!mode.equals("ECB")) {  
            byte[] ivBytes = new byte[8];  
            SecureRandom random = new SecureRandom();  
            random.nextBytes(ivBytes);  
            iv = new IvParameterSpec(ivBytes);  
            System.out.println("IV (Base64): " + Base64.getEncoder().encodeToString(ivBytes));  
        }  
  
        // Cipher  
        Cipher cipher = Cipher.getInstance("DES/" + mode + "/PKCS5Padding");  
        if (iv != null) cipher.init(Cipher.ENCRYPT_MODE, desKey, iv);  
        else cipher.init(Cipher.ENCRYPT_MODE, desKey);  
  
        // Cifra  
        byte[] encrypted = cipher.doFinal(plaintext.getBytes());  
        String encBase64 = Base64.getEncoder().encodeToString(encrypted);  
        System.out.println("Encrypted (Base64): " + encBase64);  
  
        // Decifra  
        if (iv != null) cipher.init(Cipher.DECRYPT_MODE, desKey, iv);  
        else cipher.init(Cipher.DECRYPT_MODE, desKey);  
        byte[] decrypted = cipher.doFinal(encrypted);  
        String decryptedText = new String(decrypted);  
        System.out.println("Decrypted: " + decryptedText);  
    }  
}
```

Usiamo una Engine Class (**KeyGenerator**) per generare una chiave simmetrica casuale, usando un provider che supporti l'algoritmo DES. Il codice itera con quattro modalità operative del DES diverse e usa la stessa chiave per tutte e quattro le modalità. Usiamo **SecureRandom** per la generazione di numeri random in maniera sicura e impacchettiamo i byte in un che è il metodo standardizzato da Java per passare i parametri agli algoritmi. Inoltre, poiché il DES esegue a blocchi multipli di 8 byte eseguiamo il padding (**Cipher.getInstance("DES/" + mode + "/PKCS5Padding")**).



JCA: CRITTOGRAFIA SIMMETRICA DES

Il testo in chiaro è:

Testo di prova

=== DES/ECB ===

Encrypted (Base64): BUxuy2lS4ENmwpQdBh4pww==

Decrypted: Testo di prova

=== DES/CBC ===

IV (Base64): z2YMEDZ4TbM=

Encrypted (Base64): cKYtz8a9/7JZFZQXPjKpJQ==

Decrypted: Testo di prova

=== DES/CFB ===

IV (Base64): nzFKxxWmqp4=

Encrypted (Base64): IWR9n136WPZ8rmIC1fv3sg==

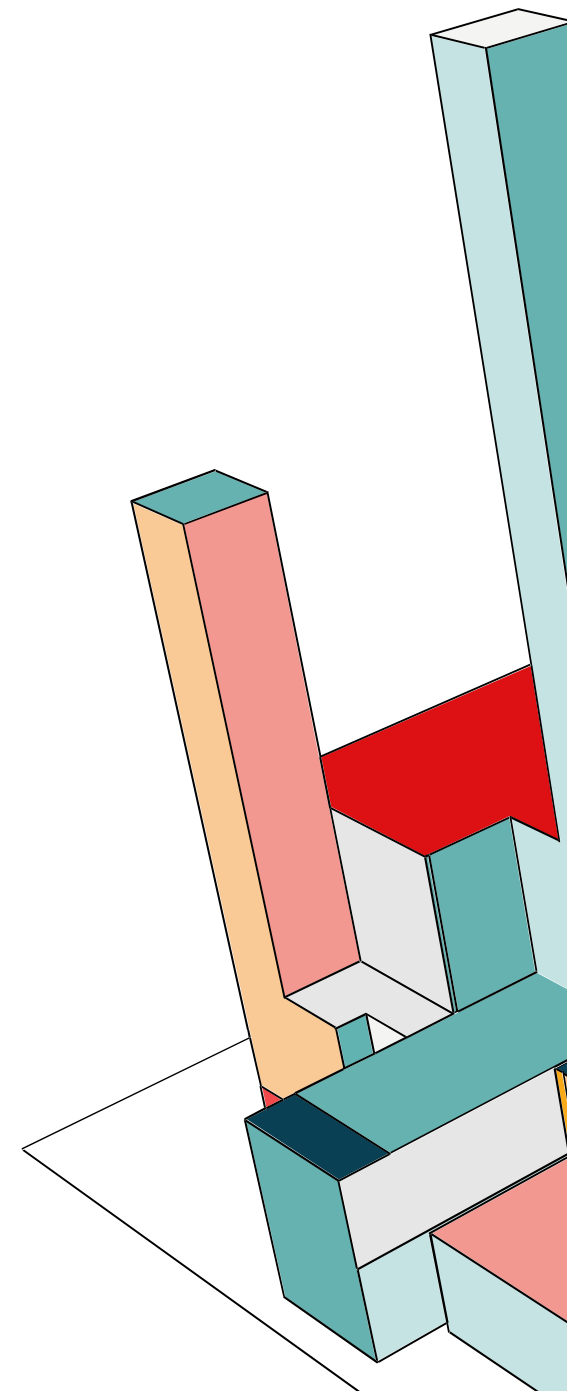
Decrypted: Testo di prova

=== DES/OFB ===

IV (Base64): wYjiNvv8MvA=

Encrypted (Base64): buxUKldUSMGm49m5uD9QJw==

Decrypted: Testo di prova



JCA: CRITTOGRAFIA SIMMETRICA TRIPLE-DES

```
public static void main(String[] args) throws Exception {

    // Leggiamo il file "testo.txt" e stampiamo il testo in chiaro
    String plaintext = new String(Files.readAllBytes(Paths.get("testo.txt")));
    System.out.println("Il testo in chiaro è:\n" + plaintext);

    // Genera chiave
    KeyGenerator keyGen = KeyGenerator.getInstance("DESede");
    SecretKey des3Key = keyGen.generateKey();

    for (String mode : modes) {
        System.out.println("\n=== 3DES/" + mode + " ===");

        IvParameterSpec iv = null;
        if (!mode.equals("ECB")) {
            byte[] ivBytes = new byte[8];
            SecureRandom random = new SecureRandom();
            random.nextBytes(ivBytes);
            iv = new IvParameterSpec(ivBytes);
            System.out.println("IV (Base64): " + Base64.getEncoder().encodeToString(ivBytes));
        }

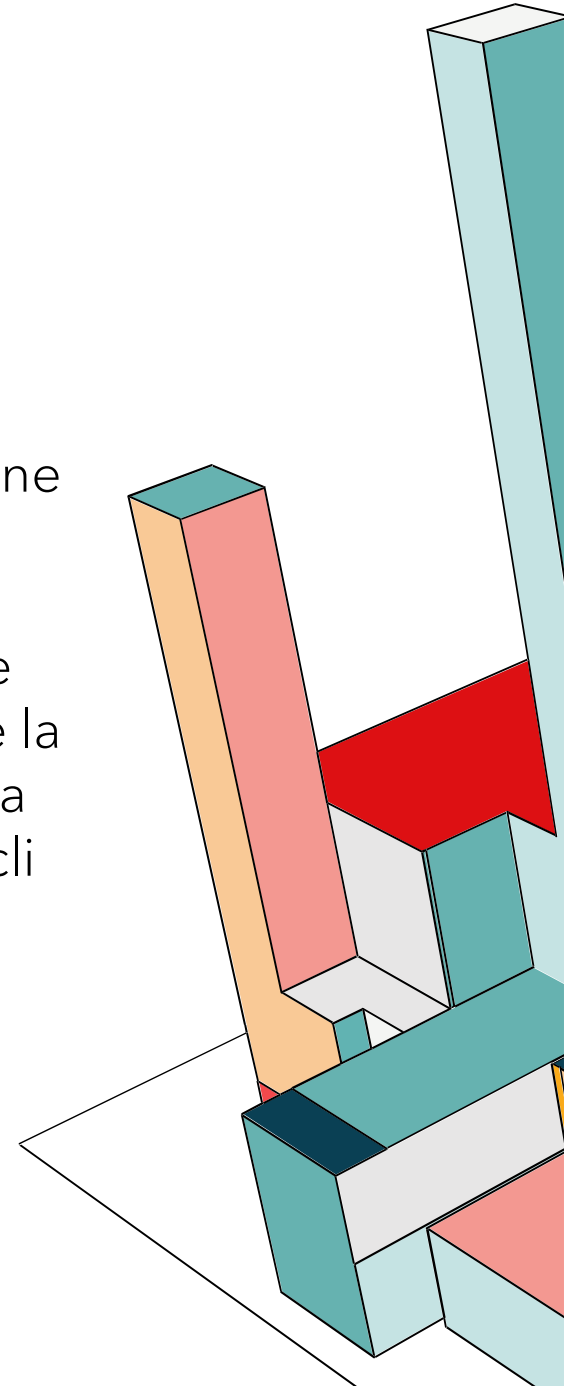
        // Cipher
        Cipher cipher = Cipher.getInstance("DESede/" + mode + "/PKCS5Padding");
        if (iv != null) cipher.init(Cipher.ENCRYPT_MODE, des3Key, iv);
        else cipher.init(Cipher.ENCRYPT_MODE, des3Key);

        // Cifra
        byte[] encrypted = cipher.doFinal(plaintext.getBytes());
        String encBase64 = Base64.getEncoder().encodeToString(encrypted);
        System.out.println("Encrypted (Base64): " + encBase64);

        // Decifra
        if (iv != null) cipher.init(Cipher.DECRYPT_MODE, des3Key, iv);
        else cipher.init(Cipher.DECRYPT_MODE, des3Key);
        byte[] decrypted = cipher.doFinal(encrypted);
        String decryptedText = new String(decrypted);
        System.out.println("Decrypted: " + decryptedText);
    }
}
```

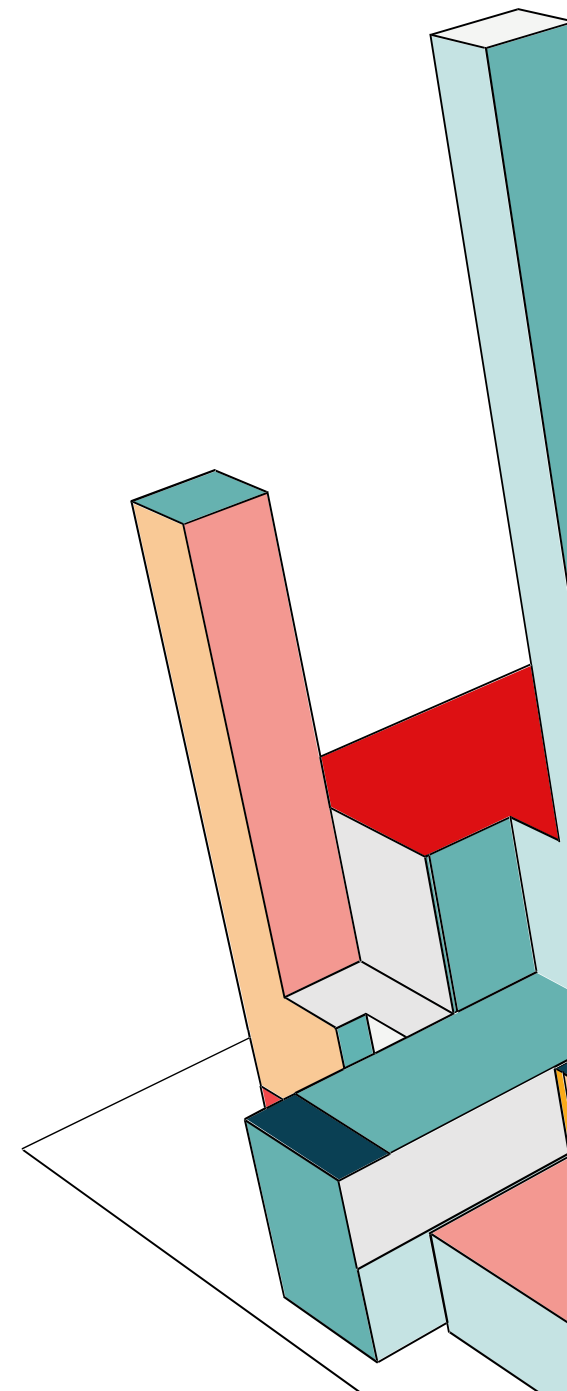
60

In questo caso eseguiamo gli stessi passi del codice precedente, ma cambia la Engine Class, che in questo caso è **DESede**. La dimensione dell'IV rimane sempre di 8 byte, anche se la chiave è più lunga, perché la dimensione del blocco è ancora un multiplo di 8. Facciamo 4 cicli per le diverse modalità di esecuzione del DES.



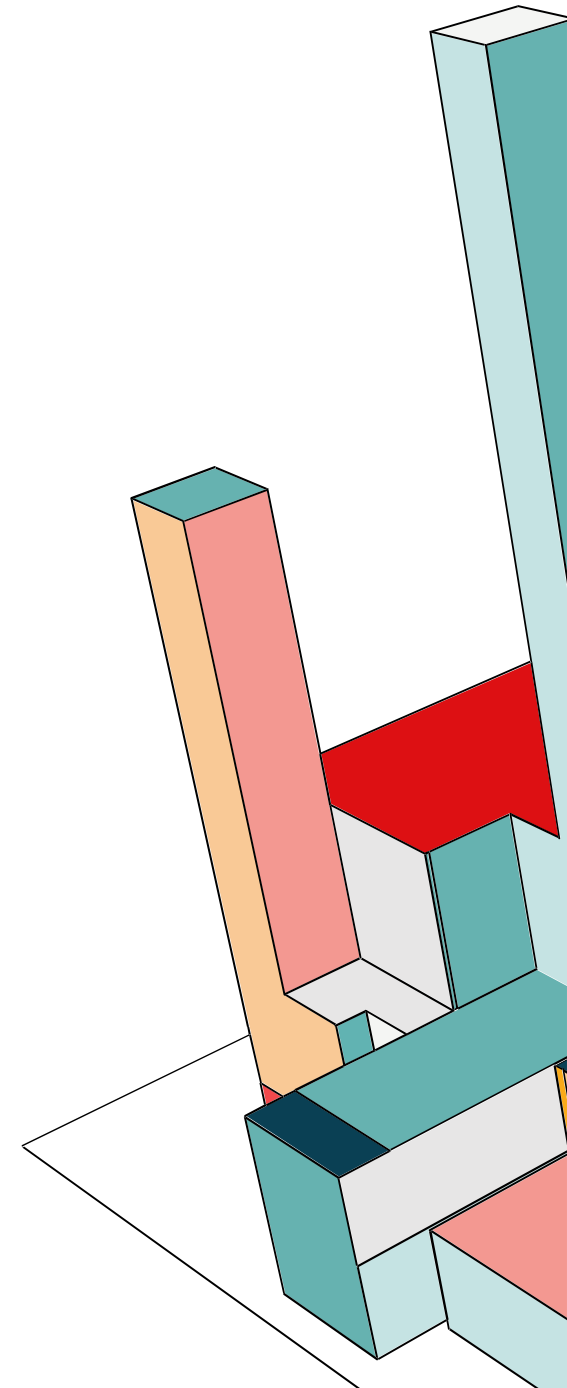
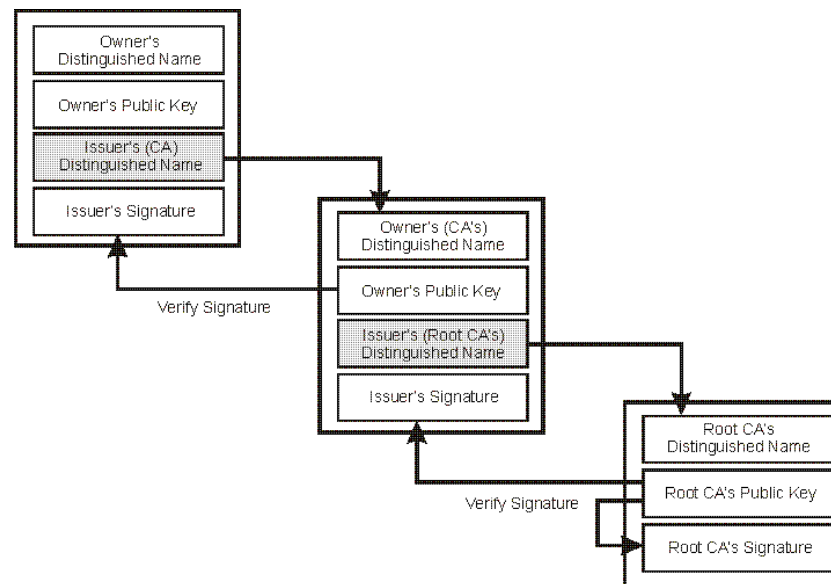
JCA: CRITTOGRAFIA SIMMETRICA TRIPLE-DES

```
=== 3DES/ECB ===  
Encrypted (Base64): SjJDIqeb6fao8mzn9k9avg==  
Decrypted: Testo di prova  
  
=== 3DES/CBC ===  
IV (Base64): qRfuLjdn5k=  
Encrypted (Base64): mAi3MFMxoZZ9To67fl105g==  
Decrypted: Testo di prova  
  
=== 3DES/CFB ===  
IV (Base64): Bble8s82TTw=  
Encrypted (Base64): 1LFz4oUeuPii+Bo0CNhTxA==  
Decrypted: Testo di prova  
  
=== 3DES/OFB ===  
IV (Base64): s/7eaP0JAYo=  
Encrypted (Base64): 2rj5XxkKKWYR2Q2Iorqn5w==  
Decrypted: Testo di prova
```



JCA: CATENE CERTIFICATI X.509 V3

Una **catena di certificati** è una sequenza di certificati digitali che verifica l'autenticità di un certificato finale. Ogni certificato nella catena è firmato dal certificato precedente, fino ad arrivare alla **Certificate Authority (CA)**, che è l'entità di fiducia principale. Una CA è considerata altamente affidabile e garantisce che i certificati emessi siano validi e autentici. Il certificato delle CA è generalmente preinstallato nei browser e nei sistemi operativi.



JCA: CATENE CERTIFICATI X.509 V3

Il **certificato intermedio** rappresenta un ponte tra la CA e il certificato finale.
Questo approccio:

- Garantisce maggiore sicurezza, dato che la chiave privata della CA è protetta ed è raro che venga utilizzata direttamente
- Facilita gestione e distribuzione dei certificati, dato che possiamo utilizzare i certificati intermedi per emettere certificati finali senza coinvolgere direttamente la CA

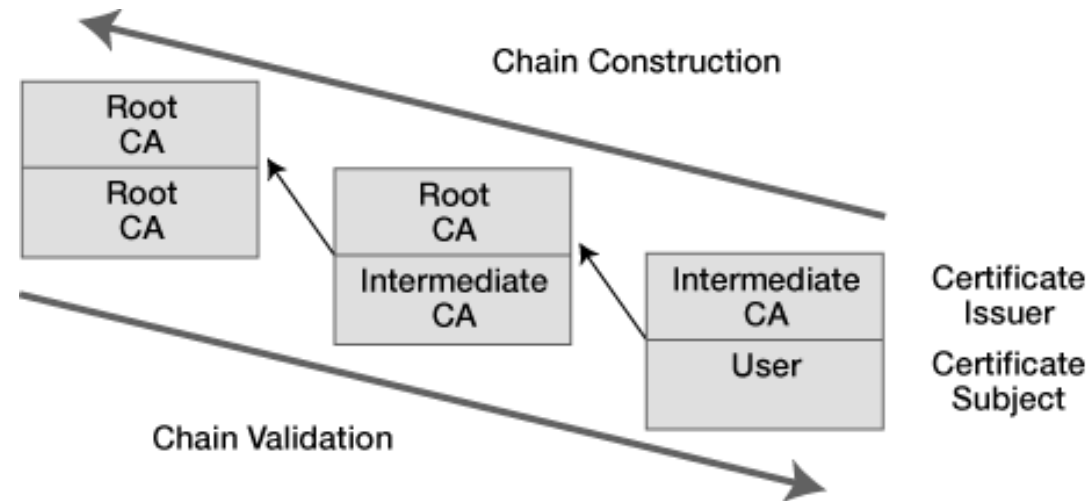
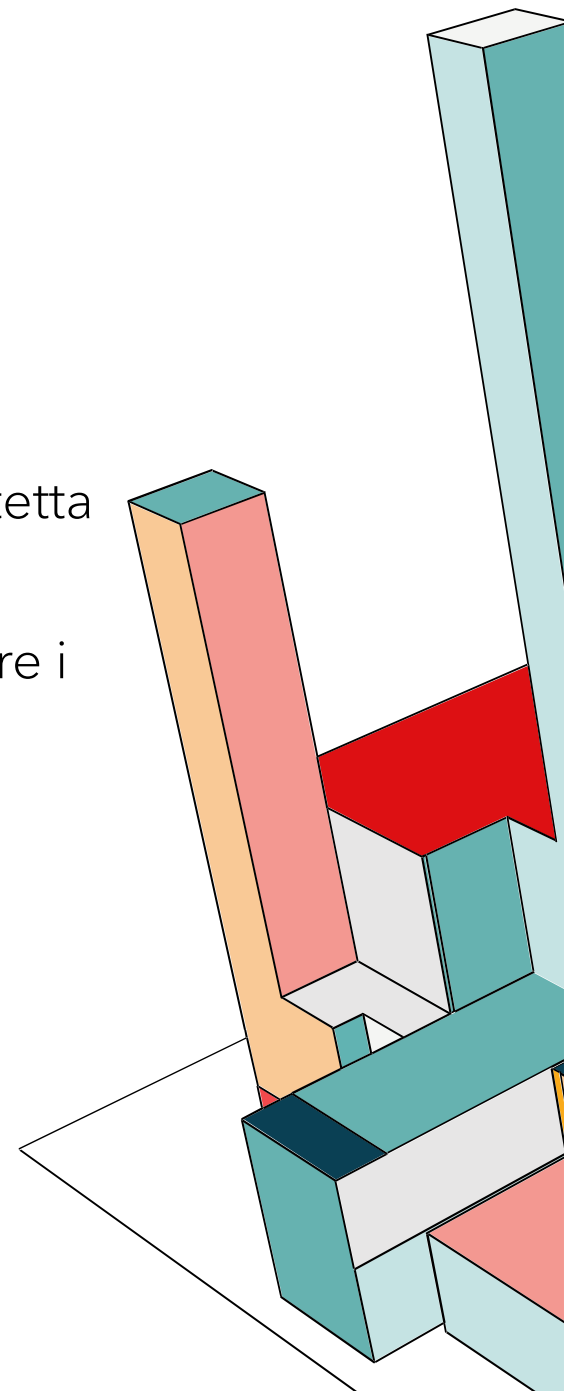
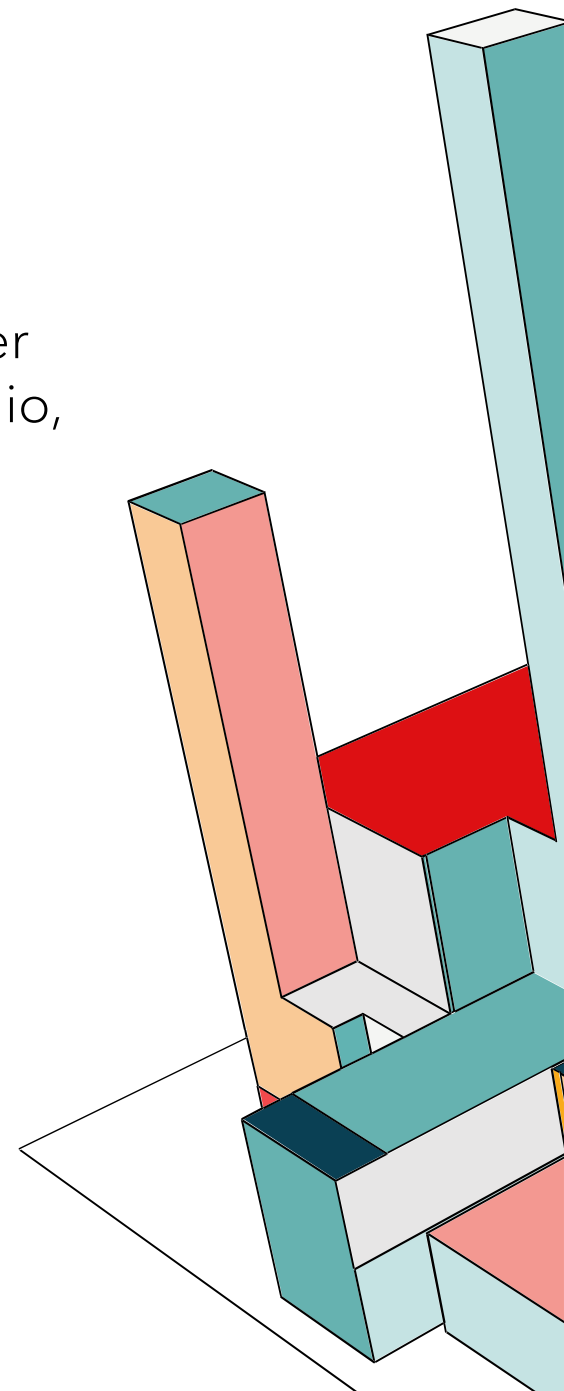
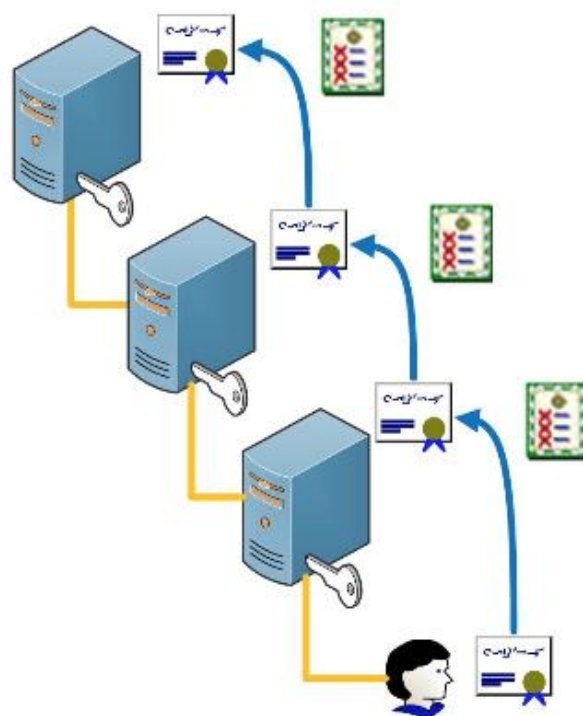


FIGURE 1: Certificate-chain-processing overview



JCA: CATENE CERTIFICATI X.509 V3

Il **certificato finale** è quello utilizzato dalle applicazioni (siti web, server...) per garantire la sicurezza delle comunicazioni. È firmato dal certificato intermedio, che a sua volta è firmato dalla CA. Si crea in tal modo un **sistema di fiducia gerarchico**, fondamentale per la sicurezza delle comunicazioni digitali.

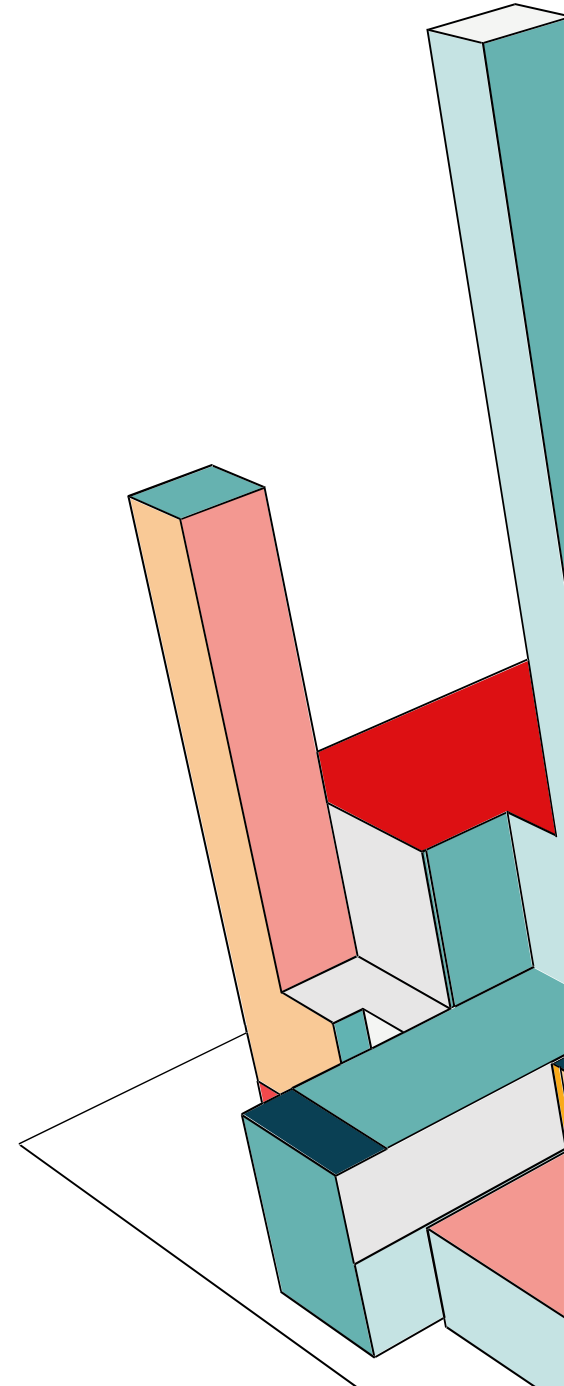


JCA: CATENE CERTIFICATI X.509 V3

Per la generazione dei certificati X.509 utilizzeremo **Bouncy Castle**, una libreria crittografica open-source che estende le funzionalità della JCA. Essa fornisce API avanzate e specifiche, necessarie per la creazione di certificati e chiavi.

Per il corretto funzionamento è necessario importare due librerie Java:

- **bcprov-jdk18on-1.82.jar**: È il provider crittografico principale, che contiene le implementazioni degli algoritmi come RSA, AES, SHA.
- **bcpkix-jdk18on-1.82.jar**: Contiene le estensioni PKIX specifiche per generare e gestire i certificati X.509



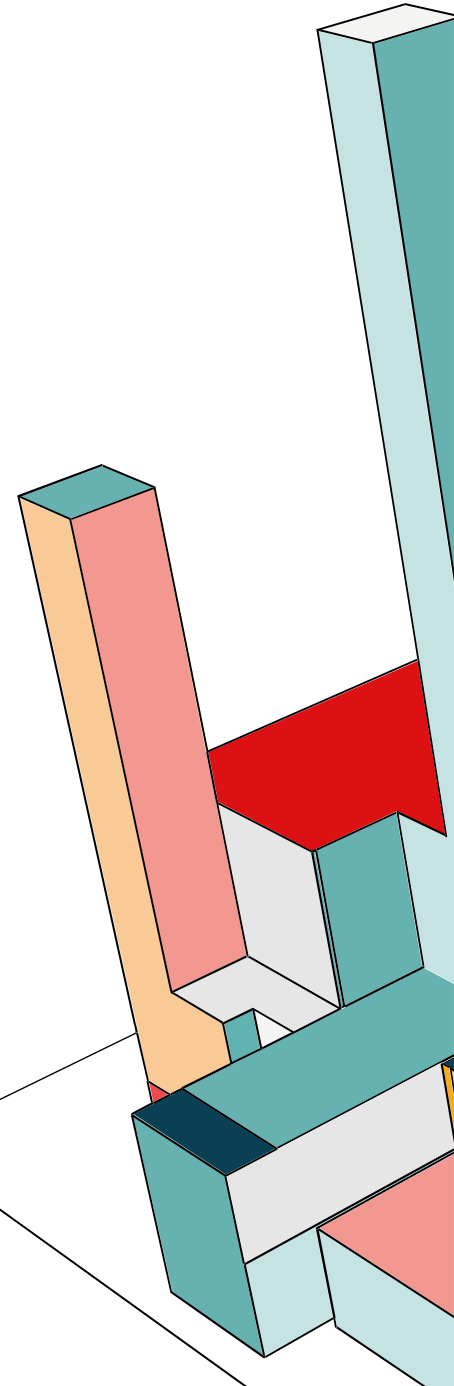
JCA: CATENE CERTIFICATI X.509 V3

Importiamo **Bouncy Castle** come provider esterno, che fornisce le classi "Builder" (JcaX509v3CertificateBuilder) necessarie per costruire certificati.

```
22 public class CertGenerator {
23
24     public static void main(String[] args) throws Exception {
25         // Aggiungo Bouncy Castle come provider
26         Security.addProvider(new BouncyCastleProvider());
27     }
```

Creiamo la Certification Authority, che è Auto-firmata, infatti nel metodo createCertificate che non passiamo un certificato "issuer" (è null). Abbiamo impostato una validità di 10 anni. Inoltre abbiamo impostato un vincolo (isCA = true), ovvero che ha il potere di firmare altri certificati.

```
28 // ===== ROOT CA =====
29 KeyPair rootKey = generateKeyPair();
30 X509Certificate rootCert = createCertificate(rootKey, rootKey.getPrivate(), null, "CN=Root CA, O=Universita', C=IT", 3650, true);
31 savePEM(rootKey.getPrivate(), "root.key");
32 savePEM(rootCert, "root.pem");
33
```



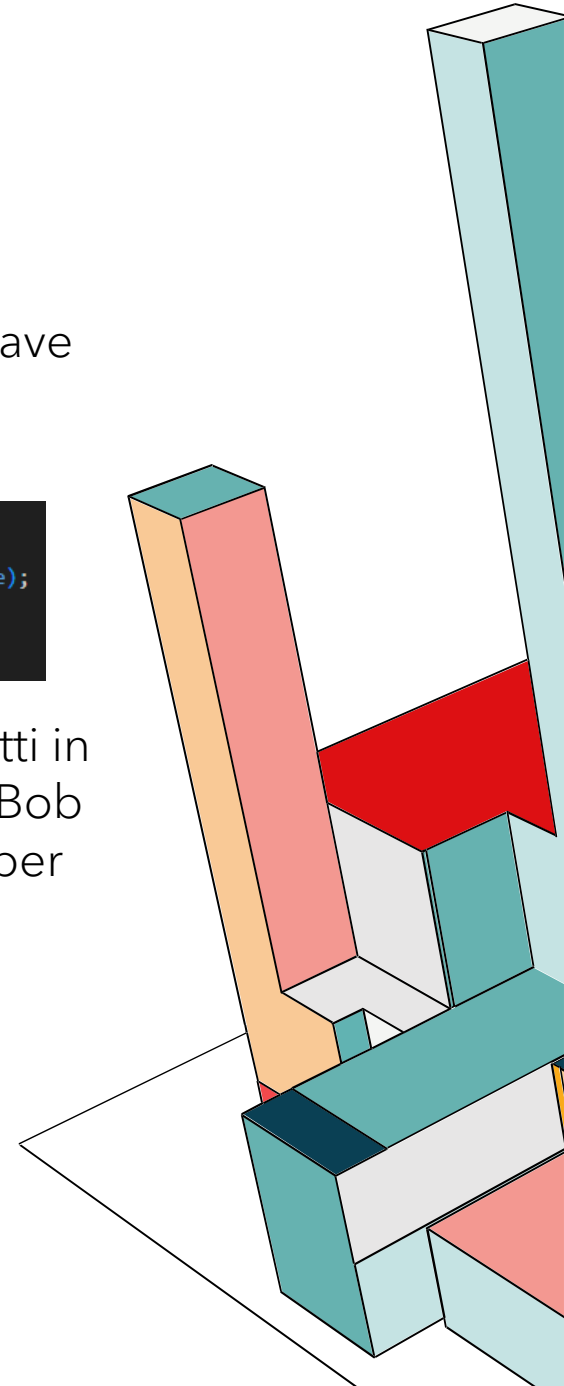
JCA: CATENE CERTIFICATI X.509 V3

Definiamo la Intermediate CA, anch'essa può firmare i certificati. Infatti anche qui abbiamo impostato il vincolo (**isCA = true**). Ma questo certificato è firmato dalla chiave privata della Root. Chiunque si fidi della Root, verificherà la firma e si fiderà automaticamente anche della Intermediate.

```
35 // ===== INTERMEDIATE CA =====
36 KeyPair interKey = generateKeyPair();
37 X509Certificate interCert = createCertificate(interKey, rootKey.getPrivate(), rootCert, "CN=Intermediate CA, O=Corso, C=IT", 1825, true);
38 savePEM(interKey.getPrivate(), "intermediate.key");
39 savePEM(interCert, "intermediate.pem");
40
```

Poi abbiamo gli utenti finali i cui certificati sono firmati dalla Intermediate CA. E infatti in questo caso il vincolo isCA è false, quindi non possono emettere certificati. Alice e Bob possono usare il certificato per identificarsi o criptare dati, ma non possono usarlo per firmare altri certificati. Infine scriviamo le chiavi e i certificati in file .PEM.

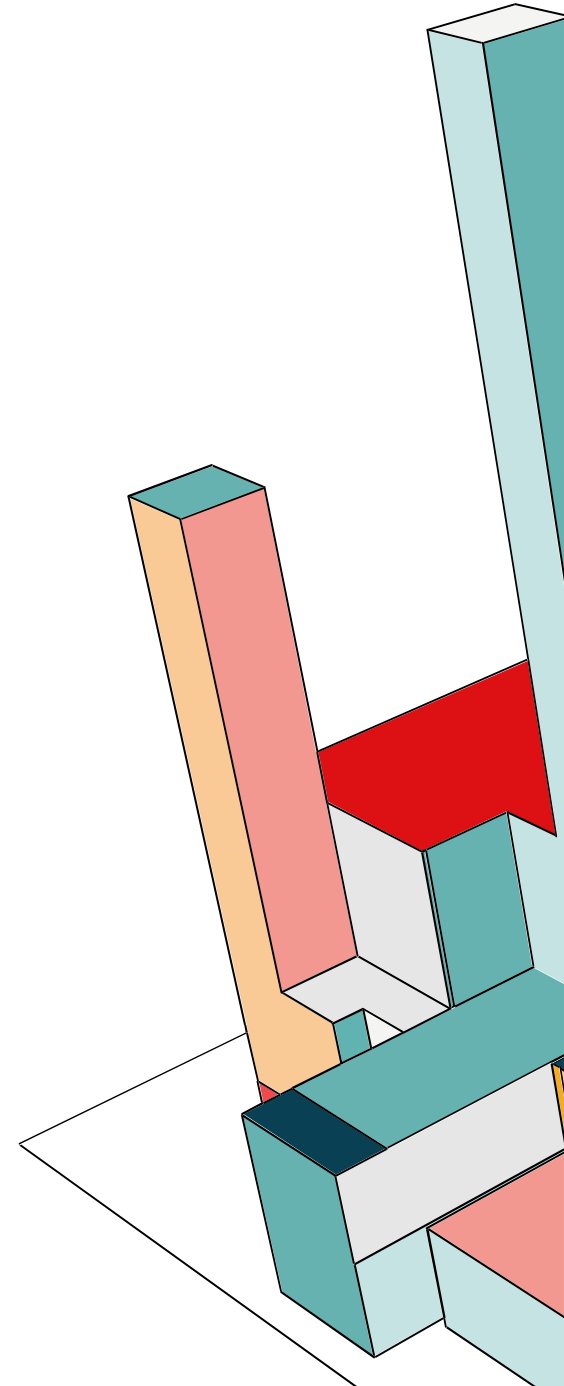
```
41 // ===== END ENTITY BOB =====
42 KeyPair endKey_Bob = generateKeyPair();
43 X509Certificate endCert_Bob = createCertificate(endKey_Bob, interKey.getPrivate(), interCert, "CN=Studenti, O=Bob, C=IT", 825, false);
44 savePEM(endKey_Bob.getPrivate(), "end_Bob.key");
45 savePEM(endCert_Bob, "end_Bob.pem");
46 // ===== END ENTITY ALICE =====
47 KeyPair endKey_Alice = generateKeyPair();
48 X509Certificate endCert_Alice = createCertificate(endKey_Alice, interKey.getPrivate(), interCert, "CN=Studenti, O=Alice, C=IT", 825, false);
49 savePEM(endKey_Alice.getPrivate(), "end_Alice.key");
50 savePEM(endCert_Alice, "end_Alice.pem");
```



JCA: CATENE CERTIFICATI X.509 V3

In questa funzione non usiamo Bouncy Castle. Utilizziamo l'algoritmo RSA (`KeyPairGenerator.getInstance("RSA")`). Inoltre usiamo una chiave a 2048 bit (`gen.initialize(2048)`). Il risultato è una `KeyPair`, che è un contenitore che ha dentro sia la chiave pubblica che la chiave privata.

```
1  
2 public static KeyPair generateKeyPair() throws Exception {  
3     KeyPairGenerator gen = KeyPairGenerator.getInstance("RSA");  
4     gen.initialize(2048);  
5     return gen.generateKeyPair();  
6 }
```



JCA: CATENE CERTIFICATI X.509 V3

Qui usiamo Bouncy Castle per assemblare la struttura dati X.509 v3. Come prima cosa ci occupiamo dell'identità del soggetto, quindi convertiamo le stringhe in formato X.500, comprensibile dai sistemi crittografici. Ogni certificato emesso da una CA deve avere un numero seriale unico. Usare un numero casuale enorme (64 bit) è una best practice per evitare collisioni (`new BigInteger(64, new SecureRandom())`). Gestiamo l'issuer del certificato e in particolare due casi: certificato normale, se passi un `issuerCert` (es. Intermediate CA), l'Issuer del nuovo certificato sarà il Subject di quello vecchio, self-signed (Root CA), se `issuerCert` è null, significa che nessuna autorità sta garantendo per quel certificato. Quindi, l'Issuer diventa uguale al Subject.

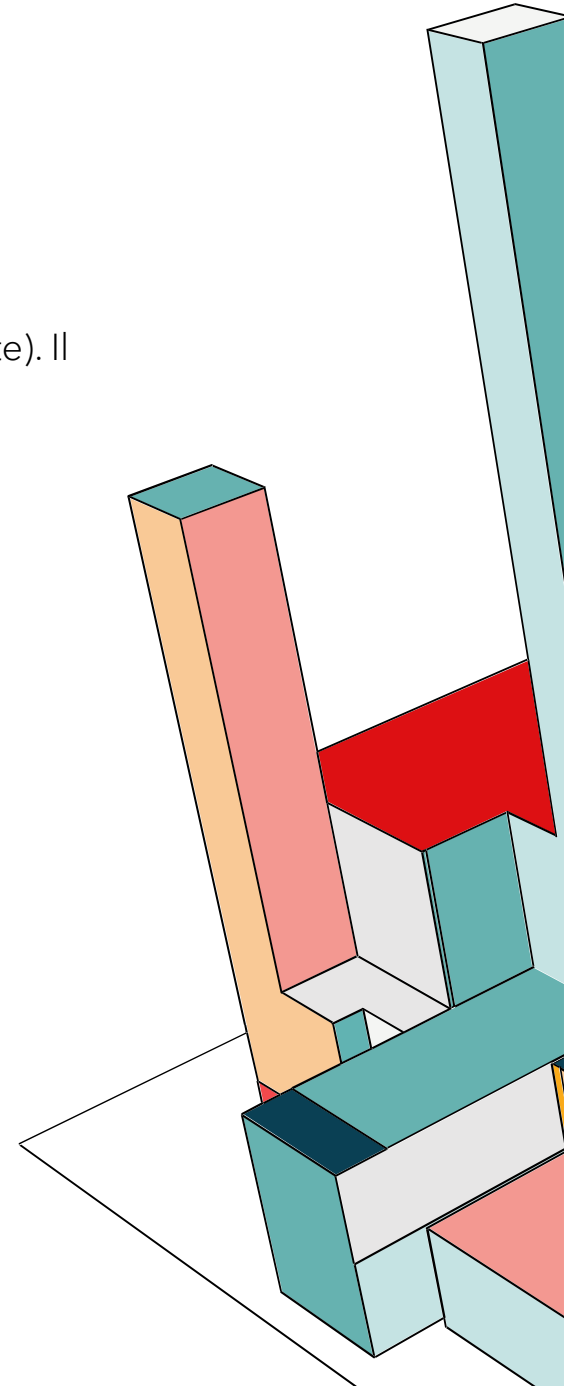
```
8 public static X509Certificate createCertificate(KeyPair subjectKey, PrivateKey issuerKey,
9 X509Certificate issuerCert, String dnString, int validityDays, boolean isCA) throws Exception {
10
11     X500Name subjectDN = new X500Name(dnString);
12
13     // Data di validità
14     Date notBefore = new Date();
15     Calendar cal = Calendar.getInstance();
16     cal.setTime(notBefore);
17     cal.add(Calendar.DAY_OF_YEAR, validityDays);
18     Date notAfter = cal.getTime();
19
20     BigInteger serial = new BigInteger(64, new SecureRandom());
21     X500Name issuerDN = (issuerCert != null)
22         ? new X500Name(issuerCert.getSubjectX500Principal().getName())
23         : subjectDN;
24
25     JcaX509v3CertificateBuilder builder = new JcaX509v3CertificateBuilder(
26         issuerDN,
27         serial,
28         notBefore,
29         notAfter,
30         subjectDN,
31         subjectKey.getPublic()
32     );
```

JCA: CATENE CERTIFICATI X.509 V3

Infine gestiamo le estensioni:

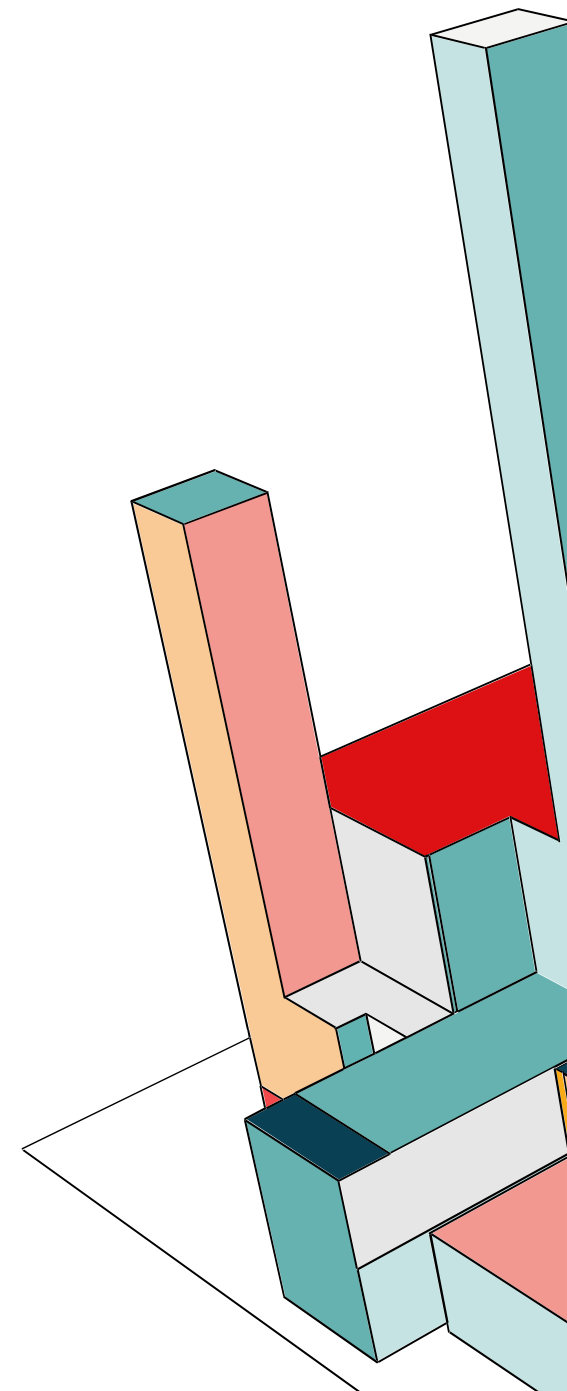
BasicConstraints (isCA), **KeyUsage** (per CA, per firmare certificati e cRLSign per firmare liste di revoca, **digitalSignature** per autenticazione di utenti e **keyEncipherment** per scambio chiavi crittografate). Il builder prende tutto il blocco di dati che gli abbiamo fornito e lo firma usando la chiave privata dell'Issuer (**JcaContentSignerBuilder**).

```
34     builder.addExtension(Extension.basicConstraints, true, new BasicConstraints(isCA));
35     if (isCA) {
36         builder.addExtension(Extension.keyUsage, true,
37             new KeyUsage(KeyUsage.keyCertSign | KeyUsage.cRLSign));
38     } else {
39         builder.addExtension(Extension.keyUsage, true,
40             new KeyUsage(KeyUsage.digitalSignature | KeyUsage.keyEncipherment));
41     }
42
43     // Firma del certificato
44     ContentSigner signer = new JcaContentSignerBuilder("SHA256withRSA").build(issuerKey);
45     X509CertificateHolder holder = builder.build(signer);
46
47     return new JcaX509CertificateConverter().setProvider("BC").getCertificate(holder);
48 }
49
50 // Salva chiave privata o certificato in formato PEM
51 public static void savePEM(Object obj, String filename) throws Exception {
52     try (JcaPEMWriter writer = new JcaPEMWriter(new FileWriter(filename))) {
53         writer.writeObject(obj);
54     }
55 }
```



JCA: CATENE CERTIFICATI X.509 V3

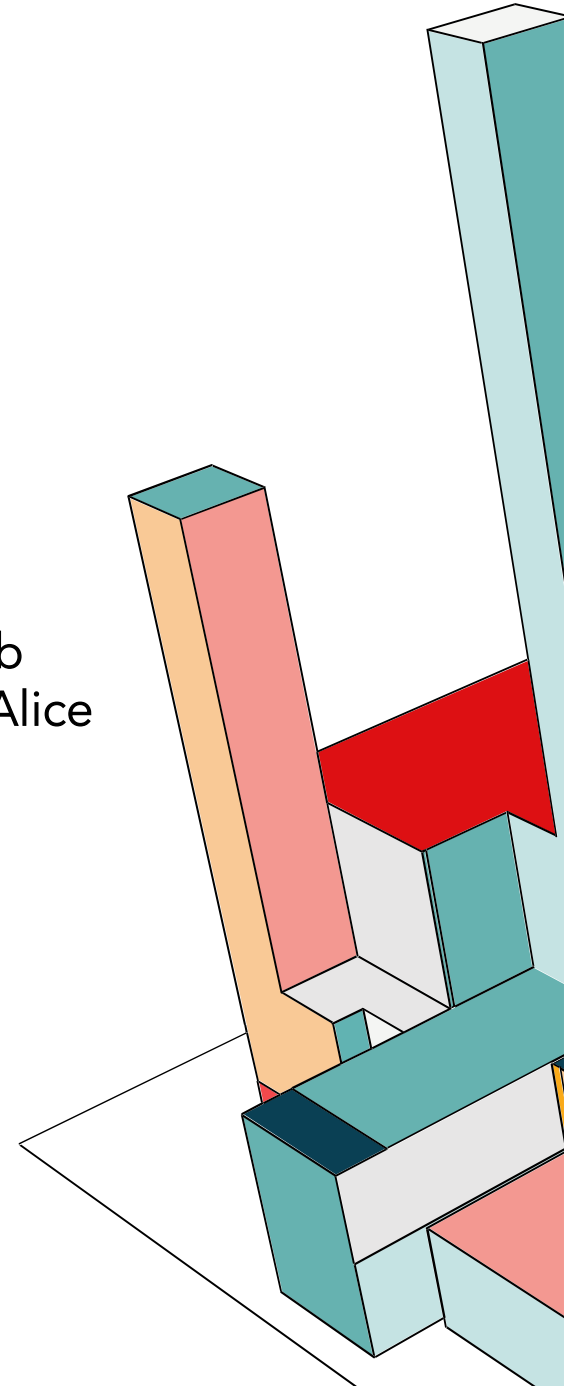
```
Certificate:
Data:
  Version: 3 (0x2)
  Serial Number:
    86:87:50:29:21:ae:09:a2
  Signature Algorithm: sha256WithRSAEncryption
  Issuer: C=IT, O=Universita', CN=Root CA
  Validity
    Not Before: Nov 24 21:33:24 2025 GMT
    Not After : Nov 23 21:33:24 2030 GMT
  Subject: CN=Intermediate CA, O=Corso, C=IT
  Subject Public Key Info:
    Public Key Algorithm: rsaEncryption
    Public-Key: (2048 bit)
    Modulus:
      00:c0:40:7b:b8:a9:2c:3d:39:d9:44:71:c7:8c:8b:
      2e:56:aa:d4:ac:0b:06:9a:68:7a:1b:de:82:b2:9a:
      31:d8:61:e5:27:37:35:b8:dc:5d:8e:c6:ca:e6:6d:
      ee:f7:cb:d6:82:ff:32:32:02:d0:a5:7a:1d:b7:6d:
      ea:86:0f:56:ca:d2:f7:c6:a9:3e:7e:bf:a2:a2:bd:
      63:e5:8b:25:4a:1f:94:3c:ba:6b:79:03:8e:bf:dc:
      1c:db:72:dc:a8:1e:28:52:13:a2:8d:01:85:9d:73:
      38:e1:0e:d3:21:83:5d:96:d4:c5:28:93:52:48:89:
      bb:89:df:32:b5:af:32:7a:3a:93:3e:19:26:80:90:
      56:66:7c:0a:1a:f7:3d:9b:2f:61:95:2e:5e:55:9e:
      b2:95:0e:a0:42:e0:2d:1e:da:23:1b:b8:62:37:df:
      5d:47:d3:9f:92:d8:27:e5:ba:d4:db:7c:13:a8:20:
      f5:ef:cd:0c:54:7d:44:a7:7a:36:84:bc:f6:cf:93:
      42:6a:74:a7:8a:55:06:7b:60:fd:82:fe:e4:b0:0d:
      66:8d:c1:a9:9f:83:74:5a:cb:7f:5a:1d:72:1d:ad:
      99:10:0b:a1:de:23:26:0a:ca:6b:e5:2f:ee:cd:e3:
      65:5f:aa:6e:95:f5:ae:6a:36:ad:60:62:a1:45:a2:
      00:0d
    Exponent: 65537 (0x10001)
  X509v3 extensions:
    X509v3 Basic Constraints: critical
      CA:TRUE
    X509v3 Key Usage: critical
      Certificate Sign, CRL Sign
  Signature Algorithm: sha256WithRSAEncryption
  Signature Value:
    1f:30:46:c6:40:8d:16:e2:6f:39:18:a4:dd:4f:00:05:ac:10:
    ef:3a:4c:cd:58:07:75:7f:36:0b:54:c7:0c:e1:50:c2:f8:26:
    1a:9a:05:76:e1:c4:38:c3:77:fa:f3:41:b8:6a:4c:71:e2:a5:
    3d:f1:e8:cb:a1:40:12:3d:34:42:c9:08:b3:9f:83:d0:89:08:
    ff:86:89:26:d7:3b:48:7a:6f:0d:99:05:a8:62:c8:50:33:d9:
    2b:82:13:af:69:85:bb:71:2f:2b:99:2d:c2:3b:65:7f:1c:1b:
    ca:22:55:d0:6c:3f:d0:79:6b:48:42:84:ac:02:65:85:46:f0:
    f2:a3:c1:f6:e6:08:97:08:61:b0:9e:85:58:c5:eb:d0:a4:37:
    7d:5c:8a:50:3f:90:37:4b:b2:39:ce:8b:44:6d:72:9f:fa:67:
    e3:ed:e8:60:22:1b:18:ee:ee:e5:ae:68:a0:9e:82:d4:5c:d3:
    d4:71:84:e4:bb:c4:d9:06:27:a3:ce:8c:d7:b7:e6:22:5e:1e:
    24:02:cc:18:16:47:57:b0:f8:7f:d3:38:18:71:f6:97:c0:53:
    26:19:c5:74:92:a1:60:a7:53:ce:68:ea:34:7a:44:47:f4:
    ad:8f:6f:ad:d8:95:a9:ec:0d:38:68:d8:1d:6b:a8:77:ed:64:
    7c:e0:ec:6d
```



JCA: CATENE CERTIFICATI X.509 V3

A questo punto è possibile creare un keystore .p12, sia per Bob che per Alice, unendo le rispettive chiavi private e certificati. Lo faremo con openssl:

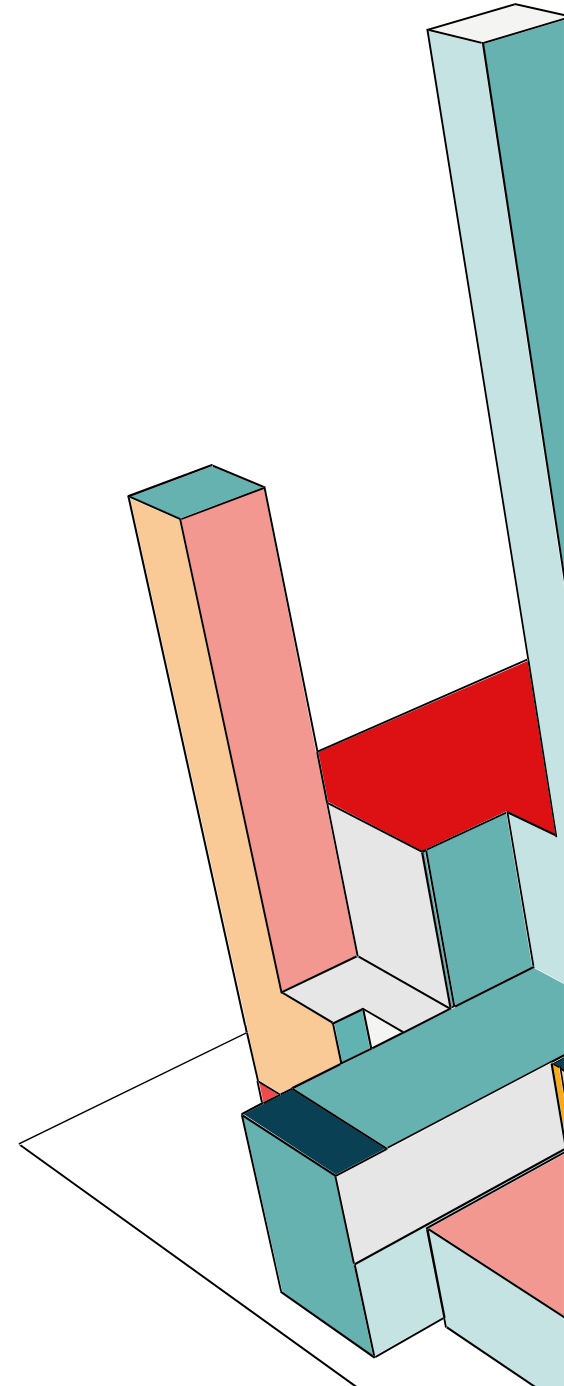
```
openssl pkcs12 -export -in end_Bob.pem -inkey end_Bob.key -out Bob.p12 -name Bob  
openssl pkcs12 -export -in end_Alice.pem -inkey end_Alice.key -out Alice.p12 -name Alice
```



JCA: FIRMA E VERIFICA MESSAGGIO

Simuliamo ora un caso d'uso concreto: Bob vuole inviare un messaggio segreto a Alice assicurandosi che nessuno possa leggerlo e che Alice sappia che proviene da lui. Per farlo, Bob invierà un messaggio cifrato e firmato. Per soddisfare la richiesta Bob dovrà proteggere il messaggio su due livelli:

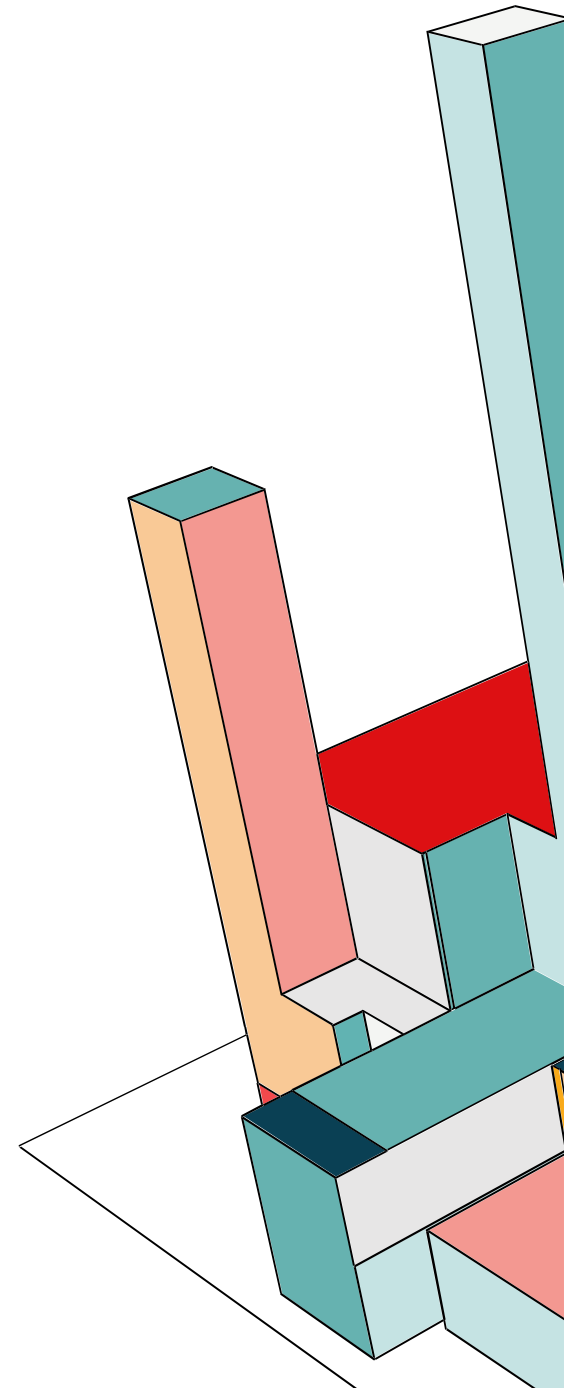
- Lo firma usando la propria chiave privata: questo sigilla il contenuto (tramite hash) garantendone l'**integrità** e ne certifica l'**autenticità** (solo Bob poteva firmarlo).
- Lo cifra usando la chiave pubblica di Alice: questo garantisce la **riservatezza**, poiché solo la chiave privata di Alice potrà sbloccare il contenuto.



JCA: FIRMA E VERIFICA MESSAGGIO

Lato mittente il processo si divide in tre fasi logiche:

- **Firma:** calcolo dell'hash del messaggio e cifratura del risultato con la sua chiave privata, garantendo autenticità ed integrità
- **Cifratura:** utilizzo di una chiave di sessione simmetrica AES per cifrare il payload, ovvero la concatenazione del messaggio con la firma
- **Protezione della chiave:** per condividere la chiave AES in sicurezza, questa viene cifrata con la chiave pubblica di Alice, garantendo riservatezza



JCA: BOB SENDER

```
12 public static void main(String[] args) throws Exception {
13
14     // Carica chiave privata di Bob e certificato
15     KeyStore ksBob = KeyStore.getInstance("PKCS12");
16     try (FileInputStream fis = new FileInputStream("Bob.p12")) {
17         ksBob.load(fis, "Bob".toCharArray());
18     }
19
20     PrivateKey bobPrivateKey = (PrivateKey) ksBob.getKey("bob", "Bob".toCharArray());
21     Certificate bobCert = ksBob.getCertificate("bob");
22
23     // Carica chiave pubblica di Alice da Alice.p12
24     KeyStore ksAlice = KeyStore.getInstance("PKCS12");
25     try (FileInputStream fis = new FileInputStream("Alice.p12")) {
26         ksAlice.load(fis, "Alice".toCharArray()); /*
27     }
28
29     String aliasAlice = ksAlice.aliases().nextElement();
30     Certificate aliceCert = ksAlice.getCertificate(aliasAlice);
31     PublicKey alicePublicKey = aliceCert.getPublicKey();
32 }
```

Carichiamo i portachiavi (Keystore PKCS#12) di:

Bob: carichiamo Bob.p12 per estrarre la sua chiave privata che poi servirà per firmare il messaggio.

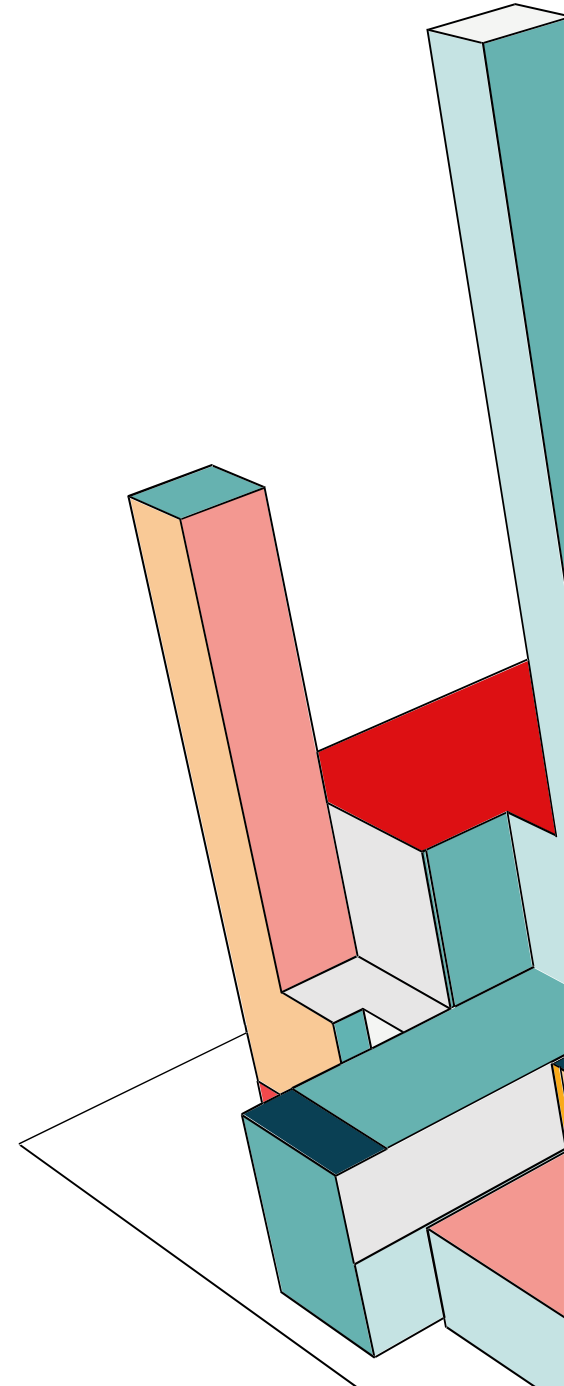
Alice: carichiamo il Keystore di Alice per estrarre la sua chiave pubblica, che servirà a Bob per cifrare la chiave di sessione.



JCA: BOB SENDER

Bob prende il messaggio in chiaro (text.txt), calcola un hash e lo cripta con la sua chiave privata. Infine crea una digitalSignature. Se Alice riuscirà a validarla con la chiave pubblica di Bob, avrà la certezza matematica che il messaggio viene da Bob e non è stato modificato.

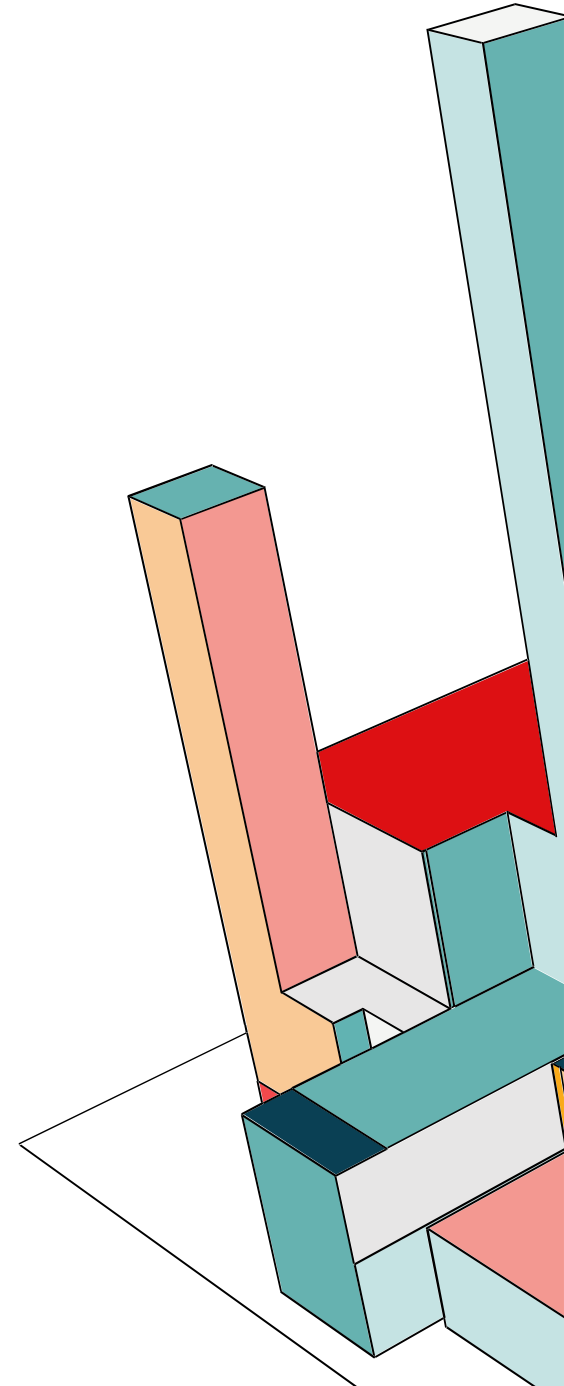
```
33 // Leggi messaggio in chiaro
34 byte[] messageBytes = Files.readAllBytes(Paths.get("text.txt"));
35 String messageString = new String(messageBytes);
36 System.out.println("Bob sta per mandare questo messaggio:");
37 System.out.println(messageString);
38
39 // Calcola hash del messaggio e stampalo
40 MessageDigest digest = MessageDigest.getInstance("SHA-256");
41 byte[] hash = digest.digest(messageBytes);
42 System.out.println("Hash del messaggio (SHA-256): " + bytesToHex(hash));
43
44 // Firma il messaggio con chiave privata di Bob
45 Signature signature = Signature.getInstance("SHA256withRSA");
46 signature.initSign(bobPrivateKey);
47 signature.update(messageBytes);
48 byte[] digitalSignature = signature.sign();
```



JCA: BOB SENDER

Bob unisce il messaggio originale e la firma in un unico pacchetto di byte. Questo è ciò che verrà spedito e protetto. In questo caso usiamo la crittografia ibrida, ovvero Bob non usa RSA per cifrare tutto il messaggio perché sarebbe troppo lento per file grandi. Invece, genera una chiave AES 256-bit casuale (chiave di sessione). Cifra il payload ottenendo encryptedPayload.

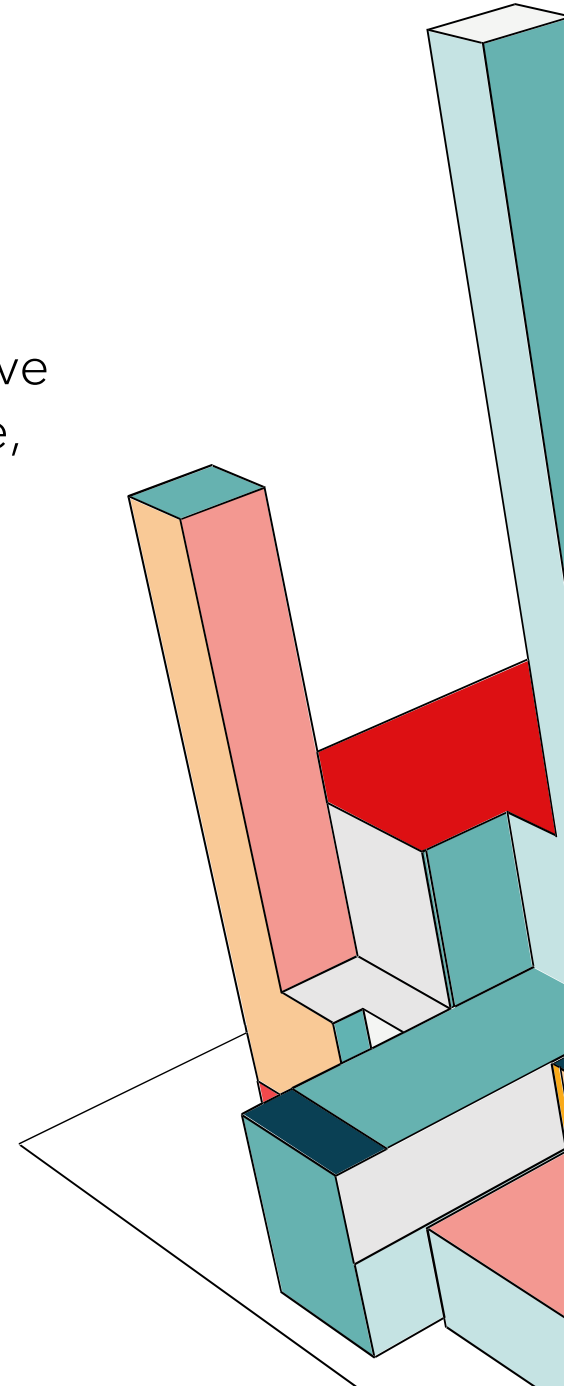
```
50      // Concatena messaggio + firma
51      ByteArrayOutputStream baos = new ByteArrayOutputStream();
52      baos.write(messageBytes);
53      baos.write(digitalSignature);
54      byte[] payload = baos.toByteArray();
55
56      // Cifra con AES
57      KeyGenerator keyGen = KeyGenerator.getInstance("AES");
58      keyGen.init(256);
59      SecretKey aesKey = keyGen.generateKey();
60      Cipher aesCipher = Cipher.getInstance("AES/GCM/NoPadding");
61      aesCipher.init(Cipher.ENCRYPT_MODE, aesKey);
62      byte[] encryptedPayload = aesCipher.doFinal(payload);
63      byte[] iv = aesCipher.getIV();
64
```



JCA: BOB SENDER

Adesso Bob deve inviare ad Alice la chiave AES casuale che ha appena generato, ovviamente senza mandarla in chiaro. Quindi, Bob prende la chiave AES e la cifra usando la chiave pubblica di Alice. In questo modo solo Alice, che possiede la chiave privata corrispondente, potrà recuperare la chiave AES.

```
65 // Cifra la chiave AES con pubblica di Alice
66 Cipher rsaCipher = Cipher.getInstance("RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
67 rsaCipher.init(Cipher.ENCRYPT_MODE, alicePublicKey);
68 byte[] encryptedAesKey = rsaCipher.doFinal(aesKey.getEncoded());
69
```



JCA: BOB SENDER

```

70 // Salva pacchetto cifrato su file
71 try (FileOutputStream fos = new FileOutputStream("packet.bin")) {
72     fos.write(intToBytes(encryptedAesKey.length));
73     fos.write(encryptedAesKey);
74     fos.write(intToBytes(iv.length));
75     fos.write(iv);
76     fos.write(encryptedPayload);
77 }
78
79 // Salva certificato di Bob
80 try (FileOutputStream fos = new FileOutputStream("bob_cert.pem")) {
81     fos.write(bobCert.getEncoded());
82 }
83
84 System.out.println("Pacchetto cifrato creato: packet.bin");
85 System.out.println("Certificato di Bob salvato: bob_cert.pem");
86
87
88 // converte int in byte[]
89 private static byte[] intToBytes(int value) {
90     return new byte[] {
91         (byte)(value >>> 24),
92         (byte)(value >>> 16),
93         (byte)(value >>> 8),
94         (byte)value
95     };
96 }
97
98 // converte byte[] in stringa esadecimale
99 private static String bytesToHex(byte[] bytes) {
100     StringBuilder sb = new StringBuilder();
101     for(byte b : bytes) {
102         sb.append(String.format("%02x", b));
103     }
104     return sb.toString();
105 }

```

Bob salva tutto in un file unico. La struttura del file binario creato è:

- **4 byte**: Lunghezza della Chiave AES Cifrata.
- **N byte**: La Chiave AES Cifrata (RSA).
- **4 byte**: Lunghezza dell'IV.
- **N byte**: L'IV.
- **N byte**: Il Payload Cifrato.



JCA: ALICE RECEIVER

Alice carica il suo portachiavi (Alice.p12), perché ha bisogno della sua chiave privata per decifrare la chiave di sessione che ha inviato Bob. Dopodiché Alice carica il file bob_cert.pem, usa CertificateFactory per trasformare i byte grezzi in un oggetto X509Certificate. Da questo estrae la Chiave Pubblica di Bob che userà alla fine per verificare la firma.

```
13     public static void main(String[] args) throws Exception {
14
15         // Carica chiave privata di Alice
16         KeyStore ksAlice = KeyStore.getInstance("PKCS12");
17         try (FileInputStream fis = new FileInputStream("Alice.p12")) {
18             ksAlice.load(fis, "Alice".toCharArray());
19         }
20
21         PrivateKey alicePrivateKey = (PrivateKey) ksAlice.getKey("alice", "Alice".toCharArray());
22
23         // Carica certificato di Bob e ricava la chiave pubblica
24         Certificate bobCert;
25         try (FileInputStream fis = new FileInputStream("bob_cert.pem")) {
26             byte[] certBytes = fis.readAllBytes();
27             bobCert = java.security.cert.CertificateFactory.getInstance("X.509")
28                 .generateCertificate(new ByteArrayInputStream(certBytes));
29         }
30         PublicKey bobPublicKey = bobCert.getPublicKey();
```

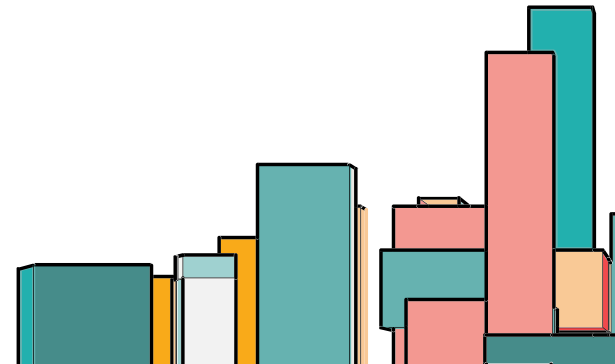


JCA: ALICE RECEIVER

Usando `DataInputStream`, Alice legge i byte che indicano le lunghezze e poi legge i rispettivi array di byte. Così facendo riesce a ricavare la chiave AES cifrata con RSA, il vettore di inizializzazione e il messaggio vero e proprio ancora cifrato.

Alice usa la sua chiave privata per decifrare la chiave AES, quindi ora possiede la stessa chiave simmetrica generata da Bob.

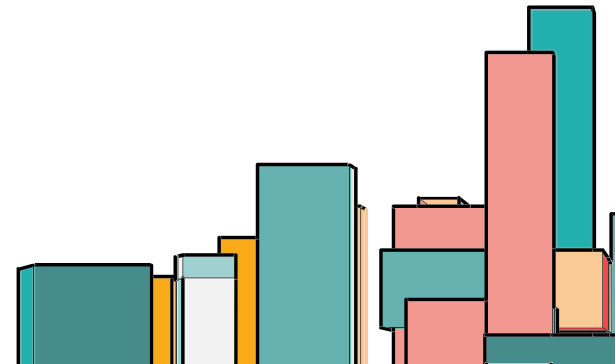
```
32 // Leggi pacchetto cifrato
33 byte[] fileData = Files.readAllBytes(Paths.get("packet.bin"));
34 DataInputStream dis = new DataInputStream(new ByteArrayInputStream(fileData));
35
36 int lenAesKey = dis.readInt();
37 byte[] encryptedAesKey = new byte[lenAesKey];
38 dis.readFully(encryptedAesKey);
39
40 int lenIv = dis.readInt();
41 byte[] iv = new byte[lenIv];
42 dis.readFully(iv);
43
44 byte[] encryptedPayload = dis.readAllBytes();
45
46 // Decifra chiave AES con chiave privata di Alice
47 Cipher rsaCipher = Cipher.getInstance("RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
48 rsaCipher.init(Cipher.DECRYPT_MODE, alicePrivateKey);
49 byte[] aesKeyBytes = rsaCipher.doFinal(encryptedAesKey);
50 SecretKey aesKey = new javax.crypto.spec.SecretKeySpec(aesKeyBytes, "AES");
51
```



JCA: ALICE RECEIVER

Ora che ha la chiave AES, Alice può decifrare il contenuto. Se il messaggio fosse stato manomesso durante il transito, il metodo `doFinal` lancerebbe una `AEADBadTagException`, avvisando Alice che l'integrità è compromessa. Il payload decifrato contiene tutti i byte attaccati, quindi li divide in modo da stampare il messaggio corretto.

```
52 // Decifra payload con AES
53 Cipher aesCipher = Cipher.getInstance("AES/GCM/NoPadding");
54 GCMParameterSpec spec = new GCMParameterSpec(128, iv);
55 aesCipher.init(Cipher.DECRYPT_MODE, aesKey, spec);
56 byte[] payload = aesCipher.doFinal(encryptedPayload);
57
58 // Separa messaggio e firma
59 int sigLength = 256;
60 int msgLength = payload.length - sigLength;
61
62 byte[] messageBytes = new byte[msgLength];
63 byte[] digitalSignature = new byte[sigLength];
64
65 System.arraycopy(payload, 0, messageBytes, 0, msgLength);
66 System.arraycopy(payload, msgLength, digitalSignature, 0, sigLength);
67
68 // Calcola e stampa hash del messaggio decifrato
69 MessageDigest digest = MessageDigest.getInstance("SHA-256");
70 byte[] hash = digest.digest(messageBytes);
71 System.out.println("Hash calcolato sul messaggio decrittato (SHA-256): " + bytesToHex(hash));
72
```



JCA: ALICE RECEIVER

```
73 // Verifica la firma
74 Signature signature = Signature.getInstance("SHA256withRSA");
75 signature.initVerify(bobPublicKey);
76 signature.update(messageBytes);
77
78 boolean isValid = signature.verify(digitalSignature);
79
80 System.out.println("Messaggio decifrato:");
81 System.out.println(new String(messageBytes));
82 System.out.println("Firma valida? " + isValid);
83 }
84
85 // converte byte[] in stringa esadecimale
86 private static String bytesToHex(byte[] bytes) {
87     StringBuilder sb = new StringBuilder();
88     for (byte b : bytes) {
89         sb.append(String.format("%02x", b));
90     }
91     return sb.toString();
92 }
93 }
```

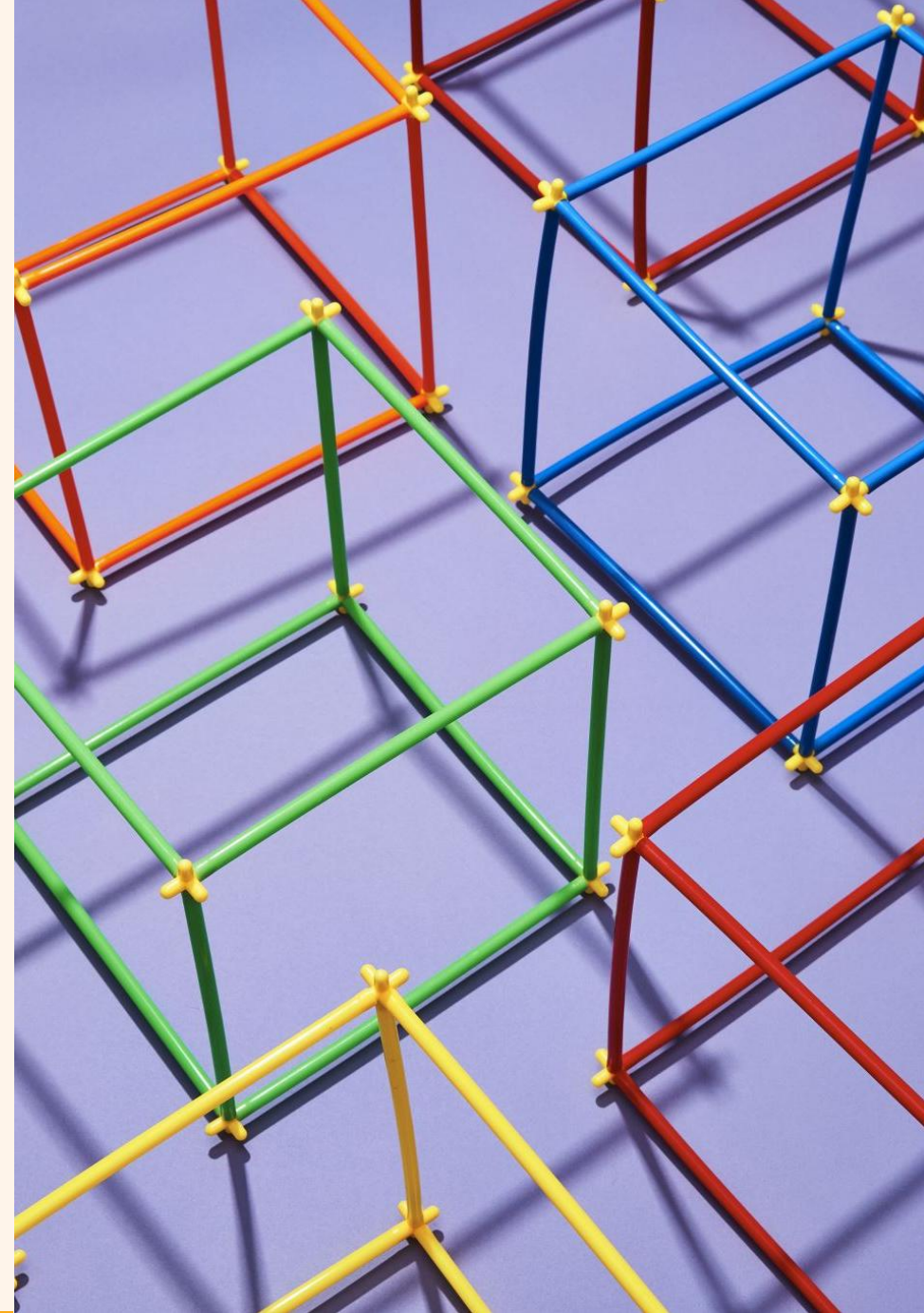
Alice prende il messaggio in chiaro e verifica se usando la chiave pubblica di Bob su questi dati, si genera la stessa firma digitale.

Se **isValid** è **true**: Autenticazione confermata, quindi abbiamo la conferma che il messaggio è stato scritto da Bob e non è stato manomesso.

Se **isValid** è **false**: Autenticazione fallita, il mittente non è Bob, o il messaggio è diverso da quello firmato.

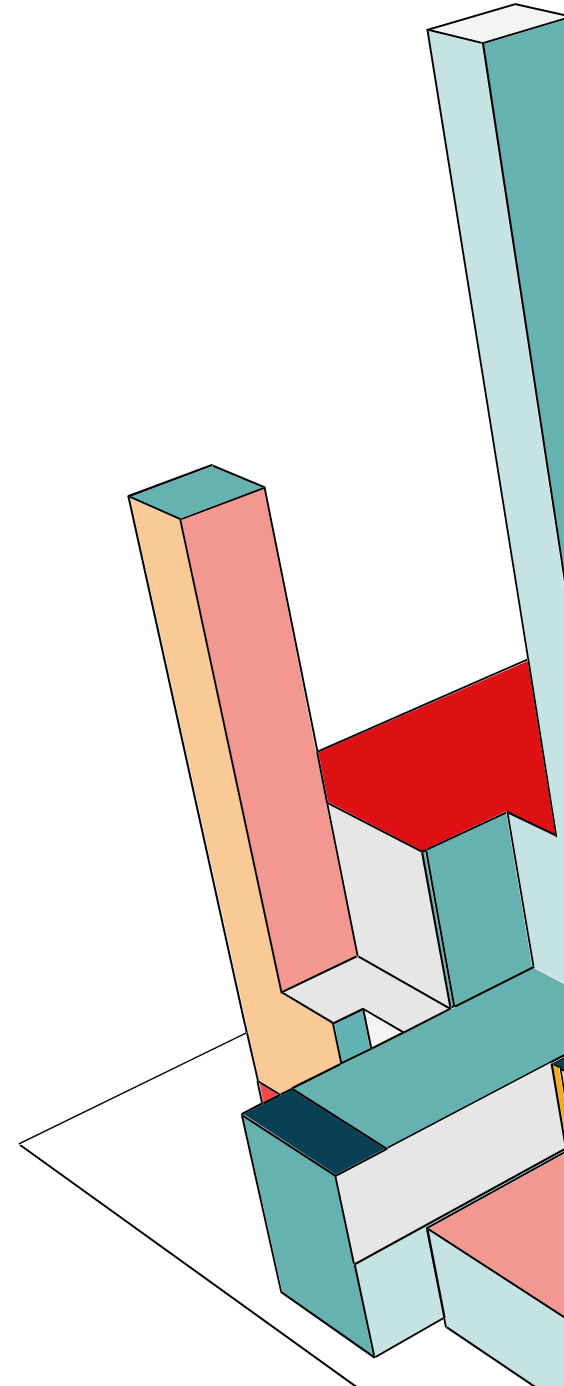


3) CONFIGURAZIONE APACHE/TOMCAT CON HTTPS



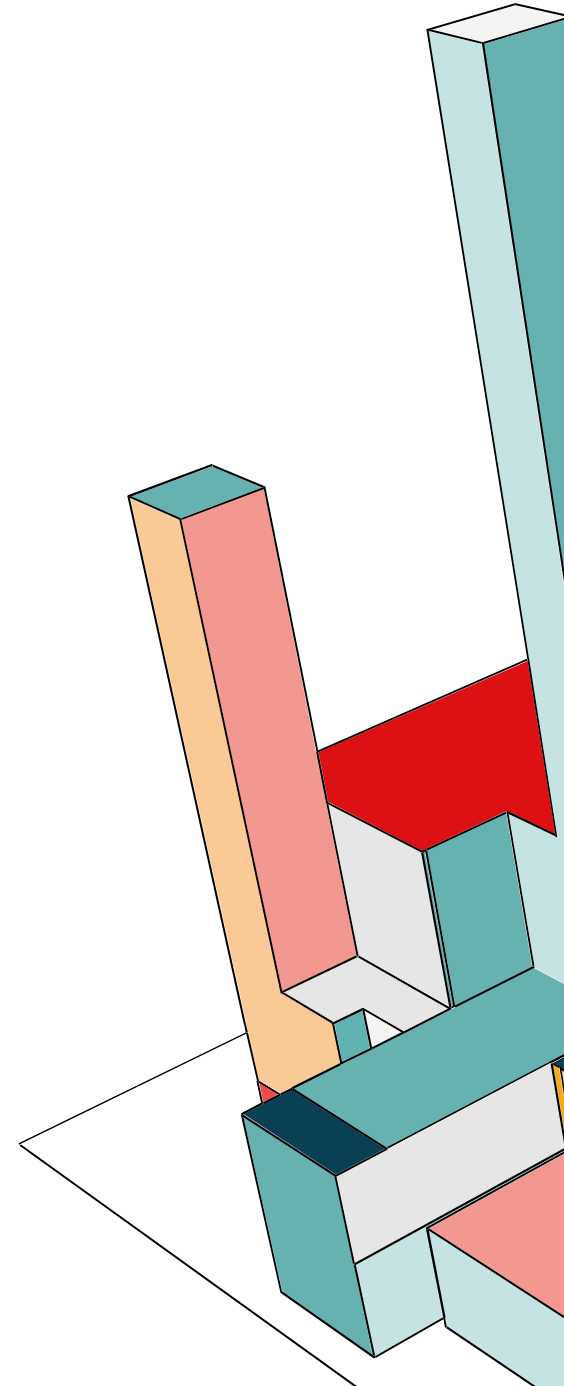
HTTPS

HTTPS è la versione sicura del protocollo HTTP che utilizza TLS (Transport Layer Security) per ottenere crittografia (il traffico tra client e server è cifrato, evitando che qualcuno possa leggerlo), autenticazione (il browser può verificare che il server sia veramente quello giusto tramite il certificato TLS), integrità (i dati non possono essere modificati mentre viaggiano). Per usare HTTPS serve un certificato digitale installato sul server, rilasciato da un'autorità (CA), nel nostro caso usiamo un certificato self-signed. Noi useremo Apache.



APACHE

Apache è un web server ad alte prestazioni che offre funzionalità di load balancing, server caching e reverse proxy. Abbiamo scelto di impiegarlo come intermediario nella comunicazione col server di back-end, configurandolo come reverse proxy per HTTPS: le comunicazioni tra i client e Apache sono crittografate, proteggendo i dati in transito, mentre il back-end può continuare a comunicare in HTTP.



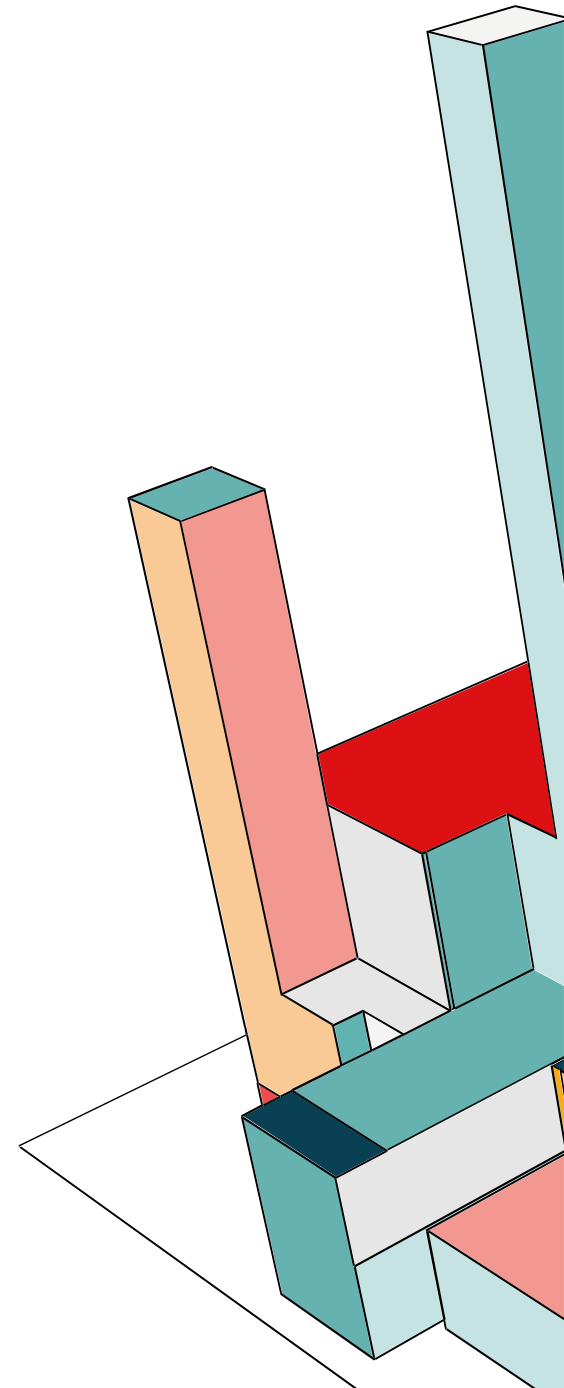
APACHE: DEPLOYMENT

Il back-end Node.js e il reverse proxy Apache sono stati deployati su Docker e collegati alla stessa rete condivisa. Abbiamo creato:

- Il file di configurazione httpd.conf
- la cartella certs, dove abbiamo inserito il certificato (certificato.pem) e chiave privata (chiave_privata.pem)
- Il file Dockerfile per specificare la versione dell'immagine da usare, i vari file da copiare nel container, i volumi, ecc.

```
1  services:
2    # --- FRONTEND (REACT) ---
3    >Run Service
4    web:
5      build: ./frontend
6      expose:
7        - "3000"
8      networks:
9        - network-backend
10     deploy:
11       resources:
12         limits:
13           cpus: '0.50'      # Massimo 50% di un core
14           memory: 512M     # Massimo 512MB di RAM
```

```
91  # --- REVERSE PROXY (APACHE) ---
92  >Run Service
93  apache:
94    build: ./apache
95    container_name: apache_secure
96    ports:
97      - "80:8080"      # HTTP
98      - "443:8443"    # HTTPS
99    networks:
100     - network-backend
101    deploy:
102     resources:
103       limits:
104         cpus: '0.50'
105         memory: 256M
```



APACHE: CONFIGURAZIONE

```
1  ServerRoot "/usr/local/apache2"
2  Listen 8080
3  Listen 8443
4
5  # --- MODULI (Standard) ---
6  LoadModule mpm_event_module modules/mod_mpm_event.so
7  LoadModule authz_core_module modules/mod_authz_core.so
8  LoadModule authz_host_module modules/mod_authz_host.so
9  LoadModule log_config_module modules/mod_log_config.so
10 LoadModule env_module modules/mod_env.so
11 LoadModule headers_module modules/mod_headers.so
12 LoadModule setenvif_module modules/mod_setenvif.so
13 LoadModule version_module modules/mod_version.so
14 LoadModule ssl_module modules/mod_ssl.so
15 LoadModule proxy_module modules/mod_proxy.so
16 LoadModule proxy_http_module modules/mod_proxy_http.so
17 LoadModule proxy_wstunnel_module modules/mod_proxy_wstunnel.so
18 LoadModule socache_shmcb_module modules/mod_socache_shmcb.so
19 LoadModule unixd_module modules/mod_unixd.so
20 LoadModule alias_module modules/mod_alias.so
21 LoadModule rewrite_module modules/mod_rewrite.so
22
23 User daemon
24 Group daemon
25
26 ServerTokens Prod
27 ServerSignature Off
28 TraceEnable Off
29
30 SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1
31 SSLCipherSuite HIGH:!aNULL:!MD5:!3DES
32 SSLHonorCipherOrder on
33 SSLSessionCache "shmcb:/usr/local/apache2/logs/ssl_scache(512000)"
34 SSLSessionCacheTimeout 300
35
```

Impostiamo le porte su cui il server sarà in ascolto, importiamo i moduli necessari.

- **ServerTokens Prod / ServerSignature Off:** Nasconde la versione di Apache e del sistema operativo nelle pagine di errore.
- **TraceEnable Off:** Disabilita il metodo HTTP TRACE, prevenendo attacchi di tipo XST (Cross-Site Tracing).
- **SSLProtocol & SSLCipherSuite:** Disabilita vecchi protocolli insicuri (SSLv3, TLS 1.0, 1.1) e accetta solo cifrari forti ("HIGH").



APACHE: CONFIGURAZIONE

- Porta 8080: qui arriva traffico HTTP non sicuro che viene reindirizzata verso la versione sicura HTTPS sulla porta mappata del server.
- Porta 8443:
 - `SSLEngine on` attiva la crittografia
 - `SSLCertificateFile` / `KeyFile` specifica dove si trovano il certificato pubblico e la chiave privata.
 - `X-Forwarded-For`, `X-Forwarded-Proto`: i servizi interni alla rete Docker vedrebbero tutte le richieste provenire dall'IP di Apache, perché è lui che le intercetta tutte. Questi header servono a passare ai servizi interni il **vero IP** dell'utente e il protocollo originale, in modo che i log e i controlli di sicurezza interni siano corretti.

```
# --- REDIRECT HTTP -> HTTPS ---
<VirtualHost *:8080>
    ServerName localhost
    Redirect permanent / https://localhost/
</VirtualHost>

# --- SERVER HTTPS (SSL Termination) ---
<VirtualHost *:8443>
    ServerName localhost
    # 1. Crittografia
    SSLEngine on
    SSLCertificateFile "/usr/local/apache2/conf/certs/certificato.pem"
    SSLCertificateKeyFile "/usr/local/apache2/conf/certs/chiave_privata.pem"

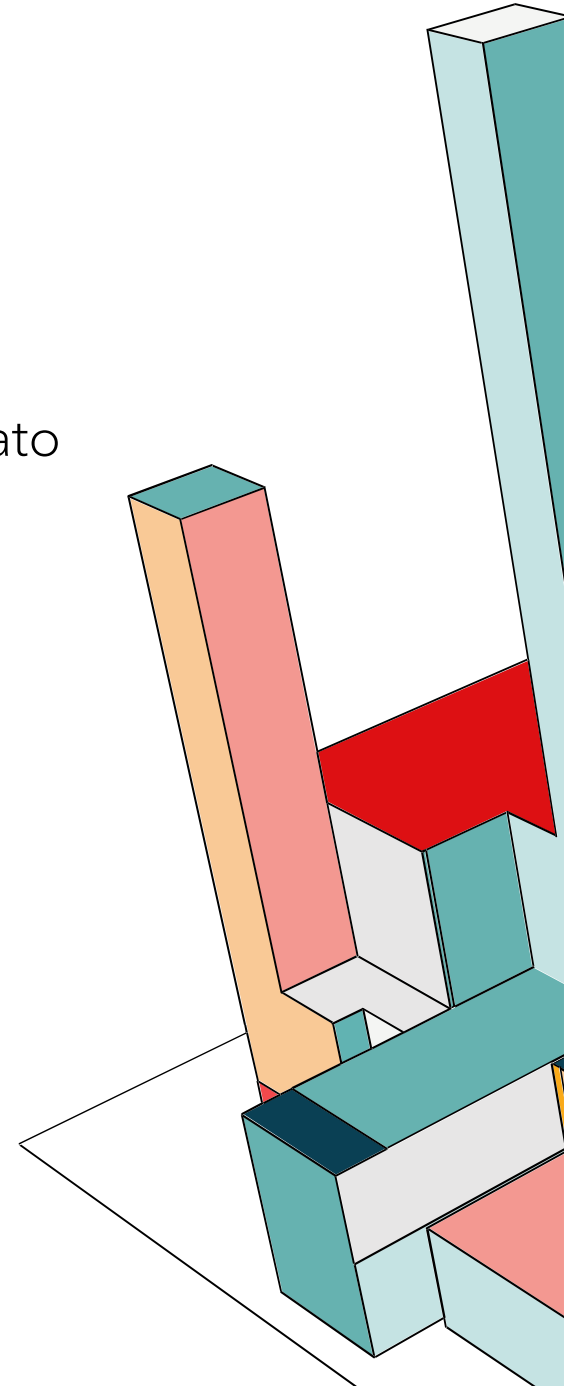
    ProxyRequests Off
    ProxyPreserveHost On
    ProxyAddHeaders On
    SSLProxyEngine on

    RequestHeader set X-Forwarded-Proto "https"
    RequestHeader set X-Forwarded-Port "443"
    RequestHeader set X-Forwarded-Host "localhost"
    RequestHeader set X-Forwarded-Server "localhost"
    RequestHeader set X-Forwarded-For "%{REMOTE_ADDR}s"
```

APACHE: CONFIGURAZIONE

Di seguito riportiamo la regola di instradamento, che intercetta le richieste WebSocket e le invia al container web. Tutto ciò che non corrisponde alle regole precedenti (quindi la navigazione normale dell'utente) viene mandato al Frontend React.

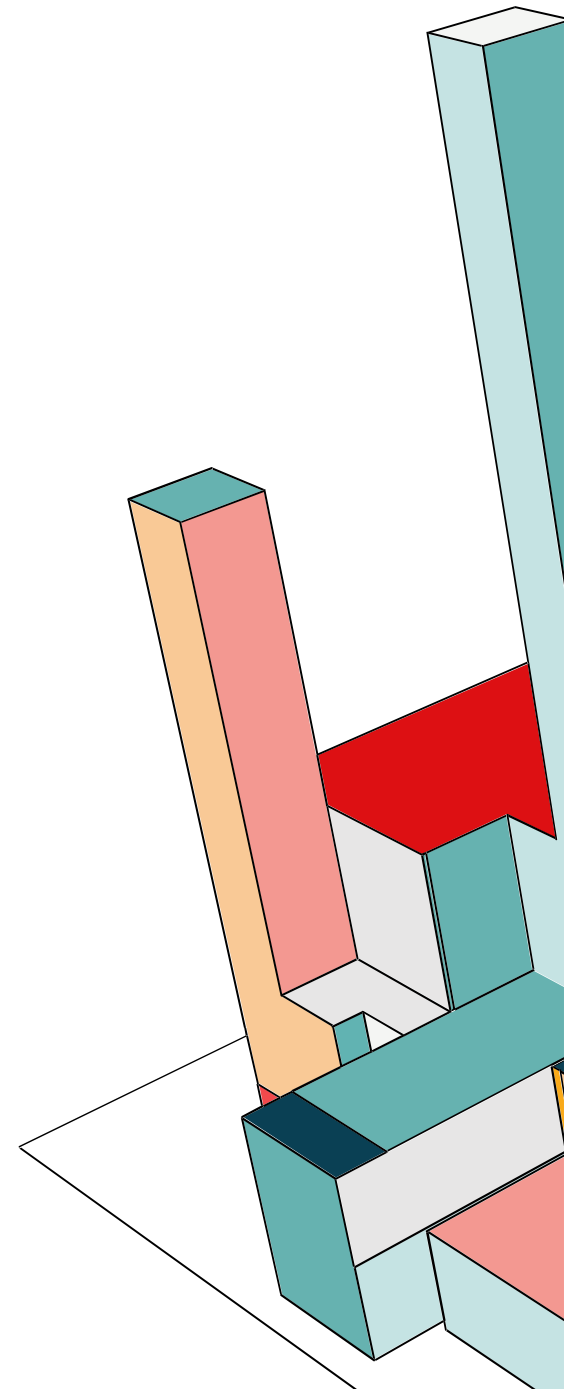
```
63 <Location "/ws">
64     ProxyPass ws://web:3000/ws
65     ProxyPassReverse ws://web:3000/ws
66 </Location>
67
68 ProxyPass /api !
69 ProxyPass /ws !
70
71 ProxyPass / http://web:3000/
72 ProxyPassReverse / http://web:3000/
73
74 </VirtualHost>
```



APACHE: VULNERABILITÀ NOTE

Abbiamo condotto un'analisi storica delle versioni recenti di Apache (2.4.x) tramite il sito ufficiale alla ricerca di vulnerabilità note che richiedono mitigazione. Il team di Apache ha assegnato ad ognuna di esse, a seconda del rischio, un livello di impatto variabile (low, moderate, important). Per ogni vulnerabilità, identificata dal relativo CVE, riportiamo:

- Severity e score assegnati dal **CVSS** (Common Vulnerability Scoring System) per capire quanto è oggettivamente grave la vulnerabilità.
- Dati forniti dall'**SSVC** (Stakeholder-Specific Vulnerability Categorization) per conoscere se la vulnerabilità è attualmente sfruttata, se è possibile automatizzare il processo di attacco e che tipo di danno potrebbe causare.



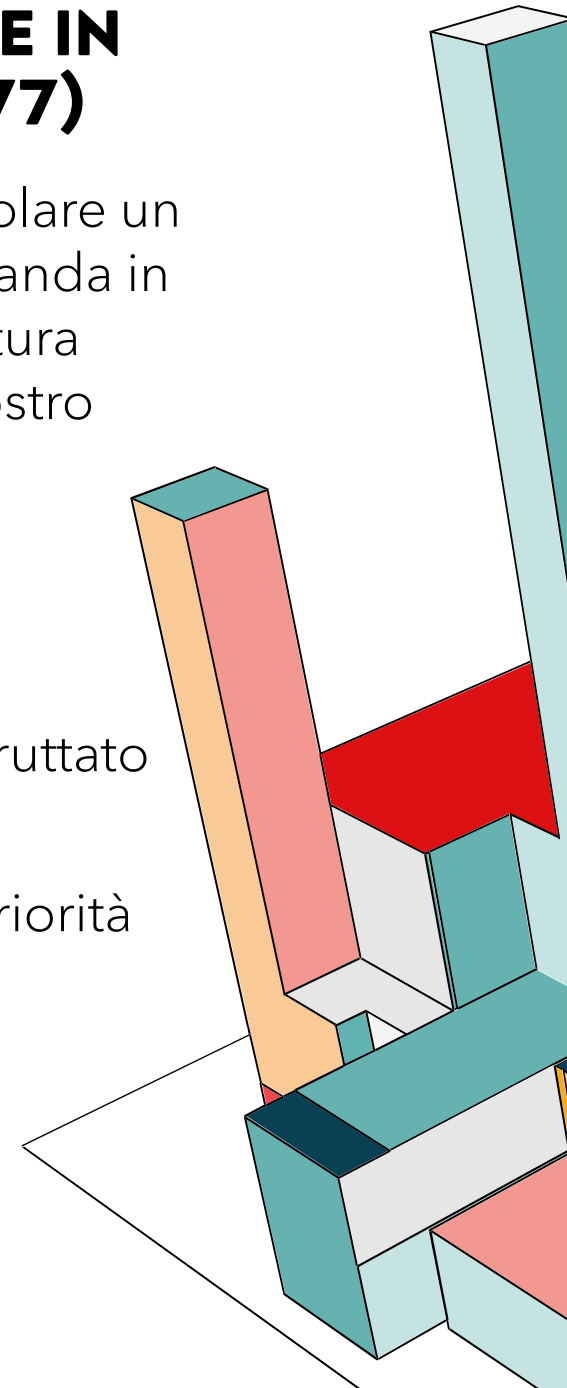
VULNERABILITÀ: CRASH RESULTING IN DENIAL OF SERVICE IN MOD_PROXY VIA A MALICIOUS REQUEST (CVE-2024-38477)

Questa vulnerabilità riguarda il crash critico del modulo `mod_proxy`, in particolare un attaccante può inviare una richiesta malevola appositamente costruita che manda in tilt il processo, causando un **Denial of Service (DoS)**. Poiché la nostra architettura utilizza Apache proprio come **Reverse Proxy**, questo modulo è il cuore del nostro sistema: se questo modulo crasha, l'intero servizio diventa irraggiungibile.

- **Classificazione team di Apache:** important
- **CVSS:** score 7.5 con severity HIGH
- **Analisi SSVC:** sebbene l'attacco non sia attualmente automatizzabile né sfruttato su larga scala, l'impatto sulla disponibilità è diretto.

Per questo motivo, consideriamo la mitigazione di questa vulnerabilità una priorità assoluta per garantire la continuità del servizio.

Reported to security team	2024-07-01
fixed by r1918839 in 2.4.x	2024-07-03
Update 2.4.61 released	2024-07-03
Affects	2.4.60



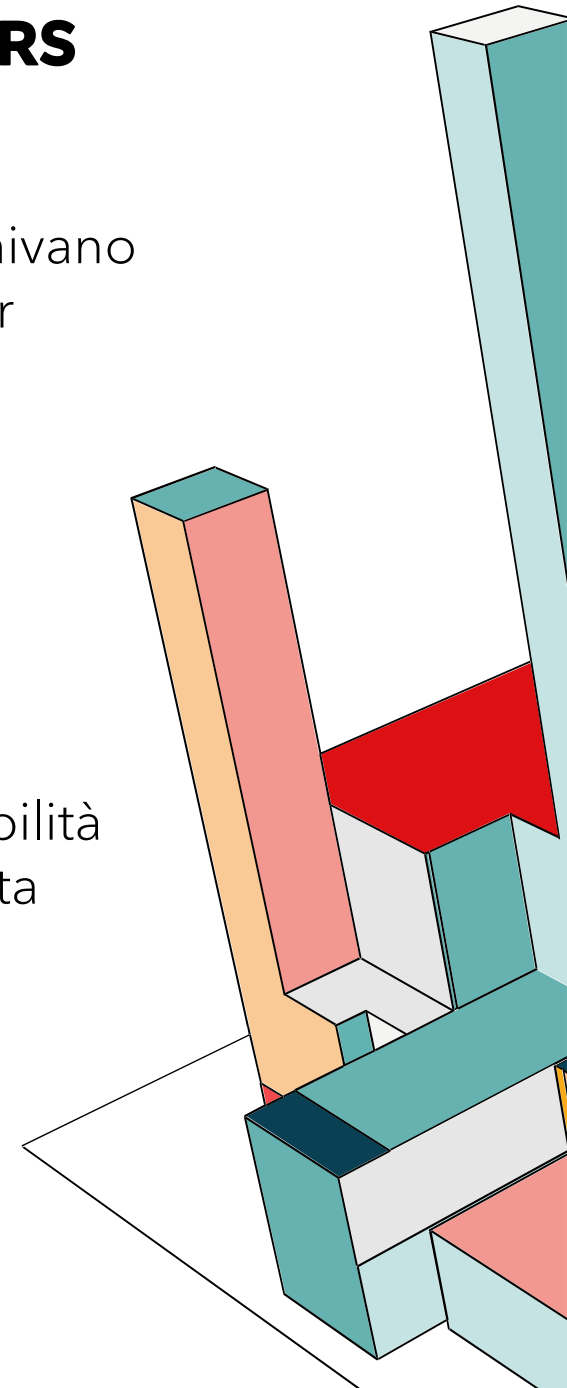
VULNERABILITÀ: SOURCE CODE DISCLOSURE WITH HANDLERS CONFIGURED VIA ADDTYPE (CVE-2024-39884)

In determinate configurazioni (es. **AddType**), in alcune circostanze, dei file venivano chiesti in maniera indiretta, permettendo ad un potenziale attaccante di poter leggere dei file, invece di eseguirli.

- **Classificazione team di Apache:** important
- **CVSS:** score 6.2 con severity MEDIUM
- **Analisi SSVC:** attualmente questa vulnerabilità non è sfruttata come fonte d'attacco e non è automatizzabile

Le conseguenze di un potenziale attacco eseguito sfruttando questa vulnerabilità avrebbe un impatto tecnico parziale. Possiamo dunque concludere che questa vulnerabilità ha una bassa priorità, ma va comunque mitigata per sicurezza.

Reported to security team	2024-07-01
fixed by r1918839 in 2.4.x	2024-07-03
Update 2.4.61 released	2024-07-03
Affects	2.4.60



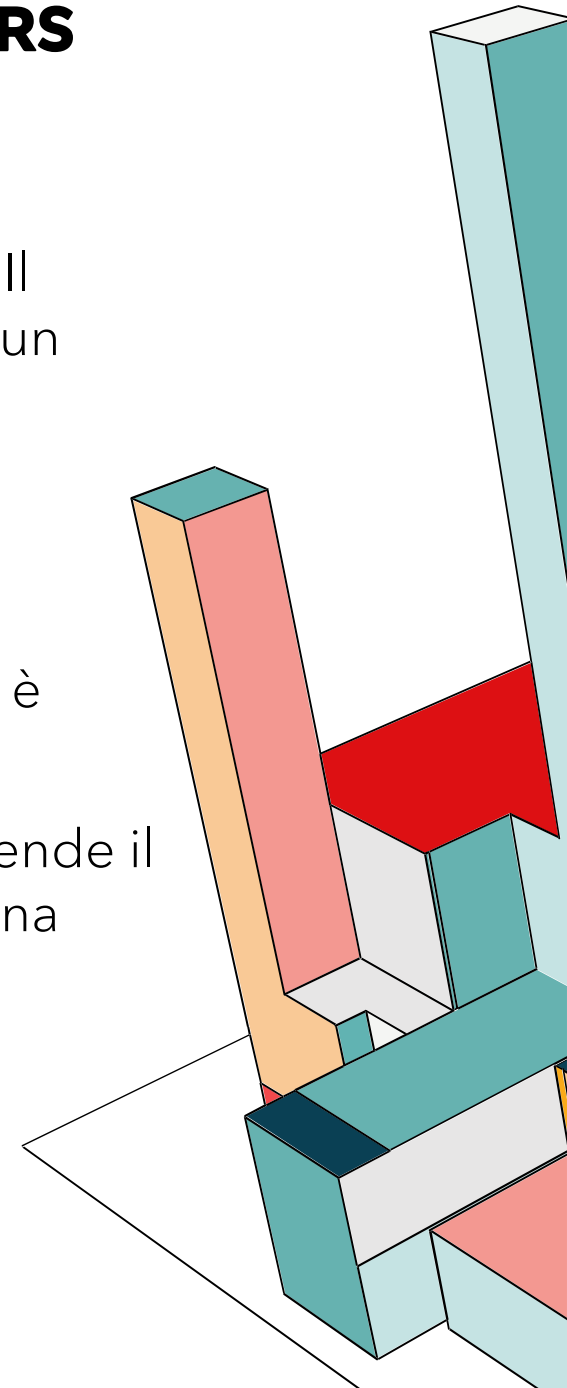
VULNERABILITÀ: SOURCE CODE DISCLOSURE WITH HANDLERS CONFIGURED VIA ADDTYPE (CVE-2024-40725)

Simile alla precedente, anche questa vulnerabilità riguarda l'esposizione involontaria del codice sorgente, dovuta a una gestione errata degli handler. Il rischio è che file contenenti informazioni sensibili vengano scaricati e letti da un attaccante invece di essere eseguiti dal server.

- **Classificazione team di Apache:** important
- **CVSS:** score 5.3 con severity MEDIUM
- **Analisi SSVC:** attualmente questa vulnerabilità ha un impatto parziale, non è sfruttata ma è automatizzabile

Nonostante lo score CVSS sia numericamente più basso, l'automatizzabilità rende il rischio molto più concreto. Riteniamo quindi che questa vulnerabilità abbia una priorità di intervento maggiore rispetto alla precedente

Reported to security team	2024-07-09
fixed by r1919249 in 2.4.x	2024-07-15
Update 2.4.62 released	2024-07-17
Affects	2.4.60 through 2.4.61



MITIGAZIONE: VERSIONE DI APACHE E BEST PRACTICES

Il primo step per la mitigazione è quella di utilizzare la versione più aggiornata e stabile di Apache (2.4.65), che ha corretto ufficialmente le falle citate nelle slide precedenti, nelle versioni 2.4.60 e 2.4.62.

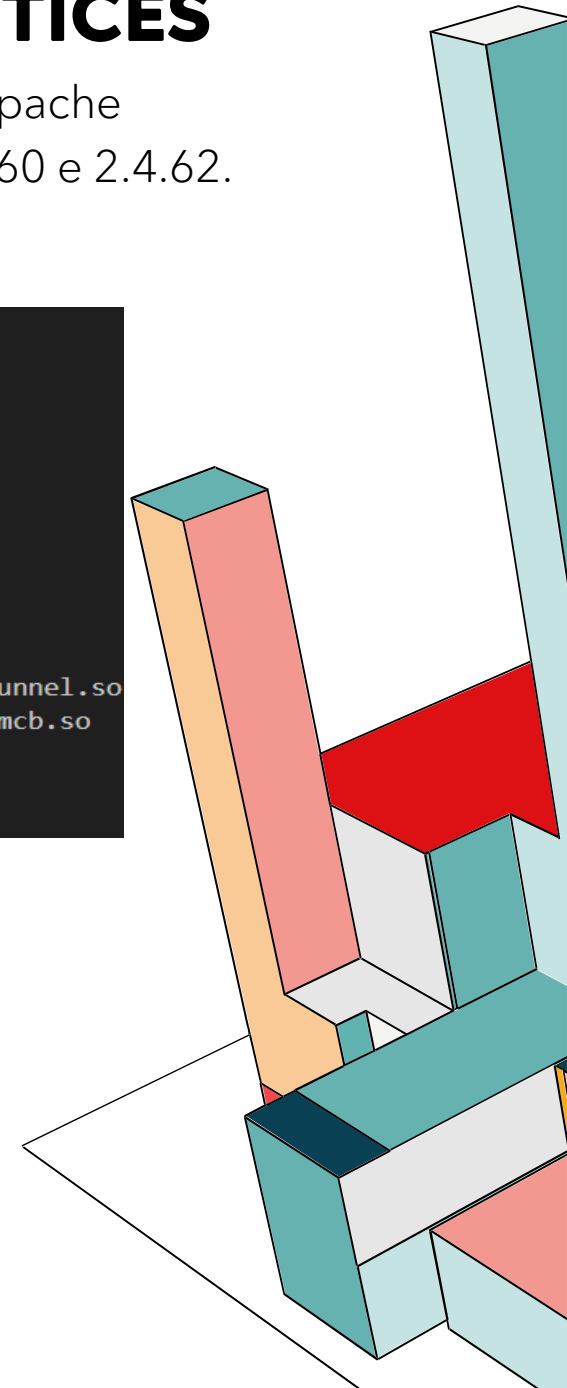
Oltre all'aggiornamento abbiamo adottato le best practices di configurazione:

- **Moduli essenziali:** abbiamo attivato solo i moduli strettamente necessari, disabilitando funzionalità superflue, riducendo così la superficie di attacco
- **Privilegio minimo:** il server viene eseguito come utente non root (daemon), limitando i danni in caso di intrusione. Di conseguenza le porte su cui è in ascolto Apache devono essere non privilegiate, quindi >1024

```
LoadModule mpm_event_module modules/mod_mpm_event.so
LoadModule authz_core_module modules/mod_authz_core.so
LoadModule authz_host_module modules/mod_authz_host.so
LoadModule log_config_module modules/mod_log_config.so
LoadModule env_module modules/mod_env.so
LoadModule headers_module modules/mod_headers.so
LoadModule setenvif_module modules/mod_setenvif.so
LoadModule version_module modules/mod_version.so
LoadModule ssl_module modules/mod_ssl.so
LoadModule proxy_module modules/mod_proxy.so
LoadModule proxy_http_module modules/mod_proxy_http.so
LoadModule proxy_wstunnel_module modules/mod_proxy_wstunnel.so
LoadModule socache_shmcb_module modules/mod_socache_shmcb.so
LoadModule unixd_module modules/mod_unixd.so
LoadModule alias_module modules/mod_alias.so
LoadModule rewrite_module modules/mod_rewrite.so
```

```
Listen 8080
Listen 8443

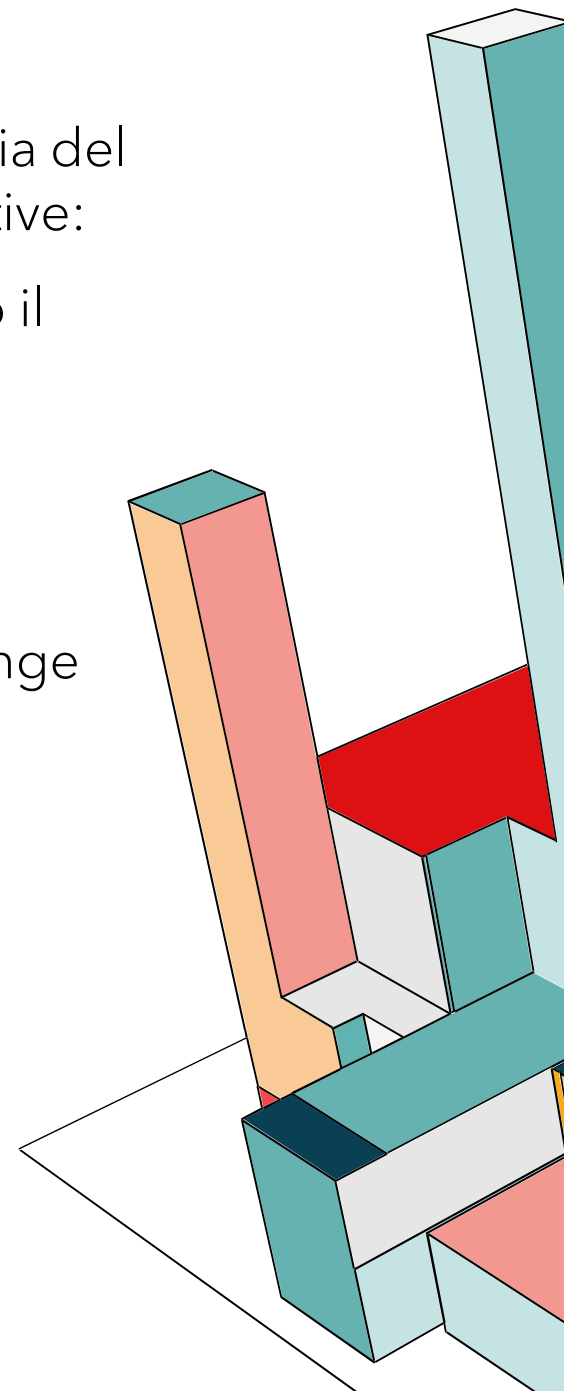
User daemon
Group daemon
```



MITIGAZIONE: INFORMATION DISCLOSURE

Per mitigare il rischio di Information Disclosure, una delle categorie di minaccia del modello STRIDE, abbiamo inserito, sempre nel file `httpd.conf` le seguenti direttive:

- **ServerTokens Prod:** istruisce Apache a restituire nell'header HTTP Server solo il nome del prodotto ("Apache"), invece di rivelare la versione esatta, il sistema operativo e i moduli installati (es. *"Apache/2.4.52 (Ubuntu) OpenSSL/1.1.1"*), fornendo all'attaccante una mappa precisa per cercare exploit mirati (Fingerprinting).
- **ServerSignature Off:** rimuove la riga alla fine della pagina che Apache aggiunge automaticamente alle pagine generate dal server, che di solito contiene la versione del server e l'indirizzo email dell'amministratore.

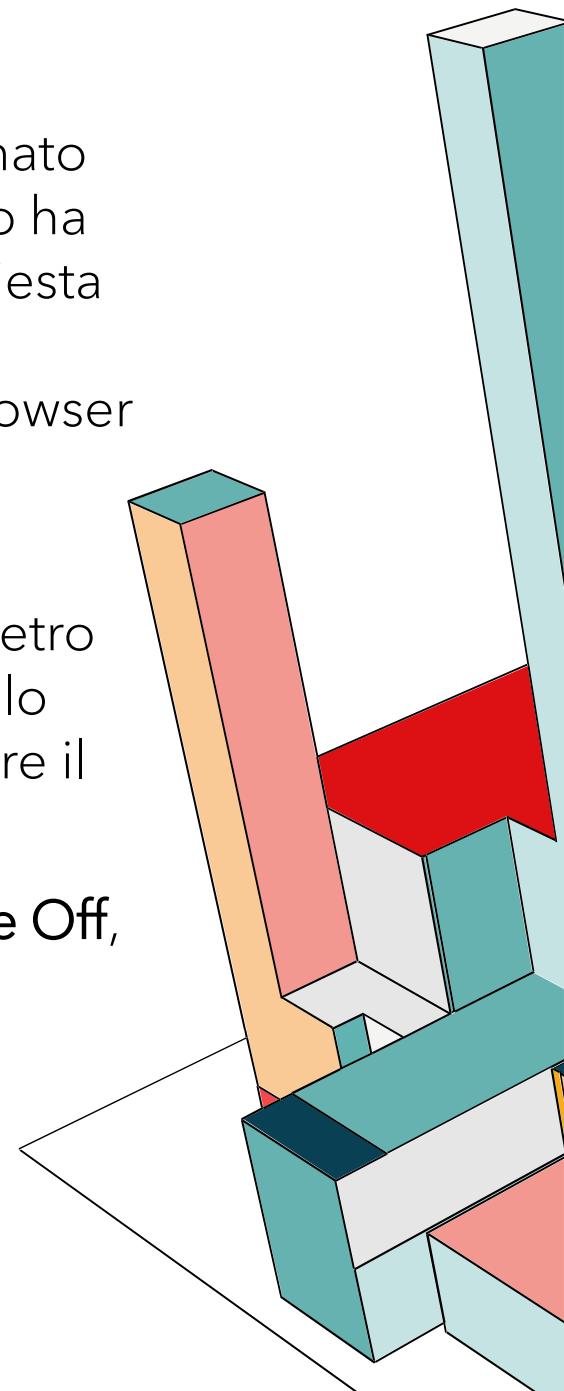


MITIGAZIONE: CROSS-SITE TRACING

Il **Cross-Site Tracing** è una tecnica di attacco che sfrutta il metodo HTTP Trace (nato per il debug) per aggirare le protezioni di sicurezza dei cookie. Questo metodo ha come scopo quello di rispondere al client facendo un semplice 'eco' della richiesta ricevuta. Tuttavia, questo comportamento diventa critico se combinato con un attacco **Cross-Site Scripting**, il quale permette di iniettare script malevoli nel browser della vittima.

Sebbene esista il flag **HttpOnly**, che impedisce agli script (come JavaScript) di leggere direttamente i dati di sessione, forzando il metodo Trace si ottiene indietro l'intera richiesta, compresi i cookie che dovevano essere protetti. A quel punto lo script malevolo non deve fare altro che leggere la risposta del server per estrarre il cookie.

Per neutralizzare questa minaccia alla radice, utilizziamo la direttiva **TraceEnable Off**, in modo tale che il server rifiuti categoricamente le richieste di tipo Trace.



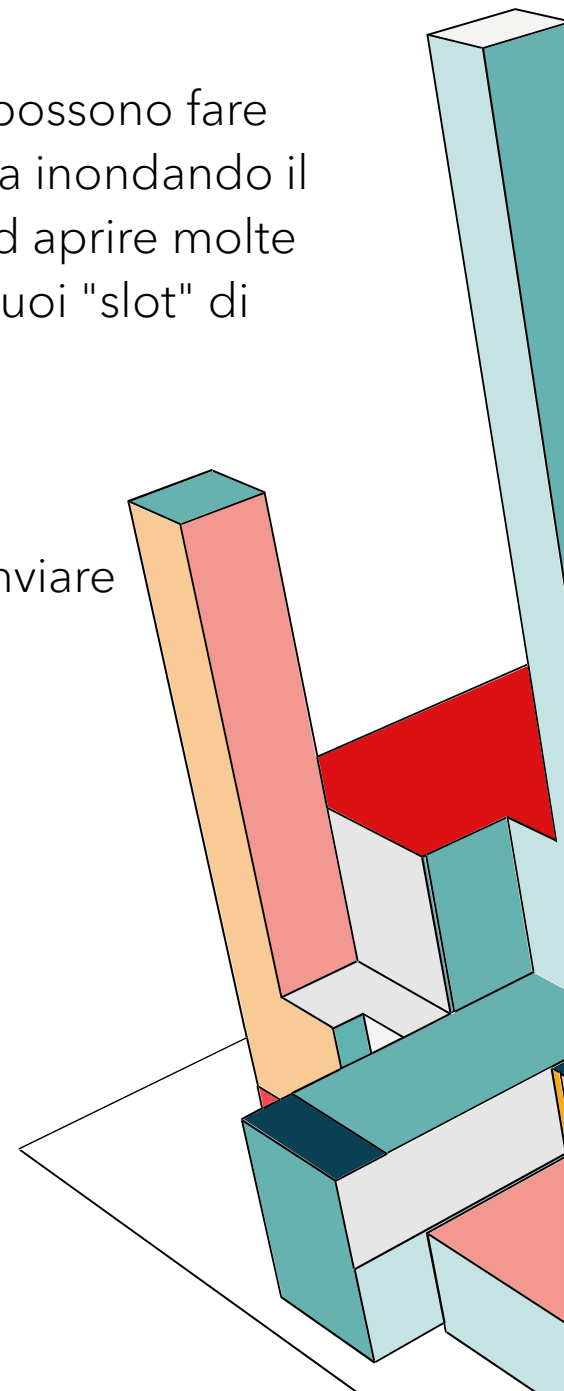
MITIGAZIONE: DOS

Gli attacchi di tipo Denial Of Service puntano a rendere un servizio irraggiungibile e lo possono fare seguendo due approcci. Il primo è **volumetrico**, cercando di saturare CPU, RAM o banda inondando il server con una mole enorme di traffico, il secondo invece detto "**Low & Slow**" e punta ad aprire molte connessioni inviando dati lentissimamente, costringendo il server a tenere impegnati i suoi "slot" di connessione finché non può più accettare utenti legittimi.

Una prima mitigazione consiste nel definire le seguenti direttive:

- **Timeout 60**: Definisce il tempo massimo che il server aspetta per ricevere dati o per inviare una conferma, impedendo di tenere una connessione aperta per minuti interi
- **KeepAlive On**: Mantiene la connessione aperta per più richieste dallo stesso client
- **MaxKeepAliveRequests 100**: Limita il numero di richieste per singola connessione persistente, impedendo di monopolizzare una connessione all'infinito
- **KeepAliveTimeout 5**: Se un client non invia una nuova richiesta entro 5 secondi dalla precedente, chiude la connessione

```
# -- Mitigazione DoS --  
Timeout 60  
KeepAlive On  
MaxKeepAliveRequests 100  
KeepAliveTimeout 5
```



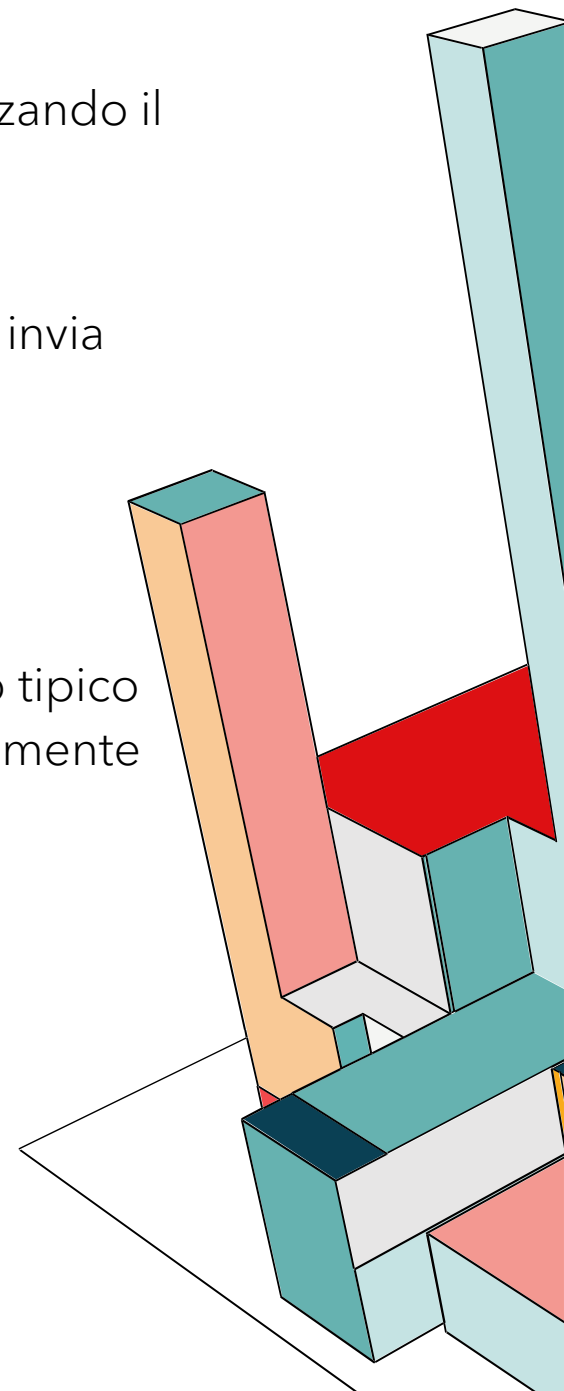
MITIGAZIONE: DOS

La seconda mitigazione, specifica per gli attacchi di tipo "Low & Slow", la facciamo utilizzando il modulo `mod_reqtimeout` per imporre una velocità minima di trasmissione:

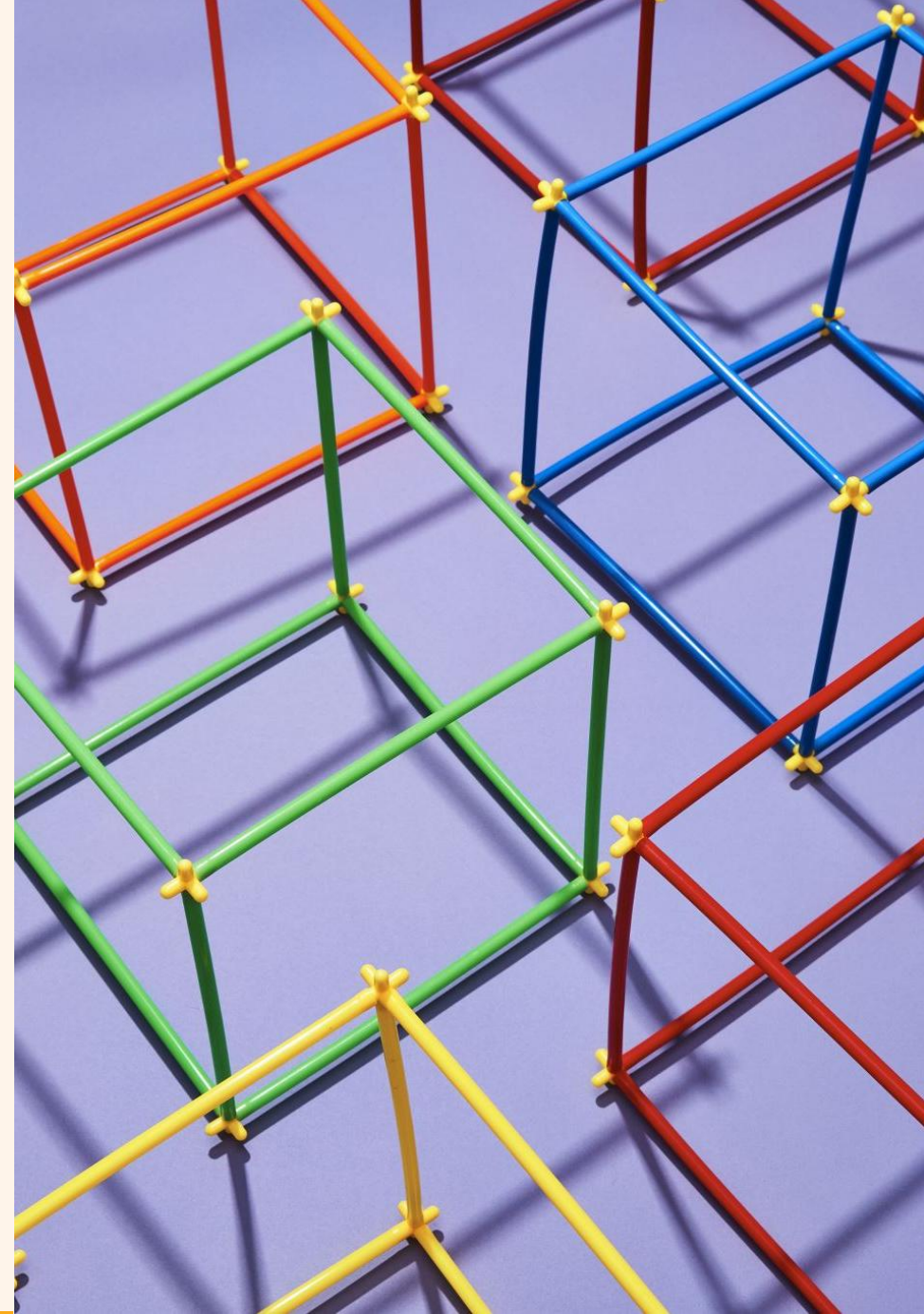
- **Header:** Abbiamo configurato una soglia dinamica. Il server inizialmente concede 20 secondi, per la lettura degli header HTTP, fino a un massimo di 40 secondi se il client invia dati a una velocità di almeno 500 byte al secondo
- **Body:** Viene applicata la stessa logica al body, con un timeout però di 20 secondi

Di conseguenza se un client trasmette più lentamente di questa soglia (comportamento tipico dell'attacco per tenere occupata la linea), il server identifica la minaccia e chiude forzatamente la connessione.

```
# Blocca connessioni troppo lente (Slow Loris Attack)
<IfModule reqtimeout_module>
  RequestReadTimeout header=20-40,MinRate=500 body=20,MinRate=500
</IfModule>
```

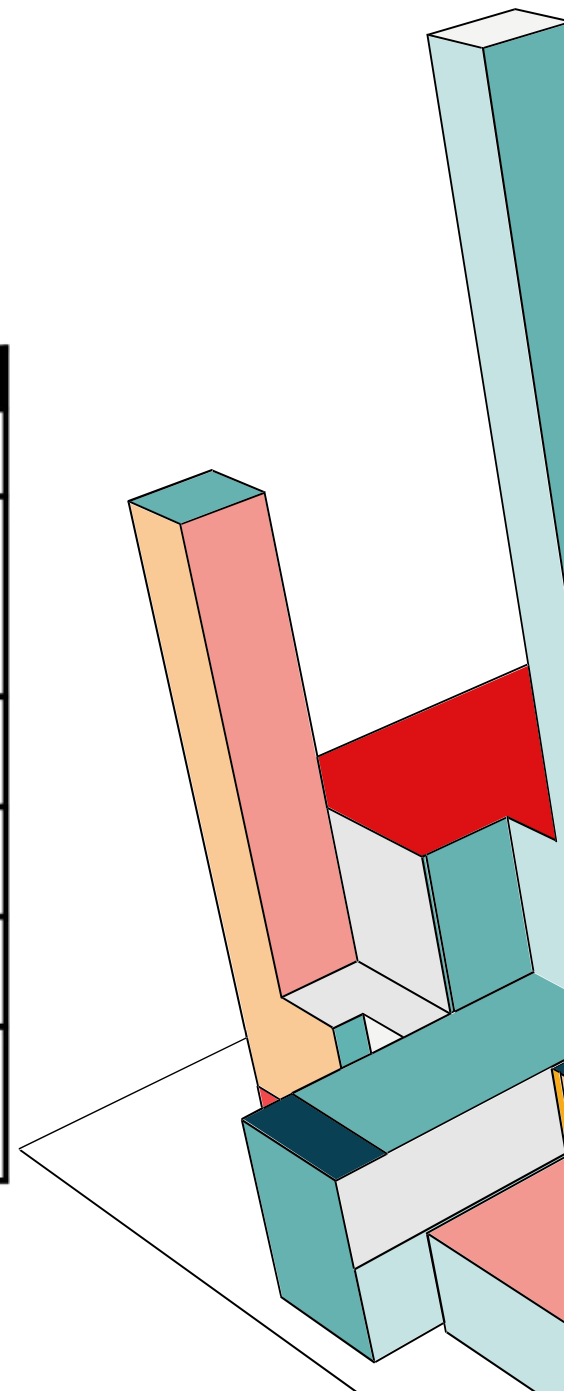


4-5) WEB APP



CLASSIFICAZIONE DEI THREATS (STRIDE)

Classe	Minaccia
Spoofing	<u>Source & Destination Spoofing</u> : falsificazione di dati e identità (IP)
Tampering	<u>Collision & Replay Attacks</u> : l'attaccante sfrutta le proprietà dei protocolli di comunicazione per sovrascrivere o riutilizzare frammenti <u>SQL injection</u> : l'attaccante modifica i dati di un database relazionale usando caratteri speciali nelle query
Repudiation	<u>Potential data Repudiation</u> : in assenza di meccanismi di logging è impossibile garantire l'accountability delle operazioni effettuate
Information Disclosure	<u>Weak Authentication Scheme & Weak Access Control</u> : assenza di meccanismi di autenticazione per il controllo degli accessi
Denial of Service	<u>Volumetric or Technical DoS</u> : l'attaccante sfrutta i meccanismi della rete per rendere inaccessibile il sistema e le sue risorse (SlowLoris, SlowDrop)
Elevation of Privilege	<u>Elevation using Remote Code Execution or Impersonation</u> : l'attaccante inietta codice malevolo, attacca la memoria del sistema o ruba credenziali di accesso con attacchi skill-based o phishing

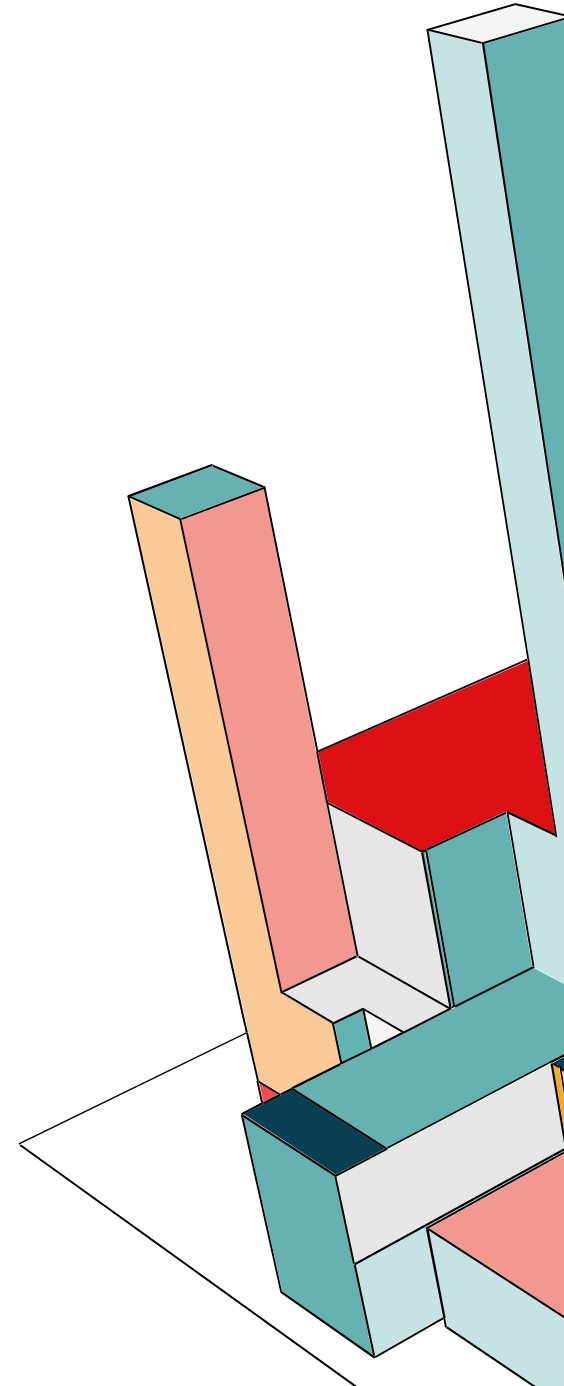


IDENTIFICARE OBIETTIVI DI SICUREZZA

Gli obiettivi di sicurezza della nostra web app sono:

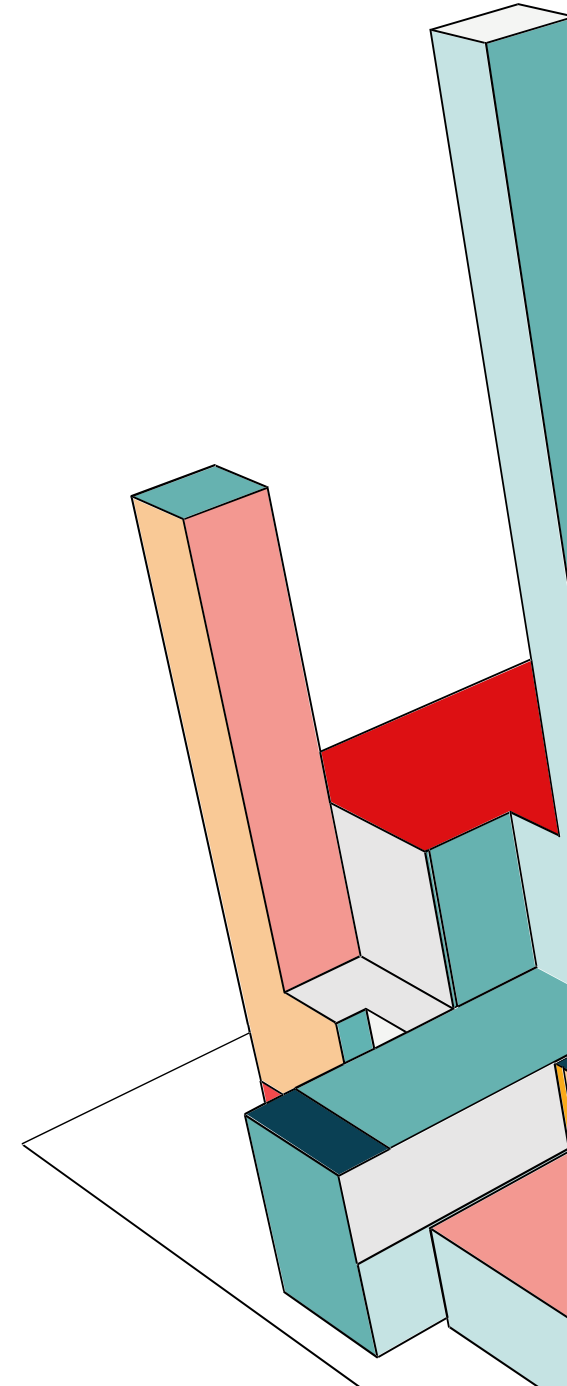
- Identità: l'applicazione deve proteggere l'identità degli utenti
- Privacy e normative: come viene trattato un utente che non rispetta le policy

Riteniamo che gli altri obiettivi non siano di nostro interesse: finanza, reputazione e garanzia e disponibilità.

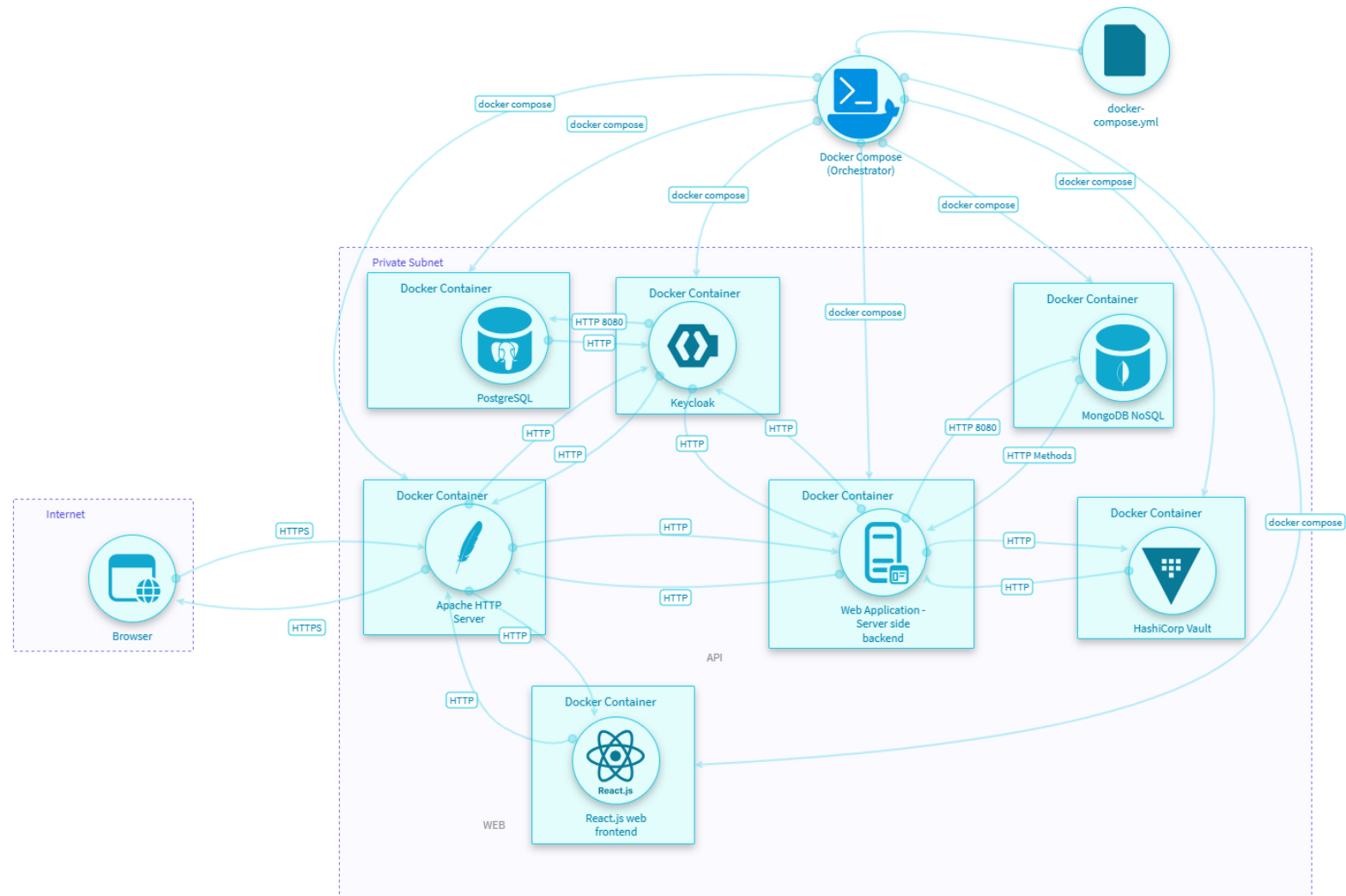


THRET MODEL: IRIUS RISK

IriusRisk è una piattaforma avanzata per la modellazione delle minacce automatizzata. Permette di disegnare l'architettura dell'applicazione attraverso componenti predefiniti. IriusRisk possiede un motore di regole che, basandosi sui componenti inclusi nel diagramma e sulle risposte ai questionari, genera automaticamente una lista di potenziali minacce e vulnerabilità. Per ogni minaccia individuata, suggerisce le contromisure tecniche specifiche da implementare per mitigare il rischio. Queste contromisure sono mappate su standard di sicurezza internazionali. Infine calcola il livello di rischio inerente e residuo, permettendo di tracciare quali minacce sono state mitigate e quali sono state accettate.

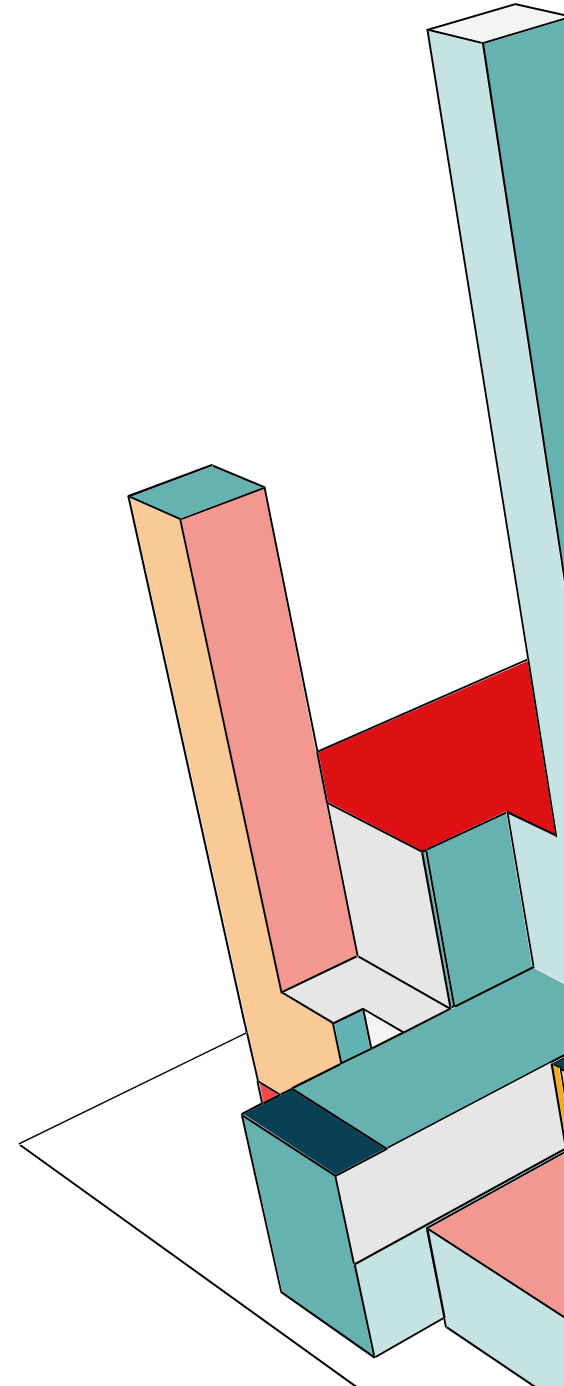


THREAT MODEL



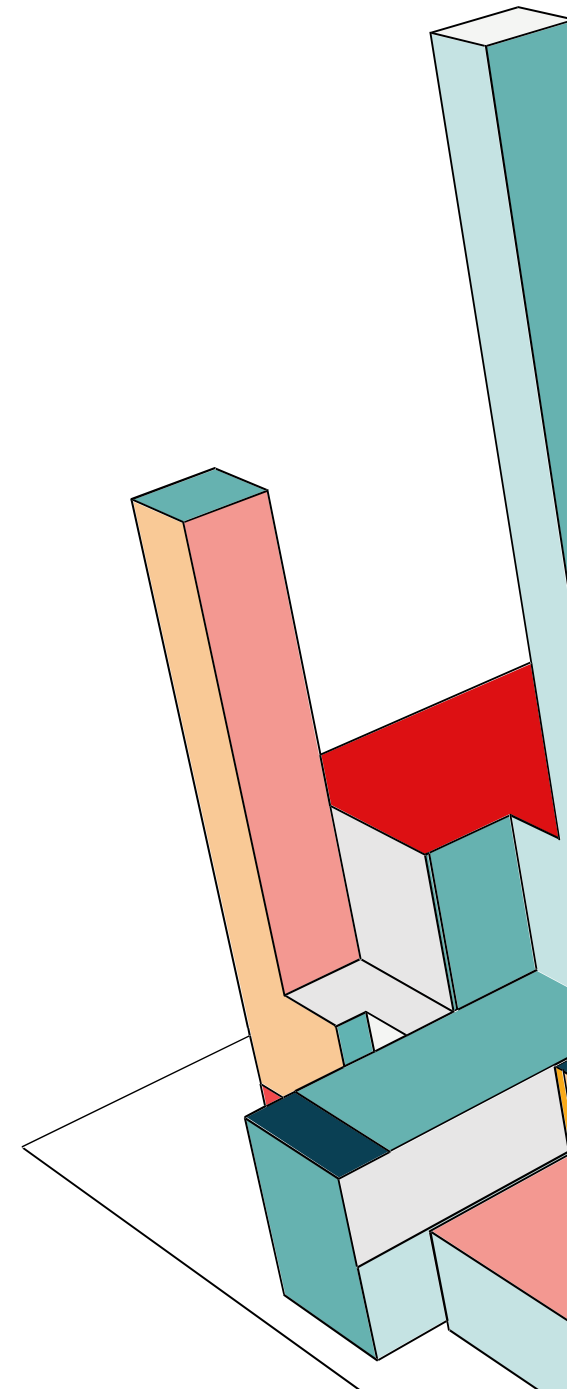
OSSERVAZIONI

L'architettura proposta adotta un approccio basato sulla Microsegmentazione e sulla riduzione della superficie d'attacco tramite containerizzazione. I componenti critici (Backend API, Keycloak, MongoDB, Vault) sono distribuiti all'interno di una Rete Docker Privata, isolata logicamente dalla rete pubblica. L'unico punto di ingresso consentito è il Reverse Proxy Apache, configurato per gestire la terminazione SSL/TLS. In questo scenario, l'adozione del protocollo HTTP per le comunicazioni tra container interni è possibile grazie al perimetro di sicurezza imposto dalla rete Docker, che agisce come una Trust Boundary. Questa configurazione implementa il controllo di sicurezza NIST SC-7 (Boundary Protection), infatti la gestione della crittografia e dell'accesso esterno sono delegate al componente di frontiera, ovvero Apache. Sebbene per questo progetto abbiamo assunto che l'isolamento della rete Docker fornisca una protezione sufficiente per il traffico HTTP interno (accettando il rischio residuo), in un ambiente di produzione Enterprise adotteremmo un approccio Zero Trust, quindi si dovrebbe garantire confidenzialità anche all'interno del perimetro, e l'applicazione di policy di Least Privilege (AC-6) sui container per prevenire movimenti laterali in caso di compromissione di un singolo nodo.



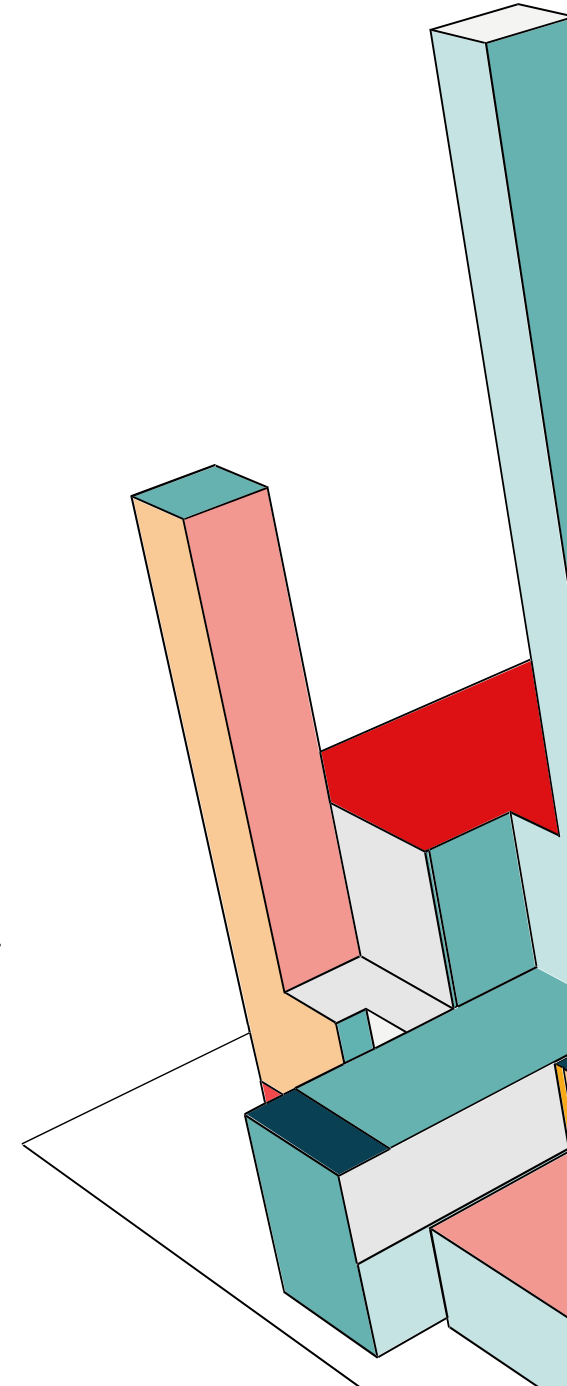
CHE COS'È KEYCLOAK?

Keycloak è una soluzione open-source di Identity and Access Management (IAM) che fornisce servizi di autenticazione e autorizzazione per applicazioni moderne. Supporta il Single Sign-On (SSO), l'autenticazione a più fattori (MFA), i social login e la federazione degli utenti attraverso molteplici sorgenti. Grazie alla sua agevole integrazione con diverse piattaforme, supporta gli sviluppatori nella messa in sicurezza delle applicazioni delegando la gestione di identità, ruoli e permessi, senza la necessità di sviluppare sistemi di autenticazione personalizzati. Keycloak è altamente flessibile e offre strumenti integrati per l'amministrazione dell'utenza e la personalizzazione dei flussi di accesso, rendendolo una scelta diffusa per le applicazioni enterprise sicure.



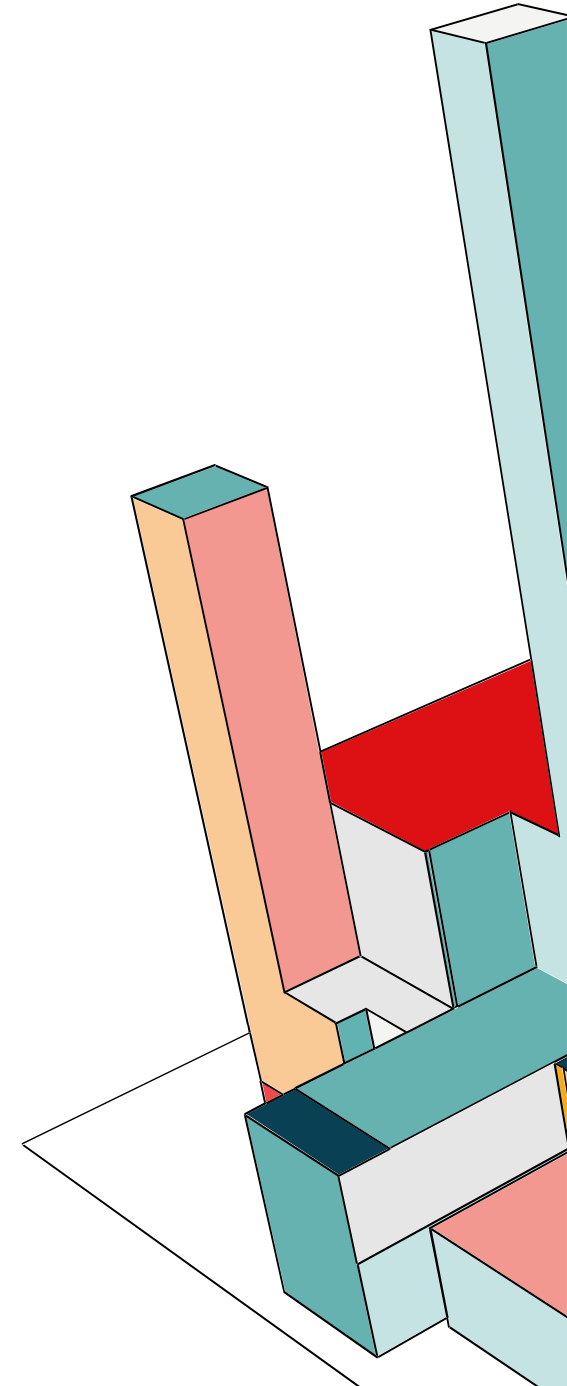
SSO & IDENTITY FEDERATION

Il SSO è un meccanismo con cui un utente può usare un singolo set di credenziali per fare il login una sola volta e accedere a diverse web application. Per diverse applicazioni che appartengono allo stesso dominio, questo è possibile condividendo le stesse informazioni di sessione utente all'interno dei cookies. Cioè le richieste di accesso vengono rediretta a un authentication server centrale (identity provider) che autentica l'utente e genera un token. Quest'ultimo viene passato dal browser all'interno di un cookie al service provider, che lo ispeziona e lo valida. Un altro concetto è l'identity federation, cioè ci deve essere un sistema di fiducia quanto più alta possibile tra service provider e identity provider allo scopo di autenticare utenti e convenire sulle informazioni necessarie ad autorizzare il loro accesso alle risorse. Quindi si estende il meccanismo del SSO abilitando la portabilità tra domini differenti. Si tratta di delegare autorizzazione e autenticazione a terzi affidabili; dunque, un Service Provider (proprietario delle risorse) delega un Identity Provider per autenticazione e access control.



OAUTH 2.0

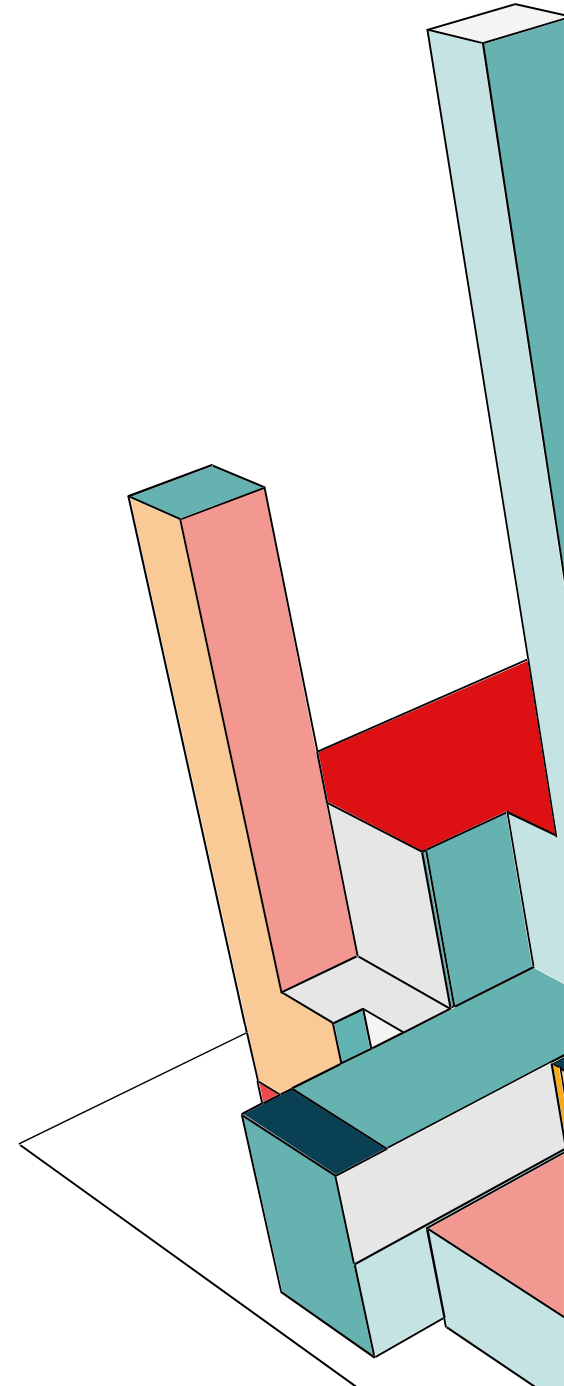
OAuth 2.0 (2012) rappresenta lo standard industriale per il protocollo di autorizzazione. Esso consente ad applicazioni di terze parti di ottenere un accesso limitato a risorse HTTP, agendo per conto di un proprietario della risorsa o consentendo all'applicazione stessa di accedere a risorse proprie.



OAuth 2.0

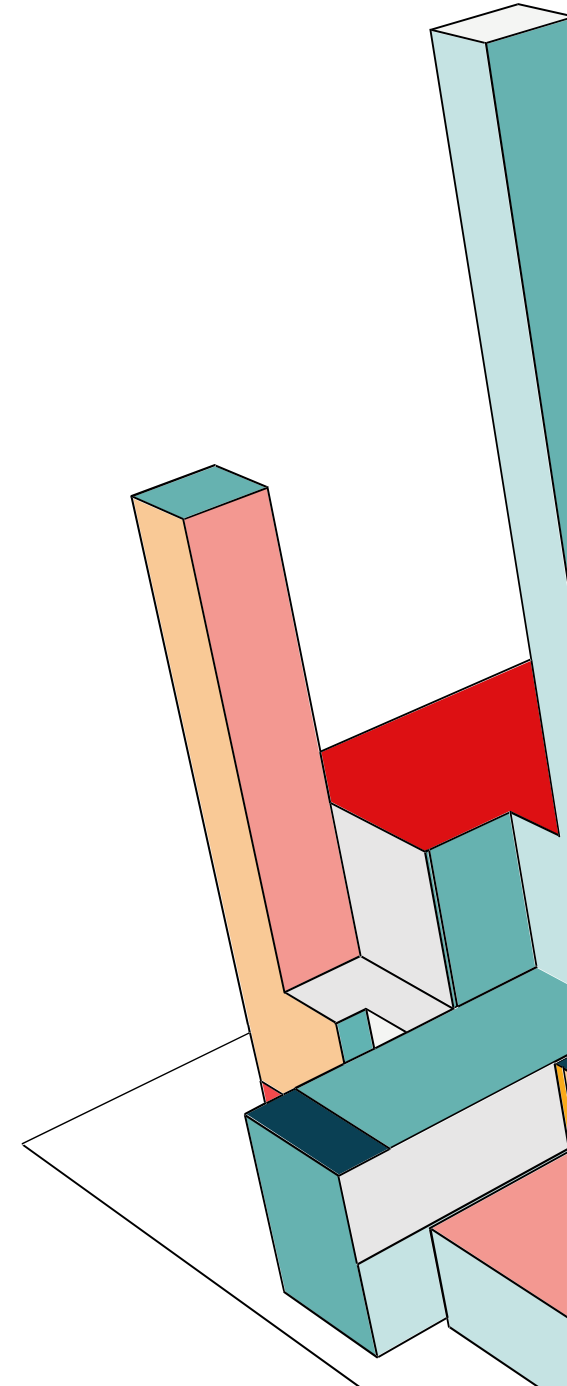
Nel modello classico client-server, il client richiede l'accesso a una risorsa protetta autenticandosi direttamente con le credenziali (username e password) del proprietario della risorsa. Questo approccio presenta vulnerabilità strutturali:

- **Gestione delle credenziali:** Le applicazioni terze sono costrette a memorizzare le password degli utenti, aumentandone l'esposizione al rischio.
- **Eccesso di privilegi:** Il client ottiene un accesso indiscriminato a tutte le risorse dell'utente, senza possibilità di limitare la durata dell'accesso o il perimetro dei dati.
- **Difficoltà di revoca:** L'utente non può revocare l'accesso a un singolo servizio senza cambiare la password principale, invalidando così l'accesso a tutti i servizi collegati.
- **Vulnerabilità a cascata:** La compromissione di un'applicazione terza espone direttamente la password dell'utente e tutti i dati correlati.



OAUTH 2.0

OAuth risolve queste criticità separando il ruolo del Client da quello del Proprietario della Risorsa (Resource Owner). In questo framework, il client non riceve le credenziali dell'utente, ma un Access Token. Questo token viene emesso da un Authorization Server (previa approvazione dell'utente) e garantisce l'accesso esclusivamente a uno specifico set di risorse per un tempo limitato.



OAUTH 2.0: CLIENTS

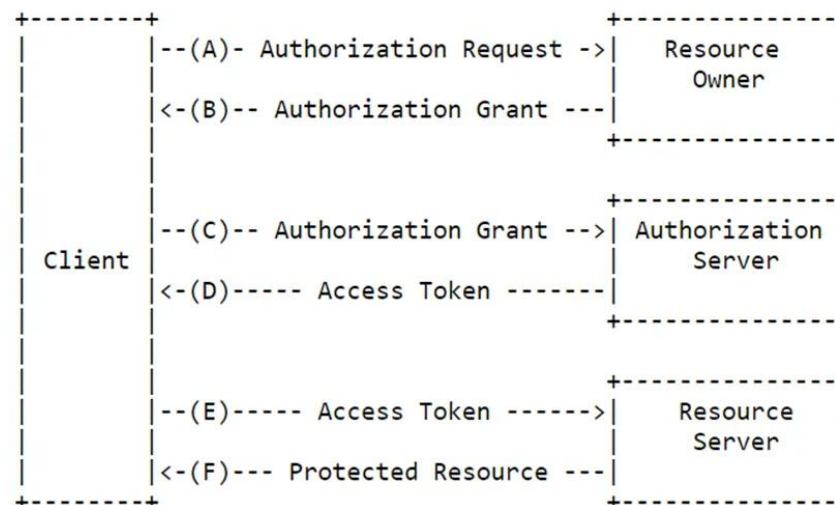
Profilo Client	Descrizione	Sicurezza
Web App	Eseguita su un server web; gestisce credenziali e token internamente.	Confidential
User-Agent App	Codice (es. JavaScript) scaricato ed eseguito nel browser dell'utente.	Public
Native App	Applicazione installata sul dispositivo dell'utente.	Public

Prima di iniziare il flusso del protocollo, ogni client deve essere registrato presso l'Authorization Server, specificando la tipologia di client (confidential o public) e le URI di reindirizzamento (Redirect URIs).



OAUTH 2.0: PROTOCOL FLOW

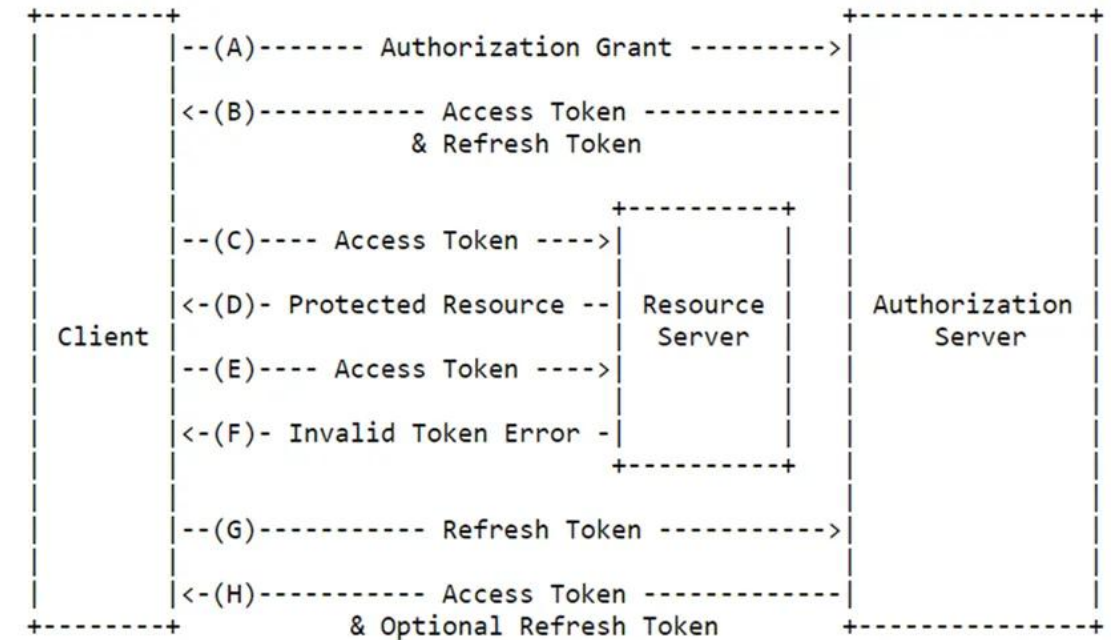
Chi è proprietario di un token sostanzialmente è ritenuto autorizzato. Per ottenere un Access Token, il client fa una richiesta di autorizzazione al resource owner, possibilmente attraverso l'authorization server, per ottenere un Authorization Grant. Quello comunemente utilizzato nelle web app e nel nostro progetto è chiamato Authorization Code Grant: il client dirige il resource owner verso un authorization server, che autentica l'owner e ottiene la sua autorizzazione ad accedere alle risorse protette.



OAUTH 2.0: TOKENS

Il framework si basa su due tipologie principali di credenziali per la gestione dell'autorizzazione:

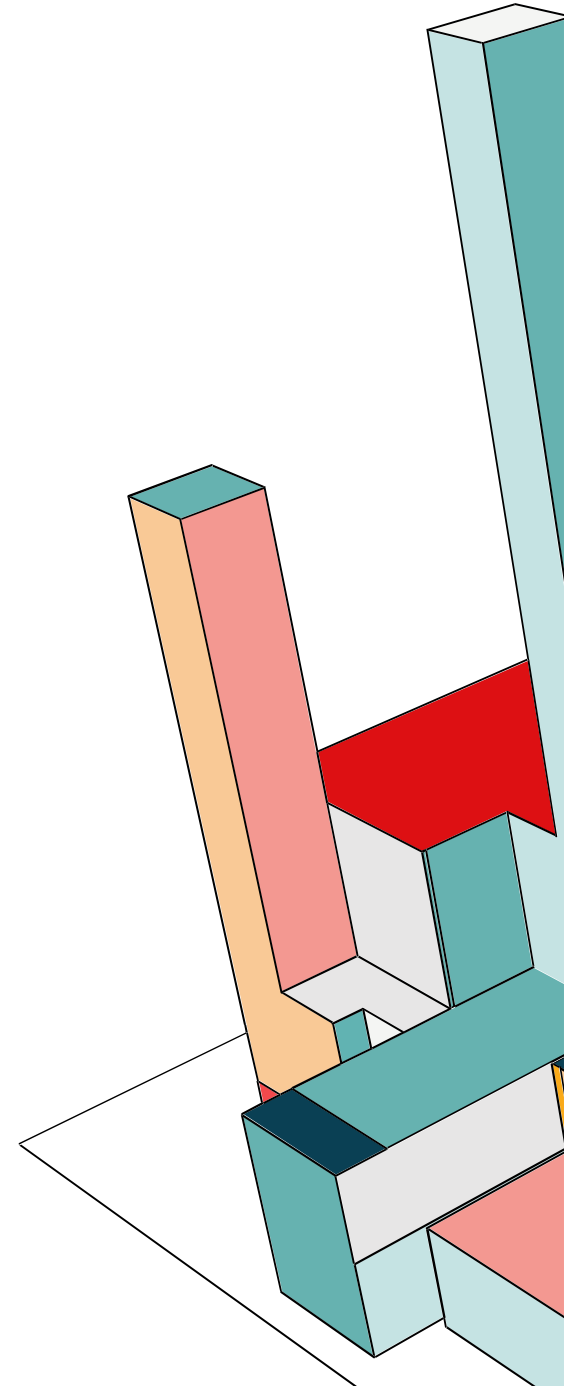
- **Access Token:** Rappresentano stringhe di autorizzazione emesse al client. Esse definiscono in modo granulare l'ambito dei privilegi (scope) e la durata temporale dell'accesso a risorse protette, secondo i parametri stabiliti dal service provider.
- **Refresh Token:** Credenziali opzionali emesse dall'Authorization Server insieme all'access token. La loro funzione esclusiva è l'ottenimento di nuovi access token qualora quelli correnti scadano o divengano invalidi, o per richiedere token con scope identico o più limitato. A differenza degli access token, i refresh token sono destinati esclusivamente all'Authorization Server e non devono mai essere trasmessi ai Resource Server.



OAuth 2.0: ENDPOINTS

Il processo di autorizzazione è strutturato su tre endpoint fondamentali:

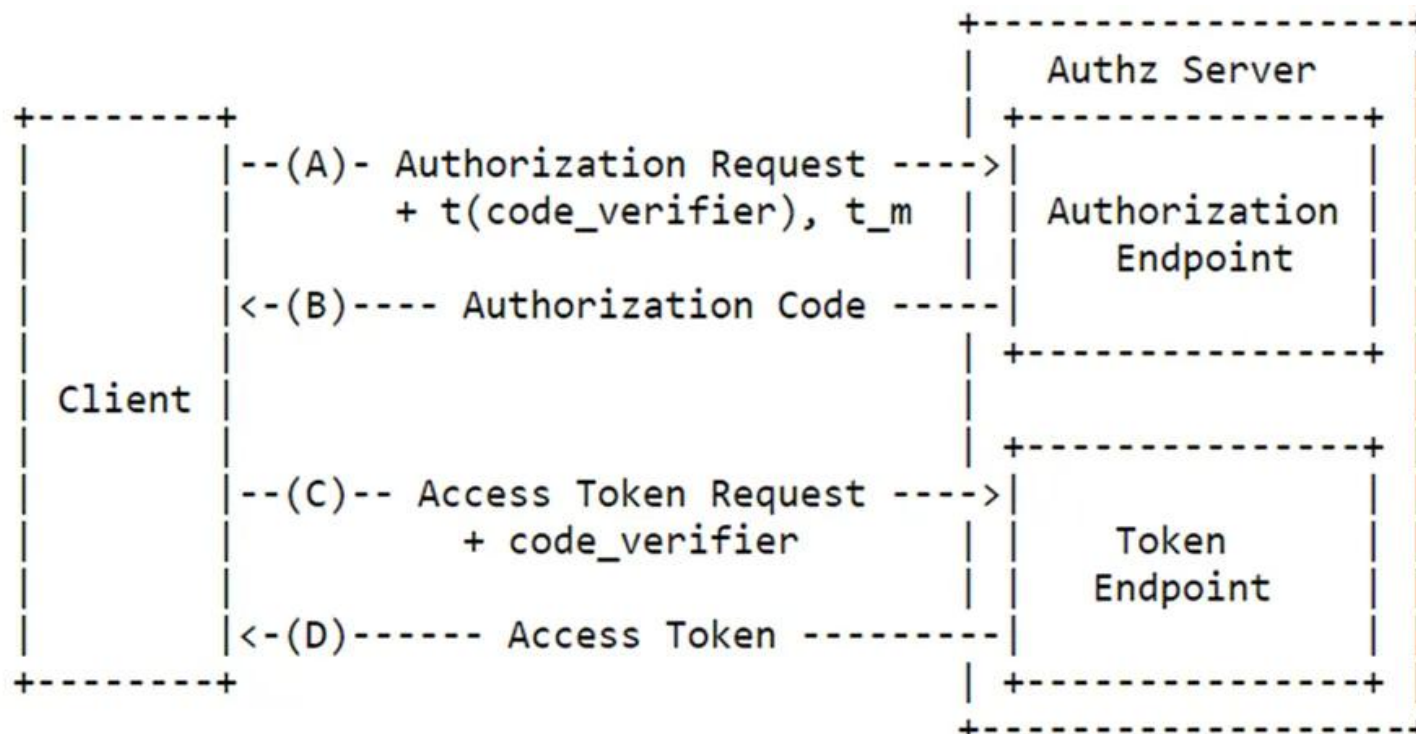
- **Authorization Endpoint:** Risorsa HTTP utilizzata dal client per ottenere l'autorizzazione dal proprietario della risorsa tramite il reindirizzamento dello user-agent. È impiegato nei flussi *Authorization Code* e *Implicit* (distinti dal parametro `response_type`). La specifica OAuth non definisce il metodo di autenticazione dell'utente su questo endpoint.
- **Token Endpoint:** Utilizzato dal client per scambiare una concessione di autorizzazione (grant) o un refresh token con un access token. Questo endpoint è centrale in tutti i flussi, eccetto quello *Implicit*. I client di tipo "confidential" devono autenticarsi obbligatoriamente presso questo endpoint.
- **Redirection Endpoint:** Endpoint lato client dove l'Authorization Server restituisce le risposte contenenti le credenziali di autorizzazione tramite lo user-agent del proprietario.



OAUTH 2.0: PKCE

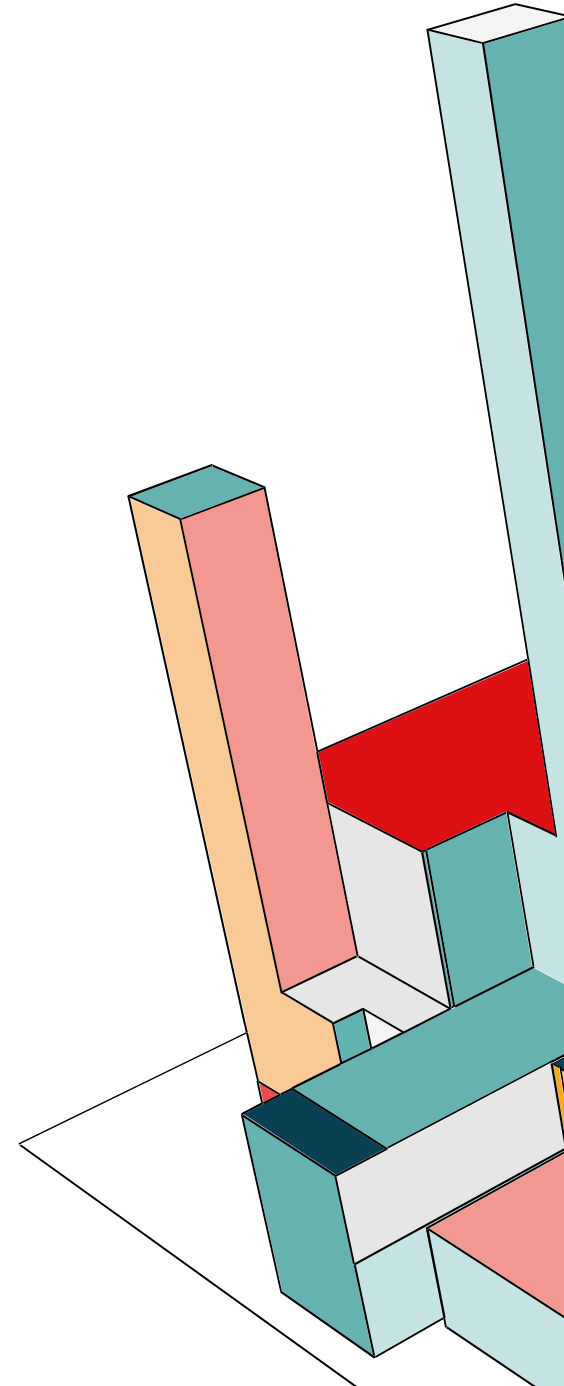
I client pubblici sono vulnerabili all'**Authorization Code Interception Attack**, dove un utente malintenzionato intercetta il codice su canali non protetti da TLS. Per mitigare questo rischio, si adotta il meccanismo **PKCE** (Proof Key for Code Exchange):

- **Generazione:** Il client genera un segreto casuale denominato **Code Verifier** e ne deriva una **Code Challenge** applicando l'algoritmo di hashing **SHA256**.
- **Validazione:** La challenge viene inviata durante la richiesta iniziale; successivamente, al momento della richiesta del token, il client presenta il **Code Verifier** originale. L'Authorization Server valida la corrispondenza per garantire che il client che richiede il token sia il medesimo che ha avviato la sessione.

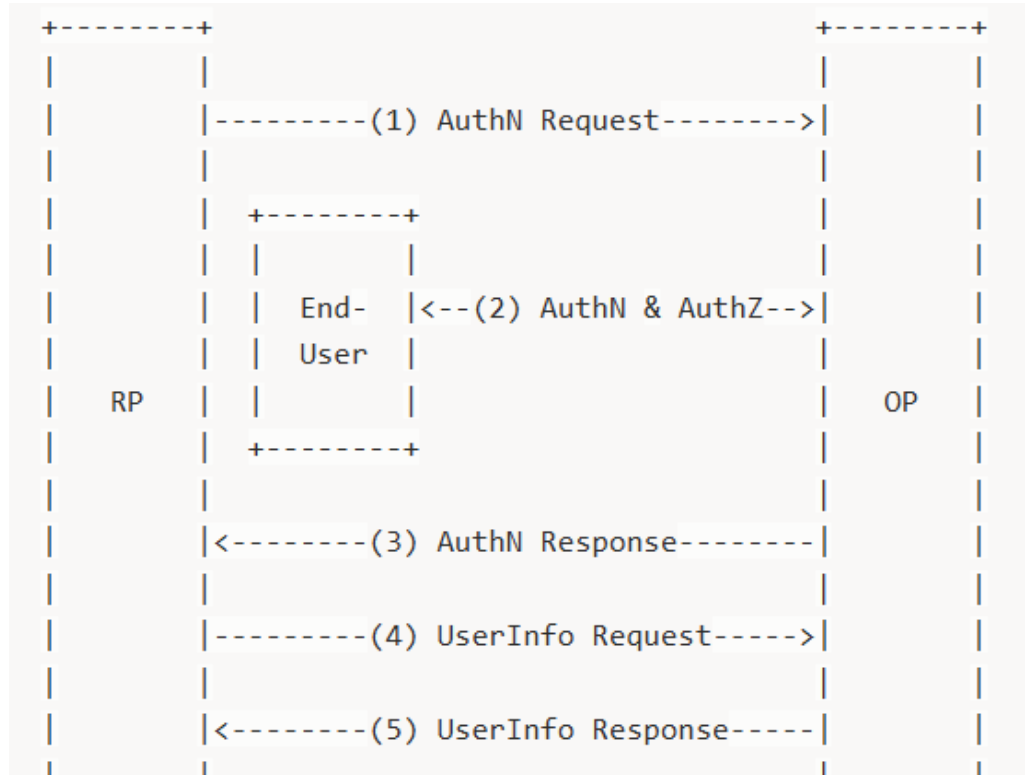


OPENID

OpenID Connect (OIDC) rappresenta lo standard aperto di terza generazione per l'**autenticazione**. A differenza dei suoi predecessori, OIDC è stato progettato per superare i limiti di interoperabilità e complessità del formato XML tipici di OpenID 2.0. OIDC è definito come un semplice strato di identità (identity layer) costruito sopra il protocollo OAuth 2.0. Consente ai **Client** (definiti **Relying Parties**) di verificare l'identità dell'utente finale basandosi sull'autenticazione eseguita da un **Authorization Server** (definito **OpenID Provider**). Utilizza il formato **JSON Web Token (JWT)**, rendendo l'implementazione più compatta e compatibile con applicazioni native e sistemi mobile rispetto ai vecchi schemi basati su XML.



OPENID



1) Authentication Request: La Relying Party (RP), agendo come client, trasmette una richiesta strutturata all'OpenID Provider (OP) per avviare il processo di identificazione dell'utente finale.

2) AuthN & AuthZ: L'OP gestisce direttamente l'autenticazione dell'End-User (tramite la verifica delle credenziali) e acquisisce l'autorizzazione specifica per la delega degli attributi richiesti dalla RP.

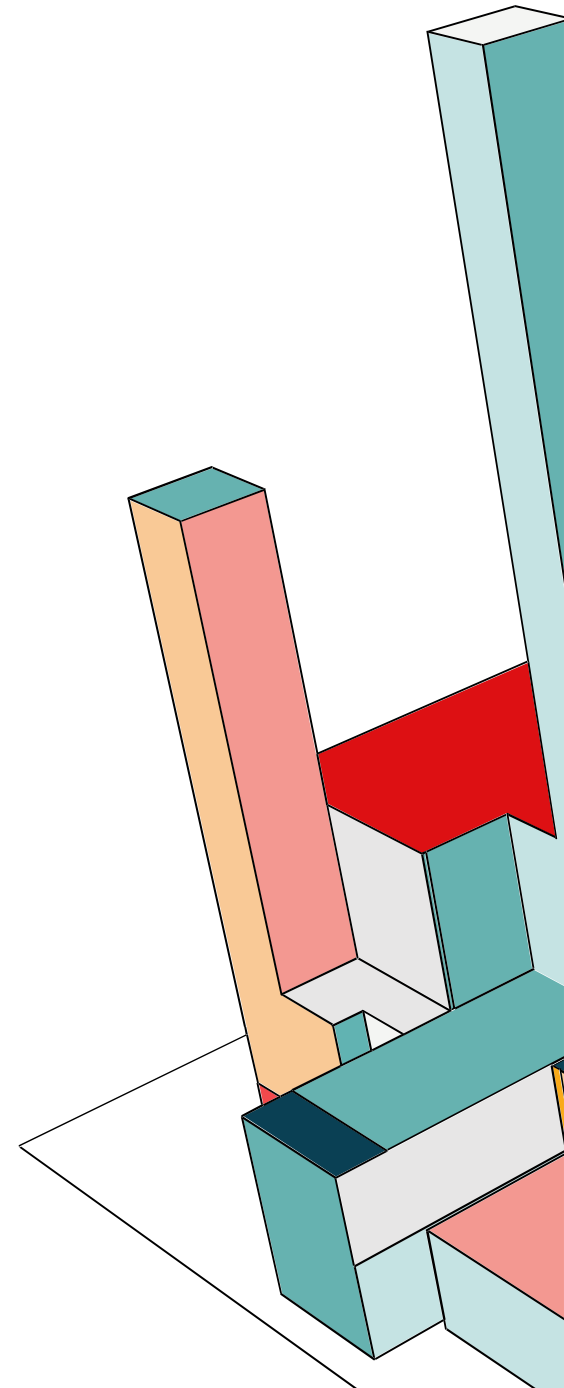
3) Authentication Response: A seguito del successo della fase precedente, l'OP rilascia alla RP un **ID Token** (asserzione di identità in formato JWT) e, nella maggior parte dei casi, un **Access Token** (necessario per le chiamate API successive).

4) UserInfo Request: La RP utilizza l'Access Token ricevuto per interrogare l'**UserInfo Endpoint**, una risorsa protetta che risiede presso l'OP.

5) UserInfo Response: L'UserInfo Endpoint convalida il token e restituisce i **Claims** (attributi del profilo come email, nome, ecc.) relativi all'utente finale in un formato interoperabile.

OPENID: ID TOKEN

L'estensione principale apportata da OIDC a OAuth 2.0 è l'introduzione dell'**ID Token**, un token di sicurezza che veicola asserzioni (claims) riguardanti l'evento di autenticazione. Esso è codificato in **BASE64URL** e si articola in tre sezioni fondamentali: **Header** (algoritmi utilizzati), **Claims** (attributi dell'utente e metadati), e **Digital Signature** (per la verifica dell'integrità). Il token specifica l'audience (client destinatario), le date di emissione e scadenza, e parametri relativi alla sessione dell'utente. Il token è obbligatoriamente firmato digitalmente per permettere al destinatario di convalidarne l'origine e l'integrità; opzionalmente, può essere crittografato per garantirne la riservatezza.



OPENID: ID TOKEN

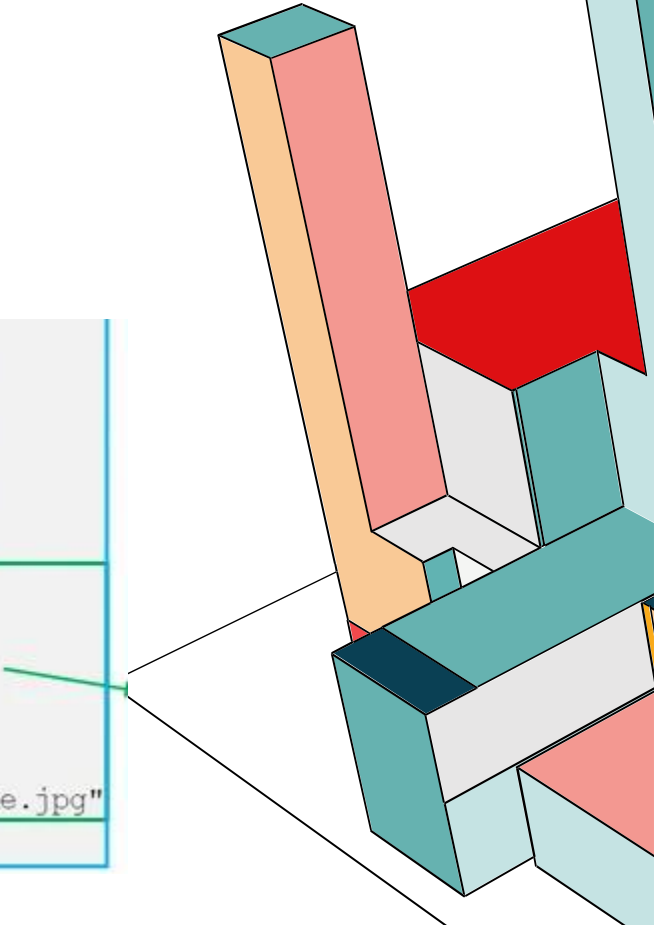
eyJhbGciOiJSUzI1NiIsImtpZCI6IjFlOWdkazcifQ.ewogImIzcyI6ICJodHRwOi8vc2VydmVyLmV4YW1wbGUuY29tIiwKICJzdWIiOiAiMjQ4Mjg5NzYxMDAxIiwKICJhdWQiOiAiczZCaGRSa3F0MyIsCiAibm9uY2UiOiAibi0wUzZfV3pBMk1qIiwKICJleHAiOiAxMzExMjg5OTcwLAogImIhdCI6IDEzMTEyODA5NzAKfQ.ggW8hZ1EuVLuxNuuIJKX_V8a_OMXzR0EHR9R6jgdqrOOF4daGU96Sr_P6qJp6IcmD3HP990bilPRs-cwh3LO-p146waJ8IhehcwL7F09JdijmBqkvPeB2T9CJNqeGpegccMg4vfKjkM8FcGvnzZUN4_KSP0aAp1tOJ1zZwgjxqGByKHioT7TpdQyHE5lcMiKPFfEIQILVq0pc_E2DzL7emopWoaoZTF_m0_N0YzFC6g6EJbOEoRoSK5hoDalrcvRYLSrQAZZKflyuVCyixEoV9GfNQC3_osjzw2PAithfubEEBLuVVk4XUVrWOLrLl0nx7RkKU8NXNHq-rvKMzqg

→ [Header] . [Claims] . [Digital Signature]

{
"iss":
"sub":
"aud":
"nonce"

→ [Header] . [Claims] . [Digital Signature]

```
{
  "iss": "http://server.example.com",
  "sub": "248289761001",
  "aud": "s6BhdRkqt3",
  "nonce": "n-0S6_WzA2Mj",
  "exp": 1311281970,
  "iat": 1311280970,
  "name": "Jane Doe",
  "given_name": "Jane",
  "family_name": "Doe",
  "gender": "female",
  "birthdate": "0000-10-31",
  "email": "janedoe@example.com",
  "picture": "http://example.com/janedoe/me.jpg"
}
```



OICD FLOWS

OIDC modella l'interazione tra le parti attraverso tre percorsi distinti, selezionati in base alla natura del client:

Authorization Code Flow:

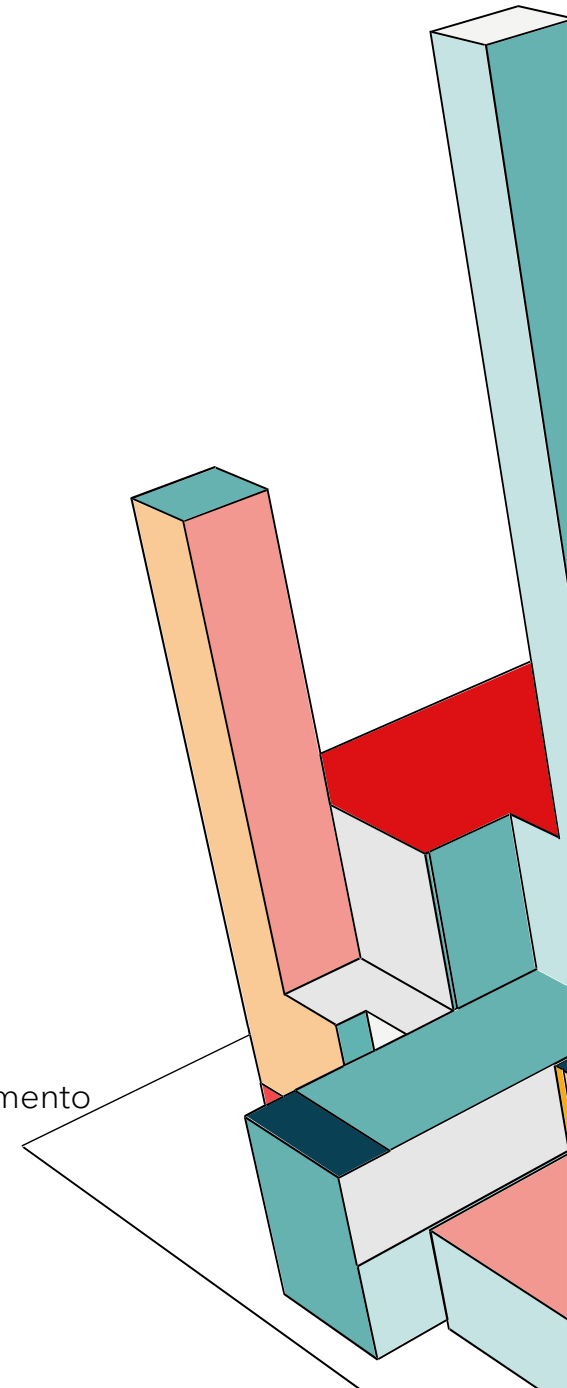
- L'OpenID Provider autentica l'utente e restituisce un codice di autorizzazione al client.
- Il client scambia il codice direttamente con l'ID Token e l'Access Token presso il **Token Endpoint**.
- È il flusso raccomandato per applicazioni web e mobile con componenti server-side (confidential clients).

Implicit Flow:

- L'ID Token e l'Access Token vengono restituiti direttamente allo user-agent dopo il consenso dell'utente.
- Utilizzato principalmente da applicazioni browser-based (public clients).

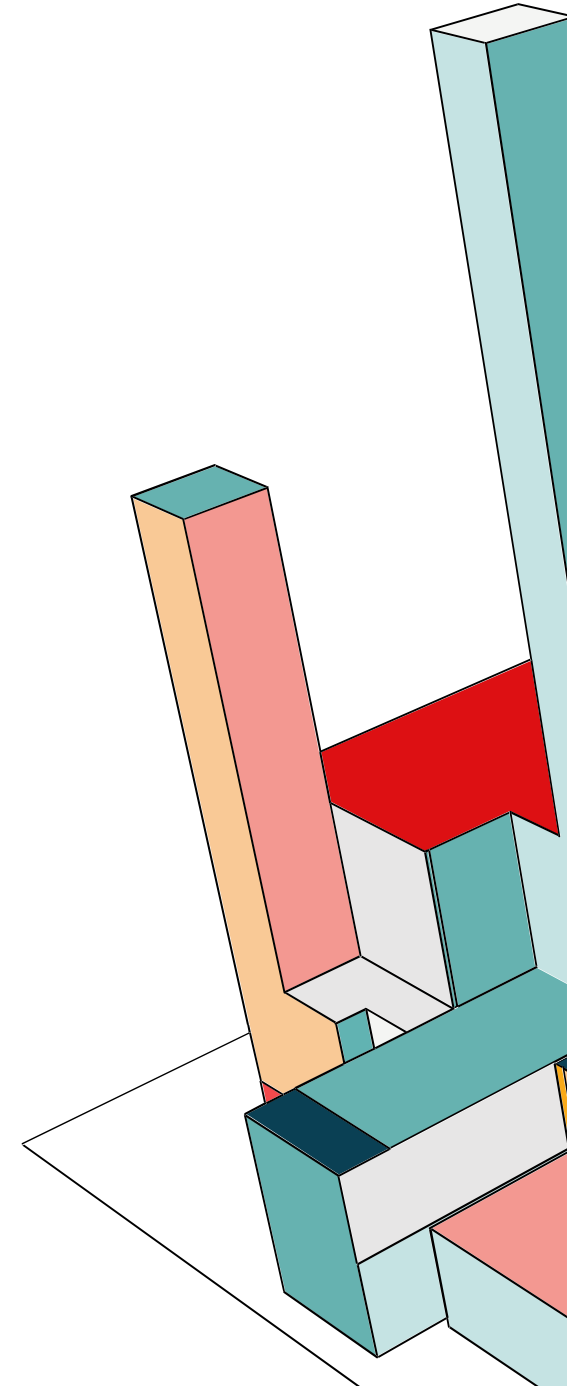
Hybrid Flow:

- Combina elementi di entrambi i flussi, permettendo di ricevere alcuni token tramite il canale di reindirizzamento e altri tramite il Token Endpoint.



OIDC USERINFO ENDPOINT

OIDC definisce una risorsa protetta specifica per l'acquisizione di claims dettagliati sull'utente autenticato. Abbiamo sia i claim standard come nome, cognome, genere, data di nascita, indirizzo, numero di telefono e stato di verifica delle credenziali di contatto (email e telefono). Ma c'è anche la possibilità di configurare la lista dei claims restituiti, nel caso servissero claims più specifici.

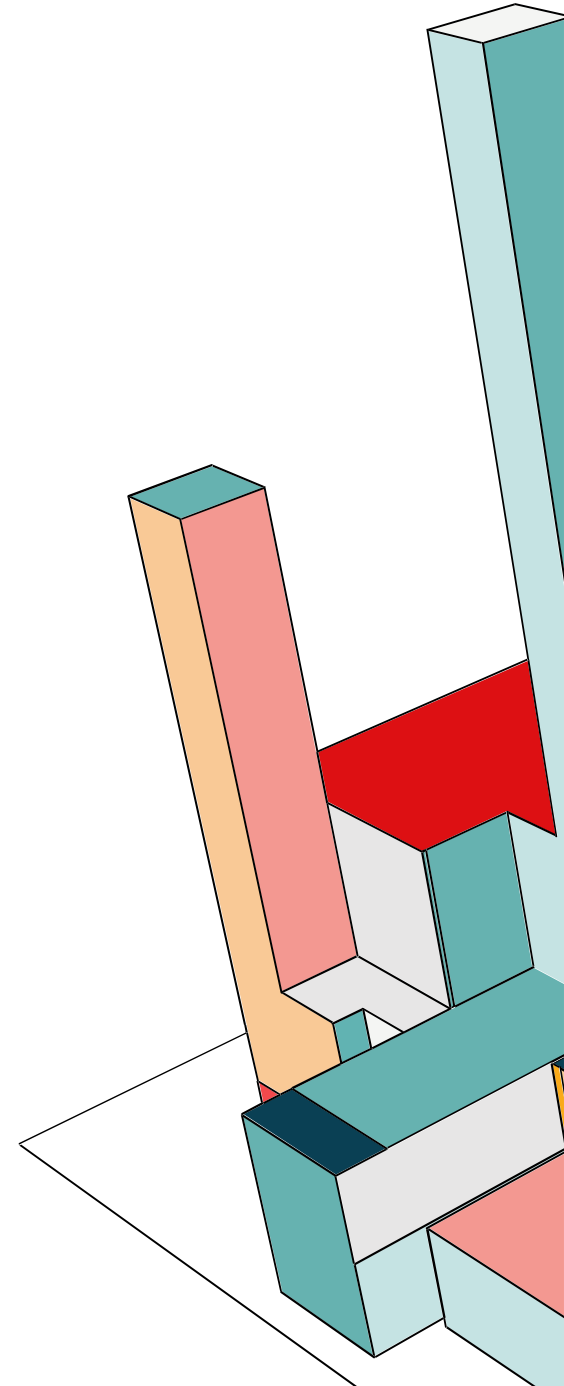


KEYCLOAK: CREAZIONE ADMIN

Abbiamo creato un utente admin definendo delle **variabili d'ambiente**:

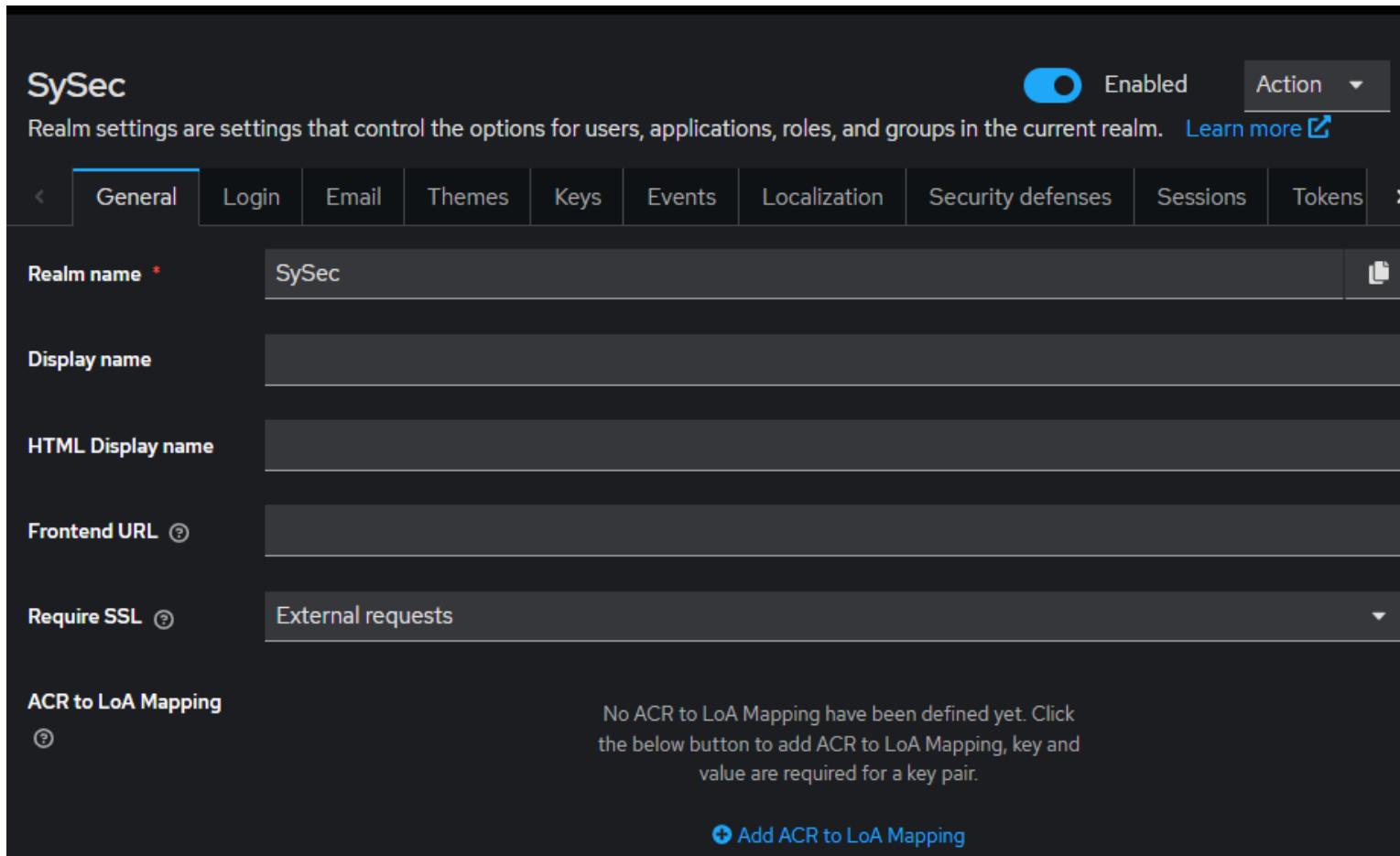
```
96 environment:  
97   KC_BOOTSTRAP_ADMIN_USERNAME: admin  
98   KC_BOOTSTRAP_ADMIN_PASSWORD: admin
```

Keycloak esegue il **parsing** di questi valori al primo avvio per generare un utente iniziale dotato di privilegi amministrativi.



KEYCLOAK: CREAZIONE REALM SYSEC

Una volta che il primo utente con diritti di amministrazione è presente, è possibile utilizzare la **Admin Console** per la creazione di ulteriori utenti.



SySec Enabled Action ▾

Realm settings are settings that control the options for users, applications, roles, and groups in the current realm. [Learn more](#)

< **General** Login Email Themes Keys Events Localization Security defenses Sessions Tokens >

Realm name * SySec 📄

Display name

HTML Display name

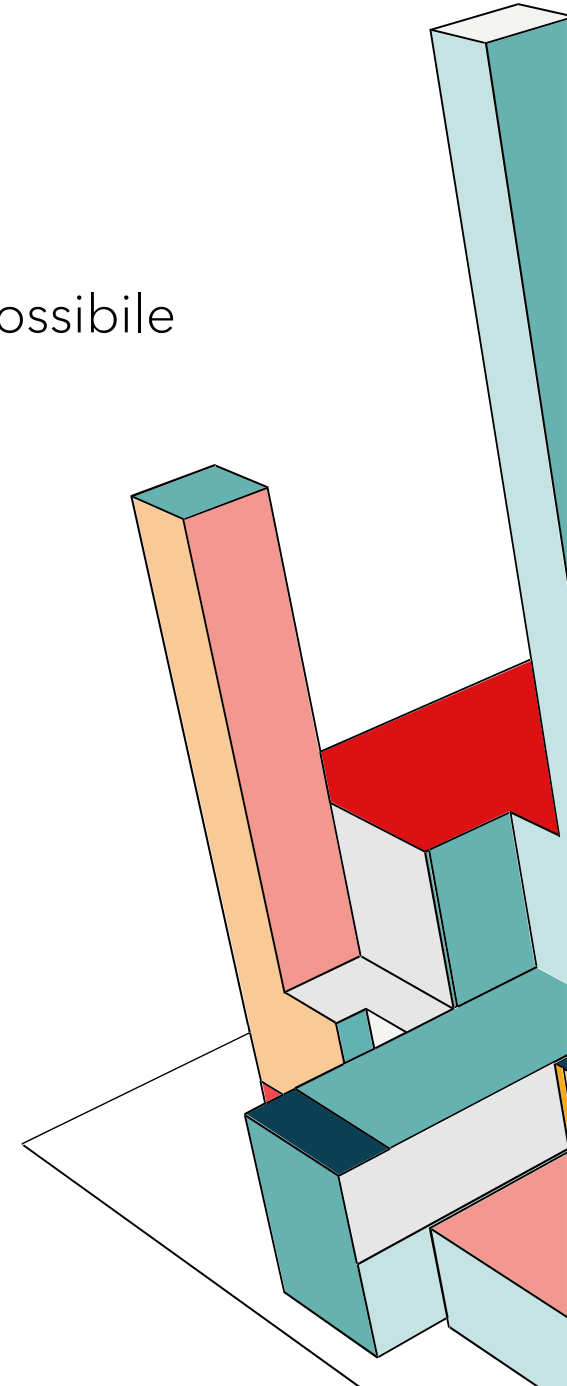
Frontend URL ?

Require SSL ? External requests ▾

ACR to LoA Mapping ?

No ACR to LoA Mapping have been defined yet. Click the below button to add ACR to LoA Mapping, key and value are required for a key pair.

[Add ACR to LoA Mapping](#)



KEYCLOAK: CREAZIONE CLIENT FRONTEND

Creiamo un utente base che userà come protocollo OpenID Connect.

Clients

Clients are applications and services that can request authentication of a user. [Learn more](#)

Clients list

Initial access token

Client registration

🔍 Search for client

→

Create client

Import client

🔄 Refresh

1-7

<

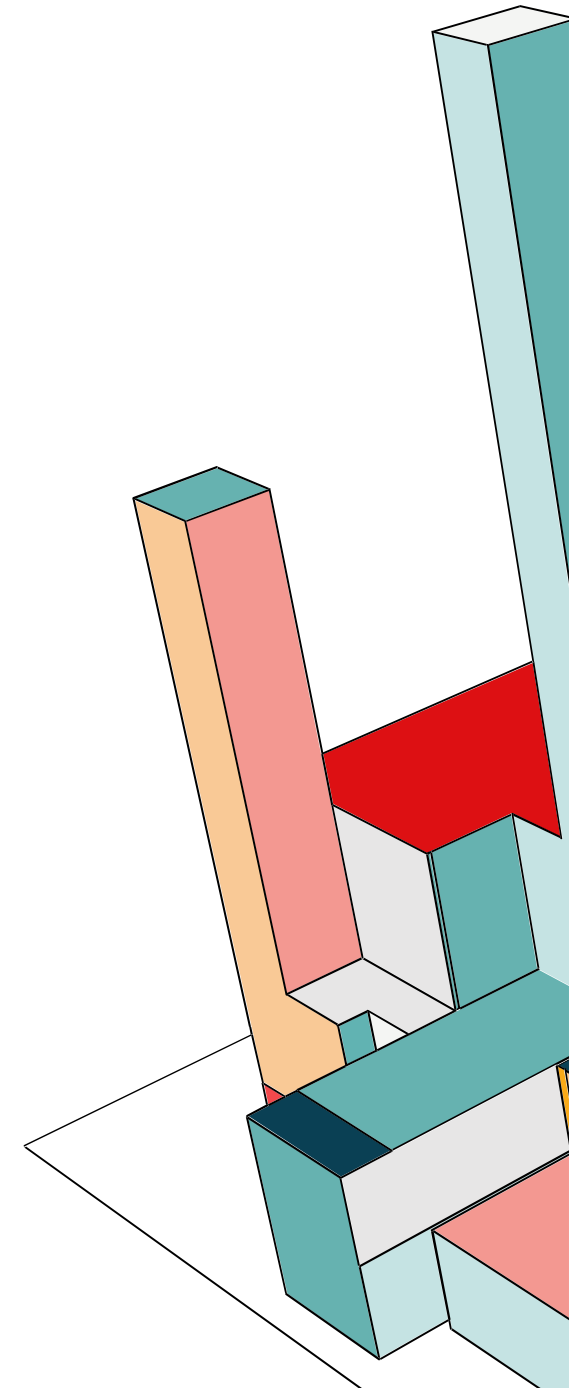
>

Client ID	Name	Type	Description	Home URL
account	Account	OpenID Connect	–	https://localhost/auth/realms/WebAppToDo/account/
account-cons...	Account Conso...	OpenID Connect	–	https://localhost/auth/realms/WebAppToDo/account/
admin-cli	Admin CLI	OpenID Connect	–	–
broker	Broker	OpenID Connect	–	–
react-app	app to do	OpenID Connect	web app to do list	–
realm-manag...	Realm Manage...	OpenID Connect	–	–
security-admi...	Security Admin...	OpenID Connect	–	https://localhost/auth/admin/WebAppToDo/console/

1-7

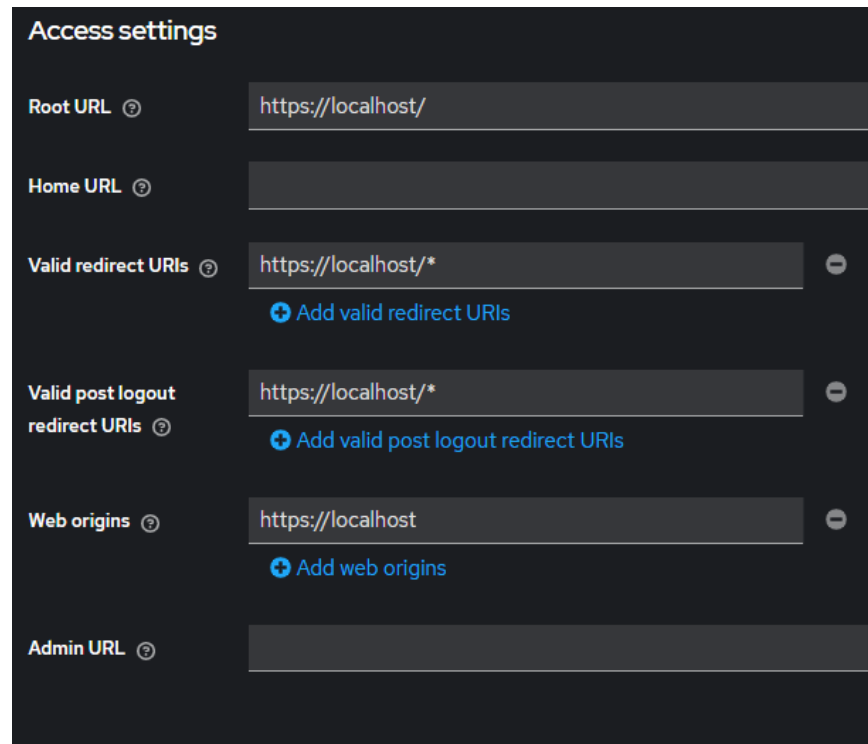
<

>



KEYCLOAK: CREAZIONE CLIENT FRONTEND

Nelle impostazioni del client, è necessario impostare la **Valid Redirect URL**, questo parametro specifica gli endpoint verso cui Keycloak è autorizzato a reindirizzare l'utente dopo una corretta autenticazione. In **Web Origins** si definiscono le origini (dominio, protocollo e porta) autorizzate a effettuare richieste HTTP cross-site verso il server Keycloak.



Access settings

Root URL ⓘ

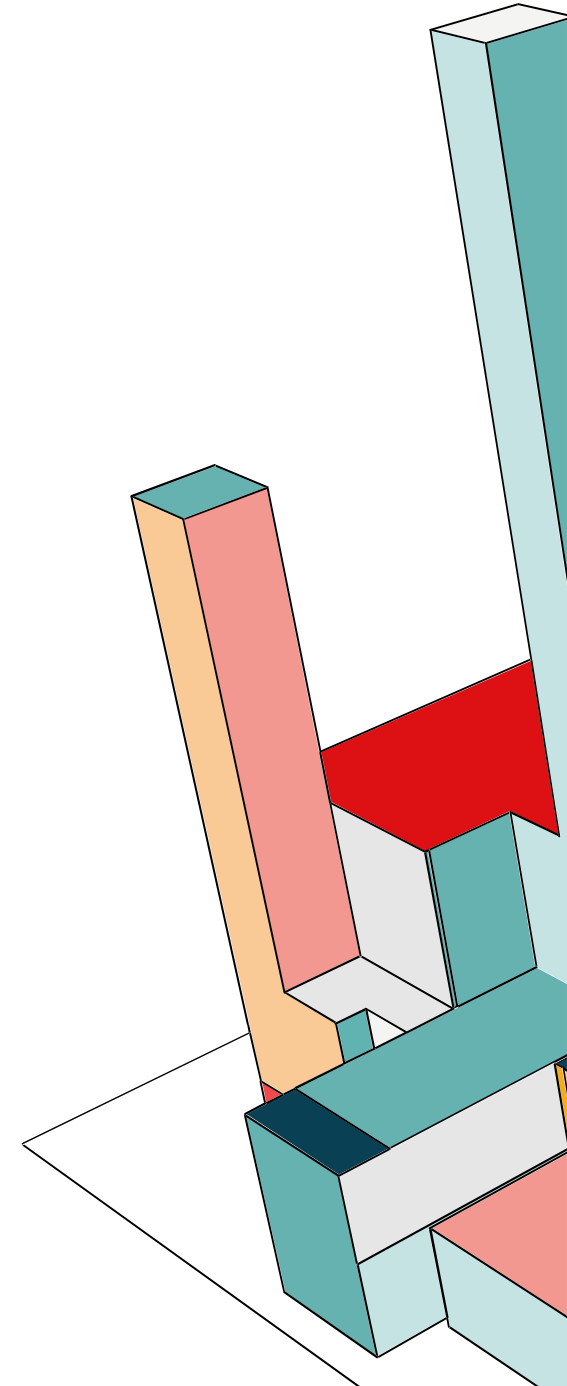
Home URL ⓘ

Valid redirect URIs ⓘ ⓘ
[+ Add valid redirect URIs](#)

Valid post logout redirect URIs ⓘ ⓘ
[+ Add valid post logout redirect URIs](#)

Web origins ⓘ ⓘ
[+ Add web origins](#)

Admin URL ⓘ



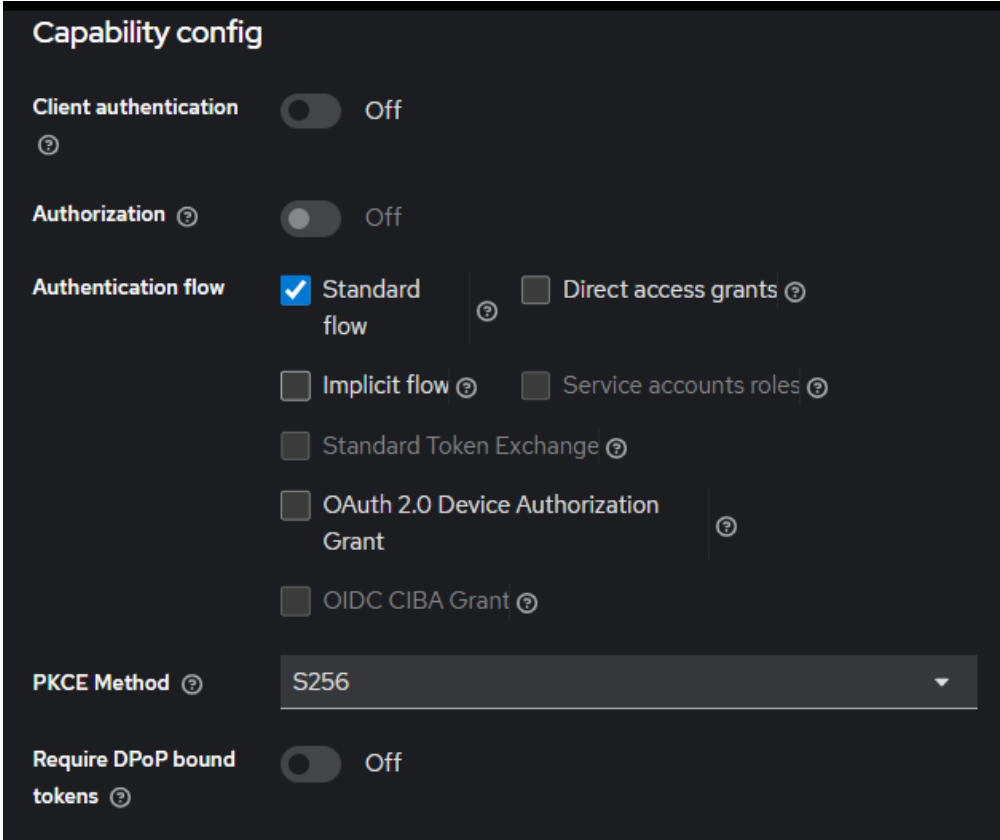
KEYCLOAK: CREAZIONE CLIENT FRONTEND

Il parametro **Client Authentication** definisce se il client è in grado di mantenere la riservatezza delle proprie credenziali.

- **ON (Confidential Client):** Utilizzato per applicazioni eseguite su server dove è possibile memorizzare in modo sicuro un Client Secret. Questo segreto è obbligatorio durante la fase di scambio dell'Authorization Code con l'Access Token.
- **OFF (Public Client):** Utilizzato per applicazioni eseguite sul lato client (SPA, Mobile) che, per loro natura, non possono proteggere una chiave segreta. La sicurezza in questo caso si basa esclusivamente sulla validazione dei Redirect URI e sull'uso di PKCE.

Authorization abilita il supporto per le **Keycloak Authorization Services**. Se attivo, permette di definire policy di autorizzazione fine-grained, gestendo permessi complessi basati su risorse, scopes e sui ruoli (RBAC).

Poi abbiamo l'**Authentication flow** che qui è **standard**, cioè basato sul reindirizzamento: l'utente viene rediretto al server di autorizzazione, si autentica e il client riceve un Authorization Code che scambierà successivamente per i token. E abbiamo abilitato anche il **Direct access grant** che permette al client di ottenere un token inviando direttamente le credenziali dell'utente (username e password) a Keycloak.



The screenshot shows the 'Capability config' section of the Keycloak administration console. It contains several settings for a client:

- Client authentication**: A toggle switch set to 'Off'.
- Authorization**: A toggle switch set to 'Off'.
- Authentication flow**: A section with multiple options. 'Standard flow' is selected with a blue checkmark. Other options include 'Direct access grants', 'Implicit flow', 'Service accounts roles', 'Standard Token Exchange', 'OAuth 2.0 Device Authorization Grant', and 'OIDC CIBA Grant', all of which are currently unchecked.
- PKCE Method**: A dropdown menu currently showing 'S256'.
- Require DPoP bound tokens**: A toggle switch set to 'Off'.



KEYCLOAK: CREAZIONE CLIENT FRONTEND

Login settings

Login theme ?

keycloak

Consent required ?

☐ Off

Display client on
screen ?

☐ Off

Consent screen text
?

Logout settings

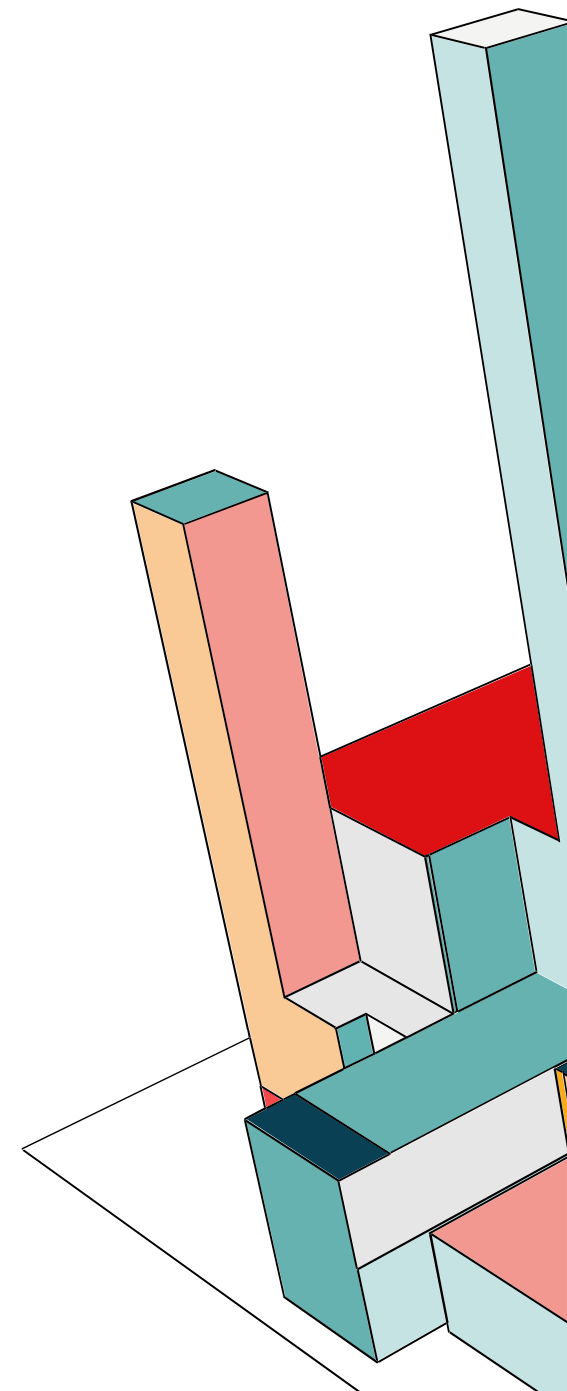
Front channel logout
?

☒ On

Front-channel logout
URL ?

Front-channel logout
session required ?

☒ On



KEYCLOAK: BRUTE FORCE DETECTION

SySec

Enabled

Action ▾

Realm settings are settings that control the options for users, applications, roles, and groups in the current realm. [Learn more](#)

<

General

Login

Email

Themes

Keys

Events

Localization

Security defenses

Sessions

Tokens

>

Headers

Brute force detection

Brute Force Mode ⓘ

Lockout permanently ▾

Max login failures ⓘ

–

10

+

Quick login check
milliseconds ⓘ

–

1000

+

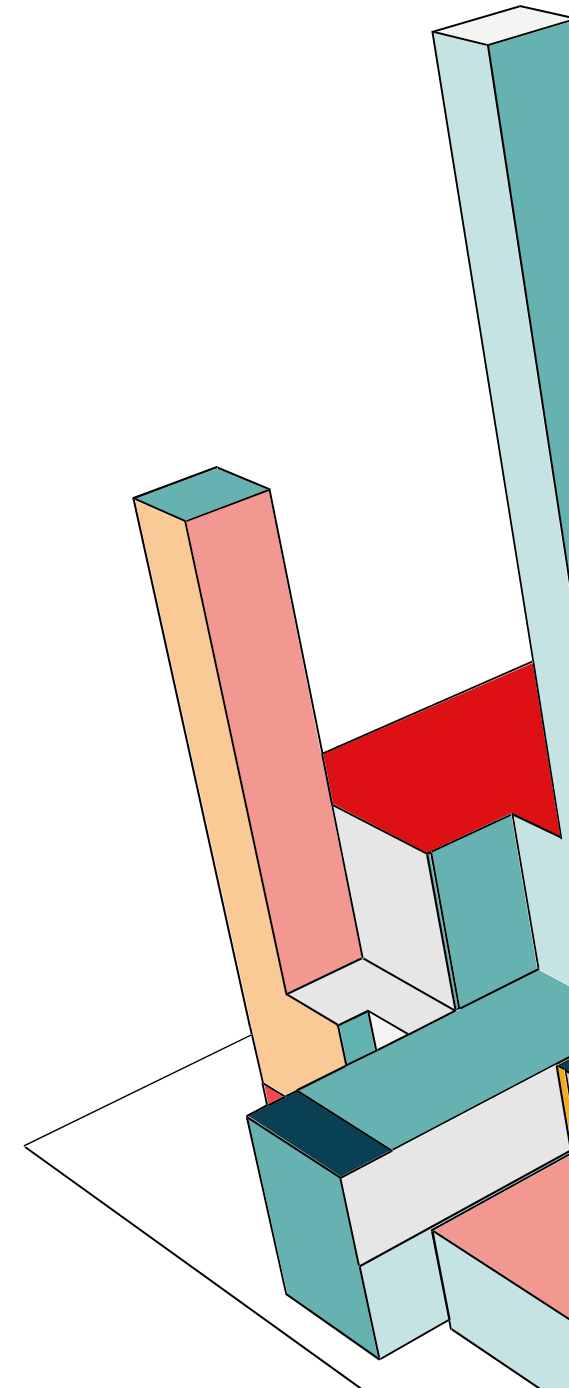
Minimum quick login
wait ⓘ

1

Minutes ▾

Save

Revert



KEYCLOAK: CREAZIONE CLIENT FRONTEND

Il **Refresh Token** è una credenziale che permette al client di ottenere nuovi Access Token senza richiedere nuovamente l'intervento dell'utente (re-authentication). Se il parametro è impostato su ON nella risposta JSON del Token Endpoint, oltre all'Access Token e all'ID Token, verrà incluso l'attributo `refresh_token`.

OpenID Connect Compatibility Modes

This section is used to configure settings for backward compatibility with older OpenID Connect / OAuth 2 adaptors. It is useful especially if your client uses an older version of Keycloak / RH-SSO adapter.

Exclude Session State
From Authentication
Response ? ☐ Off

Exclude Issuer From
Authentication
Response ? ☐ Off

Use refresh tokens ? ☒ On

Use refresh tokens for
client credentials grant
? ☐ Off

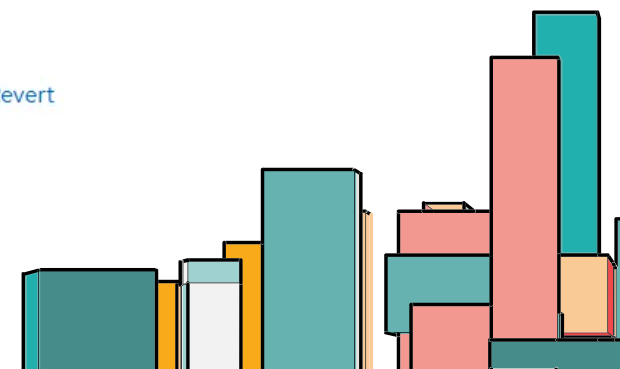
Use lower-case bearer
type in token
responses ? ☐ Off

Allow refresh token in
Standard Token
Exchange ?

Choose... ▼

Save

Revert



KEYCLOAK: CREAZIONE CLIENT BACKEND

Clients > Client details

backend-node OpenID Connect

Clients are applications and services that can request authentication of a user.

Settings Keys Credentials Roles Client scopes Authorization Service accounts roles Sessions Advanced Events

General settings

Client ID * ⓘ backend-node

Name ⓘ Backend node

Description ⓘ

Always display in UI ⓘ ☐ Off

Access settings

Root URL ⓘ

Home URL ⓘ

Valid redirect URIs ⓘ * ⓘ
[+ Add valid redirect URIs](#)

Valid post logout redirect URIs ⓘ * ⓘ
[+ Add valid post logout redirect URIs](#)

Web origins ⓘ * ⓘ
[+ Add web origins](#)



KEYCLOAK: CREAZIONE CLIENT BACKEND

Capability config

Client authentication ☒ On

Authorization ☒ On

Authentication flow

- ☒ Standard flow
- ☒ Direct access grants
- ☐ Implicit flow
- ☒ Service accounts roles
- ☐ Standard Token Exchange
- ☐ OAuth 2.0 Device Authorization Grant
- ☐ OIDC CIBA Grant

PKCE Method Choose...

Require DPoP bound tokens ☐ Off



KEYCLOAK: CLIENT BACKEND

Client scopes > Client scope details

audience-backend openid-connect Action

Settings Mappers Scope

Name * ? audience-backend

Description ?

Type ? Default

Display on consent screen ? ☒ On

Consent screen text ?

Include in token scope ? ☒ On

Include in OpenID Provider Metadata ? ☒ On

Display Order ?

Save Cancel



KEYCLOAK: MAPPERS

I mapper sono usati per mappare gli specifici claim all'interno delle Verifiable Credentials. In questo caso usiamo il mapper di tipo Audience, ovvero che aggiunge il valore specificato nel campo 'aud' del token.

Audience

94e9a4f2-25d4-4715-bef8-5cc2398d888b

Action ▾

Mapper type	Audience
Name * ⓘ	audience-mapping-backend
Included Client Audience ⓘ	backend-node × ▾
Included Custom Audience ⓘ	
Add to ID token ⓘ	☐ Off
Add to access token ⓘ	☒ On
Add to lightweight access token ⓘ	☐ Off
Add to token introspection ⓘ	☒ On



KEYCLOAK: CREAZIONE CLIENT BACKEND SCOPE

Quando viene creato un access token con OIDC, di default, tutte le mappature dei ruoli dell'utente vengono aggiunte nel token come claim. Questo vuol dire che se esiste un client malevolo registrato nel realm, possiamo fornire agli attaccanti token di accesso con una vasta gamma di permessi. Il client scope server proprio a questo: limitare i ruoli dichiarati nell'access token.

[Clients](#) > [Client details](#)

backend-node OpenID Connect

Clients are applications and services that can request authentication of a user.

Settings

Keys

Credentials

Roles

Client scopes

Authorization

Service accounts roles

Sessions

Advanced

Events

Setup

Evaluate

▼ Name

🔍 Search by name

→

Add client scope

Change type to ▼

⋮

🔄 Refresh

☐	Assigned client scope	Assigned type	Description
☐	backend-node-dedicated	None ▼	Dedicated scope and mappers for this client



KEYCLOAK: CREAZIONE CLIENT BACKEND

Clients > Client details

backend-node OpenID Connect

Clients are applications and services that can request authentication of a user.

Settings Keys Credentials Roles Client scopes **Authorization** Service accounts

Settings Resources Scopes Policies Permissions Evaluate Export

Import ⓘ

Policy enforcement mode ⓘ

☒ Enforcing

☐ Permissive

☐ Disabled

Decision strategy ⓘ

☐ Unanimous

☒ Affirmative

Remote resource management ⓘ

☐ Off



KEYCLOAK: CREAZIONE RISORSE

[Clients](#) > [Client details](#)

backend-node

OpenID Connect

Clients are applications and services that can request authentication of a user.

Settings

Keys

Credentials

Roles

Client scopes

Authorization

Service accounts roles

Sessions

Advanced

Events

Settings

Resources

Scopes

Policies

Permissions

Evaluate

Export

Search resource

Create resource

Name	Display name	Type
> Default Resource		urn:backend-node:resources:default
▼ todo	API Gestione Todo	
URIs	/api/todos/* /api/todos	
Scopes	create delete view viewuser	
Associated permission	Permesso Delete Permesso Admin View All Permesso Create Permesso User View Only	



KEYCLOAK: CREAZIONE SCOPE

Clients > Client details

backend-node

OpenID Connect

Clients are applications and services that can request authentication of a user.

Settings

Keys

Credentials

Roles

Client scopes

Authorization

Service accounts roles

Sessions

Advanced

Events

Settings

Resources

Scopes

Policies

Permissions

Evaluate

Export

Q Search by name

→

Create authorization scope

Name	Display name
▼ create	create
Resources todo	
Associated permission Permesso Create	
▼ delete	delete
Resources todo	
Associated permission Permesso Delete	
▼ view	view
Resources todo	
Associated permission Permesso Admin View All	
▼ viewuser	viewuser
Resources todo	
Associated permission Permesso User View Only	



KEYCLOAK: CREAZIONE POLICY

Clients > Client details

backend-node

OpenID Connect

Clients are applications and services that can request authentication of a user.

Settings

Keys

Credentials

Roles

Client scopes

Authorization

Service accounts roles

Sessions

Settings

Resources

Scopes

Policies

Permissions

Evaluate

Export

Search policy

Create client policy

Name	Type	Dependent permission
> Default Policy	Js	Default Permission
▼ Policy Admin	Role	Permesso Admin View All 1 more
Dependent permission	Permesso Admin View All Permesso Delete	
▼ Policy Technician	Role	
Dependent permission	None	
▼ Policy User	Role	Permesso User View Only 2 more
Dependent permission	Permesso User View Only Permesso Delete Permesso Create	



KEYCLOAK: CREAZIONE PERMESSI

Clients > Client details

backend-node

OpenID Connect

Clients are applications and services that can request authentication of a user.

Settings

Keys

Credentials

Roles

Client scopes

Authorization

Service accounts roles

Sessions

Advanced

Event

Settings

Resources

Scopes

Policies

Permissions

Evaluate

Export

Search permission

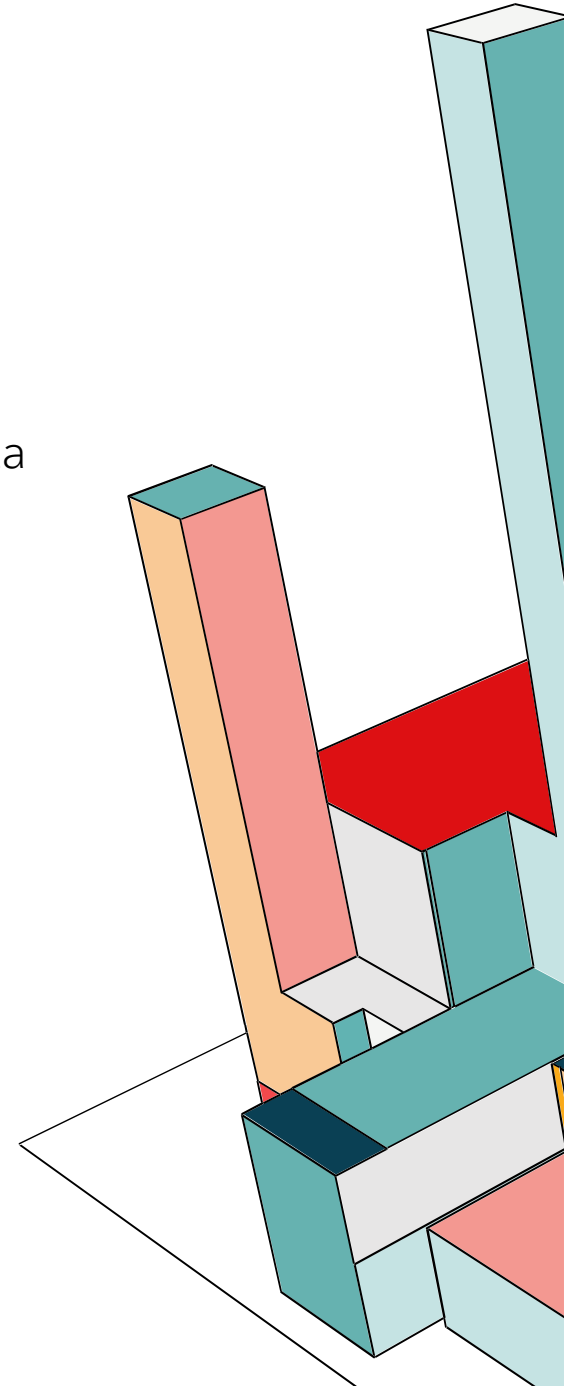
Create permission

Name	Type	Associated policy
> Default Permission	Resource-Based	Default Policy
▼ Permessi User View Only	Scope-Based	Policy User
Associated policy	Policy User	
▼ Permessi Admin View All	Scope-Based	Policy Admin
Associated policy	Policy Admin	
▼ Permessi Create	Scope-Based	Policy User
Associated policy	Policy User	
▼ Permessi Delete	Scope-Based	Policy Admin 1 more
Associated policy	Policy Admin Policy User	



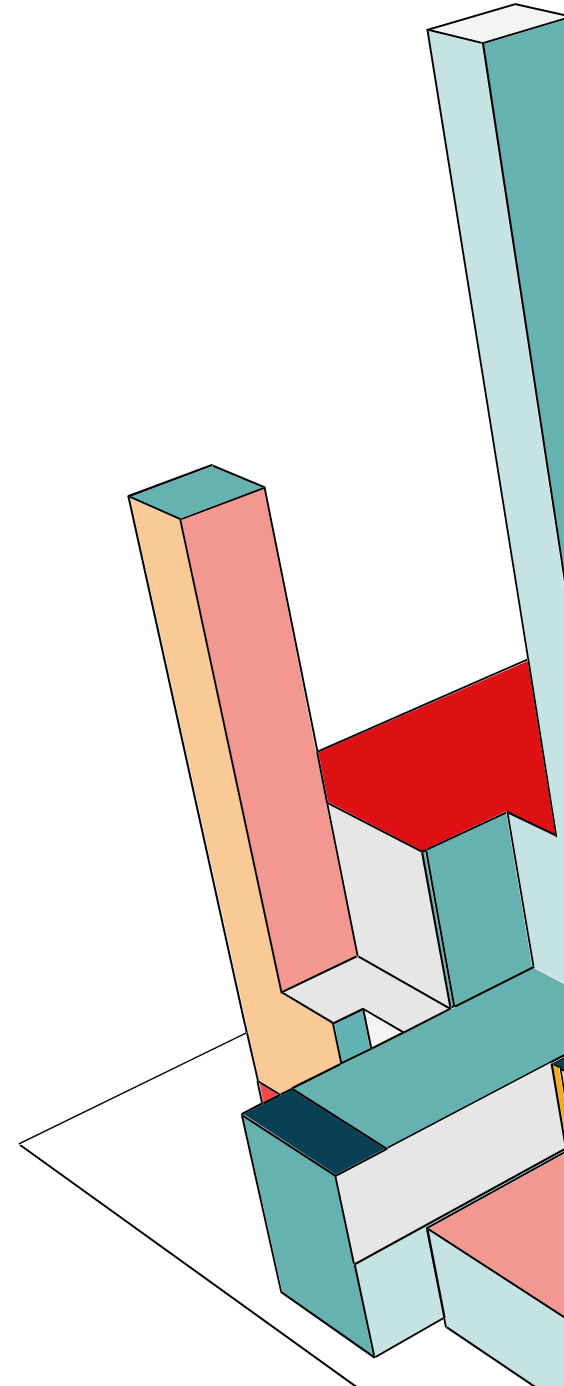
RBAC

L'RBAC è un paradigma di controllo degli accessi in cui i diritti di sistema sono mediati attraverso l'astrazione dei **ruoli**, anziché essere assegnati direttamente alle singole identità degli utenti. Questo approccio semplifica la gestione della sicurezza scalando in contesti organizzativi complessi.



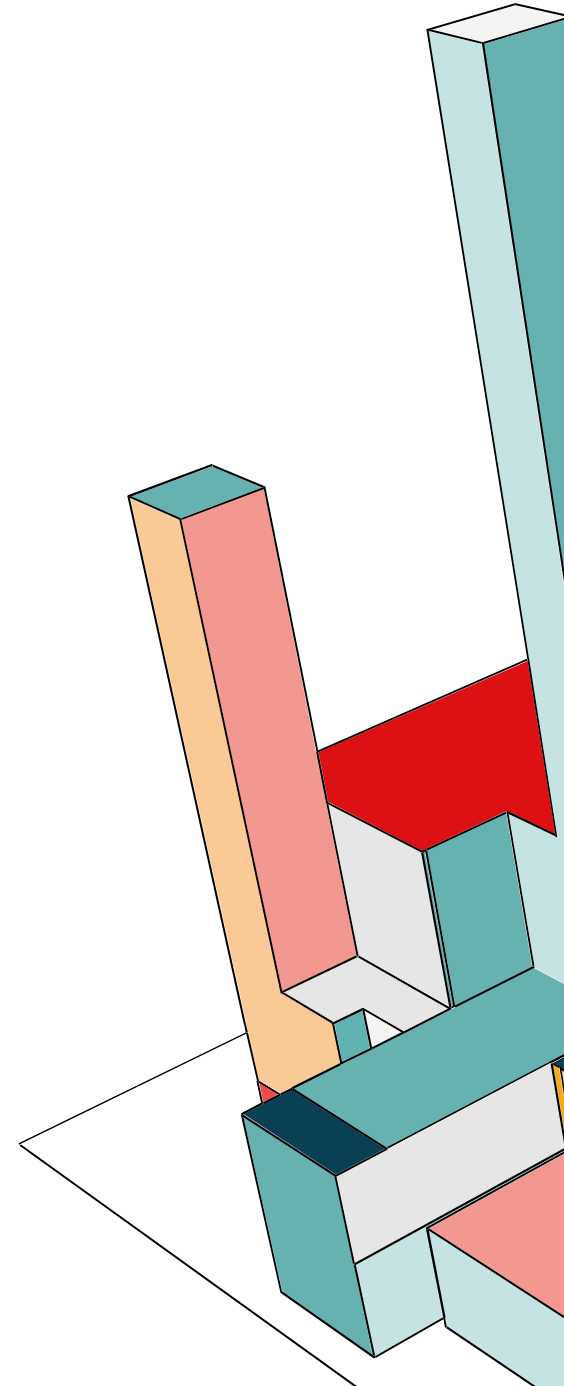
RBAC: PRINCIPI FONDAMENTALI

- **Astrazione dell'Identità:** L'accesso alle risorse (oggetti) è determinato esclusivamente dal ruolo attivo dell'utente. Il sistema non valuta l'identità del singolo (User ID), ma le prerogative associate alla sua funzione.
- **Definizione dei Ruoli:** I ruoli sono costrutti logici definiti in base alle **funzioni lavorative** (job functions) all'interno dell'organizzazione.
- **Granularità dei Permessi:** I permessi sono stabiliti in base all'autorità e alle responsabilità operative. Un permesso è l'approvazione a eseguire una determinata **operazione** su un particolare **oggetto** di sistema.
- **Invocazione delle Operazioni:** L'esecuzione di metodi o operazioni su un oggetto è vincolata all'esistenza di una relazione valida nel set dei permessi assegnati al ruolo.



RBAC: VANTAGGI

Le politiche RBAC aiutano ad affrontare le vulnerabilità di cybersecurity applicando il principio del privilegio minimo (PoLP). Secondo il PoLP, i ruoli utente concedono l'accesso al livello minimo di autorizzazioni necessarie per completare un'attività o svolgere un incarico. Limitando l'accesso ai dati sensibili, RBAC aiuta a prevenire sia la perdita accidentale di dati sia le violazioni intenzionali dei dati. Nello specifico, RBAC aiuta a limitare il movimento laterale, ovvero quando gli hacker utilizzano un vettore di accesso iniziale alla rete per espandere gradualmente il loro raggio d'azione all'interno di un sistema. Ma limitando l'accesso ai dati, RBAC rende più difficile per i dipendenti utilizzare in modo doloso o negligente i propri privilegi di accesso per danneggiare l'organizzazione, quindi mitiga anche minacce interne all'organizzazione.



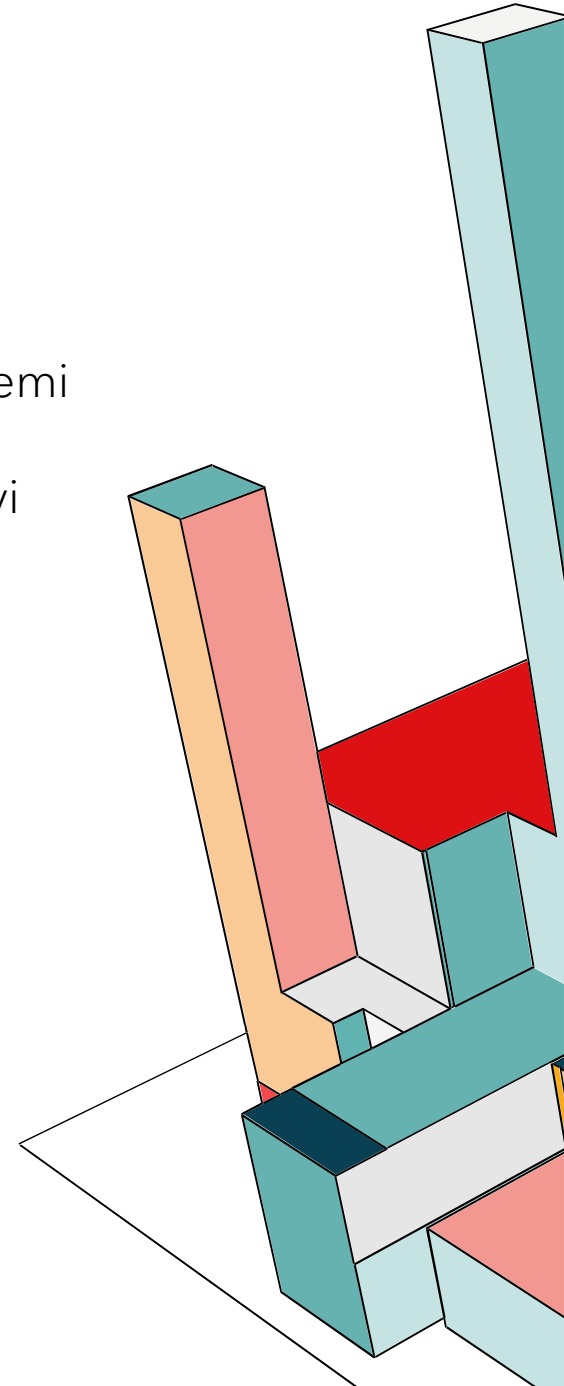
RBAC: TRE REGOLE FONDAMENTALI

Il NIST, che ha sviluppato il modello RBAC, fornisce tre regole di base per tutti i sistemi RBAC.

1. Assegnazione del ruolo: a un utente devono essere assegnati uno o più ruoli attivi per esercitare autorizzazioni o privilegi.

2. Autorizzazione del ruolo: l'utente deve essere autorizzato ad assumere il ruolo o i ruoli che gli sono stati assegnati.

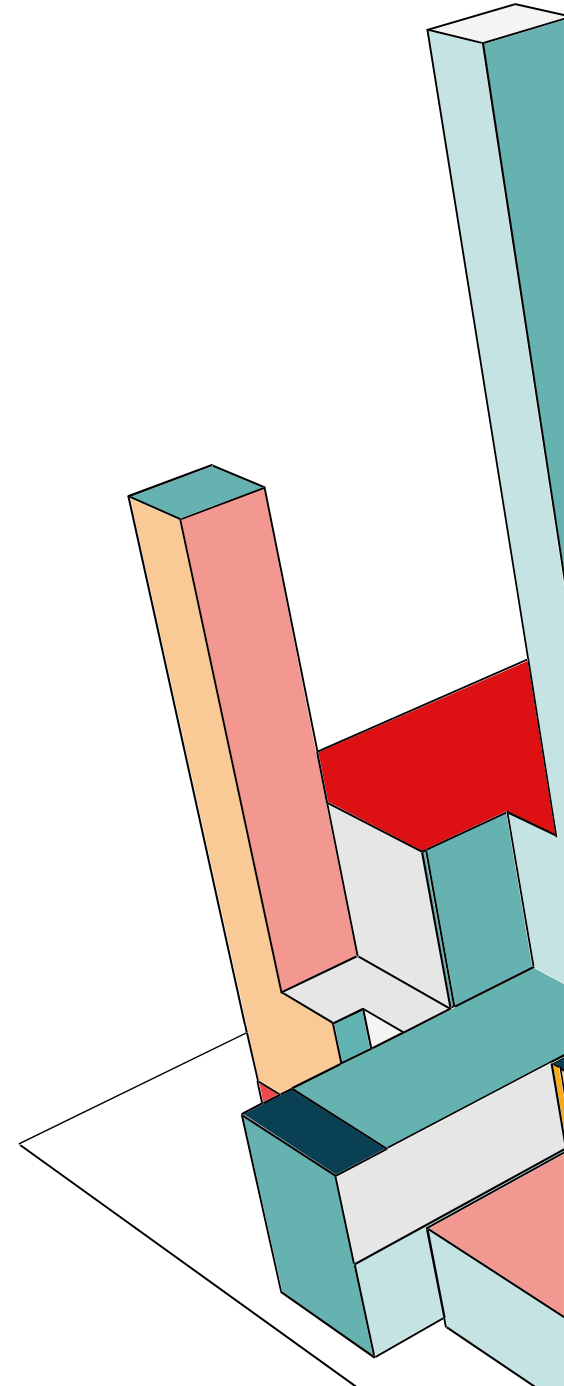
3. Autorizzazione: le autorizzazioni o i privilegi vengono concessi solo agli utenti autorizzati tramite l'assegnazione dei ruoli.



RBAC: 4 MODELLI PRINCIPALI

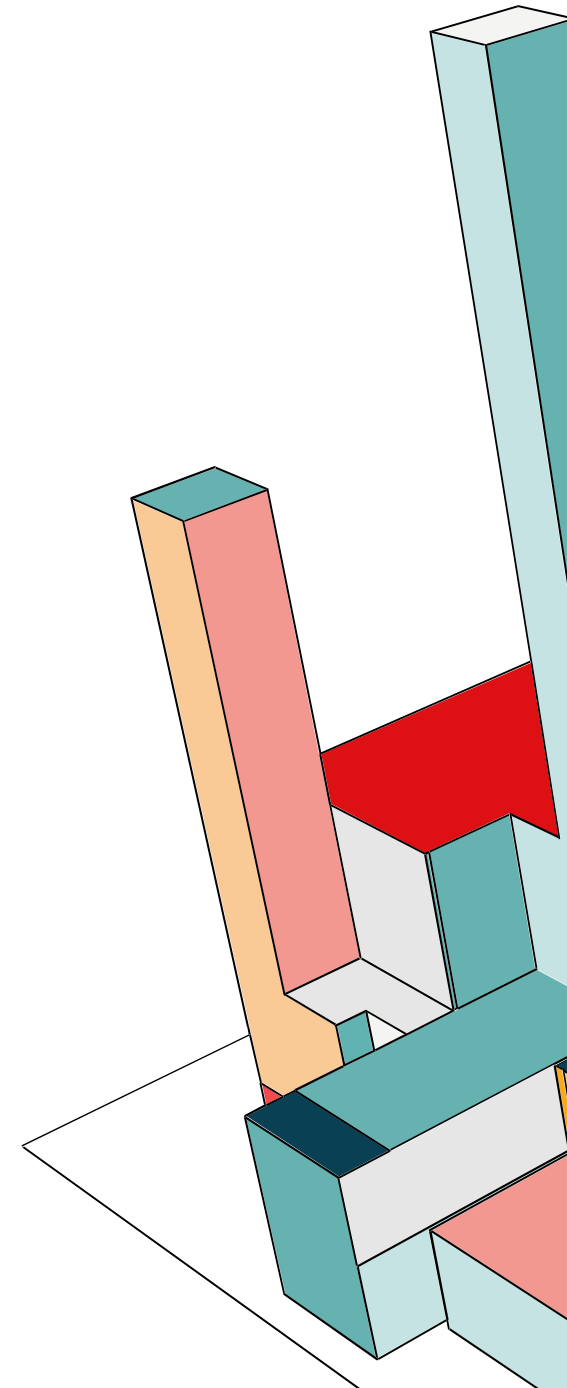
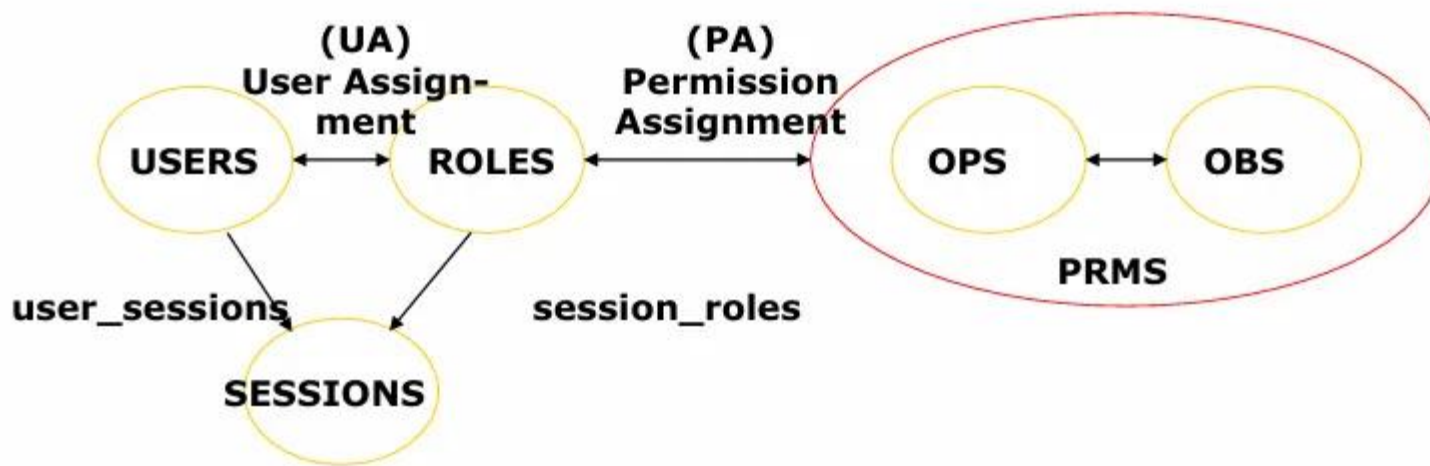
Esistono quattro modelli distinti per l'implementazione di RBAC, ma ogni modello inizia con la stessa struttura di base. Ogni modello successivo aggiunge nuove funzionalità e caratteristiche al modello precedente.

- RBAC core
- RBAC gerarchico
- RBAC vincolato
- RBAC simmetrico



RBAC CORE

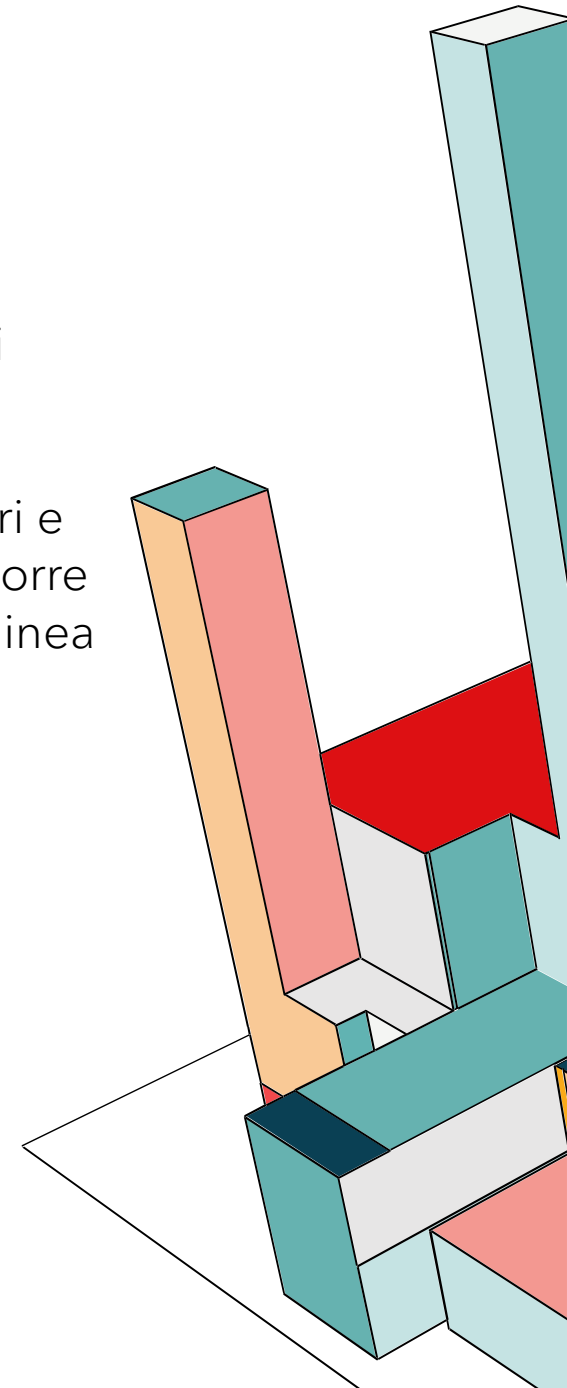
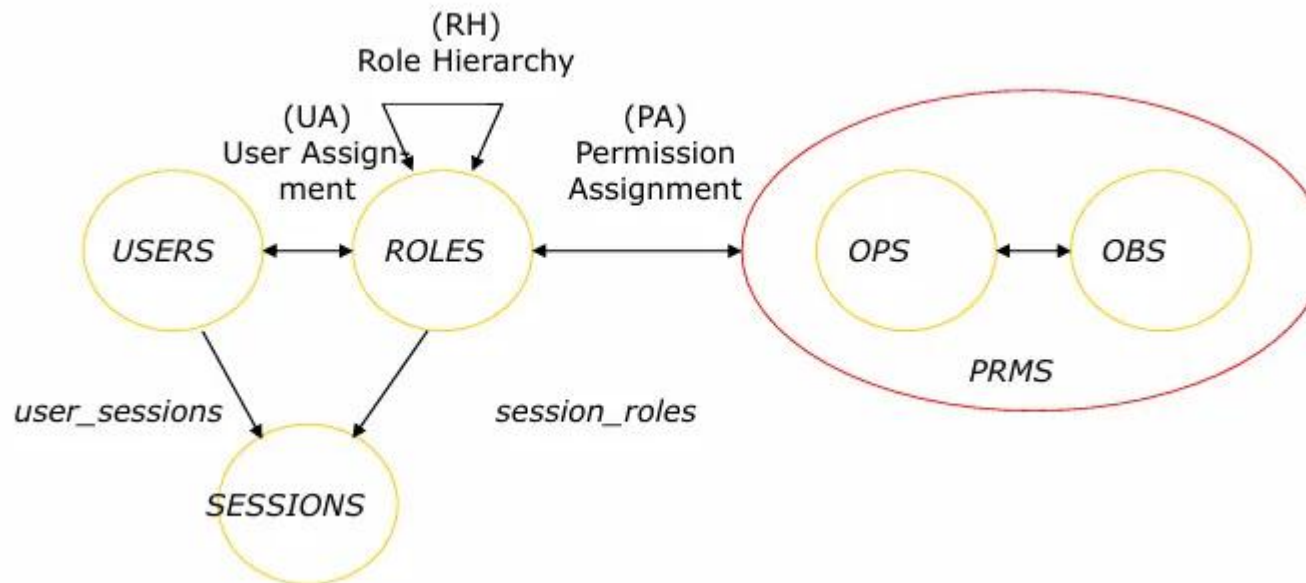
Questo modello, a volte chiamato Flat RBAC, è la base necessaria per qualsiasi sistema RBAC. Segue le tre regole base del sistema RBAC. Agli utenti vengono assegnati ruoli e tali ruoli autorizzano l'accesso a set specifici di autorizzazioni e privilegi. RBAC core può essere utilizzato come sistema principale di controllo degli accessi o come base per modelli RBAC più avanzati.



RBAC GERARCHICO

Questo modello aggiunge gerarchie di ruoli che replicano la struttura di reporting di un'organizzazione. In una gerarchia dei ruoli, ogni ruolo eredita le autorizzazioni del ruolo sottostante e ottiene nuove autorizzazioni.

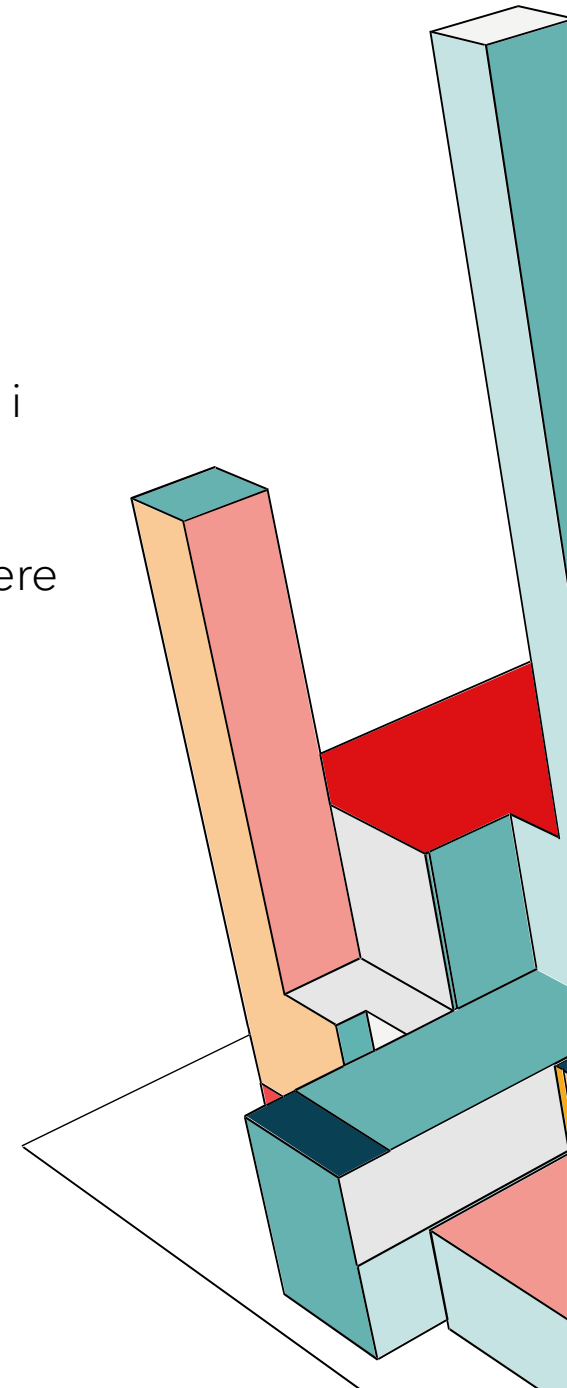
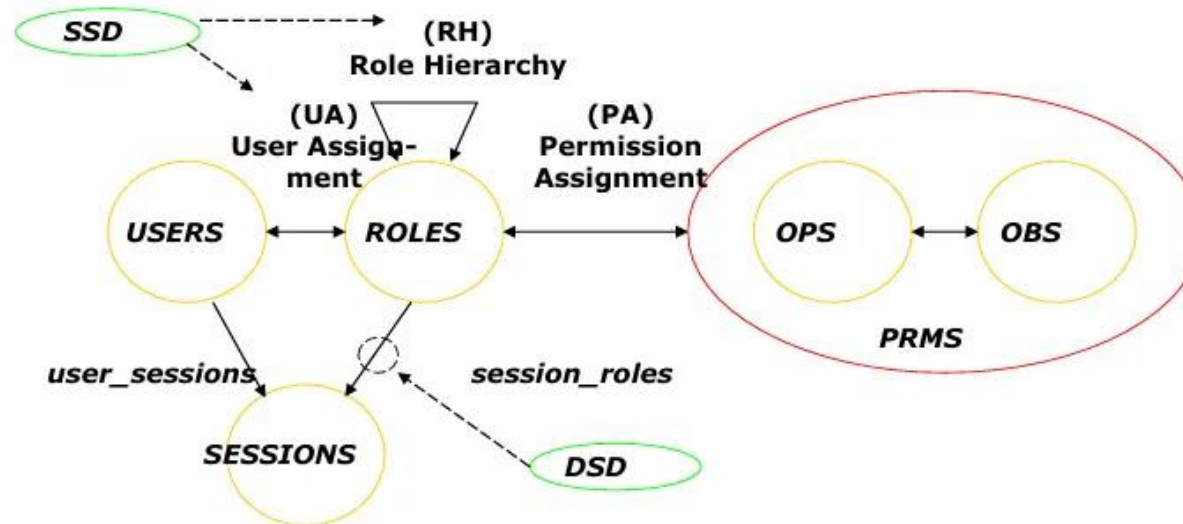
Ad esempio, una gerarchia di ruoli potrebbe includere dirigenti, manager, supervisori e dipendenti di linea. Un dirigente al vertice della gerarchia sarebbe autorizzato a disporre di un set completo di permessi, mentre ai manager, ai supervisori e ai dipendenti di linea verrebbero concessi sottoinsiemi via via più piccoli di tale set di permessi.



RBAC VINCOLATO

Oltre alle gerarchie dei ruoli, questo modello aggiunge funzionalità per imporre la separazione delle mansioni (SOD). La separazione delle mansioni aiuta a prevenire i conflitti di interesse, in quanto richiede che due persone portino a termine determinati compiti.

Ad esempio, l'utente che richiede il rimborso di una spesa aziendale non deve essere la stessa persona che approva la richiesta. I criteri RBAC vincolati garantiscono la separazione dei privilegi degli utenti per questo tipo di attività.

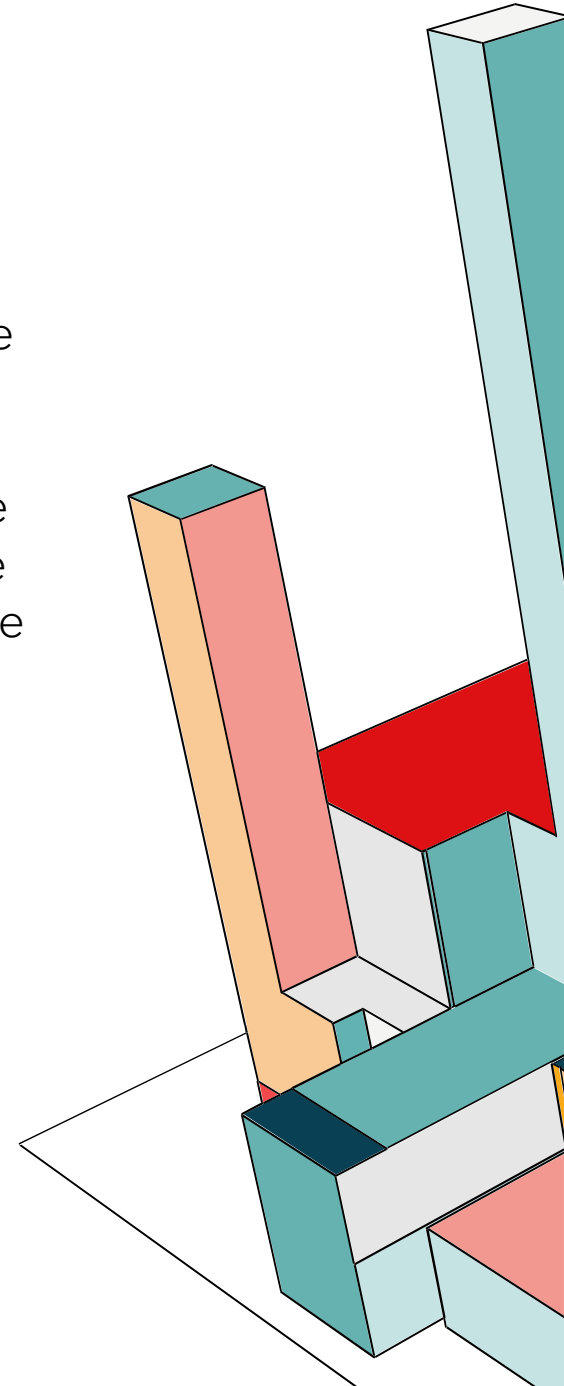


RBAC SIMMETRICO

Questo modello è la versione più avanzata, flessibile e completa di RBAC. Oltre alle funzionalità dei modelli precedenti, aggiunge una visibilità più approfondita sulle autorizzazioni in tutta l'azienda.

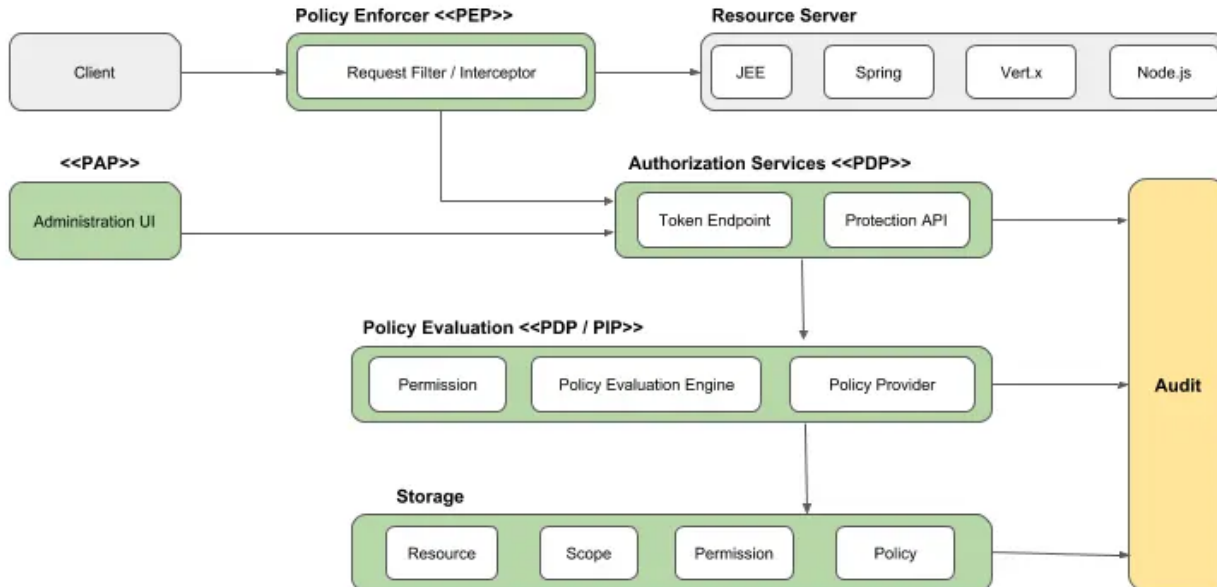
Le organizzazioni sono in grado di verificare in che modo ogni autorizzazione viene assegnata a ciascun ruolo e a ciascun utente nel sistema. Possono anche regolare e aggiornare le autorizzazioni associate ai ruoli man mano che i processi aziendali e le responsabilità dei dipendenti si evolvono.

Queste funzionalità sono particolarmente utili per le grandi organizzazioni che devono garantire che ogni ruolo e ogni utente disponga del minimo accesso necessario per svolgere le proprie attività.



AUTHORIZATION SERVICES IN KEYCLOAK

- **Policy Administration Point (PAP):** Rappresenta il piano di gestione (Control Plane). Attraverso la console di amministrazione o la **Protection API**, il PAP permette la definizione e la persistenza di entità quali *resource servers*, *resources*, *scopes* e *policies*.
- **Policy Decision Point (PDP):** Il motore computazionale responsabile della valutazione delle policy. Riceve richieste di autorizzazione, analizza il contesto e i metadati, e restituisce un verdetto (Permit/Deny) basato sui requisiti di accesso.



- **Policy Enforcement Point (PEP):** Lo strato di intercettazione situato lato *resource server*. Il PEP interroga il PDP e applica rigorosamente la decisione ricevuta, garantendo o negando l'accesso fisico alla risorsa protetta.
- **Policy Information Point (PIP):** Fornisce il contesto dinamico necessario alla valutazione. In Keycloak, il PIP estrae attributi dalle identità (Identity Attributes) e dall'ambiente di runtime per arricchire il processo decisionale.



RBAC: LIFECYCLE AUTHORIZATION PROCESS

1. Resource Management (Definizione del Perimetro)

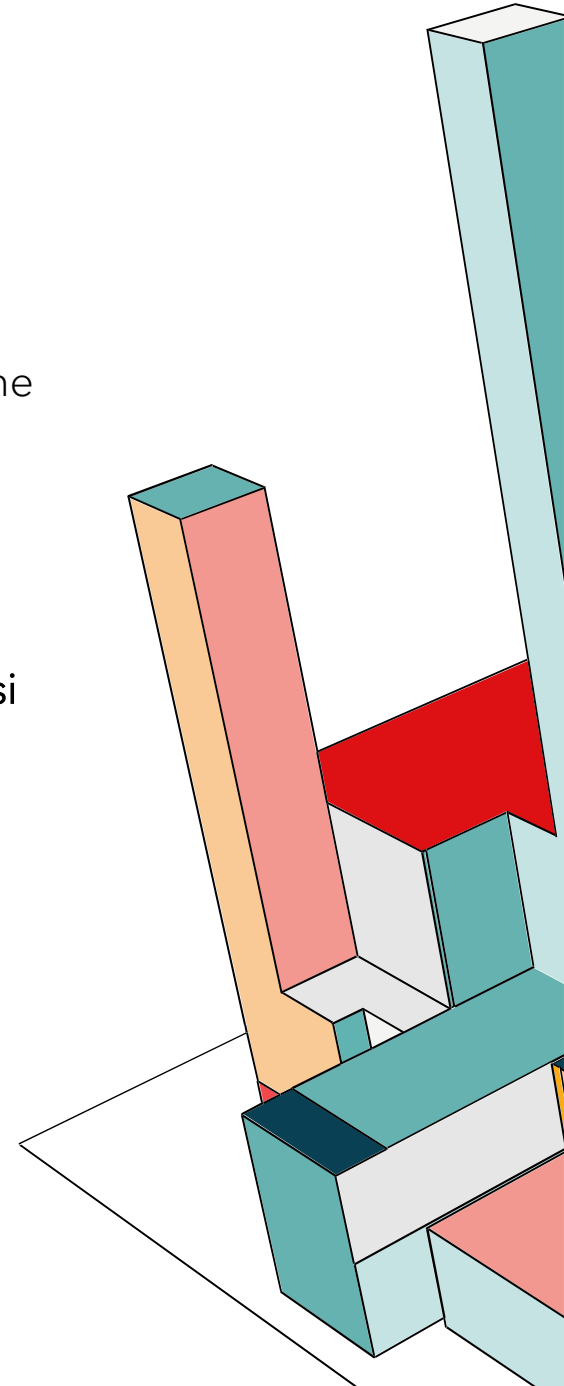
Identificazione degli asset da proteggere: **resource server** (qualsiasi servizio o applicazione che delega il controllo degli accessi a Keycloak), **resources** (astrazioni di oggetti di sistema, per esempio file o istanze di classi come "Conto Corrente", **scopes** (i bound operativi o le azioni eseguibili su una risorsa, ovvero i permessi come read, write, execute).

2. Permission & Policy Management (Logica e Vincoli)

Fase di configurazione dei criteri di accesso: **policies** basate sul ruolo in questo caso, permessi risultanti dall'associazione tra una risorsa (o scope) e una o più policy.

3. Policy Enforcement

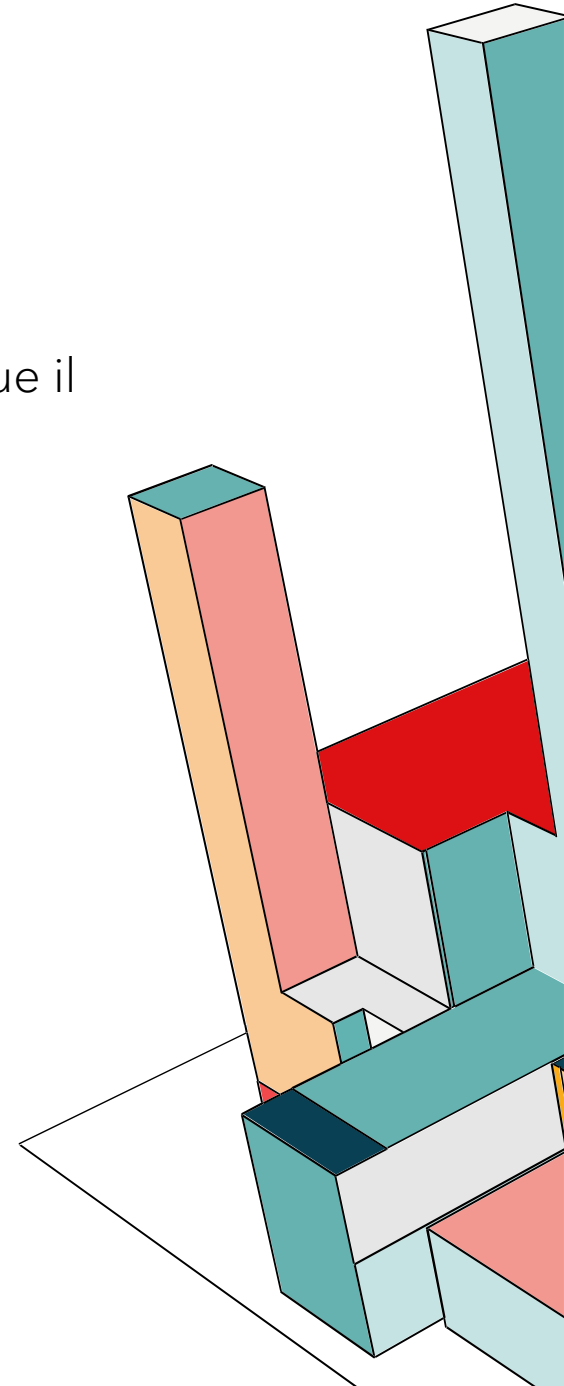
L'integrazione del PEP all'interno dell'applicazione per validare le richieste in transito. Il PEP comunica con il server di autorizzazione per ottenere i metadati decisionali e proteggere gli endpoint in modo granulare.



KEYCLOAK: POLICY ENFORCER

Il Policy Enforcement Point (PEP) è un design pattern fondamentale nei sistemi di controllo degli accessi. In Keycloak, la logica di autorizzazione è centralizzata e segue il modello di separazione delle responsabilità:

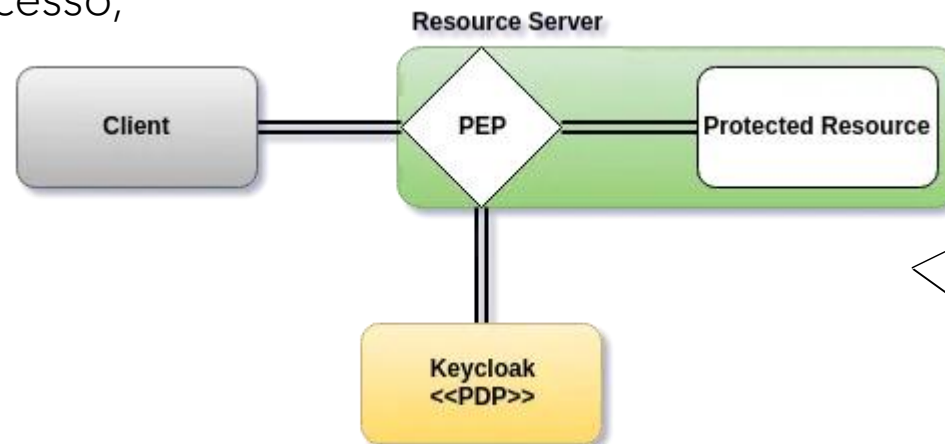
- **PDP (Policy Decision Point):** Keycloak stesso, che valuta le policy e decide se l'accesso è consentito.
- **PEP (Policy Enforcement Point):** Un componente integrato nell'applicazione che riceve ed esegue (applica) tali decisioni. Keycloak implementa i servizi di autorizzazione tramite **API RESTful** e sfrutta le estensioni di **OAuth2** per fornire un'autorizzazione a grana fine (**fine-grained**).



KEYCLOAK: POLICY ENFORCER

Il PEP agisce come un **intercettore** o un **filtro HTTP** all'interno dell'applicazione. Il suo ciclo di esecuzione prevede:

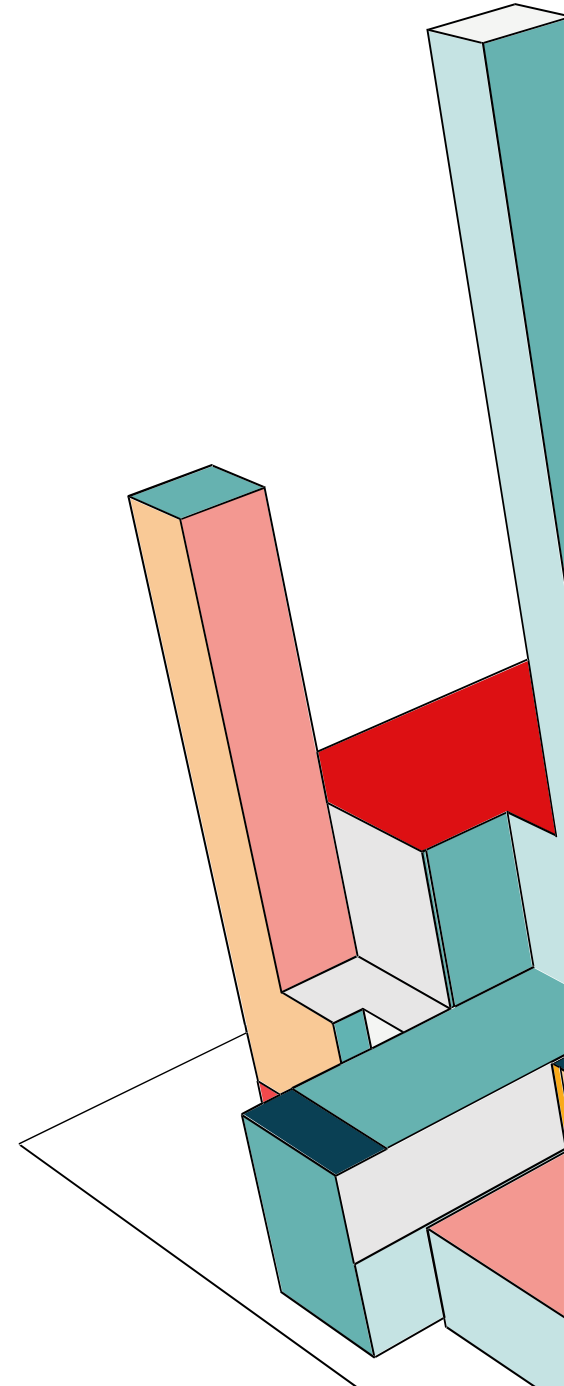
- 1.L'intercettazione di ogni richiesta entrante verso una risorsa protetta.
- 2.L'invio delle informazioni di contesto (identità dell'utente, risorsa richiesta, scope) al server Keycloak.
- 3.L'applicazione della decisione ricevuta: se il permesso è negato, la richiesta viene bloccata prima di raggiungere la logica di business; se concesso, l'elaborazione prosegue.



CONFIGURAZIONE RBAC

Per il nostro progetto vorremmo implementare tre ruoli:

- 1. app-User:** può aggiungere nuovi task, può visualizzare solo i propri task (filtro basato sull'ID utente nel database), può modificare o segnare come completati i propri task e eliminare i propri task. In questo modo garantiamo la riservatezza dei dati tra utenti diversi.
- 2. app-admin:** è il responsabile della gestione degli utenti e della supervisione dell'applicazione. Può creare, sospendere o eliminare gli account degli utenti, può vedere la dashboard con il numero totale di task creati nel sistema, e può eliminare task inappropriati creati da qualsiasi utente.
- 3. app-tech:** il tecnico non deve vedere i dati privati né gestire gli utenti, ma deve assicurarsi che il sistema funzioni. Infatti può accedere ai log di Keycloak e del backend per monitorare tentativi di accesso falliti o errori di sistema, può eseguire operazioni di manutenzione tecnica (es. pulizia cache o reset dei log). In questo modo implementiamo la Separation of Duties: un tecnico può riparare il sistema senza avere accesso alle password degli utenti o ai contenuti dei loro task.



KEYCLOAK: POLICY ENFORCER

[Clients](#) > [Client details](#)

react-app OpenID Connect ☒ Enabled ? Action ▾

Clients are applications and services that can request authentication of a user.

[Settings](#) [Keys](#) [Credentials](#) [Roles](#) [Client scopes](#) **Authorization** [Service accounts roles](#) [Sessions](#) >

[Settings](#) [Resources](#) [Scopes](#) [Policies](#) [Permissions](#) [Evaluate](#) [Export](#)

[Import](#) ? [Import](#)

Policy enforcement mode ?

☒ Enforcing

☐ Permissive

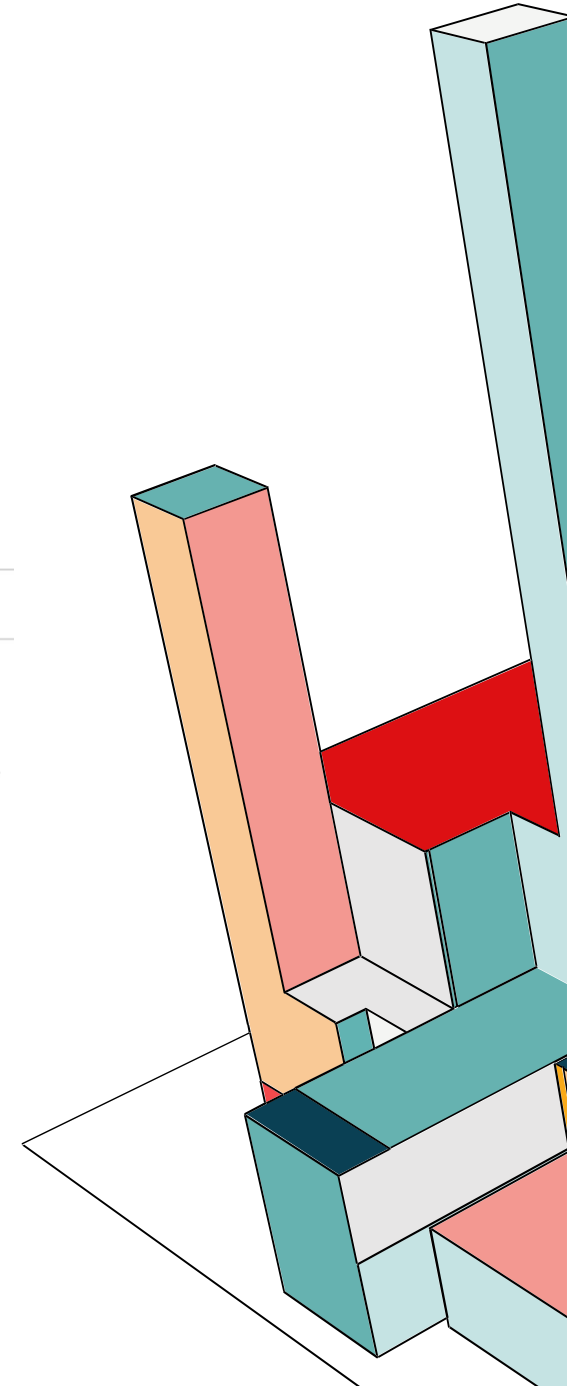
☐ Disabled

Decision strategy ?

☒ Unanimous

☐ Affirmative

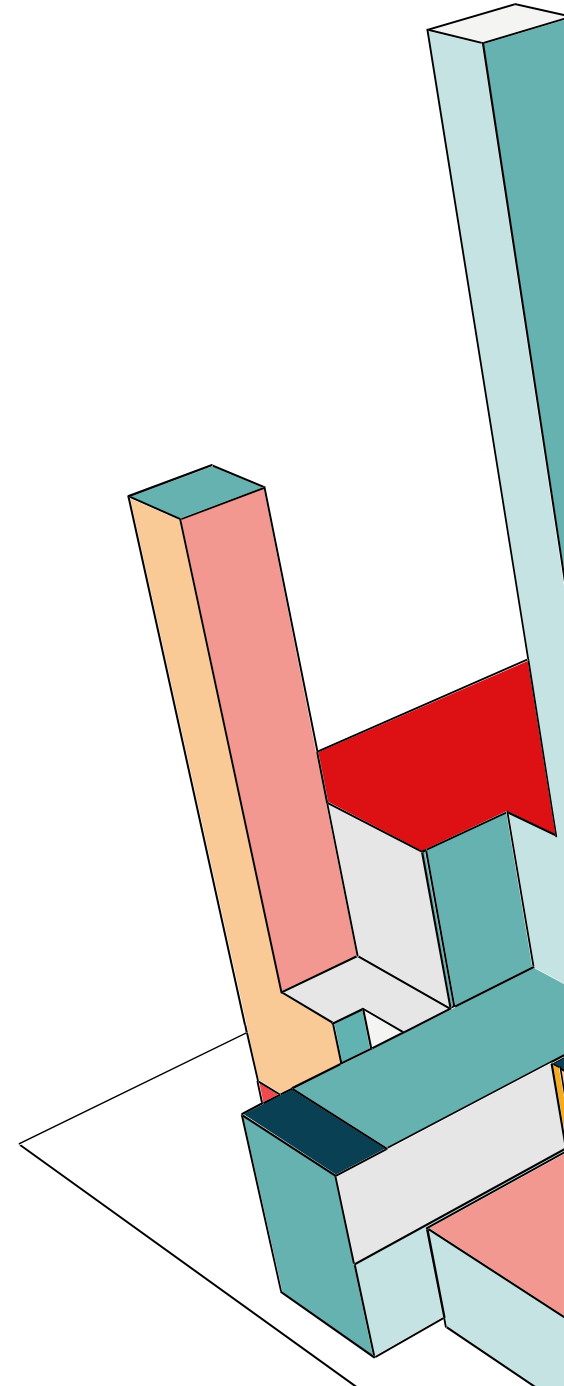
Remote resource management ? ☒ On



KEYCLOAK: POLICY ENFORCER

La direttiva `enforcement-mode` definisce il comportamento del filtro in assenza di policy o in stati specifici:

- **ENFORCING** (Default) Approccio **Deny-by-Default**. Se una risorsa non ha una policy associata o l'accesso non è esplicitamente concesso, la richiesta viene bloccata (403 Forbidden).
- **PERMISSIVE** Approccio **Allow-if-Undefined**. Le richieste sono consentite se non esiste una policy specifica associata alla risorsa. Se invece esiste una policy e questa nega l'accesso, la richiesta viene bloccata.
- **DISABLED** Disabilita l'intercettazione e la valutazione delle policy. L'accesso è libero a tutte le risorse, ma l'applicazione può comunque accedere al **Authorization Context** per recuperare i permessi concessi (utile per logiche di autorizzazione manuali interne al codice).



CREAZIONE RUOLI

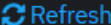
Realm roles

Realm roles are the roles that you define for use in the current realm. [Learn more](#)

Search role by name



Create role



Refresh

1-6



Role name	Composite	Description
app-admin	False	amministratore di sistema può gestire utenti e supervisionare l'applicazione (crea/sospende account di utenti, può vedere la dashboard con tutti i task creati, può eliminare task)
app-tech	False	tecnico dell'applicazione che può visualizzare i log, capire se è in corso un attacco e in generale assicurarsi che l'app funzioni correttamente.
app-user	False	—

CREAZIONE UTENTI

Users

Users are the users in the current realm. [Learn more](#)

User list

Default search

Search user

→

Add user

Delete user

Refresh

1 - 3

☐ Username	Email	Last name	First name	
☐ elenaverdi_admin@test.com	elenaverdi_admin@test.com	Verdi	Elena	⋮
☐ lucamagenta_tech@test.com	lucamagenta_tech@test.com	Magenta	Luca	⋮
☐ mariorossi_user@test.com	mariorossi_user@test.com	Rossi	Mario	⋮

ROLE MAPPING

Users > User details

mariorossi_user@test.com

Details

Credentials

Role mapping

Groups

Consents

Identity provider links

Sessions

Events

🔍 Search by name

➔

☒ Hide inherited roles

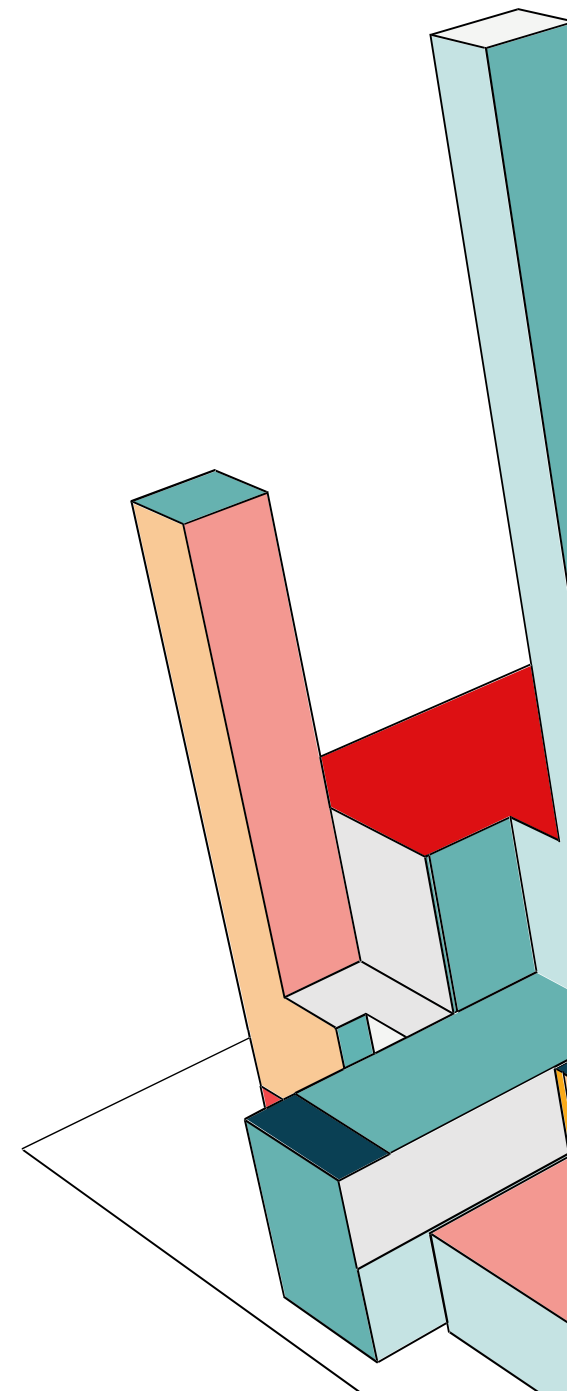
Assign role ▾

Unassign

🔄 Refresh

☐ Name	Inherited	Description
☐ default-roles-sysec	False	role_default-roles
☐ app-user	False	—

LOGIN COME USER



LOGIN AS APP-USER

SYSEC

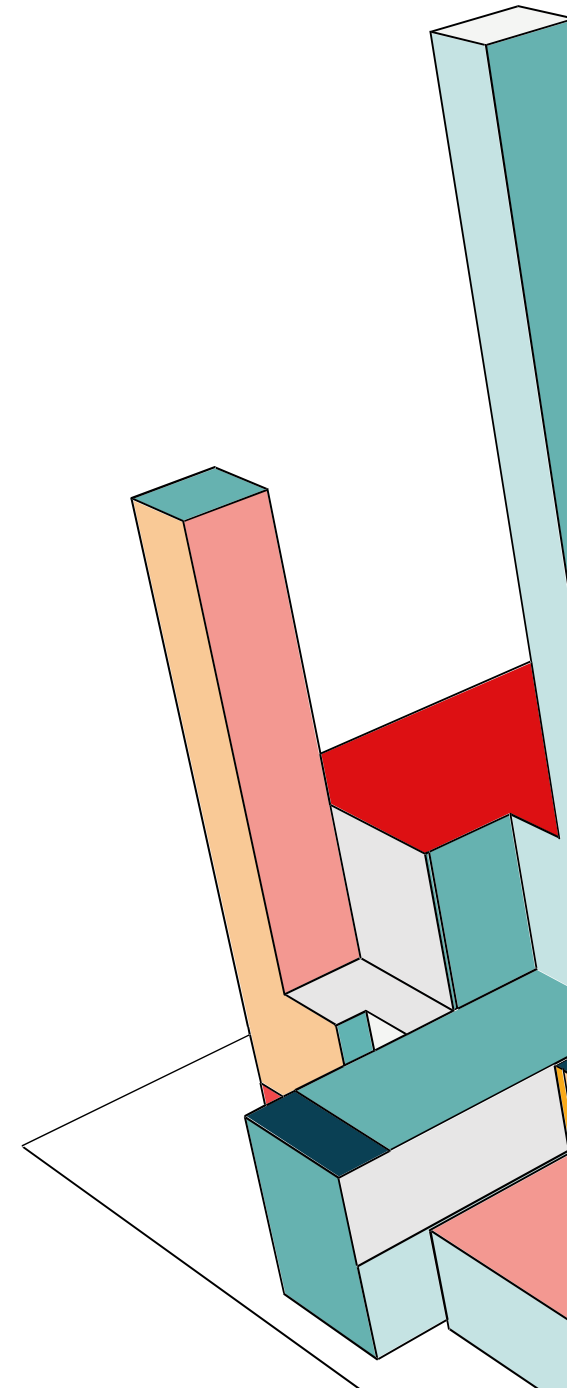
Sign in to your account

Email

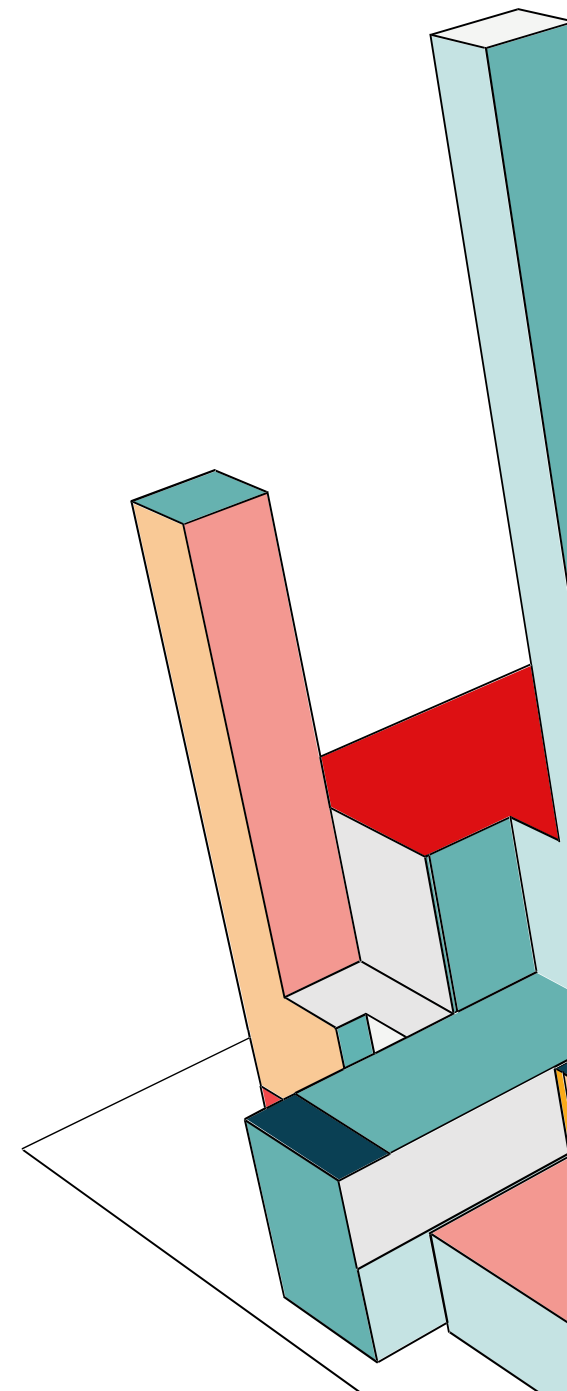
Password

Sign In



LOGIN

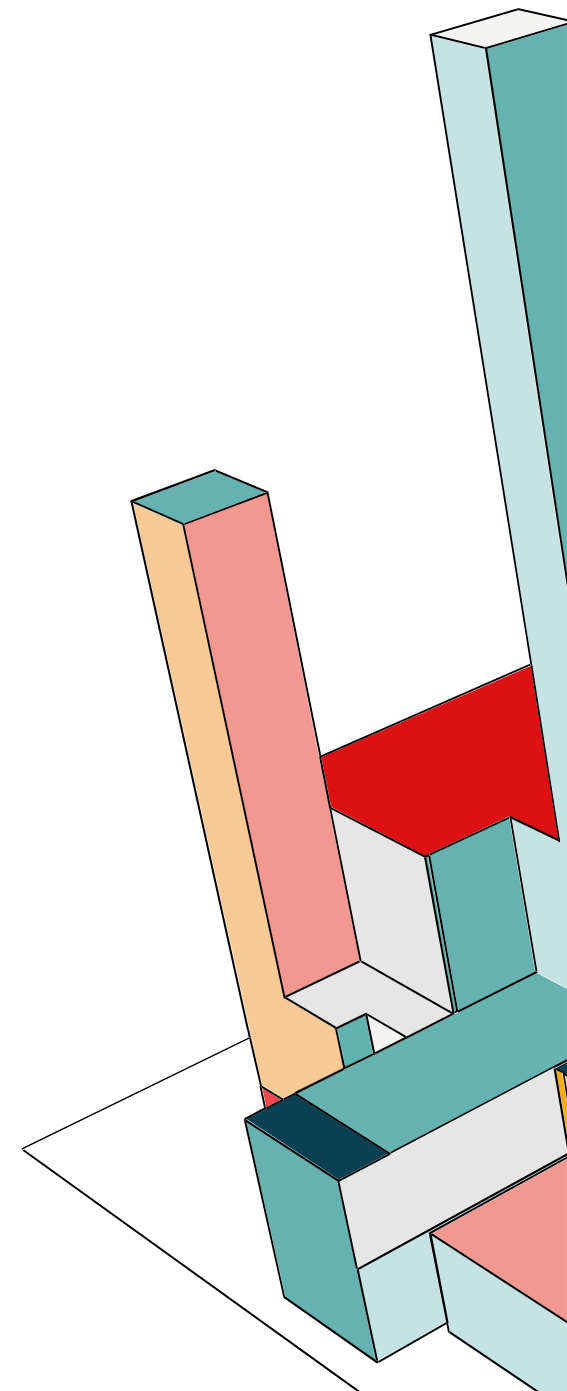


LOGOUT E LOGIN COME ADMIN

Area Amministratore

[← Torna alla Home](#)

Proprietario	Task	Stato	Azioni
mariorossi_user@test.com	Finire progetto di System Security Mancano pochi controlli	In Corso	Elimina



PROFILO TECNICO

Ciao, Luca

TECNICO

 Area Tecnica

Logout

Profilo Tecnico Rilevato

Utilizza il pulsante in alto per accedere agli strumenti di manutenzione.

CREAZIONE POLICY



Pannello Manutenzione Tecnico

[← Torna alla Home](#)

Stato Infrastruttura

Database:

OPERATIVO

Keycloak:

CONNESSO

API Backend:

ONLINE

Log di Sistema (Live)

```
[INFO] User lucamagenta_tech@test.com logged in
[INFO] Fetching todos for session...
[DEBUG] Token validation: SUCCESS
[WARN] High memory usage detected in backend...
```

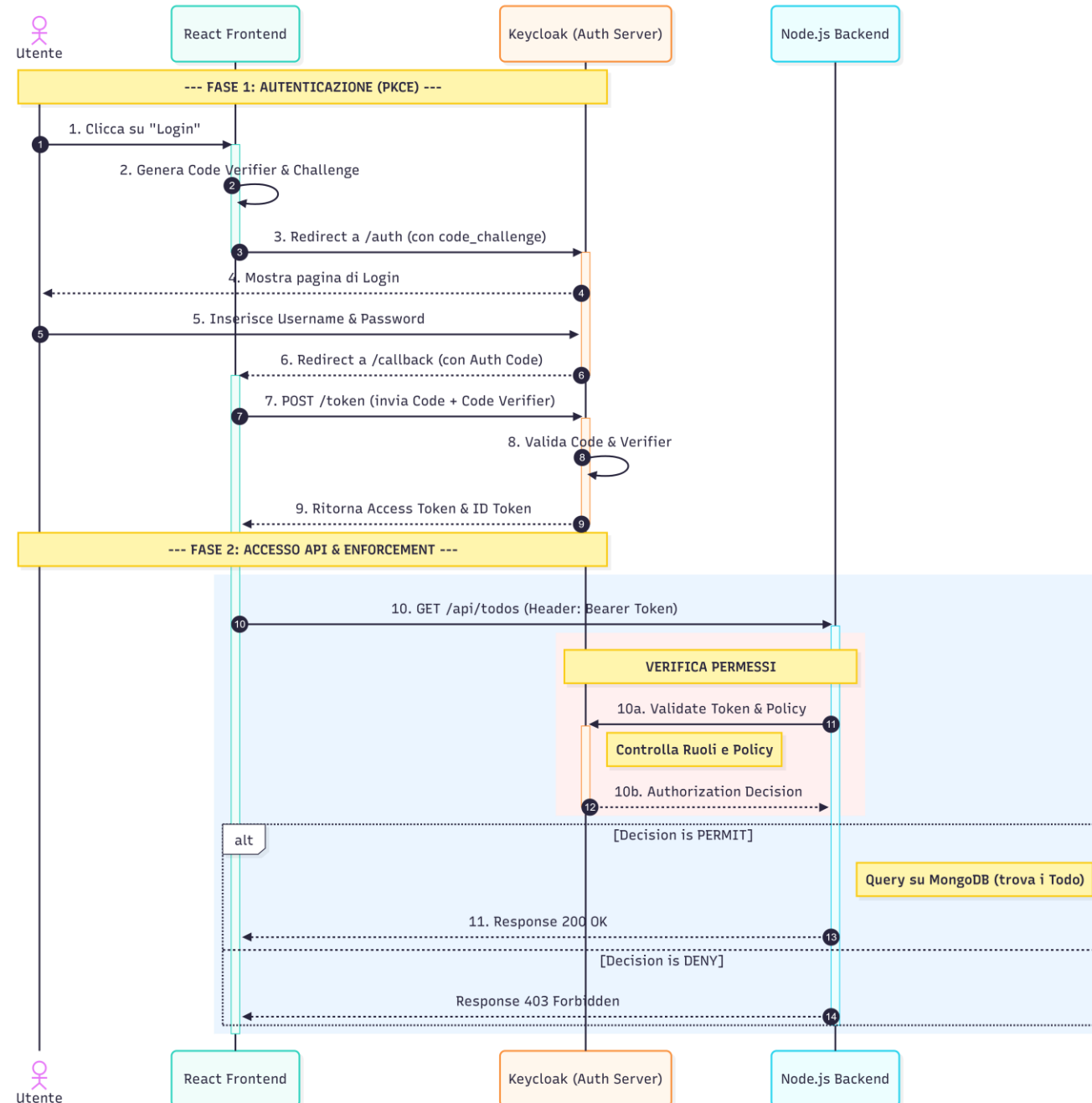

LOG IN SEQUENCE DIAGRAM

FASE 1: Authorization Code con PKCE, necessario perché il nostro frontend React è un client pubblico e quindi potenzialmente vulnerabile.

- L'utente clicca su Login. React genera Code Verifier e la sua versione hashata (Code Challenge).
- L'utente viene reindirizzato su Keycloak, si autentica, e Keycloak restituisce un Authorization Code temporaneo.
- React invia L'Authorization code più il Code Verifier a Keycloak, che verifica il Code Verifier. Se corrispondono, rilascia l'Access Token.

FASE 2: Accesso API & Policy Enforcement.

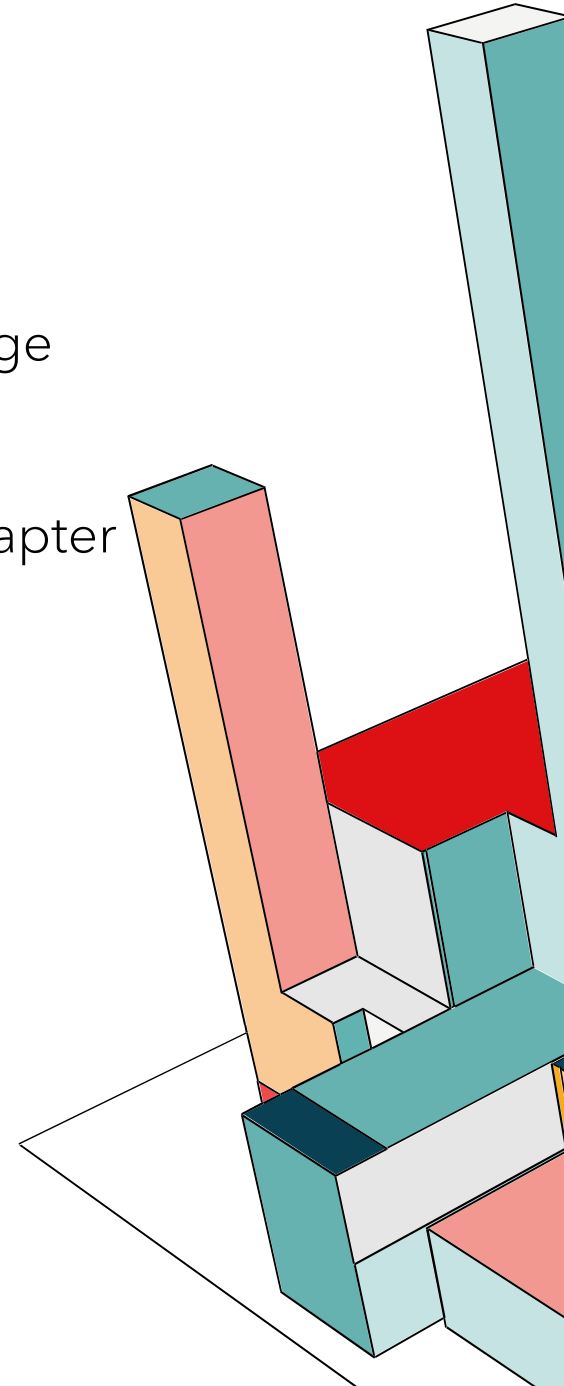
- Il Frontend chiama l'API (GET /api/todos) allegando il token nell'header Authorization.
- Il middleware del Backend (keycloak-connect) controlla i permessi dell'utente su quella risorsa.
- Keycloak valuta le Policy e risponde con PERMIT o DENY.



KEYCLOAK ADAPTER PER NODE.JS

Si tratta della libreria keycloak-connect, ovvero un componente software che funge da Middleware di Sicurezza. Si posiziona tra il server web (Express) e la logica di business (le rotte delle API). Il suo compito è nello specifico:

- Quando arriva una richiesta con un header Authorization: Bearer <token>, l'adapter intercetta quel token.
- Controlla la firma digitale del token usando la chiave pubblica di Keycloak.
- Estrae le informazioni dell'utente.
- Lo passiamo al Keycloak Enforcer, che deciderà se bloccare o meno questa richiesta.



KEYCLOAK ADAPTER PER NODE.JS

Importiamo la libreria

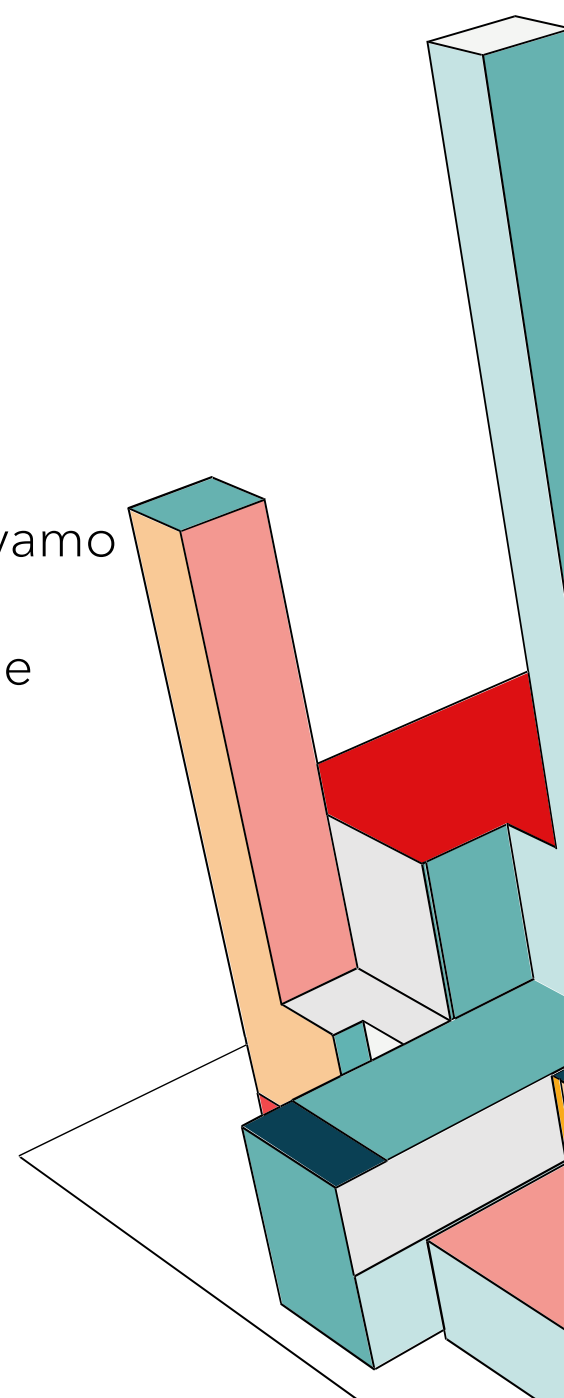
```
const Keycloak = require('keycloak-connect');
```

Dobbiamo istruire il framework Express a non ignorare quei metadati che avevamo configurato nel Reverse Proxy Apache. Con questa direttiva comunichiamo a Express che si trova dietro un Proxy sicuro e quindi può ispezionare e parsare le intestazioni X-Forwarded-* e salvarle nelle proprietà req.ip, req.protocol e req.hostname.

```
app.set('trust proxy', true);
```

L'adapter ha bisogno di un piccolo spazio di memoria per salvare le chiavi pubbliche di Keycloak

```
const memoryStore = new session.MemoryStore();  
app.use(session({  
  secret: process.env.SESSION_SECRET,  
  resave: false,  
  saveUninitialized: true,  
  store: memoryStore  
}));
```



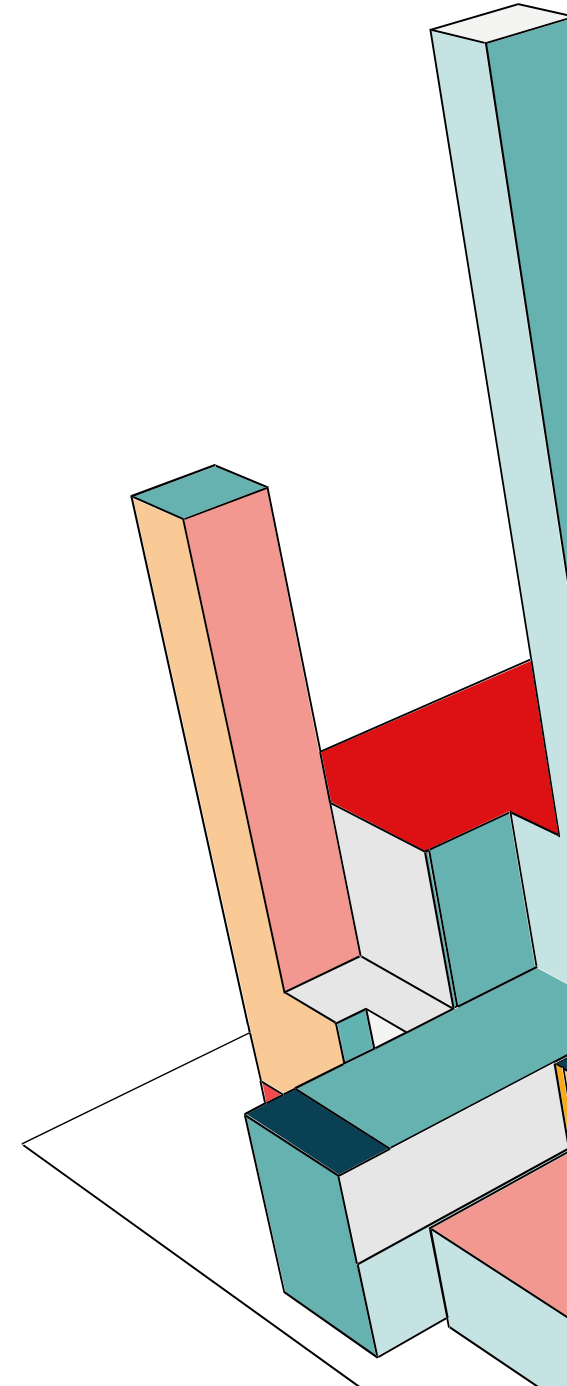
KEYCLOAK ADAPTER PER NODE.JS

Configurazione enforcer bearer only:

```
const kcConfig = {  
  "realm": "SySec",  
  "auth-server-url": "http://keycloak:8080/auth",  
  "ssl-required": "external",  
  "resource": "backend-node",  
  "public-client": false,  
  "confidential-port": 0,  
  "bearer-only": true,  
  "credentials": { "secret": "cA4senu3bvi4NBj6mtMXuOmyE8kBoWka" },  
  "policy-enforcer": { "mode": "enforcing", "lazy-load-paths": true }  
};
```

Qui creiamo fisicamente l'oggetto guardiano, collegandolo alla memoria e passandogli le istruzioni (kcConfig).

```
const keycloak = new Keycloak({ store: memoryStore }, kcConfig);
```



KEYCLOAK ADAPTER PER NODE.JS

```
app.use((req, res, next) => {
  const auth = req.headers.authorization;
  if (!auth) return next();

  const tokenStr = auth.split(' ')[1];

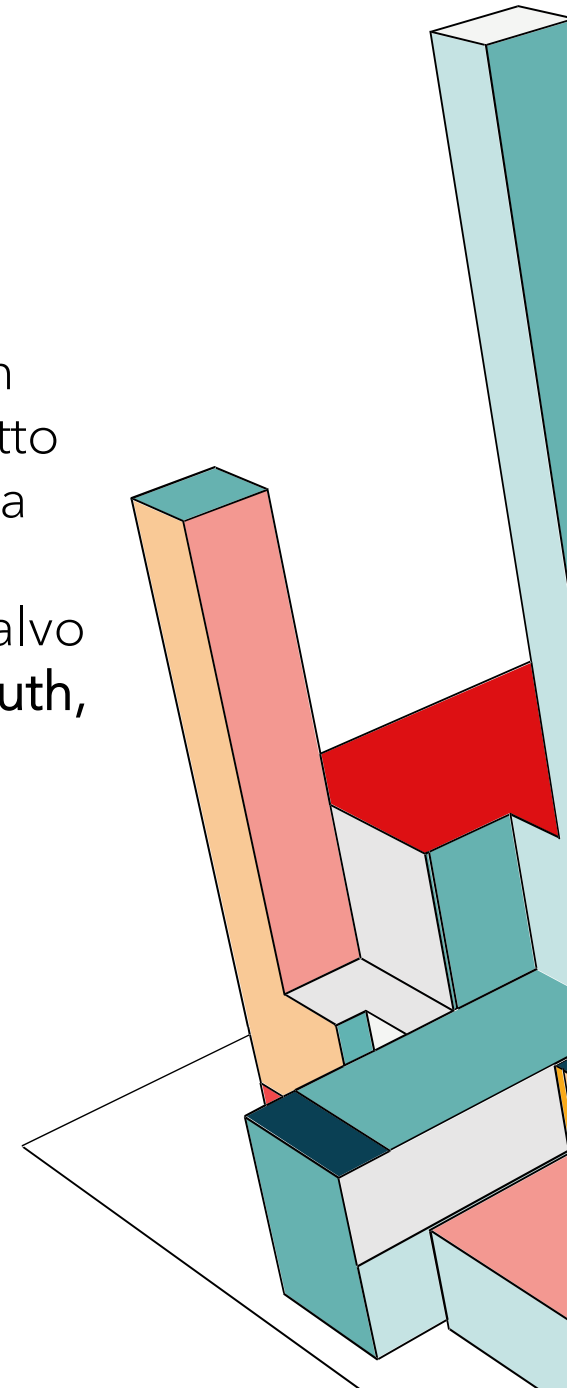
  try {
    const decoded = jwt.verify(tokenStr, PEM_KEY, { algorithms: ['RS256'] });

    req.user = decoded; //salviamo i dati dell'utente nella richiesta corrente
                        // per visualizzarlo nell'area dedicata all'utente
                        // e per il backend per capire a quale utente associare i to do
    const kToken = new Token(decoded, 'backend-node');
    kToken.token = tokenStr;

    req.kauth = { //
      grant: {
        access_token: kToken,
        toString: () => JSON.stringify(decoded)
      }
    };
  }
});
```

Verifichiamo la firma del token con `jwt.verify()`. Creo un oggetto `Token` in cui salviamo la stringa crittografata del payload del token e i dati dell'utente. Lo salvo nella richiesta corrente `req.kauth`, nel campo `access_token`.

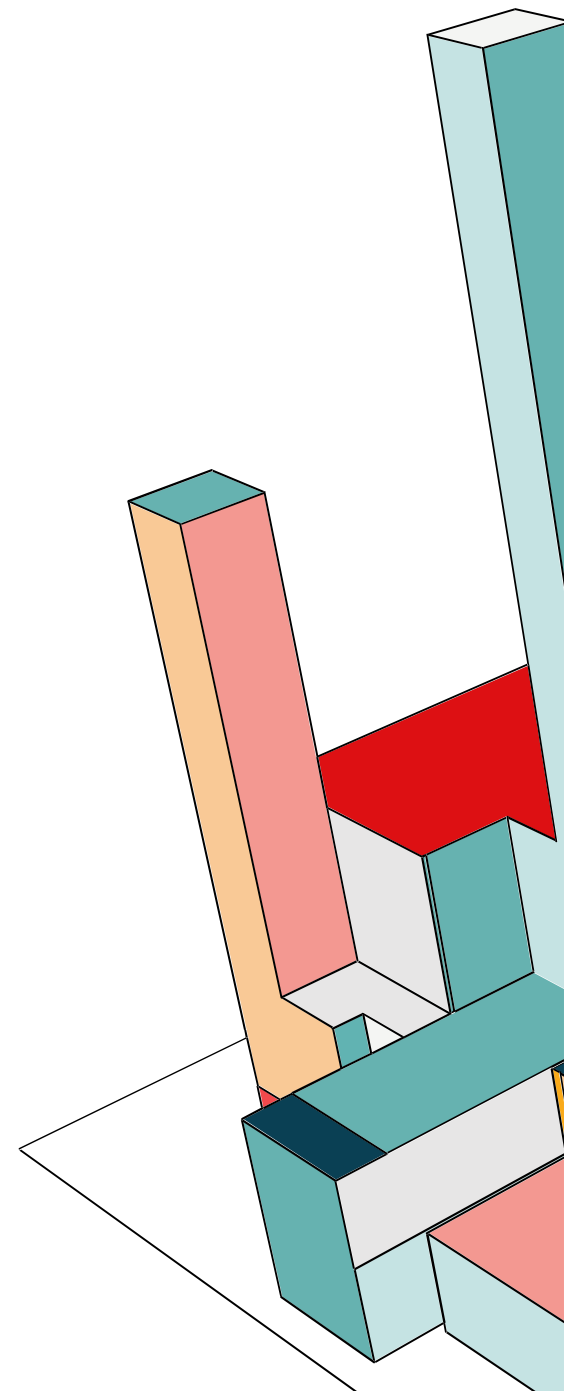
```
const todosRoutesFactory = require("../routes/todos");
const routes = todosRoutesFactory(keycloak);
app.use("/api/todos", routes);
```



KEYCLOAK ADAPTER PER NODE.JS

Settiamo le rotte

```
const todosRoutesFactory = require("../routes/todos");  
const routes = todosRoutesFactory(keycloak);  
app.use("/api/todos", routes);
```



KEYCLOAK ENFORCER: GET

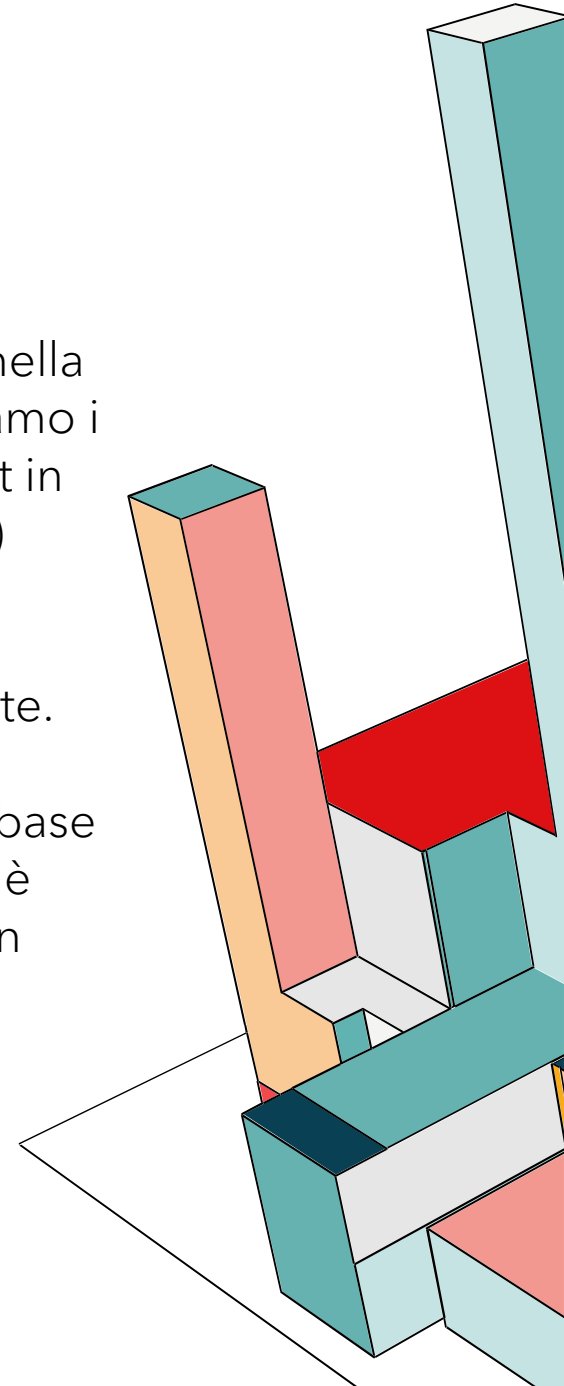
```
// GET
router.get("/", keycloak.enforcer(), async (req, res) => {
  console.log("--> [SUCCESS] GET approvato.");
  try {
    const currentUser = getUsername(req);

    // Recuperiamo i ruoli
    let roles = [];
    if (req.user && req.user.realm_access) {
      roles = req.user.realm_access.roles;
    } else if (req.kauth && req.kauth.grant && req.kauth.grant.access_token) {
      roles = req.kauth.grant.access_token.content.realm_access.roles;
    }

    const isAdmin = roles.includes('app-admin');

    let query = isAdmin ? {} : { owner: currentUser };
    const todos = await Todo.find(query);
    res.json(todos);
  } catch (err) {
    console.error("Errore GET:", err);
    res.status(500).json({ error: err.message });
  }
});
```

Carichiamo i dati dell'utente nella variabile **currentUser**. Carichiamo i ruoli dalla posizione di default in cui vengono inseriti (**req.user**) oppure, se non ci sono dall'oggetto **req.kauth** che abbiamo creato appositamente. Verifichiamo se l'utente è amministratore, se lo è il database restituisce tutti i to do, se non è admin allora filtriemo i to do in base a **currentUser**.



KEYCLOAK ENFORCER: POST

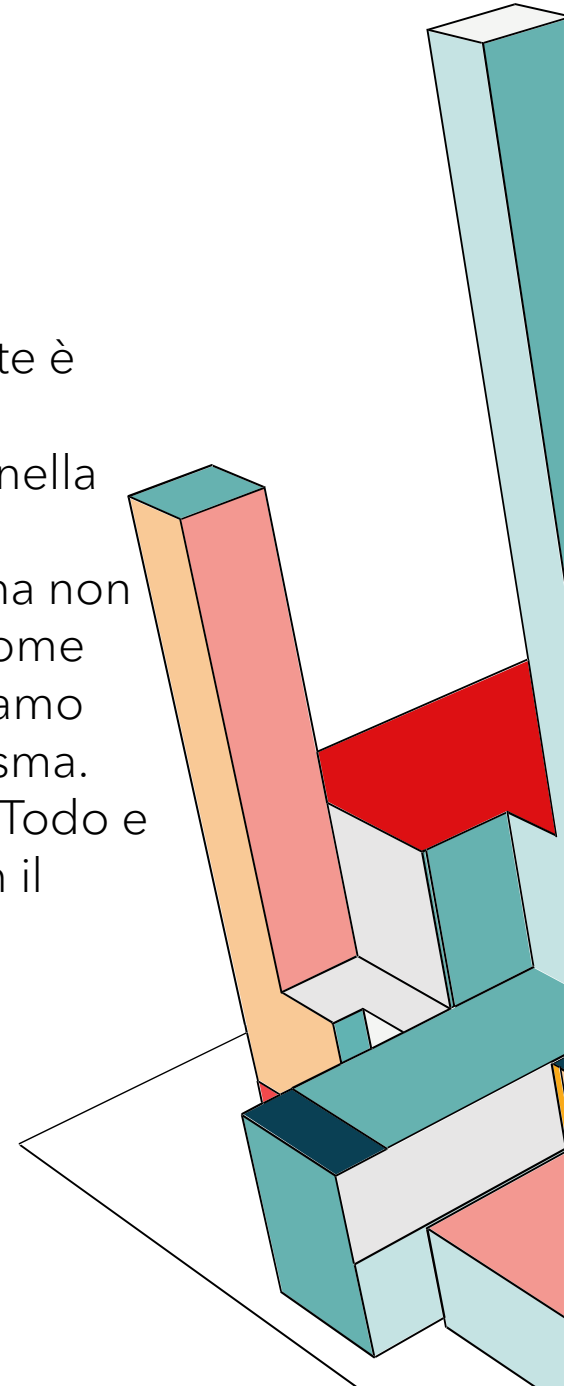
```
// POST
router.post("/", keycloak.enforcer(), async (req, res) => {
  console.log("--> [SUCCESS] POST approvato.");
  try {
    const currentUser = getUserFromToken(req);

    if (!currentUser) {
      console.error("ERRORE: Utente non identificato nel token");
      return res.status(400).json({ error: "Impossibile identificare l'utente (Owner mancante)" });
    }

    const newTodo = new Todo({
      title: req.body.title,
      description: req.body.description,
      due_date: req.body.due_date,
      completed: false,
      owner: currentUser
    });

    const savedTodo = await newTodo.save();
    res.status(201).json(savedTodo);
  } catch (err) {
    console.error("Errore POST:", err);
    res.status(500).json({ error: err.message });
  }
});
```

Controlliamo se l'utente è identificato, perché se ci troviamo nella situazione in cui il token è valido, ma non possiamo leggere il nome dell'utente, non vogliamo salvare un To do fantasma. Costruiamo un nuovo Todo e associamo l'owner con il currentUser.



KEYCLOAK ENFORCER: DELETE

```
// DELETE
router.delete("/:id", keycloak.enforcer(), async (req, res) => {
  console.log(`--> [SUCCESS] DELETE approvato.`);
  try {
    const todoToDelete = await Todo.findById(req.params.id);
    if (!todoToDelete) return res.status(404).json({ error: "Task non trovato" });

    const currentUser = getUserNames(req);

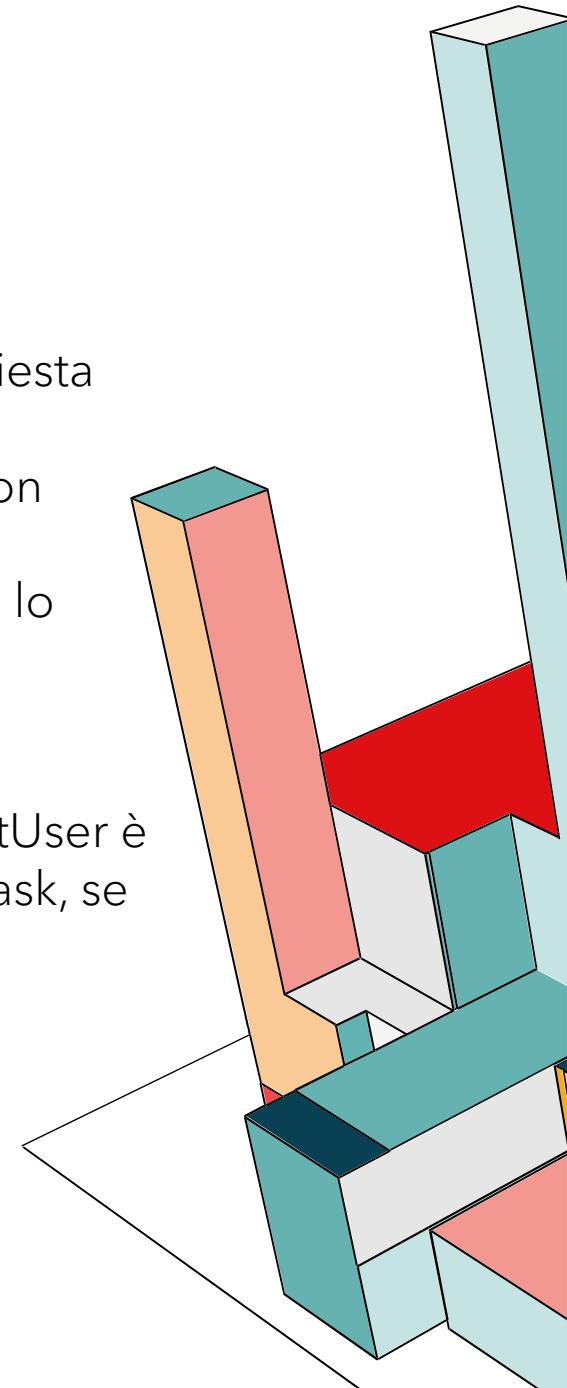
    let roles = [];
    if (req.user && req.user.realm_access) roles = req.user.realm_access.roles;

    const isAdmin = roles.includes('app-admin');
    const isOwner = todoToDelete.owner === currentUser;

    if (!isAdmin && !isOwner) return res.status(403).json({ error: "Non puoi eliminare task altrui." });

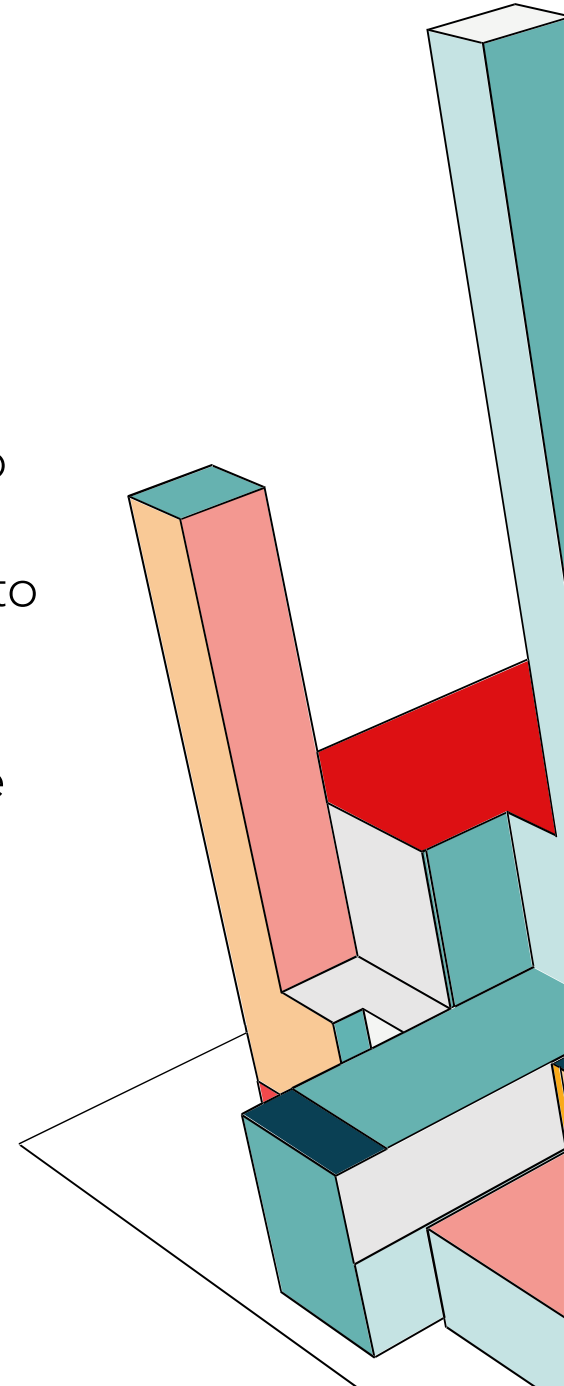
    await Todo.findByIdAndDelete(req.params.id);
    res.json({ message: "Task eliminato con successo" });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Prendiamo l'id dalla richiesta dall'URL della richiesta. Controlliamo se il task con quell'id è presente nel database, se lo troviamo lo salviamo nella variabile `todoToDelete` altrimenti restituisce null. Inoltre controlliamo se il `currentUser` è admin o il creatore del task, se non è nessuno dei due restituiamo errore.



MITIGAZIONE DEI RISCHI

Per la gestione dei rischi, abbiamo utilizzato IriusRisk non solo per identificare le minacce, ma per contestualizzarle. Abbiamo applicato il filtro normativo NIST 800-53, che ha permesso alla piattaforma di suggerirci automaticamente i controlli di sicurezza corretti per ogni minaccia. In questo modo, l'implementazione delle difese non è stata arbitraria, ma guidata direttamente dai requisiti dello standard. Per ogni minaccia inseriremo una descrizione, il controllo associato preso direttamente dal NIST SP 800-53 e una descrizione di come l'abbiamo mitigata.



THREAT: APACHE HTTP SERVER / DENIAL OF SERVICE

La minaccia rilevata da IriusRisk indica che il tuo server Apache, essendo il punto di ingresso (Gateway) della tua applicazione, è esposto al rischio di esaurimento risorse. Se un attaccante invia un numero massiccio di richieste (o richieste molto lente, come nell'attacco *Slowloris*), può saturare la capacità di Apache di aprire nuove connessioni. **Risultato:** Il server smette di rispondere agli utenti legittimi (Negazione del Servizio).

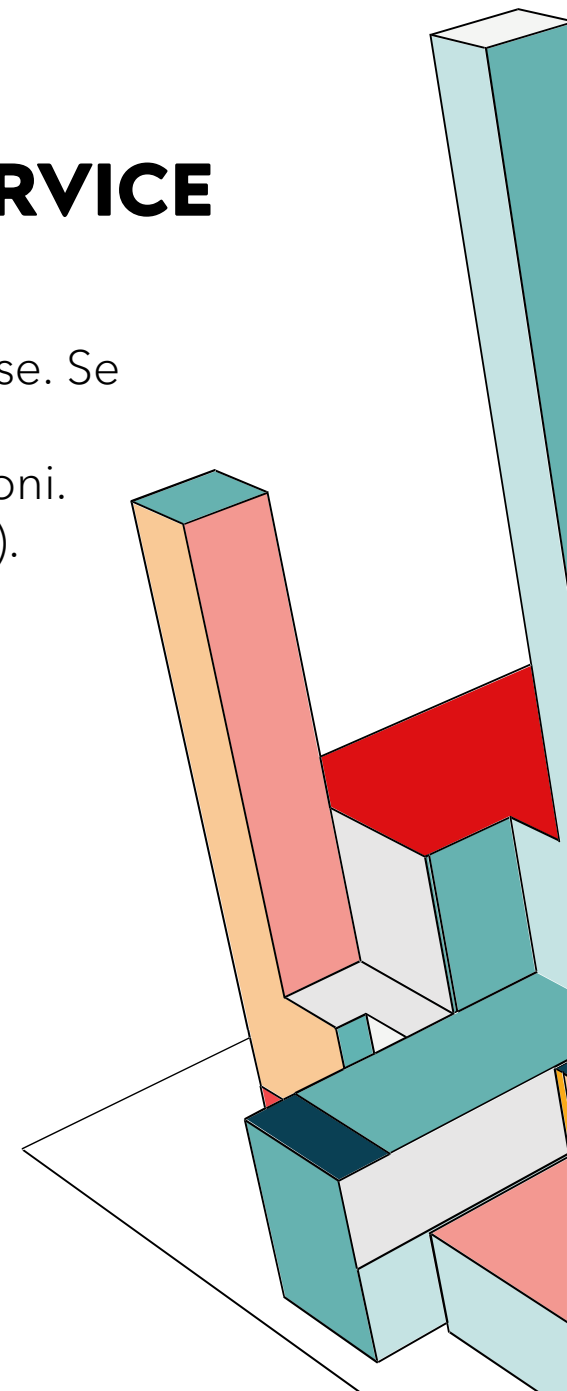
SC-5 DENIAL-OF-SERVICE PROTECTION

Control:

- [*Selection: Protect against; Limit*] the effects of the following types of denial-of-service events: [*Assignment: organization-defined types of denial-of-service events*]; and
- Employ the following controls to achieve the denial-of-service objective: [*Assignment: organization-defined controls by type of denial-of-service event*].

Discussion: Denial-of-service events may occur due to a variety of internal and external causes, such as an attack by an adversary or a lack of planning to support organizational needs with respect to capacity and bandwidth. Such attacks can occur across a wide range of network protocols (e.g., IPv4, IPv6). A variety of technologies are available to limit or eliminate the origination and effects of denial-of-service events. For example, boundary protection devices can filter certain types of packets to protect system components on internal networks from being directly affected by or the source of denial-of-service attacks. Employing increased network capacity and bandwidth combined with service redundancy also reduces the susceptibility to denial-of-service events.

Related Controls: [CP-2](#), [IR-4](#), [SC-6](#), [SC-7](#), [SC-40](#).



```
18  api:
19    build: ./backend
20    expose:
21      - "3001"
22    depends_on:
23      - mongo
24      - vault
25    deploy:
26      replicas: 3
27    cap_drop:
28      - ALL
29    environment:
30      - VAULT_ADDR=http://vault:8200
31      - VAULT_TOKEN=${VAULT_ROOT_TOKEN}
32    networks:
33      - network-backend
34    volumes:
35      - ./backend:/app
36      - /app/node_modules
```

MITIGAZIONE ANTI-DOS

Per soddisfare il requisito, abbiamo adottato una strategia basata su 3 pilastri fondamentali:

A. Architettura Ridondata

Invece di avere un singolo server backend che, se colpito, fa cadere tutto il sito, abbiamo usato Docker per creare un cluster. Quindi nel docker-compose.yml, abbiamo impostato replicas: 3 per il servizio API. In questo modo se un attacco DoS sovraccarica una replica del backend, le altre due sono ancora libere di rispondere. Abbiamo eliminato il *Single Point of Failure* (SPOF) a livello applicativo.

MITIGAZIONE ANTI-DOS

```
<Proxy "balancer://mycluster">
  BalancerMember "http://api:3001" retry=30 timeout=5
  ProxySet lbmethod=byrequests
</Proxy>
```

B. Load Balancer

Abbiamo configurato Apache non solo come proxy, ma come **bilanciatore di carico** usando il modulo `mod_proxy_balancer`. In questo modo se una delle 3 repliche va giù, Apache smette di mandarle traffico per 30 secondi, e le altre richieste vengono dirottate sulle repliche sane. Inoltre in questo modo preveniamo i blocchi perché grazie al `timeout = 5`, impediamo che Apache rimanga in attesa di un server che non risponde, liberando risorse per altri utenti.

```
<IfModule reqtimeout_module>
  RequestReadTimeout header=20-40,MinRate=500 body=20,MinRate=500
</IfModule>
```

C. Protezione Attiva (Slowloris Mitigation)

Oltre al bilanciamento, abbiamo attivato il modulo `mod_reqtimeout` per bloccare connessioni specificamente malevole (client lenti). Taglia proattivamente le connessioni che cercano di consumare risorse inviando dati troppo lentamente, una tecnica tipica degli attacchi DoS a basso volume.

MITIGAZIONE ANTI-DOS

Queste direttive servono a proteggere il server da richieste malformate o eccessivamente grandi.

- **LimitRequestBody:** Imposta la dimensione massima consentita per il corpo di una richiesta HTTP. Impedisce a un attaccante di intasare il server inviando file giganti o payload enormi.
- **LimitRequestFields:** Imposta il numero massimo di campi header HTTP in una singola richiesta.
- **LimitRequestFieldSize:** Imposta la dimensione massima (in byte) consentita per un singolo header HTTP.

```
LimitRequestBody 10485760  
LimitRequestFields 100  
LimitRequestFieldSize 8190
```


VERIFICA

Per verificare se le mitigazioni anti-dos hanno avuto effetto entriamo nel container di Apache mentre è in esecuzione, e stabiliamo una connessione sulla porta 443 (openssl s_client -connect localhost:443). A questo punto simuliamo un attacco slowloris, quindi non facciamo niente e dopo 20s vediamo che la connessione si chiude da sola.

```
PS C:\Users\Francesca\docker-frontend-backend-db> docker exec -it apache_secure sh
# openssl s_client -connect localhost:443
Connecting to 127.0.0.1
CONNECTED(00000003)
Can't use SSL_get_servername
depth=0 CN=localhost
verify error:num=18:self-signed certificate
verify return:1
depth=0 CN=localhost
verify return:1
---
Certificate chain
```

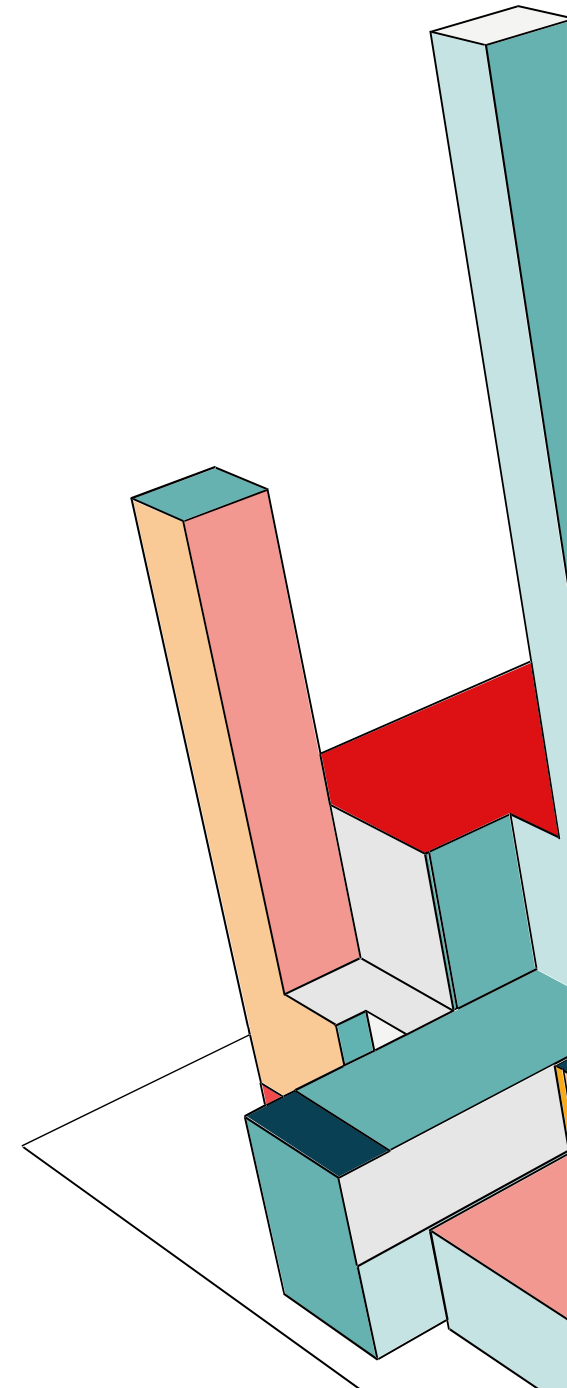
VERIFICA

```
Start Time: 1767519248  
Timeout   : 7200 (sec)  
Verify return code: 18 (self-signed certificate)  
Extended master secret: no  
Max Early Data: 0
```

```
read R BLOCK
```

```
closed
```

```
# |
```



THREAT: WEB APPLICATION - SERVER SIDE BACKEND / REPUDIATION

L'assenza di un sistema di registrazione degli eventi e di monitoraggio impedisce la rilevazione di attività sospette, tentativi di intrusione o guasti. Potenzialmente le violazioni dei dati potrebbero rimanere nascoste, sarebbe impossibile determinare con certezza l'impatto del danno o il vettore di attacco e attribuire azioni specifiche (es. accesso a dati sensibili) a utenti specifici. Irius Risk associa il seguente controllo a questa minaccia.

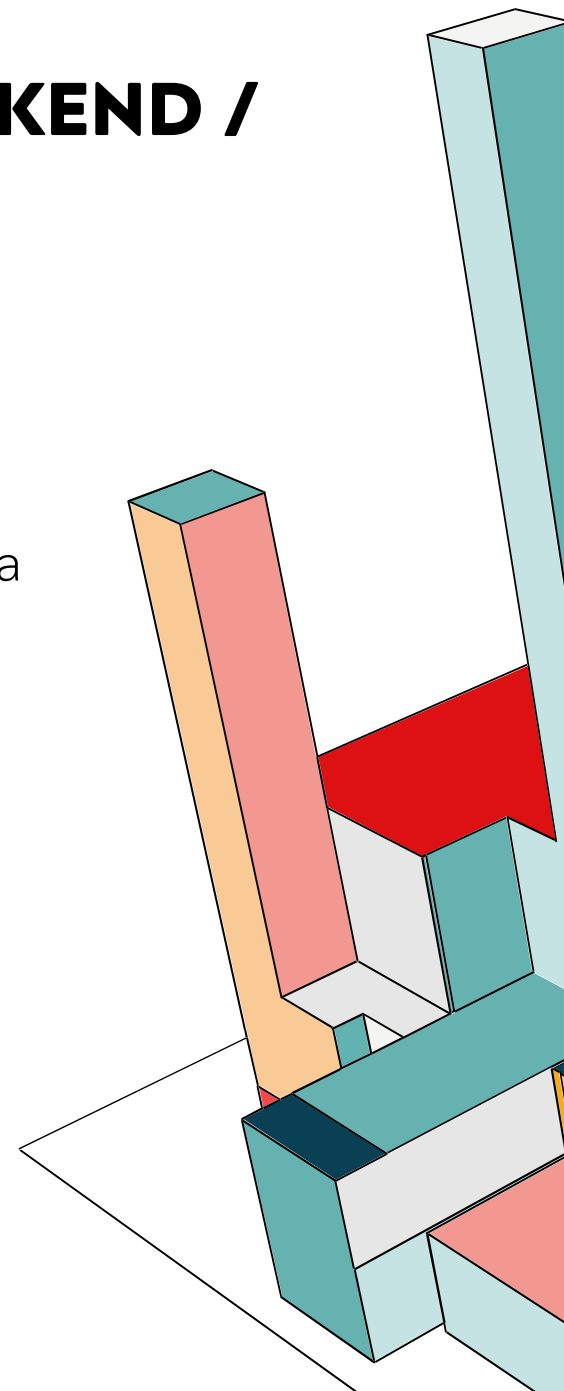
AU-12 AUDIT RECORD GENERATION

Control:

- a. Provide audit record generation capability for the event types the system is capable of auditing as defined in [AU-2a](#) on [Assignment: organization-defined system components];
- b. Allow [Assignment: organization-defined personnel or roles] to select the event types that are to be logged by specific components of the system; and
- c. Generate audit records for the event types defined in [AU-2c](#) that include the audit record content defined in [AU-3](#).

Discussion: Audit records can be generated from many different system components. The event types specified in [AU-2d](#) are the event types for which audit logs are to be generated and are a subset of all event types for which the system can generate audit records.

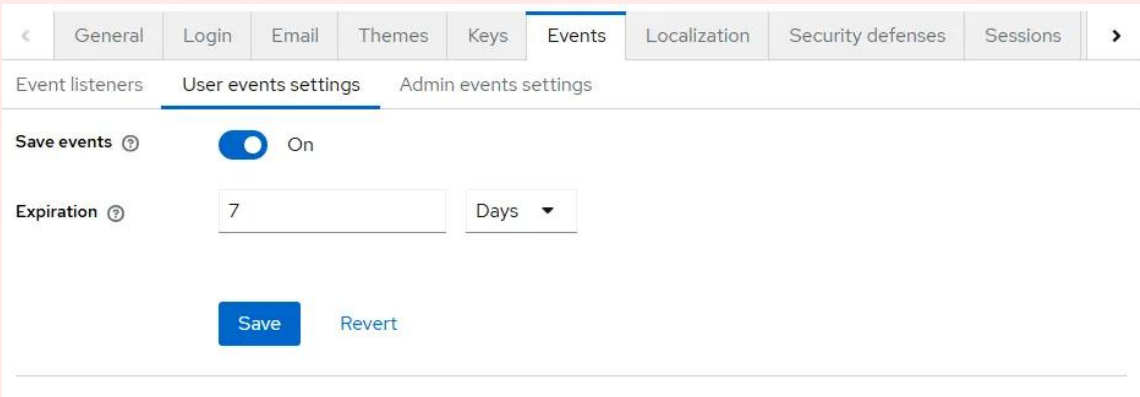
181 Related Controls: [AC-6](#), [AC-17](#), [AU-2](#), [AU-3](#), [AU-4](#), [AU-5](#), [AU-6](#), [AU-7](#), [AU-14](#), [CM-5](#), [MA-4](#), [MP-4](#), [PM-12](#), [SA-8](#), [SC-18](#), [SI-3](#), [SI-4](#), [SI-7](#), [SI-10](#).



MITIGAZIONE LOGGING SU KEYCLOAK

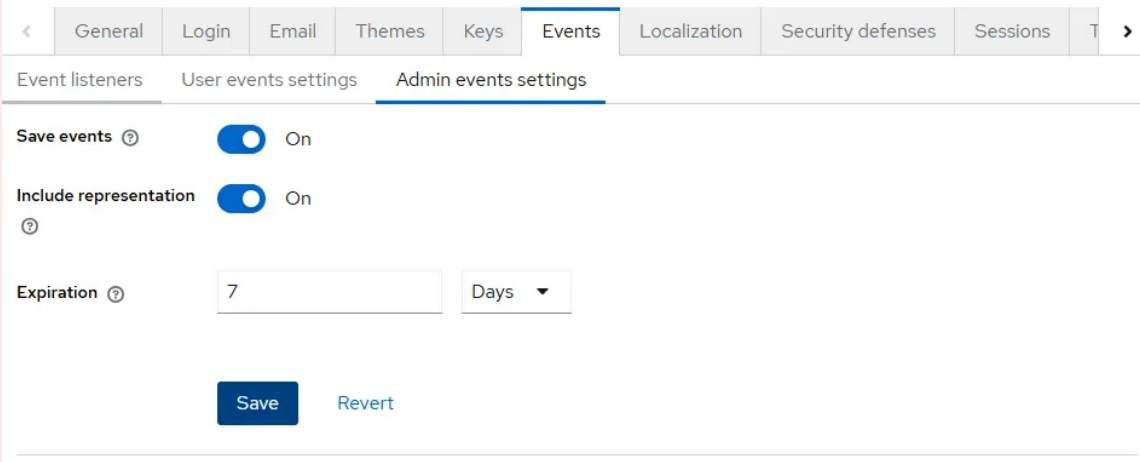
Abbiamo abilitato due livelli di auditing:

- **User Events:** Ogni volta che un utente fa login o scambia un codice per un token, Keycloak registra l'evento, l'indirizzo IP, il Client ID e il Timestamp. Questi log sono mantenuti con persistenza nel database PostgreSQL.



The screenshot shows the 'Events' tab in the Keycloak Admin Console. The 'User events settings' sub-tab is active. The 'Save events' toggle is turned 'On'. The 'Expiration' is set to 7 days. There are 'Save' and 'Revert' buttons at the bottom.

Tab	Save events	Expiration
User events settings	On	7 Days



The screenshot shows the 'Events' tab in the Keycloak Admin Console. The 'Admin events settings' sub-tab is active. The 'Save events' toggle is turned 'On'. The 'Include representation' toggle is also turned 'On'. The 'Expiration' is set to 7 days. There are 'Save' and 'Revert' buttons at the bottom.

Tab	Save events	Include representation	Expiration
Admin events settings	On	On	7 Days

- **Admin Events:** Abbiamo attivato l'opzione 'Include Representation', in questo modo se un amministratore cambia un ruolo o una policy, il sistema salva uno snapshot JSON dell'oggetto modificato. . Questi dati sono visibili direttamente dalla admin console di Keycloak.

VISUALIZZAZIONE EVENTI

Events

Events are records of user and admin events in this realm. To configure the tracking of these events, go to [Event configs](#). [Learn more](#)

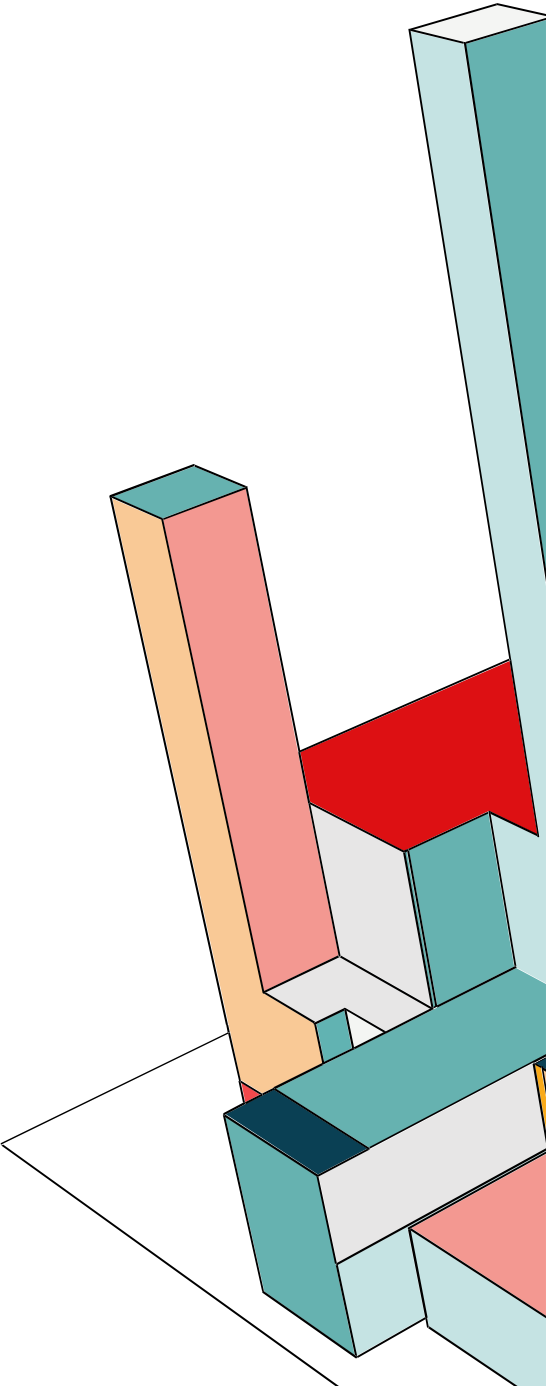
- User events
- Admin events

Search events

Refresh

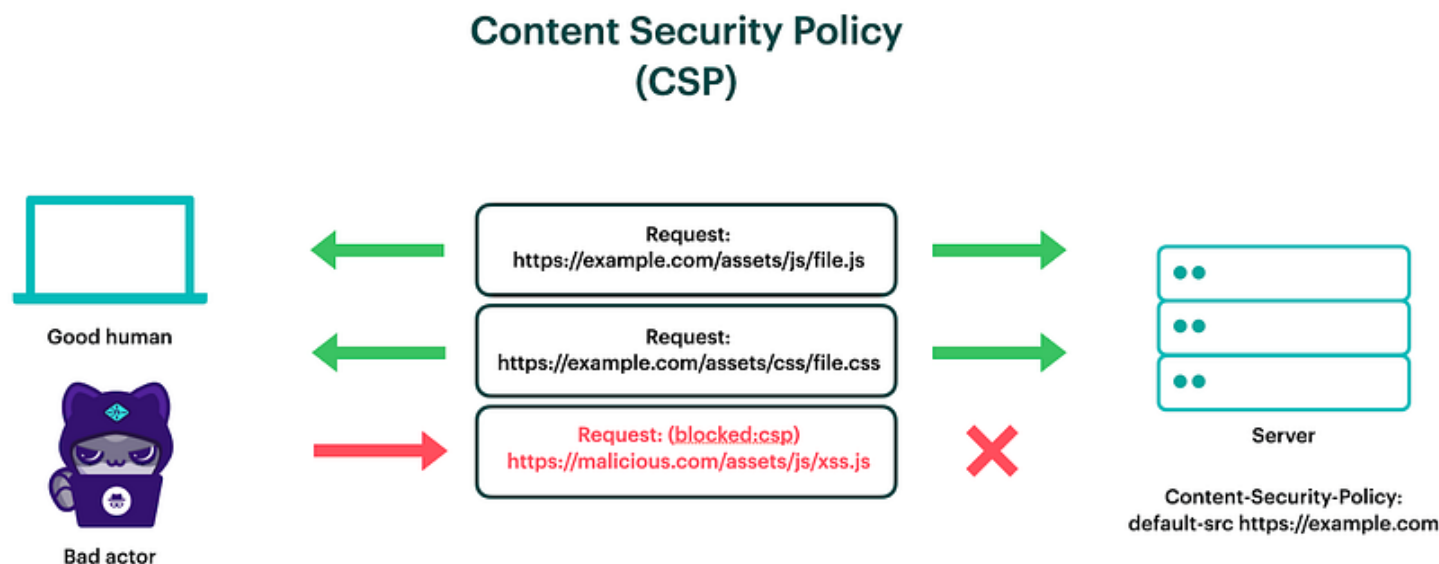
1 - 4

Time	User ID	Event saved type	IP address	Client
> December 30, 2025 at 11:37 AM	6102a488-772e-4a86-b789-8aabbfd8283	✔CODE_TO_TOKEN	172.18.0.1	security-admin-console
< December 30, 2025 at 11:37 AM	6102a488-772e-4a86-b789-8aabbfd8283	✔LOGIN	172.18.0.1	security-admin-console
auth_method openid-connect auth_type code response_type code redirect_uri https://localhost/auth/admin/master/console/#/master/events/user-events consent no_consent_required code_id 04e93b28-d02b-9f7e-038d-18f95f7fe7fb response_mode query username fran_b				
> December 30, 2025 at 11:36 AM	6102a488-772e-4a86-b789-8aabbfd8283	✔CODE_TO_TOKEN	172.18.0.1	security-admin-console
< December 30, 2025 at 11:36 AM	6102a488-772e-4a86-b789-8aabbfd8283	✔LOGIN	172.18.0.1	security-admin-console
auth_method openid-connect auth_type code				



THREAT: REACT.JS WEB FRONTEND / ELEVATION OF PRIVILEGE

Le minacce principali mitigate sono: Cross-Site Scripting (XSS), Data injection e Clickjacking. In un'applicazione moderna basata su JavaScript (come React), il browser esegue codice client-side. Se un attaccante riesce a iniettare uno script malevolo all'interno di una pagina visitata dalla vittima, quel codice viene eseguito nel contesto di sicurezza dell'utente fidato.



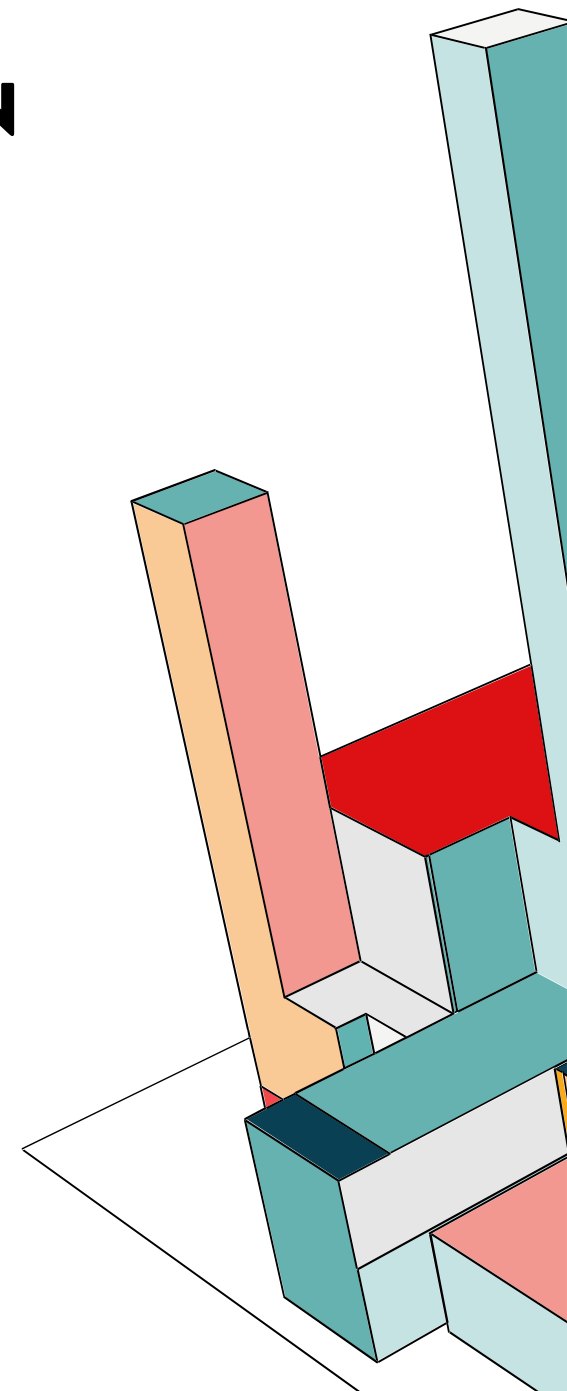
THREAT: REACT.JS WEB FRONTEND / ELEVATION OF PRIVILEGE

CM-6 CONFIGURATION SETTINGS

Control:

- a. Establish and document configuration settings for components employed within the system that reflect the most restrictive mode consistent with operational requirements using [Assignment: organization-defined common secure configurations];
- b. Implement the configuration settings;
- c. Identify, document, and approve any deviations from established configuration settings for [Assignment: organization-defined system components] based on [Assignment: organization-defined operational requirements]; and
- d. Monitor and control changes to the configuration settings in accordance with organizational policies and procedures.

Related Controls: [AC-3](#), [AC-19](#), [AU-2](#), [AU-6](#), [CA-9](#), [CM-2](#), [CM-3](#), [CM-5](#), [CM-7](#), [CM-11](#), [CP-7](#), [CP-9](#), [CP-10](#), [IA-3](#), [IA-5](#), [PL-8](#), [PL-9](#), [RA-5](#), [SA-4](#), [SA-5](#), [SA-8](#), [SA-9](#), [SC-18](#), [SC-28](#), [SC-43](#), [SI-2](#), [SI-4](#), [SI-6](#).



MITIGAZIONE CONFIGURAZIONE CSP

La policy è stata applicata a livello di **Apache Reverse Proxy**, poiché Apache è l'unico punto di ingresso e uscita per il traffico esterno alla rete Docker, quindi configurare gli header qui garantisce una **protezione centralizzata**. Questo copre sia il frontend React che le pagine fornite da Keycloak, e segue un approccio basato su whitelist.

- default-src 'self': impedisce il caricamento di qualsiasi risorsa da domini diversi da quello di origine (https://localhost).
- connect-src 'self': definisce a quale URL l'applicazione si può connettere
- script-src 'self': definisce quale sorgente può caricare codice (se stessa). Qui abbiamo dovuto abilitare anche 'unsafe-inline' e 'unsafe-eval' perché Keycloak utilizza scripti inline nelle pagine di login e funzioni di valutazione dinamica.
- img-src 'self': specifica chi è abilitato a caricare immagini.
- frame-src 'self': permette all'applicazione di caricare se stessa in un frame.
- frame-ancestorse 'self': impedisce che altri siti possano inglobare la nostra applicazione in un iframe.

```
Header always set Content-Security-Policy "default-src 'self';\nscript-src 'self' 'unsafe-inline' 'unsafe-eval';\nstyle-src 'self' 'unsafe-inline';\nconnect-src 'self' https://localhost:8443 wss://localhost;\nimg-src 'self' data:;\nfont-src 'self' data:;\nframe-src 'self' https://localhost:8443;\nframe-ancestors 'self' https://localhost:8443;\nobject-src 'none';"
```


MITIGAZIONE HARDENING AGGIUNTIVO

Oltre alla CSP, abbiamo implementato header specifici per chiudere vulnerabilità storiche dei browser:

- **X-Frame-Options: DENY:** può essere utilizzato per indicare se al browser è consentito renderizzare una pagina all'interno di tag <frame>, <iframe>, <embed> o <object>. I siti web possono utilizzare questo strumento per prevenire attacchi di clickjacking, assicurandosi che i propri contenuti non vengano incorporati in altri siti. Nel nostro caso la pagina non può essere visualizzata in un frame, a prescindere dal sito che tenta di farlo.
- **X-Content-Type-Options: nosniff:** Un attaccante potrebbe caricare sul server un file malevolo contenente script JavaScript, ma camuffarlo per esempio con un'estensione .png. Senza protezione, se un utente visita quel file, il browser potrebbe analizzare i primi byte, vedere qualche tag Java o TML ed eseguirlo come codice. Invece con questo filtro il server tratterà quel file esclusivamente come un'immagine.
- **Referrer-Policy: strict-origin-when-cross-origin:** quando un utente clicca su un link esterno, l'URL di provenienza viene inviato al sito di destinazione. Questa policy assicura che, uscendo dal nostro dominio, venga inviato solo il nome del dominio e non il percorso completo che potrebbe contenere informazioni sensibili nei parametri.

```
Header always set X-Frame-Options "DENY"  
Header always set X-Content-Type-Options "nosniff"  
Header always set Referrer-Policy "strict-origin-when-cross-origin"
```

VERIFICA

Possiamo verificare se la CSP è scritta in modo corretto oppure se ci esponga a delle vulnerabilità.

Content Security Policy

[Sample unsafe policy](#)[Sample safe policy](#)

```
default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval'; style-src 'self'
'unsafe-inline'; connect-src 'self' https://localhost:8443 wss://localhost; img-src
'self' data:; font-src 'self' data:; frame-src 'self' https://localhost:8443;
frame-ancestors 'self' https://localhost:8443;
```

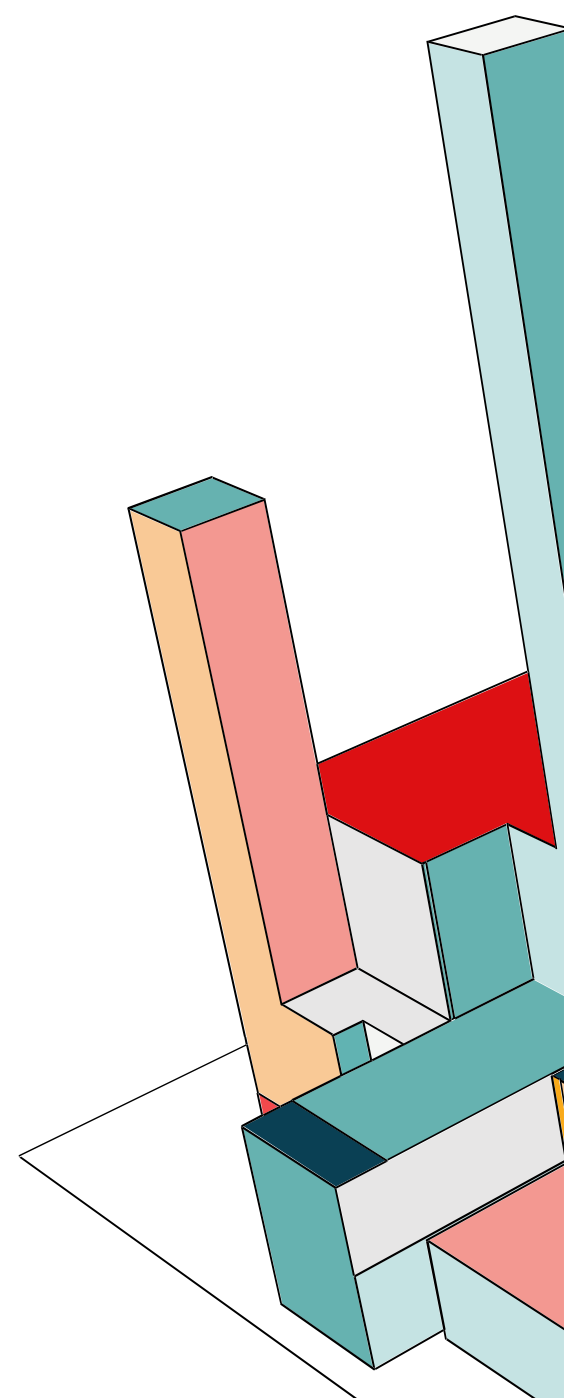
CSP Version 3 (nonce based + backward compatibility checks) ▼ ⓘ

CHECK CSP

Evaluated CSP as seen by a browser supporting CSP Version 3

[expand/collapse all](#)

✓ default-src	▼
❗ script-src	▲
❓ 'self'	'self' can be problematic if you host JSONP, AngularJS or user uploaded files.
❗ 'unsafe-inline'	'unsafe-inline' allows the execution of unsafe in-page scripts and event handlers.
❓ 'unsafe-eval'	'unsafe-eval' allows the execution of code injected into DOM APIs such as eval().
✓ style-src	▼
✓ connect-src	▼
✓ img-src	▼
✓ font-src	▼
✓ frame-src	▼
✓ frame-ancestors	▼



THREAT: MONGODB NOSQL / INFORMATION DISCLOSURE

All'interno della rete Docker la comunicazione tra Backend e il Database avviene tramite protocollo TCP, quindi senza crittografia. Se un attaccante ottenesse l'accesso alla nostra rete interna potrebbe fare packet sniffing e leggere il flusso di rete (username e password del database, dettagli dei To do). Questo comprometterebbe la Confidenzialità e l'Integrità del nostro sistema.

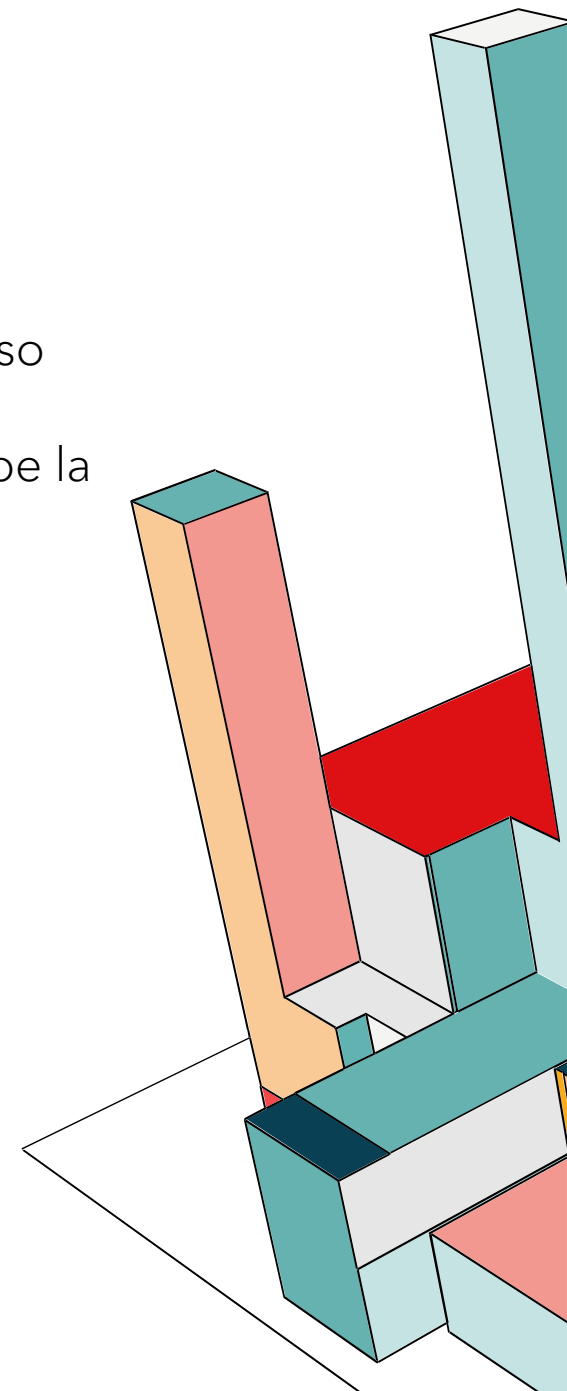
SC-28 PROTECTION OF INFORMATION AT REST

Control: Protect the [Selection (one or more): confidentiality; integrity] of the following information at rest: [Assignment: organization-defined information at rest].

Discussion: Information at rest refers to the state of information when it is not in process or in transit and is located on system components. Such components include internal or external hard disk drives, storage area network devices, or databases. However, the focus of protecting information at rest is not on the type of storage device or frequency of access but rather on the state of the information. Information at rest addresses the confidentiality and integrity of

information and covers user information and system information. System-related information that requires protection includes configurations or rule sets for firewalls, intrusion detection and prevention systems, filtering routers, and authentication information. Organizations may employ different mechanisms to achieve confidentiality and integrity protections, including the use of cryptographic mechanisms and file share scanning. Integrity protection can be achieved, for example, by implementing write-once-read-many (WORM) technologies. When adequate protection of information at rest cannot otherwise be achieved, organizations may employ other controls, including frequent scanning to identify malicious code at rest and secure offline storage in lieu of online storage.

Related Controls: [AC-3](#), [AC-4](#), [AC-6](#), [AC-19](#), [CA-7](#), [CM-3](#), [CM-5](#), [CM-6](#), [CP-9](#), [MP-4](#), [MP-5](#), [PE-3](#), [SC-8](#), [SC-12](#), [SC-13](#), [SC-34](#), [SI-3](#), [SI-7](#), [SI-16](#).



MITIGAZIONE TLS

Abbiamo implementato la crittografia Transport Layer Security (TLS). Dal lato database nel `docker-compose.yml`, abbiamo riconfigurato il servizio mongo per rifiutare connessioni non sicure.

- **--tlsMode requireTLS**: Questa è la direttiva che configura il database in una modalità "Deny All Non-Encrypted". Se un client tenta di connettersi senza avviare un handshake TLS, il server MongoDB chiude la connessione.
- **/etc/ssl/mongodb.pem**: Abbiamo generato un certificato X.509 tramite Vault che funge da identità digitale del server database. Il file è montato nel container in modalità read-only (`:ro`). In questo modo impediamo che, anche in caso di compromissione del traffico Mongo, il certificato possa essere alterato o cancellato.

```
mongo:
  image: mongo
  restart: always
  environment:
    MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USER}
    MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASS}
  command: [
    "--tlsMode", "requireTLS",
    "--tlsCertificateKeyFile", "/etc/ssl/mongodb.pem",
    "--tlsCAFile", "/etc/ssl/mongodb.pem",
    "--tlsAllowConnectionsWithoutCertificates"
  ]
  volumes:
    - mongodb_data:/data/db
    - ./mongodb-certs/mongodb.pem:/etc/ssl/mongodb.pem:ro
  networks:
    - network-backend
```

MITIGAZIONE TLS

Abbiamo aggiornato il lato client nel codice server.js: **tls=true** forza il driver Node.js a incapsulare la connessione TCP standard all'interno di un tunnel TLS cifrato in questo modo anche se i pacchetti venissero intercettati non sarebbero comprensibili, **tlsAllowInvalidCertificates=true** questa flag istruisce il client a **procedere con la crittografia** ignorando l'errore di validazione dell'autorità emittente. Questo lo dobbiamo fare perché siamo in ambiente di sviluppo e abbiamo un certificato emesso dal Vault che è la nostra PKI, ed è self-signed, ma in produzione si dovrebbe validare la cateni di certificati.

```
async function main() {
  try {
    const vaultRes = await vault.read("secret/data/db");
    const { username, password } = vaultRes.data.data;

    const mongoUri = `mongodb://${username}:${password}@mongo:27017/todos?tls=true&tlsAllowInvalidCertificates=true&authSource=admin`;

    await mongoose.connect(mongoUri, {
      useUnifiedTopology: true,
      useNewUrlParser: true,
    });
    console.log("Connesso a MongoDB con TLS e credenziali da Vault!");
  } catch (err) {
    console.error("Errore critico nella configurazione iniziale:", err);
    process.exit(1);
  }
}
```

AC-1: POLICY AND PROCEDURES

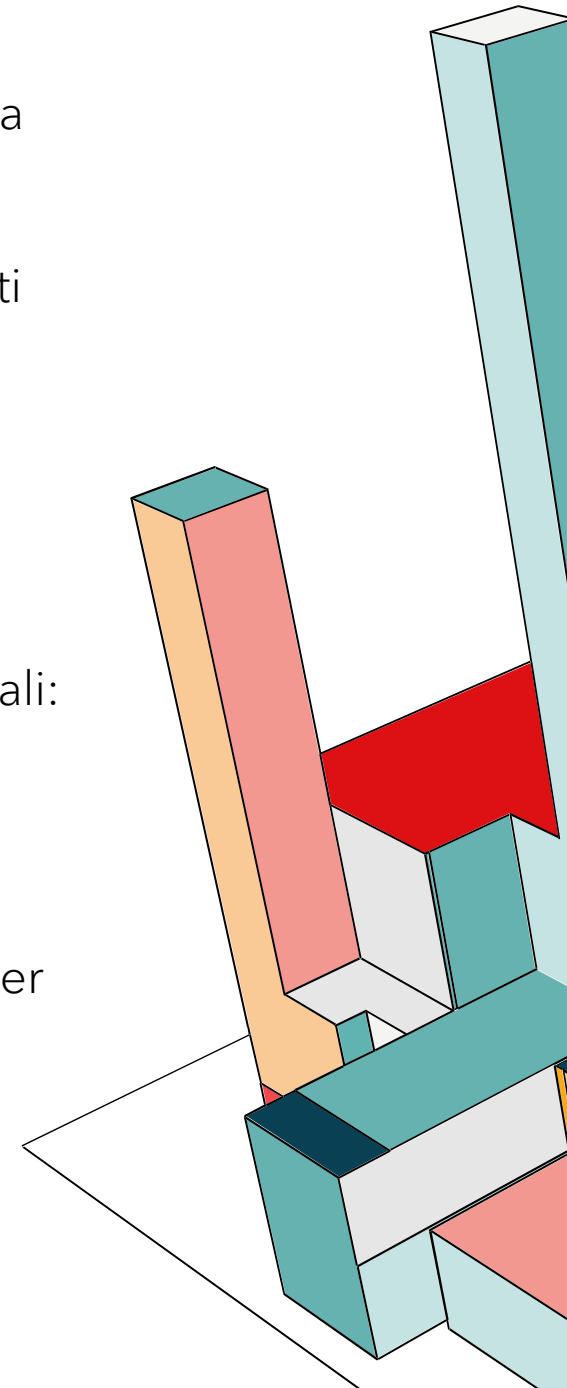
Per soddisfare il requisito, nel progetto si adotta una **politica di controllo degli accessi** basata sul modello **RBAC** e sul principio del privilegio minimo.

1. Policy: L'accesso alle risorse è consentito esclusivamente previa autenticazione e sono stati definiti tre ruoli:

- **Impiegato (User):** Accesso in lettura/scrittura limitato ai propri dati personali
- **Amministratore (Admin):** Accesso gestionale globale sui dati di tutti gli utenti
- **Tecnico (Tech):** Accesso limitato al pannello di monitoraggio infrastrutturale

2. Procedure: L'applicazione di queste regole è delegata ai seguenti componenti architetturali:

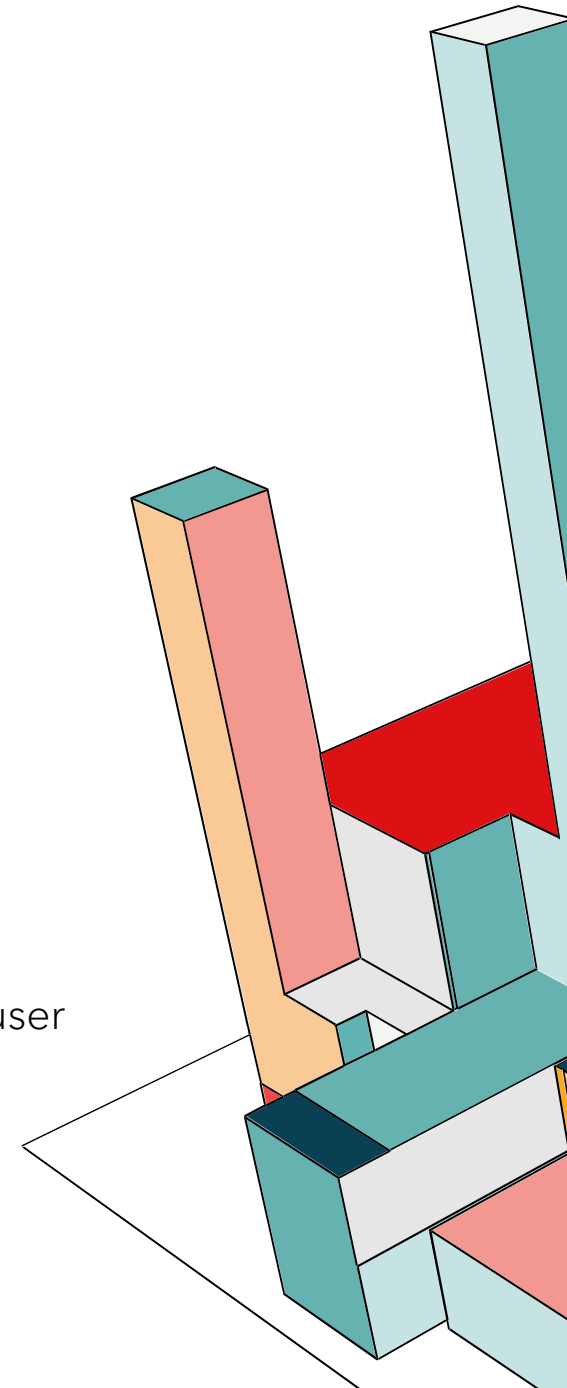
- **Keycloak (Identity Provider):** Gestione centralizzata di utenti, ruoli ed emissione dei token di accesso (JWT)
- **Frontend (React):** Rendering condizionale dell'interfaccia (mostra/nasconde menu) basato sui ruoli presenti nel token
- **Backend (Node.js):** Verifica crittografica del token JWT a ogni singola richiesta API, per impedisce accessi diretti non autorizzati



AC-2: ACCOUNT MANAGEMENT

La gestione delle identità è centralizzata utilizzando **Keycloak** e questo garantisce un punto unico di controllo per creare, modificare o rimuovere gli accessi.

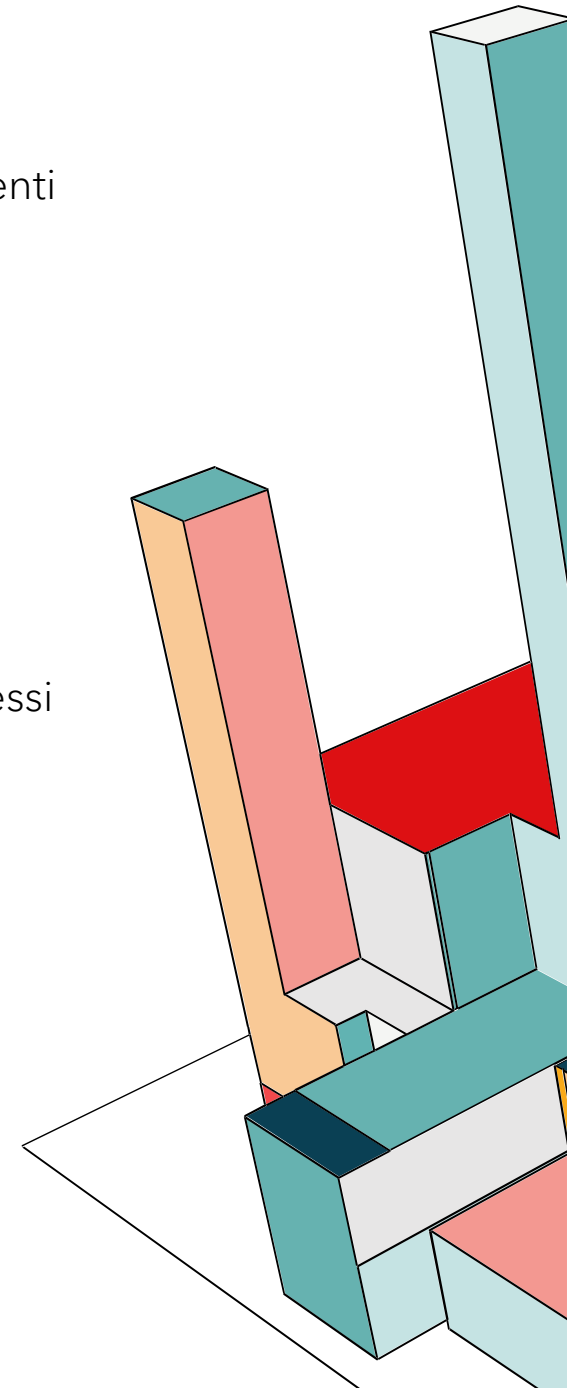
1. **Tipi di Account Ammessi:** Nel sistema esistono solo due categorie di account:
 - **Account Utente (Interattivi):** Identità personali gestite da Keycloak per accedere all'interfaccia della web-app via browser
 - **Account di Sistema (Non-Interattivi):** Credenziali tecniche utilizzate per la comunicazione tra i microservizi
2. **Definizione dei Ruoli:** Il sistema utilizza i **Ruoli (RBAC)** definiti su Keycloak:
 - **app-user:** Accesso limitato alle proprie risorse
 - **app-admin:** Accesso alle proprie risorse e a quelle degli altri user
 - **app-technician:** Accesso alla dashboard tecnica di monitoraggio
3. **Gestione del Ciclo di Vita:** La creazione e la rimozione di un account, che sia un impiegato, amministratore o tecnico, viene gestita tramite Keycloak, mediante un intervento manuale dell'amministratore di sistema
4. **Monitoraggio:** Come già detto nelle slide precedenti sono stati abilitati su Keycloak gli auditing per user ed admin



AC-3: ACCESS ENFORCEMENT

L'access enforcement è stato implementato su 3 livelli:

1. **Livello di Presentazione:** Sebbene non costituisca un perimetro di sicurezza invalicabile, il Frontend esegue un **Rendering condizionale**, ovvero, l'applicazione renderizza dinamicamente solo gli elementi dell'interfaccia consentiti
2. **Livello Applicativo:** La protezione delle risorse di business viene effettuata tramite l'**enforcer di keycloak**, contenuto nella libreria `keycloak_enforcer`. Il quale interroga keycloak per verificare se l'utente possiede i permessi necessari per effettuare l'azione richiesta.
3. **Livello di Sistema:** Tutti i container sono inseriti in una rete privata isolata (network-backend) e ad ognuno vengono applicate le direttive **cap_drop: - ALL** e **security_opt: - no-new-privileges:true**, che rimuovo tutte le capability del kernel Linux e impediscono l'escalation dei privilegi, assicurando che, anche in caso di compromissione, l'attaccante non possa ottenere permessi di root sul sistema ospite.



AC-7: UNSUCCESSFUL LOGON ATTEMPTS

L'implementazione del controllo sui tentativi di accesso falliti è stata delegata a Keycloak, sfruttando il modulo nativo di **Brute Force Detection**.

Nello specifico, la policy di sicurezza applicata nel realm prevede:

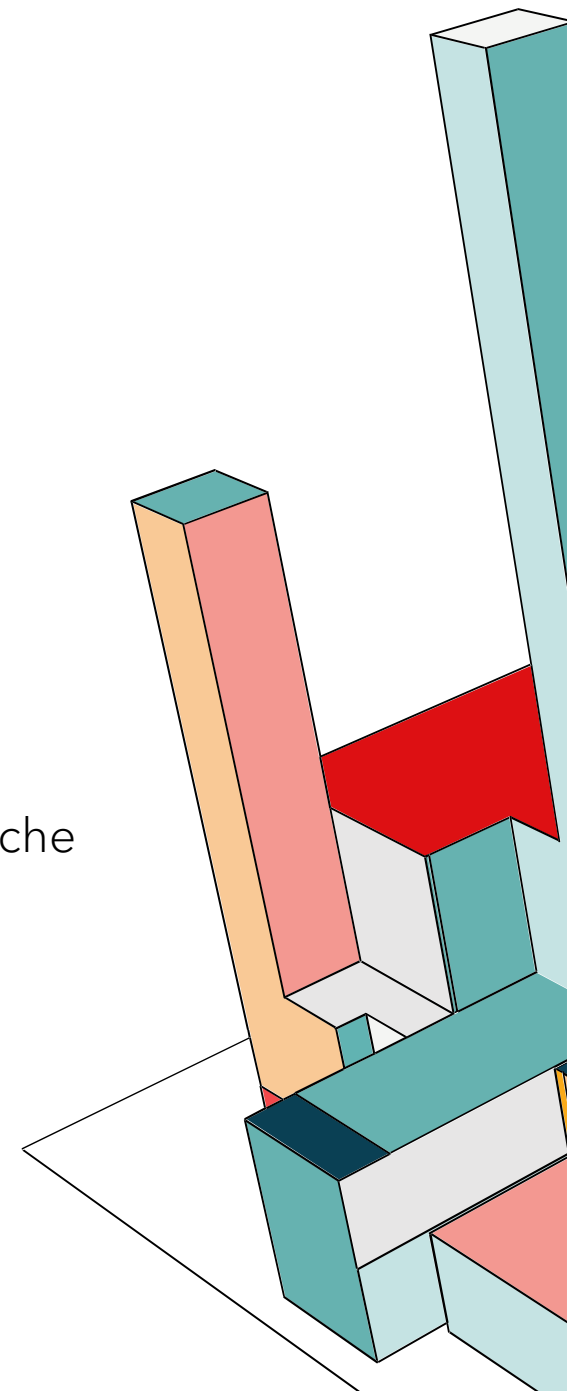
1. Un intervento automatico dopo una soglia di **5 tentativi di login falliti consecutivi**, al superamento dei quali, l'account viene posto in stato di disabilitazione temporanea.
 - o Questa strategia è stata preferita al blocco permanente per mitigare il rischio di Denial of Service (DoS) verso utenti legittimi.
2. È stato impostato un incremento progressivo del tempo di attesa pari a **1 minuto** per ogni ulteriore tentativo errato, rendendo di fatto computazionalmente impraticabili gli attacchi di forza bruta prolungati.
3. Ed infine è attivo il controllo *Quick Login Check* con soglia a **1000ms**, che blocca istantaneamente i tentativi di accesso effettuati con frequenze tipiche di script automatizzati o botnet, indipendentemente dalla correttezza delle credenziali



AC-8: SYSTEM USE NOTIFICATION

L'obbligo di notifica delle condizioni d'uso è stato implementato attraverso un banner che viene visualizzato prima di reindirizzare l'utente all'Identity Provider per l'inserimento delle credenziali. Questo garantisce che nessun utente possa accedere al sistema senza aver visualizzato le clausole legali richieste dalla normativa di sicurezza:

- **Restrizione d'uso:** Specifica chiaramente che l'accesso è riservato esclusivamente al personale autorizzato.
- **Monitoraggio e Audit:** Informa l'utente che le attività svolte sul sistema sono oggetto di monitoraggio e registrazione per fini di sicurezza.
- **Sanzioni:** Avvisa esplicitamente delle conseguenze civili e penali in caso di accesso non autorizzato o uso improprio.
- **Consenso Implicito:** Il pulsante di accesso è etichettato o accompagnato da una dicitura che lega l'azione di login all'accettazione formale delle condizioni di monitoraggio.

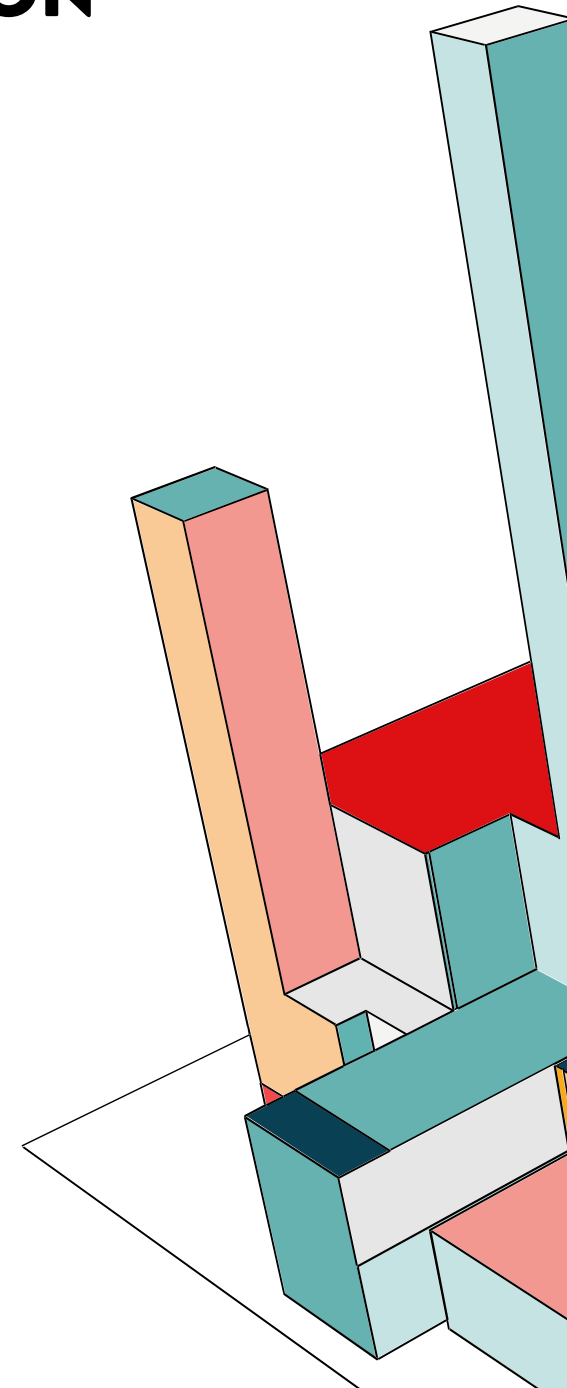


AC-14: PERMITTED ACTIONS WITHOUT IDENTIFICATION OR AUTHENTICATION

L'accesso al sistema è completamente inibito agli utenti non autenticati, le uniche eccezioni strettamente necessarie per permettere l'ingresso, sono:

- **Visualizzazione della Landing Page:** Per caricare l'interfaccia grafica iniziale.
- **Consultazione del Warning Banner:** Per permettere all'utente di leggere e accettare le condizioni legali prima del login.
- **Inizializzazione del Login:** Per reindirizzare l'utente verso l'Identity Provider (Keycloak) e avviare la procedura di autenticazione.

Qualsiasi altro tentativo di accesso alle risorse o ai dati viene immediatamente bloccato dal sistema.



AC-17: REMOTE ACCESS

L'accesso remoto al sistema è stato regolamentato attraverso una rigorosa architettura di rete basata su container Docker, che distingue tra superficie di attacco pubblica e privata.

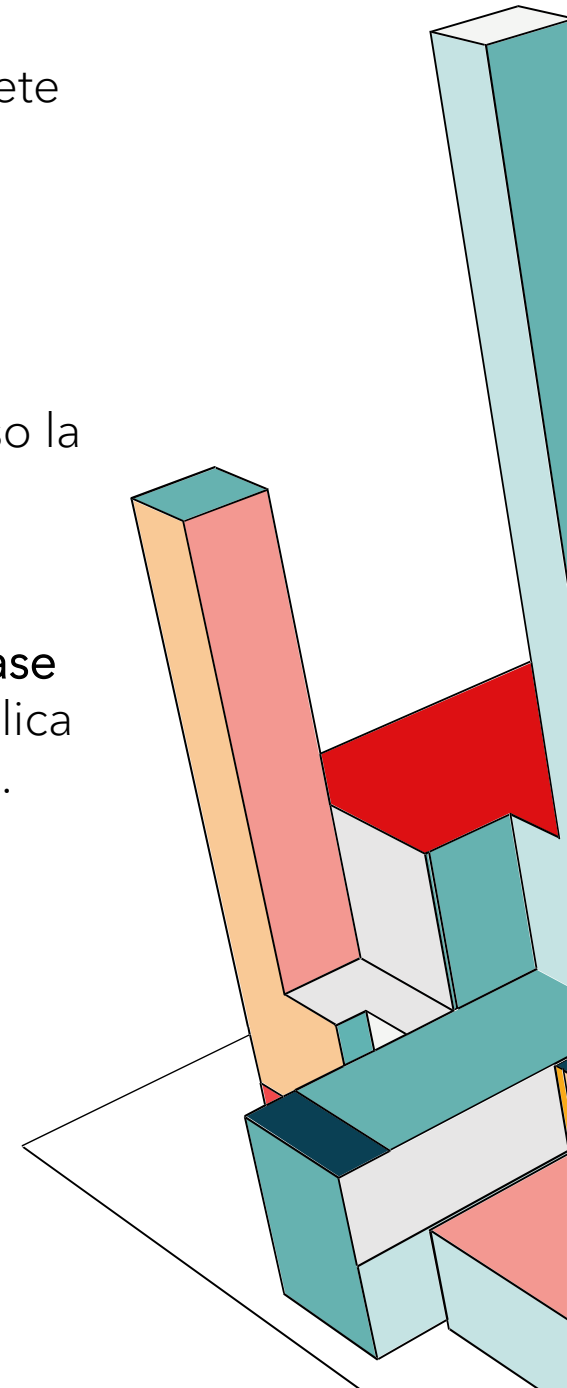
1. Unico Punto di Ingresso Pubblico: L'accesso dall'esterno è consentito esclusivamente attraverso il container **Apache**, che funge da **reverse proxy**.

- Nel file **docker-compose.yml**, questo è l'unico servizio configurato con la direttiva **ports** (mappatura 80/443).
- La porta 80 è mantenuta aperta solo per forzare il reindirizzamento automatico verso la porta 443, obbligando di fatto tutto il traffico operativo a passare attraverso la crittografia TLS.

2. Segregazione dei Servizi Interni: I container che gestiscono dati sensibili, come il **Database (MongoDB)**, il **Gestore Segreti (Vault)** e il **Backend API**, non hanno alcuna mappatura pubblica (**ports**), ma solo l'esposizione interna (**expose**) sulla rete virtuale privata **network-backend**.

3. Flusso di Accesso Autorizzato: L'unica modalità di accesso ai dati è indiretta:

- L'utente remoto si collega al Proxy (Apache) via HTTPS.
- Il Proxy, se la richiesta è valida, la inoltra al Backend sulla rete interna.
- Solo il Backend ha i privilegi di rete per dialogare con il Database.



AC-18: WIRELESS ACCESS

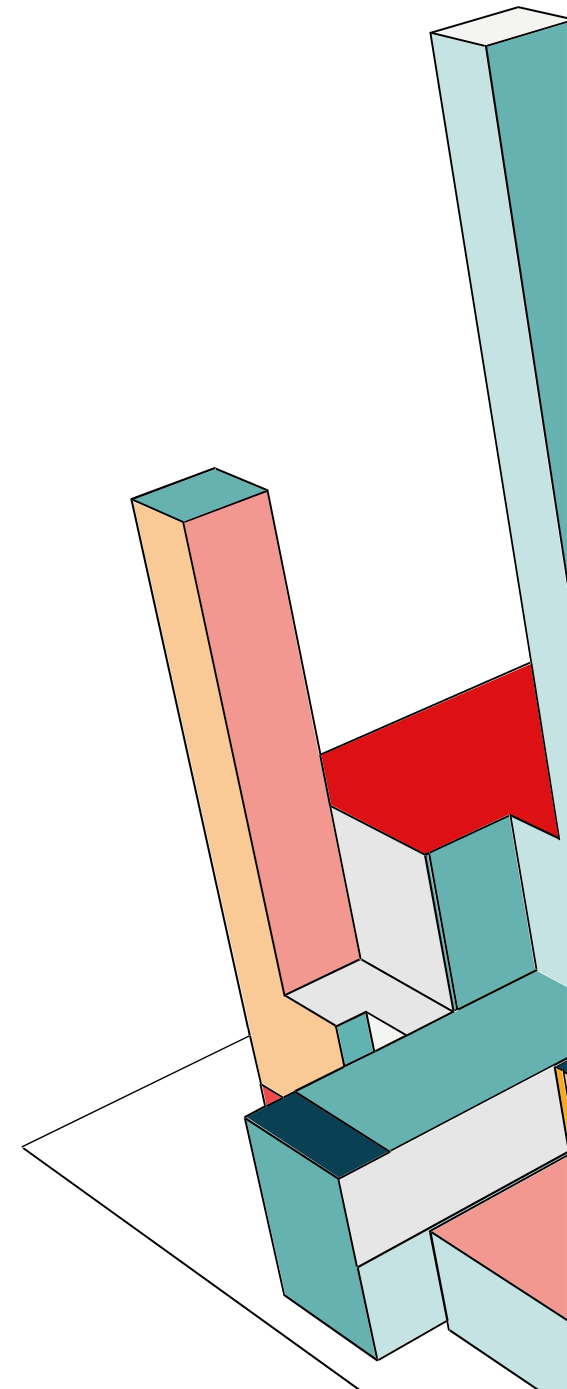
Non Applicabile (N/A)

Il sistema non prevede né permette l'accesso tramite tecnologie wireless. L'applicazione è configurata per rispondere solo alle richieste interne alla macchina stessa (localhost).

AC-19: ACCESS CONTROL FOR MOBILE DEVICES

Non Applicabile (N/A)

L'accesso tramite dispositivi mobili non è autorizzato ed è tecnicamente impedito dalla configurazione architetturale.



AC-20: USE OF EXTERNAL SYSTEMS

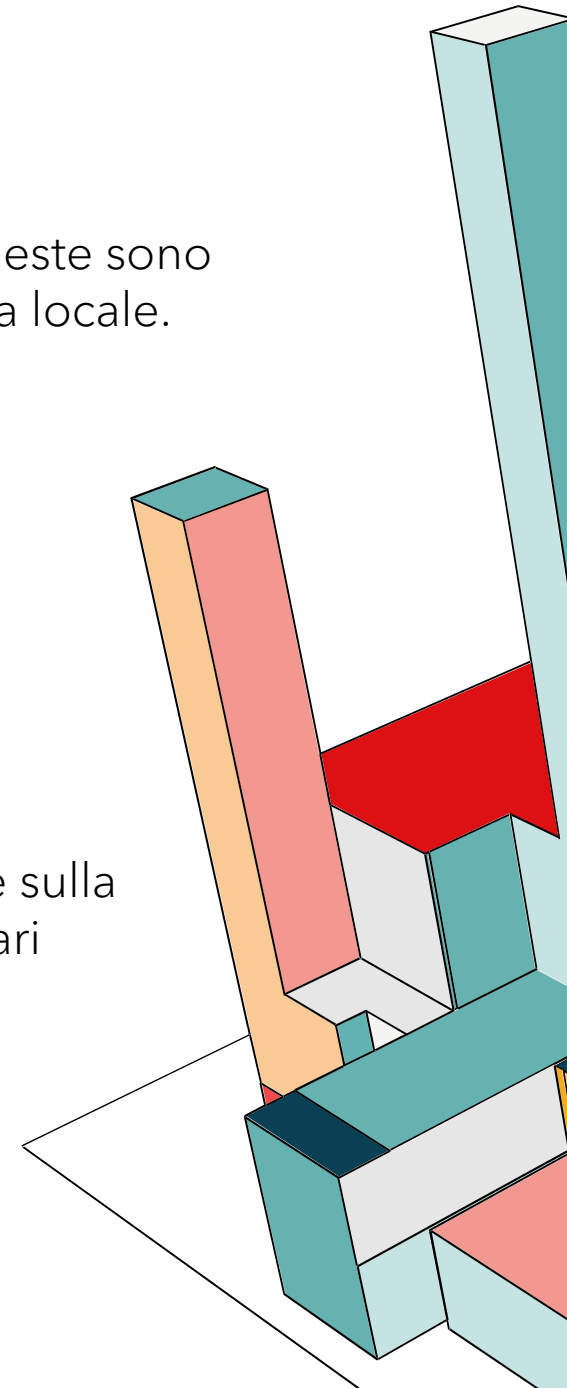
Non Applicabile (N/A)

L'applicazione non utilizza sistemi esterni per l'elaborazione o l'archiviazione dei dati. Sebbene l'architettura utilizzi tecnologie di terze parti (come MongoDB, Vault, Keycloak), queste sono distribuite esclusivamente come istanze containerizzate all'interno del perimetro di sicurezza locale.

AC-22: PUBLICLY ACCESSIBLE CONTENT

Non Applicabile (N/A)

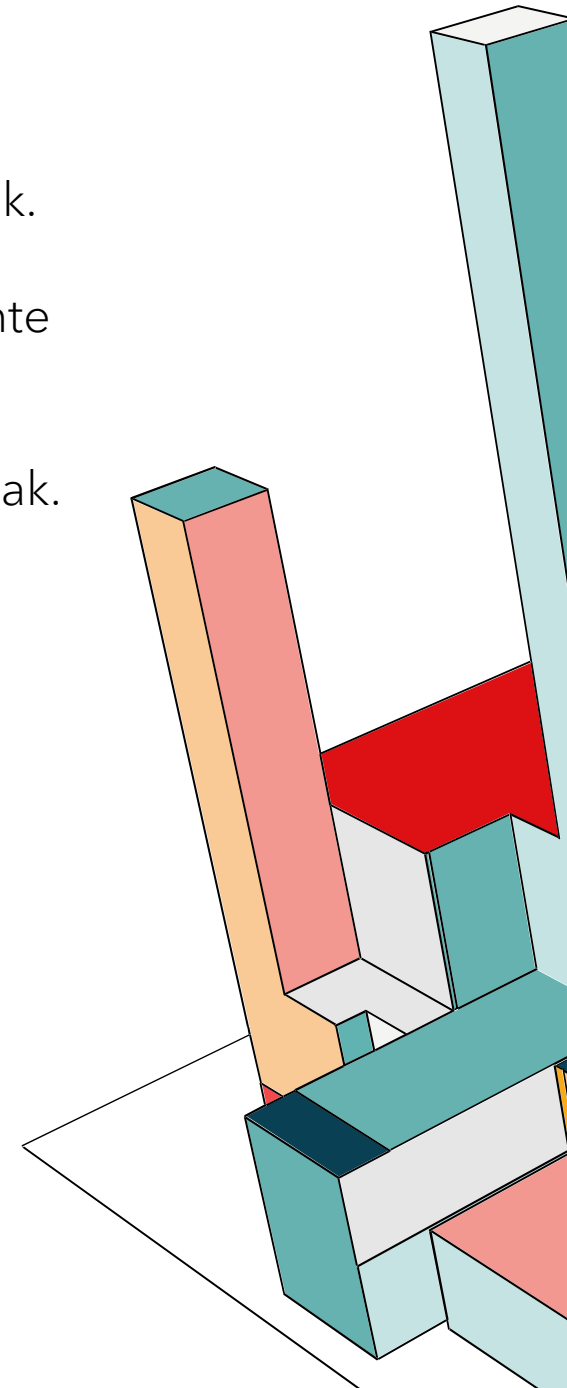
Il sistema non è esposto su Internet né su reti esterne; è accessibile esclusivamente in locale sulla macchina host. Non essendoci un'interfaccia aperta al pubblico generico, non sono necessari processi di gestione o revisione dei contenuti pubblici.



IA-1: POLICY AND PROCEDURES

Il controllo degli accessi è affidato all'Amministratore di Keycloak, che segue queste linee guida:

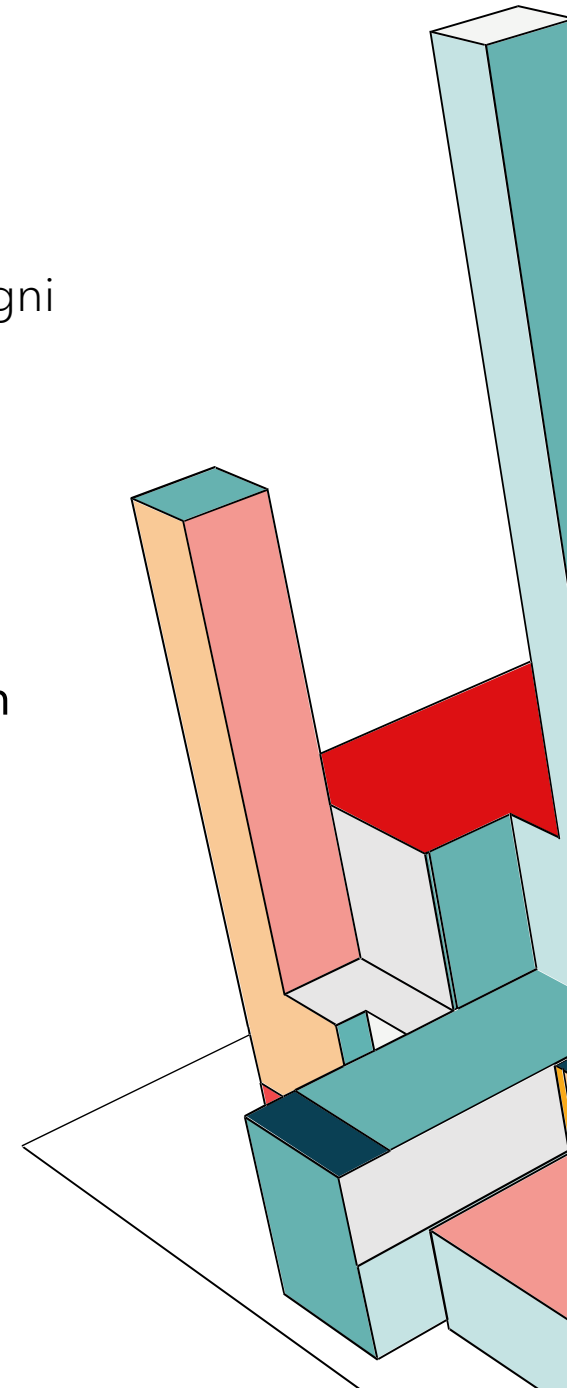
- **Identificazione Obbligatoria:** Ogni utente deve avere un account unico creato su Keycloak.
- **Sicurezza dei Dati:** Le password sono salvate nel database dell'app, ma gestite interamente da Keycloak.
- **Gestione Utenze:** Solo l'Admin può creare o eliminare utenti usando la console di Keycloak.



IA-2: IDENTIFICATION AND AUTHENTICATION (ORGANIZATIONAL USERS)

Il sistema implementa l'identificazione univoca e l'autenticazione degli utenti delegando il processo all'Identity Provider centralizzato **Keycloak**, secondo lo standard OpenID Connect.

- **Identificazione Univoca:** Per garantire che ogni identità sia distinta e non duplicabile, a ogni utente registrato nel sistema, Keycloak assegna in automatico un identificativo univoco immutabile (UUID)
- **Autenticazione:** L'autenticazione avviene tramite la verifica di email e password, gestite sempre in modo sicuro da Keycloak.
- **Associazione Utente-Processo:** L'esito del flusso OIDC è il rilascio di un **JSON Web Token (JWT)** crittografato. Questo token accompagna ogni richiesta HTTP verso il Backend, fungendo da "passaporto digitale" che attesta l'identità dell'utente in modo sicuro e verificabile senza dover reinserire la password.



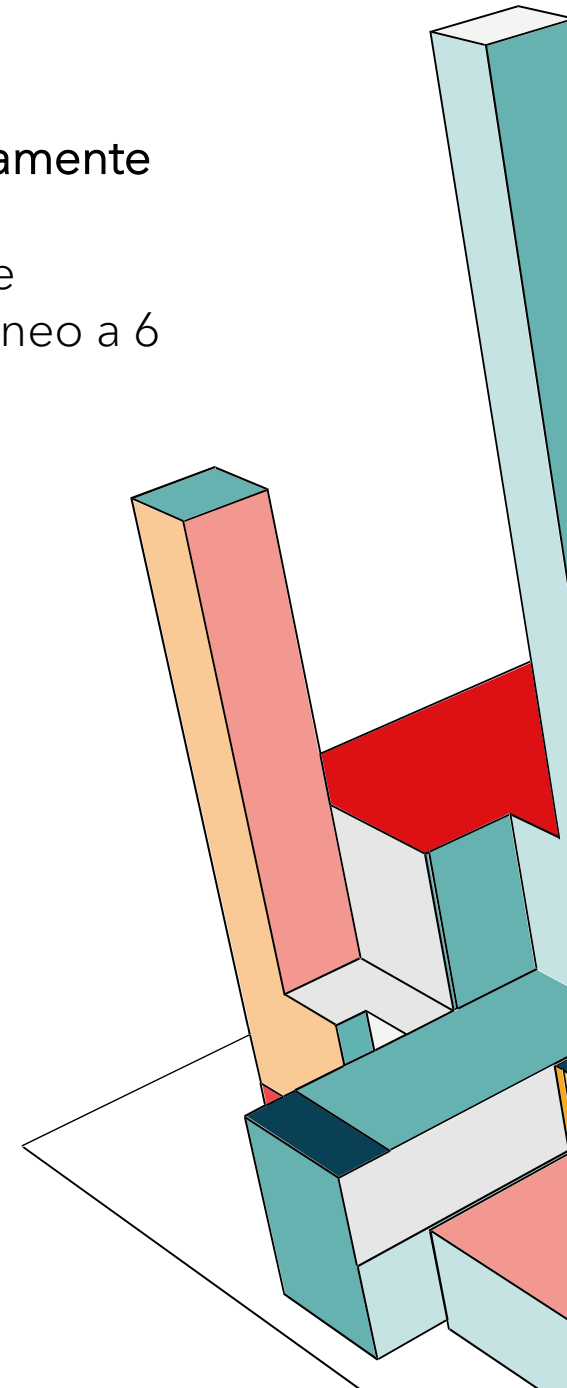
IA-2 (1): MULTI-FACTOR AUTHENTICATION TO PRIVILEGED ACCOUNTS

Per elevare la sicurezza degli account amministrativi e tecnici, il sistema **impone obbligatoriamente** l'utilizzo dell'**autenticazione a due fattori**.

Keycloak è stato configurato per richiedere, oltre alla password standard, un secondo fattore basato sullo standard TOTP (Time-based One-Time Password: un codice numerico temporaneo a 6 cifre generato dall'applicazione mobile Google Authenticator).

IA-2 (2): MULTI-FACTOR AUTHENTICATION TO NON-PRIVILEGED ACCOUNTS

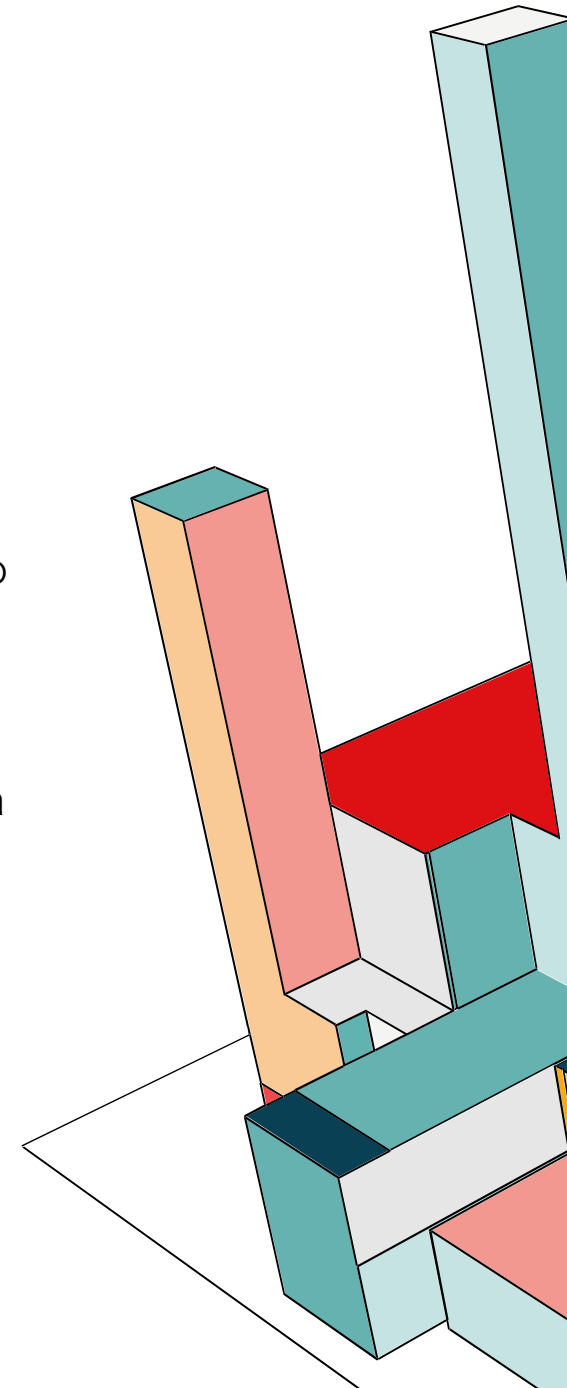
Sulla base dell'analisi dei rischi, abbiamo deciso di **non obbligare** gli impiegati a usare l'autenticazione a due fattori, per mantenere il sistema veloce e facile da usare.



IA-2 (8): ACCESS TO ACCOUNTS REPLAY RESISTANT

Il sistema implementa meccanismi di autenticazione resistenti agli attacchi di "Replay", sia per gli account privilegiati che per quelli standard.

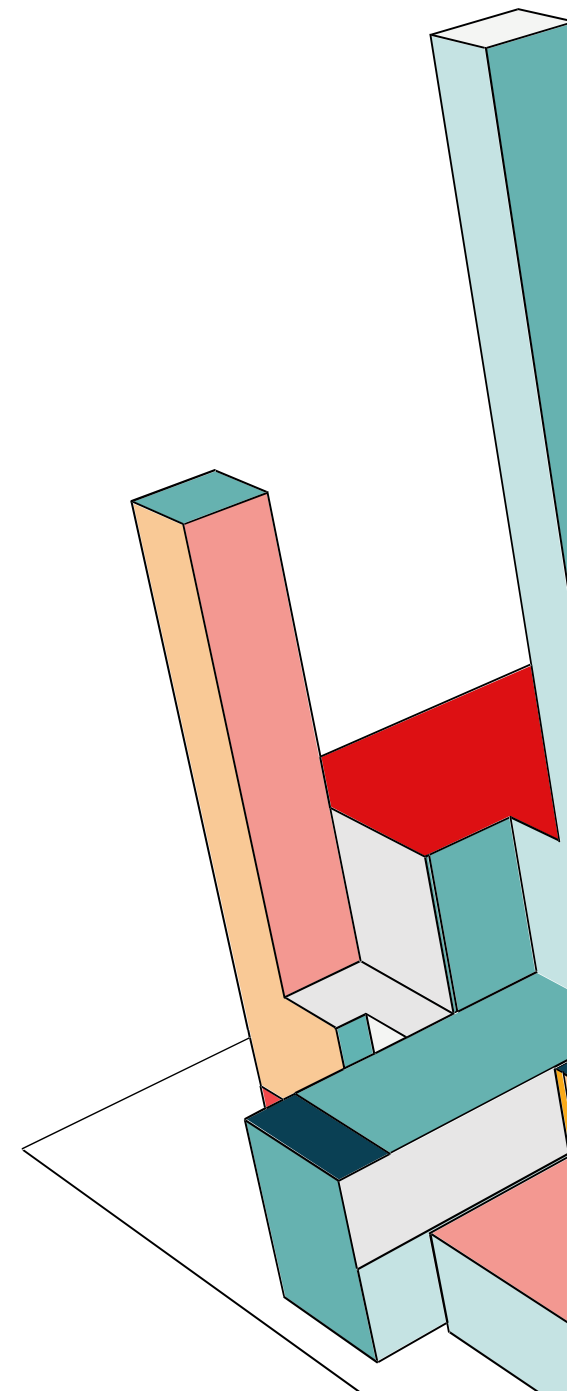
- **Protezione tramite MFA:** Per gli account amministrativi e tecnici, l'uso dell'algoritmo TOTP (Google Authenticator) garantisce la resistenza intrinseca al replay. Poiché il codice di autenticazione cambia ogni 30 secondi ed è matematicamente legato all'orario esatto, qualsiasi tentativo di riutilizzare un codice precedentemente intercettato fallisce automaticamente.
- **Protezione del Protocollo OIDC:** Per tutti gli utenti, il protocollo di autenticazione gestito da Keycloak utilizza meccanismi crittografici standard anti-replay, tra cui:
 - **Nonce (Number used ONCE):** Ogni richiesta di login include un valore casuale univoco che impedisce il riutilizzo dello stesso messaggio di autenticazione.
 - **Codici "One-Time Use":** Nel flusso di login, il codice di autorizzazione scambiato tra Frontend e Keycloak è valido per un solo utilizzo.



IA-2 (12): ACCEPTANCE OF PIV CREDENTIALS

Non Applicabile (N/A)

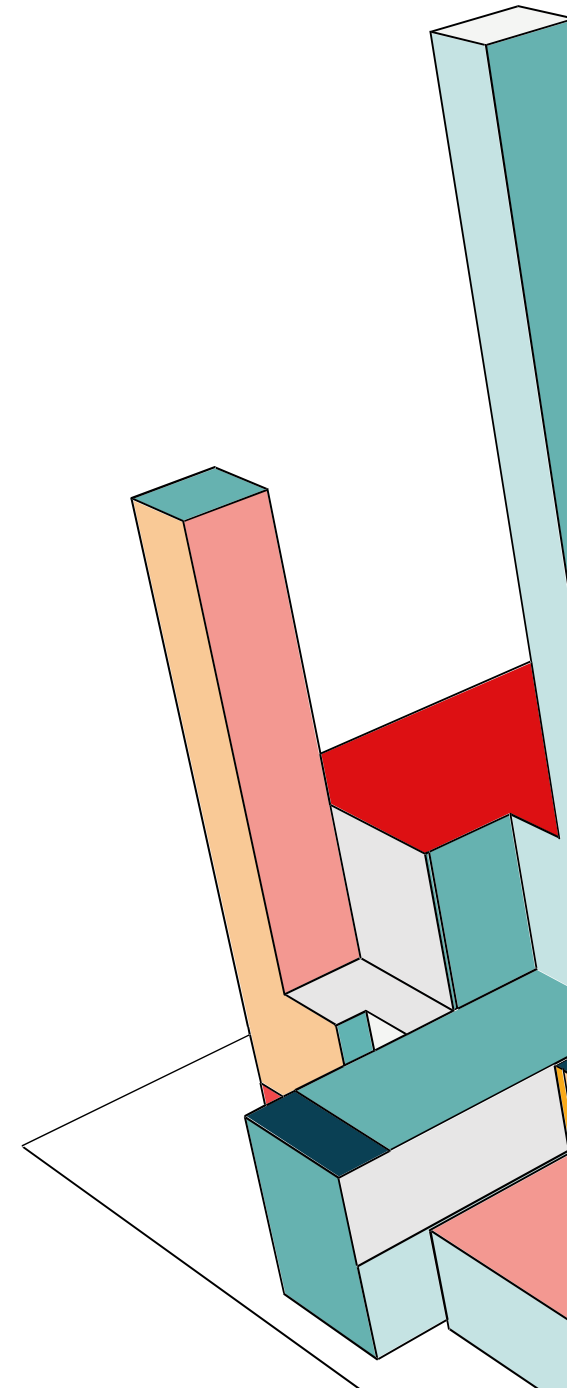
Considerato il contesto non governativo, il supporto per le credenziali fisiche PIV (Personal Identity Verification) non è stato implementato.



IA-4: IDENTIFIER MANAGEMENT

La gestione degli identificativi è centralizzata su Keycloak:

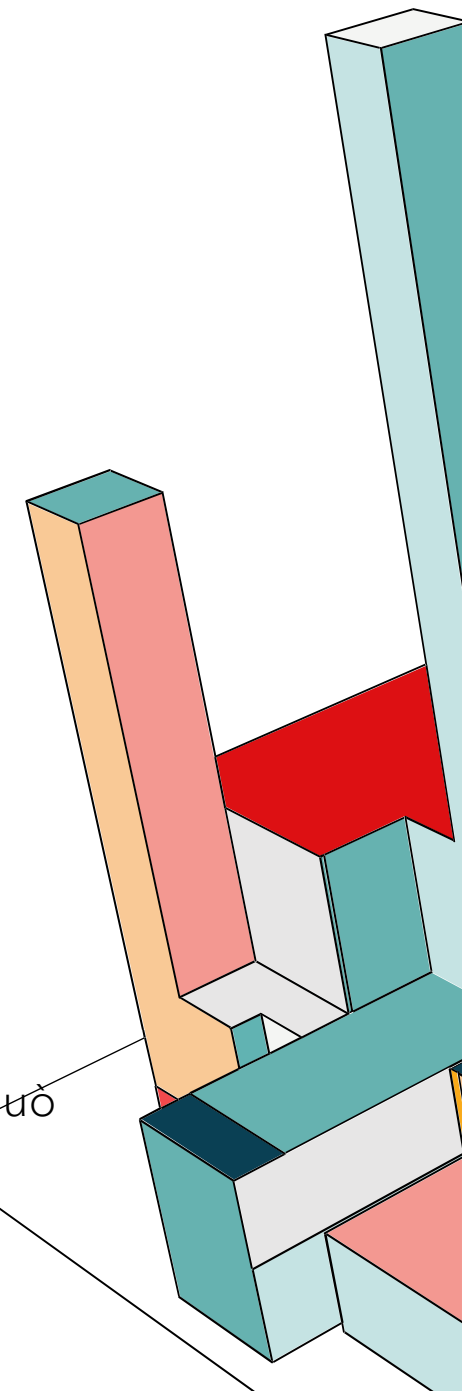
- **Autorizzazione e Assegnazione:** La creazione degli account è riservata all'amministratore del Realm, il quale inserisce i dati anagrafici verificati, garantendo l'identità del soggetto.
- **Selezione e Unicità:** Il sistema utilizza l'**indirizzo email** inserito in fase di registrazione come username univoco. Il tentativo di registrare un nuovo utente con un username già presente nel sistema viene bloccato automaticamente ("**User exists**"), prevenendo duplicati e ambiguità.
- **Prevenzione del Riutilizzo:** Il sistema gestisce il rischio di riutilizzo tramite **UUID**. Anche se un'email venisse riassegnata a un nuovo dipendente (es. dopo la cancellazione del precedente titolare), il sistema distingue le due identità grazie a un codice interno univoco diverso per ogni creazione.



IA-5: AUTHENTICATOR MANAGEMENT

La gestione delle credenziali di autenticazione è governata dalle policy di sicurezza del *Realm* di Keycloak:

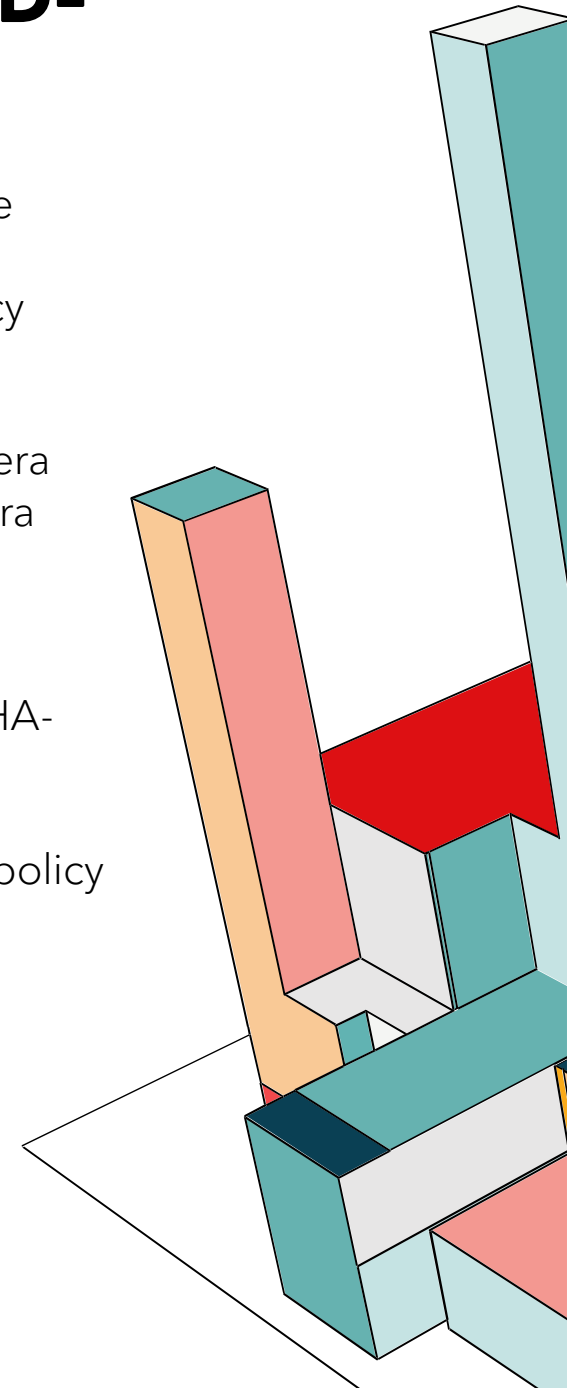
- **Distribuzione Iniziale e Primo Utilizzo:**
 - L'**Amministratore del Realm** genera manualmente le credenziali iniziali per i nuovi utenti, impostando il flag di sistema "**Temporary**" su ON.
 - Al primo accesso, il sistema intercetta il login e **obbliga** l'utente a sostituire la password provvisoria con una nuova da scegliere, garantendo che le credenziali definitive non siano mai note all'amministratore del Realm.
- **Robustezza e Complessità:** Per garantire un'adeguata resistenza agli attacchi, Keycloak applica le seguenti policy sulla scelta della password:
 - Lunghezza: Minimo **8 caratteri**.
 - Complessità: Inclusione obbligatoria di almeno **1 lettera maiuscola, 1 numero e 1 carattere speciale**.
- **Protezione Crittografica:** Le password non sono mai memorizzate in chiaro, keycloak utilizza lo standard industriale **PBKDF2** (Password-Based Key Derivation Function 2) con algoritmo di hashing SHA-256.
- **Procedure Amministrative:** In caso di smarrimento o compromissione, l'Amministratore del Realm può invalidare le sessioni attive e forzare un reset delle credenziali al successivo accesso dell'utente.



IA-5 (1): AUTHENTICATOR MANAGEMENT | PASSWORD-BASED-AUTHENTICATION

La sicurezza delle password è garantita dall'integrazione tra le policy di **Keycloak** e l'architettura di rete **Docker/Apache**:

- **Policy password** : Il sistema impedisce l'uso di password banali o legate al contesto tramite le policy "Not Username" e "Not Email".
- **Canali Protetti e Isolamento**: La riservatezza durante la trasmissione è garantita da **Apache**, che opera come *Reverse Proxy* gestendo la **TLS Termination** (HTTPS) verso l'esterno. Internamente, il traffico tra Apache e Keycloak è isolato nella rete Docker (**network-backend**), prevenendo intercettazioni non autorizzate.
- **Archiviazione Sicura**: Le credenziali sono memorizzate tramite l'algoritmo PBKDF2 con hashing SHA-256.
- **Recupero Account e Reset Password**: Poiché la funzione di recupero automatico è disabilitata per policy di sicurezza, il recupero avviene contattando l'**Amministratore del Realm**.
- **Robustezza delle password**: Il sistema impone vincoli di sicurezza discussi nella slide precedente.



IA-6: AUTHENTICATION FEEDBACK

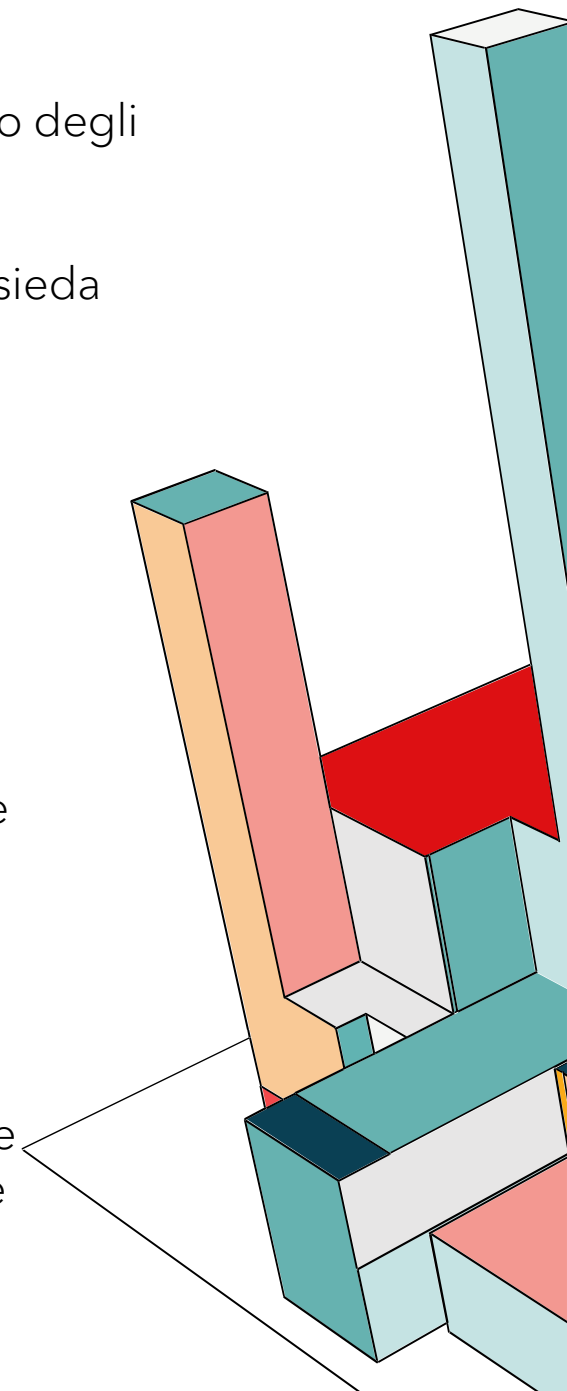
Keycloak soddisfa questo requisito in modo nativo e automatico:

- **Oscuramento dei caratteri:** I caratteri inseriti nel campo password vengono sostituiti utilizzando degli asterischi per impedirne la visualizzazione in chiaro.
- **Feedback di errore generico:** In caso di credenziali errate, il sistema non specifica se l'errore risieda nello username o nella password, limitando le informazioni utili a un potenziale attaccante.

IA-7: CRYPTOGRAPHIC MODULE AUTHENTICATION

In questo progetto, i moduli crittografici che gestiscono la sicurezza sono Keycloak e Vault, l'accesso a questi strumenti è protetto in questo modo:

- **Accesso dell'utente:** L'utente interagisce con Keycloak solo per il login. Non può accedere alle funzioni interne o alle chiavi di sicurezza.
- **Sicurezza tecnica:** Solo l'amministratore di sistema (Admin) può accedere alle console di gestione di questi moduli per cambiare le impostazioni di sicurezza.
- **Accesso del Backend:** Il server backend è l'unico autorizzato a parlare con Vault per ottenere le credenziali del database. Questa comunicazione è protetta e avviene solo all'interno della rete Docker.



IA-8: IDENTIFICATION AND AUTHENTICATION (NON-ORGANIZATIONAL USERS)

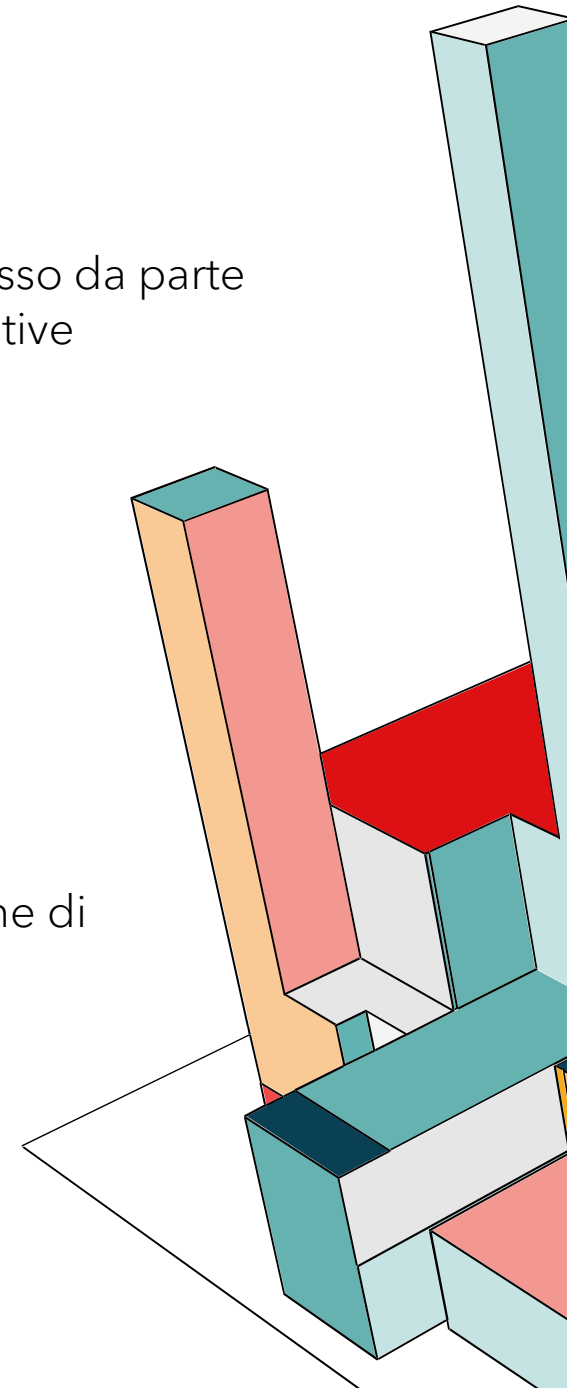
Non Applicabile (N/A)

Il sistema è ad uso strettamente interno, non essendo prevista la registrazione autonoma o l'accesso da parte di utenti esterni (come clienti o partner), non sussiste la necessità di gestire utenze non-organizzative

IA-8 (1): ACCEPTANCE OF PIV CREDENTIALS FROM OTHER AGENCIES

Non Applicabile (N/A)

Il sistema non opera all'interno di un framework federale o inter-agenzia che richieda l'accettazione di credenziali PIV.



IA-8 (2): ACCEPTANCE OF EXTERNAL AUTHENTICATORS

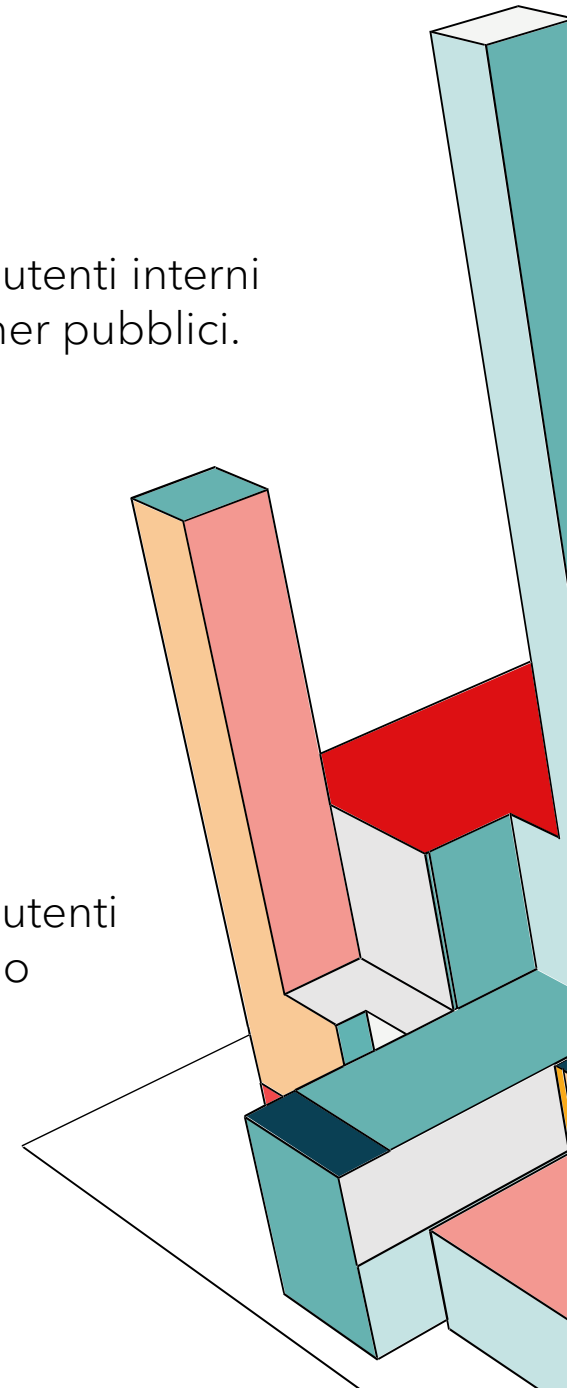
Non Applicabile (N/A)

Il controllo non si applica al progetto in quanto il sistema è configurato per l'accesso esclusivo di utenti interni (organizzativi) e non è previsto l'accesso per utenti esterni (non-organizzativi) come clienti o partner pubblici.

IA-8 (4): USE OF DEFINED PROFILES

Non Applicabile (N/A)

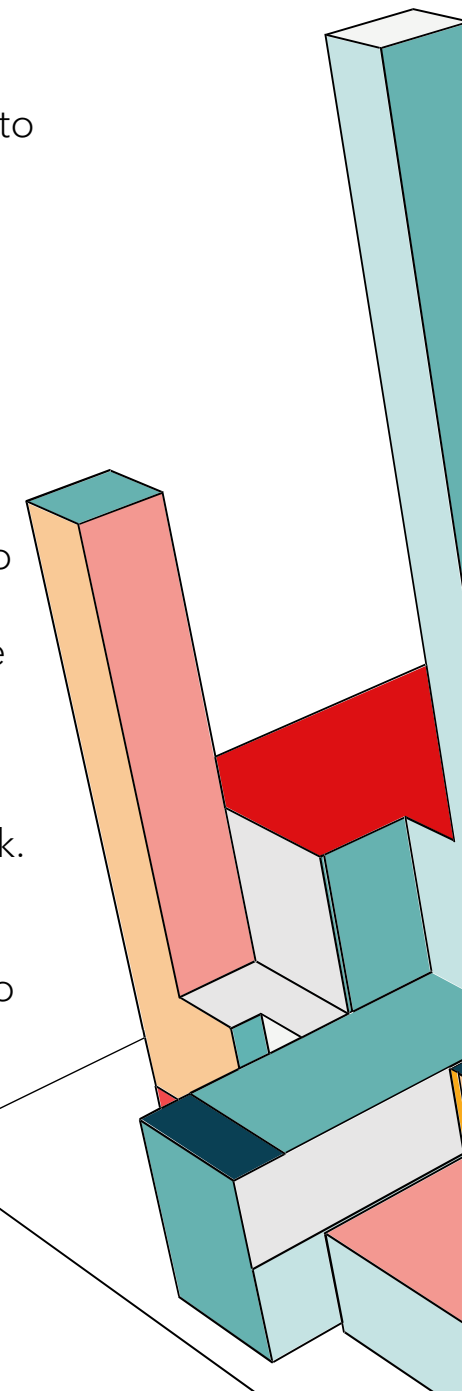
Il controllo non si applica al progetto in quanto il sistema è configurato per l'accesso esclusivo di utenti interni (organizzativi) e non è previsto l'accesso per utenti esterni (non-organizzativi) come clienti o partner pubblici.



IA-11: RE-AUTHENTICATION

Il controllo stabilisce che un'organizzazione debba richiedere agli utenti di autenticarsi nuovamente in circostanze specifiche, come la scadenza di un periodo di tempo prefissato, l'esecuzione di funzioni privilegiate o il cambiamento di ruoli e credenziali. Questo requisito è stato soddisfatto integrando le politiche di gestione delle sessioni e dei token su **Keycloak**.

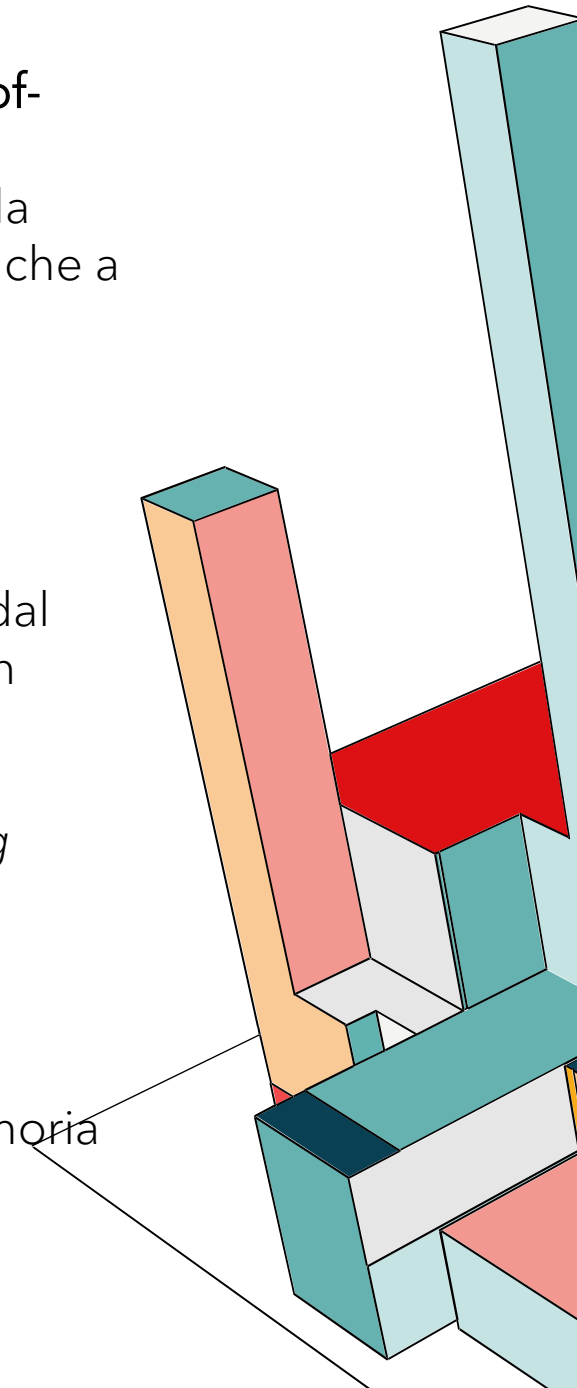
- **Durata della validità degli Access Token JWT:** Poiché il backend adotta un approccio *stateless*, esso verifica la validità del token JWT a ogni singola richiesta. L'**Access Token Lifespan** è stato configurato con una durata di **10 minuti**. Una volta trascorso questo intervallo, il token diventa automaticamente inutilizzabile, di conseguenza quando il backend si accorge della scadenza, interrompe le operazioni restituendo un errore.
- **Revoke Refresh Token:** Ogni volta che l'Access Token scade, il frontend utilizza il **Refresh Token** memorizzato nel browser per richiederne uno nuovo a Keycloak senza dover chiedere nuovamente la password all'utente. Avendo impostato che il **numero massimo di riutilizzi sia 0**, Keycloak invalida subito il vecchio Refresh Token e ne emette uno nuovo. Questo garantisce che il processo di rinnovo si interrompa immediatamente se la sessione principale viene terminata o se l'amministratore cambia le credenziali o i ruoli dell'utente, forzando quest'ultimo a una ri-autenticazione completa.
- **Gestione della Sessione SSO:** Il Refresh Token permette il rinnovo solo finché la sessione SSO è attiva su Keycloak. A tal fine, sono stati definiti due parametri critici:
 - **SSO Session Idle:** Definisce il tempo massimo di inattività. Se l'utente non interagisce con il sistema per **30 minuti**, la sessione scade automaticamente e il Refresh Token perde validità, obbligando l'utente a un nuovo login.
 - **SSO Session Max:** Rappresenta il limite massimo assoluto di durata della sessione (fissato a **4 ore**), indipendentemente dall'attività dell'utente. Raggiunto questo limite, il sistema invalida forzatamente ogni token residuo.



SC-5: DENIAL-OF-SERVICE PROTECTION

Il sistema implementa una strategia di **difesa multi-livello** per limitare gli attacchi di **Denial-of-Service**:

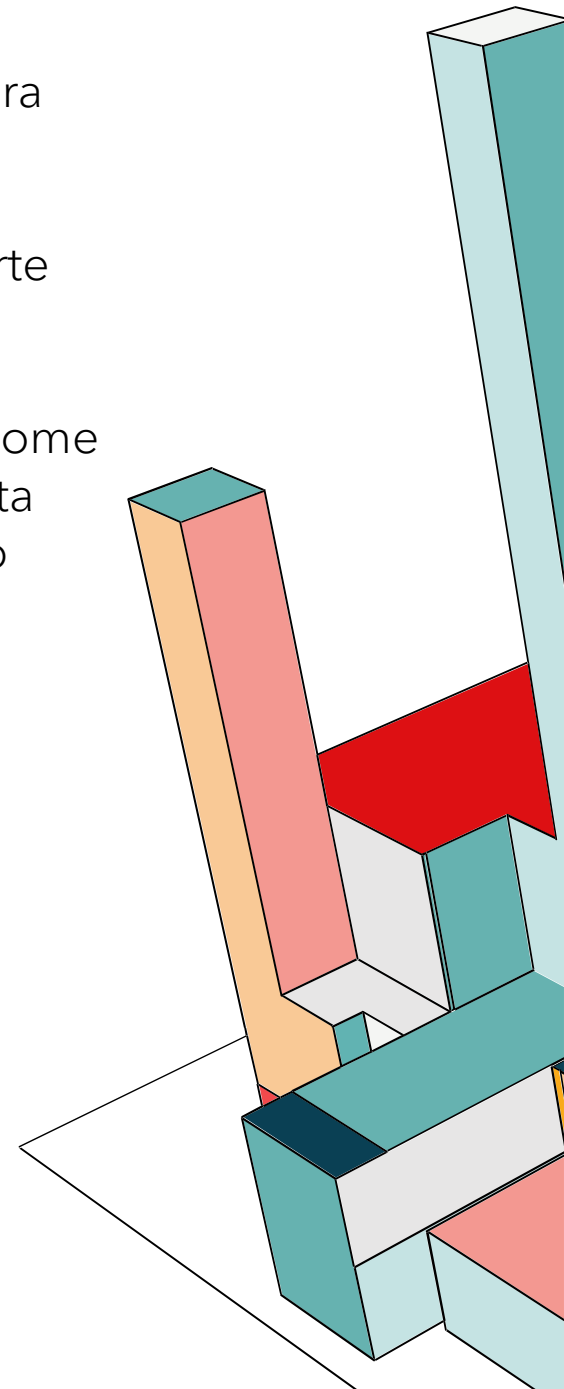
- **Protezione del Perimetro:** Apache essendo l'unico componente esposto direttamente alla rete pubblica, è stato configurato per agire da scudo contro attacchi DoS sia volumetrici che a bassa velocità
- **Protezione da attacchi brute force in Keycloak:** Già discusso nelle slide precedenti
- **Ridondanza e limitazioni del Backend:**
 - **Service Redundancy:** Il backend Node.js è configurato con **3 repliche attive** gestite dal load balancer di Apache. Se un'istanza dovesse smettere di rispondere a causa di un carico eccessivo, le restanti continuano a garantire la disponibilità del sistema.
 - **Rate Limiting:** Tramite il middleware **express-rate-limit**, viene imposto un tetto massimo di 100 richieste ogni 15 minuti per singolo IP. Questo impedisce il *flooding* delle API e protegge il database MongoDB da query massive.
- **Limiti sull'utilizzo delle Risorse Hardware:** Per garantire che nessun container possa "monopolizzare" l'intera macchina host mandando in crash gli altri servizi (Resource Exhaustion), sono stati imposti limiti, diversificati in base al servizio, sull'utilizzo della memoria della CPU tramite le direttive **deploy.resources.limits**.



SC-7: BOUNDARY PROTECTION

Per proteggere i confini del sistema e separare la rete pubblica da quella privata, l'architettura che utilizziamo si basa su:

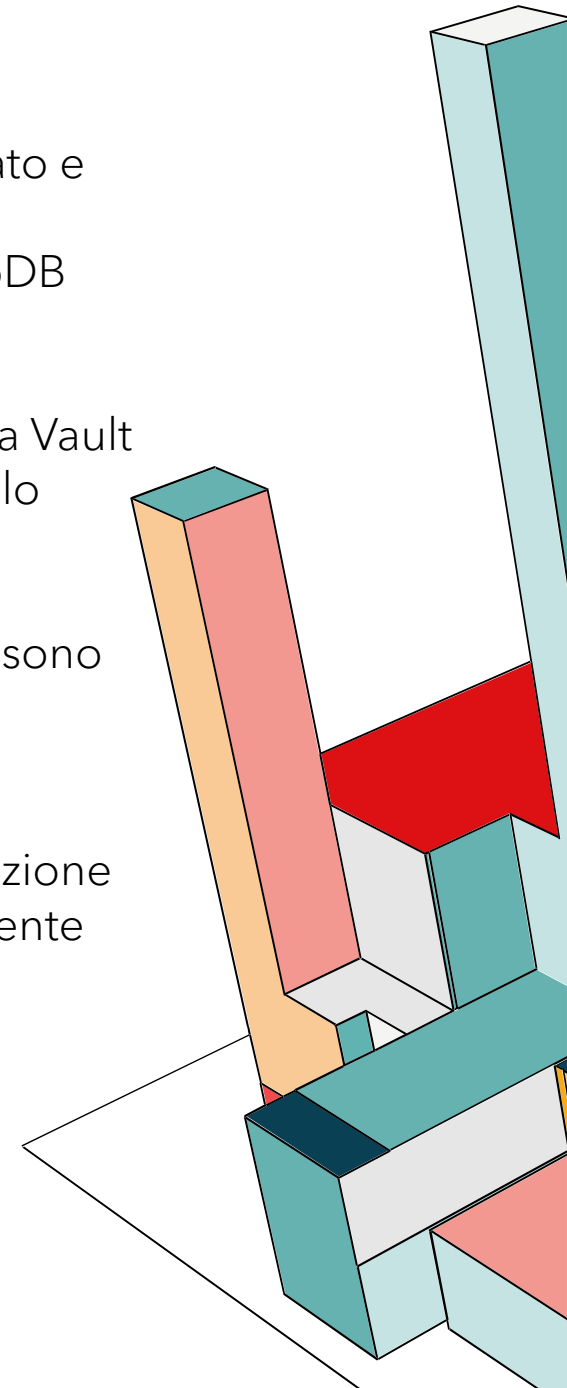
- **Unico Punto di Accesso:** Il sistema si connette con l'esterno solo tramite **Apache**, in particolare abbiamo mappato le porte esterne dell'host 80 e 443 rispettivamente alle porte 8080 e 8443, dove Apache nel container ascolta.
- **Rete Privata:** Abbiamo creato una rete interna chiamata **network-backend** che funziona come una zona riservata. Tutti i servizi (Web, Api, Mongo, Keycloak, Vault) sono collegati a questa rete tramite la voce **networks:** - **network-backend**. I container possono parlarsi tra loro usando questa rete privata, ma questa rete non è accessibile direttamente dall'esterno
- **Servizi Nascosti:** A differenza di Apache, tutti gli altri servizi **non hanno la sezione ports:** mappata verso l'esterno, quindi non visibili dall'esterno.



SC-12: CRYPTOGRAPHIC KEY ESTABLISHMENT AND MANAGEMENT

Il sistema gestisce il ciclo di vita delle chiavi e delle credenziali attraverso un approccio centralizzato e stratificato:

- **Vault:** Invece di scrivere le password nel codice sorgente, le credenziali per il database MongoDB sono memorizzate nel Vault.
- **Distribuzione Sicura:** Il Backend è l'unico processo autorizzato a richiedere queste credenziali a Vault durante l'esecuzione, quindi non sono mai salvate in chiaro sui file del server, ma trasmesse solo quando necessario.
- **Utilizzo del file .env:** Le chiavi principali, quindi la Master Key di Vault e credenziali di Keycloak sono gestite tramite variabili d'ambiente protette nel file **.env**. Questo separa la configurazione di sicurezza dall'applicazione vera e propria.
- **Sicurezza in Memoria:** Vault è configurato per bloccare la propria memoria RAM, tramite l'istruzione **IPC_LOCK** nel docker-compose, impedendo che le chiavi crittografiche vengano accidentalmente scritte sul disco rigido a causa di uno swap di memoria.



SC-13: CRYPTOGRAPHIC PROTECTION

Il sistema utilizza la crittografia per proteggere le informazioni in tre momenti chiave:

1. Protezione dei Dati in Transito: Il traffico tra il browser dell'utente e il server Apache è protetto al protocollo TLS sulla porta 443, questo garantisce che nessuno possa intercettare o leggere i dati mentre viaggiano sulla rete.

2. Gestione delle Identità e dei Token: Keycloak utilizza l'algoritmo RS256 per firmare i token JWT, evitando che vengano falsificati. Inoltre, le password degli utenti non sono salvate in chiaro, ma protette da algoritmi di hashing sicuri.

3. Protezione dei Segreti: Grazie all'algoritmo AES a 256 bit, Vault rendere i segreti illeggibili a chiunque non sia autorizzato, anche se riuscisse ad accedere fisicamente ai file.

4. Comunicazione Sicura tra i Database (MongoDB): Lo scambio di dati tra il backend e il database Mongo, che è configurato con `--tlsMode requireTLS`, rende obbligatoria la cifratura per ogni connessione interna.



SC-15: COLLABORATIVE COMPUTING DEVICES AND APPLICATIONS

Non Applicabile (N/A)

L'applicazione non include funzioni di videoconferenza, chat vocale, uso della telecamera o del microfono.

SC-20: SECURE NAME/ADDRESS RESOLUTION SERVICE (AUTHORITATIVE SOURCE)

Non Applicabile (N/A)

Il sistema non agisce come un server DNS autoritativo e non gestisce la risoluzione dei nomi a livello pubblico su Internet. Poiché l'applicazione è accessibile esclusivamente tramite indirizzo locale (**localhost**) sulla macchina host, non avvengono query di risoluzione nomi esterne.



SC-21: SECURE NAME/ADDRESS RESOLUTION SERVICE (RECURSIVE OR CACHING RESOLVER)

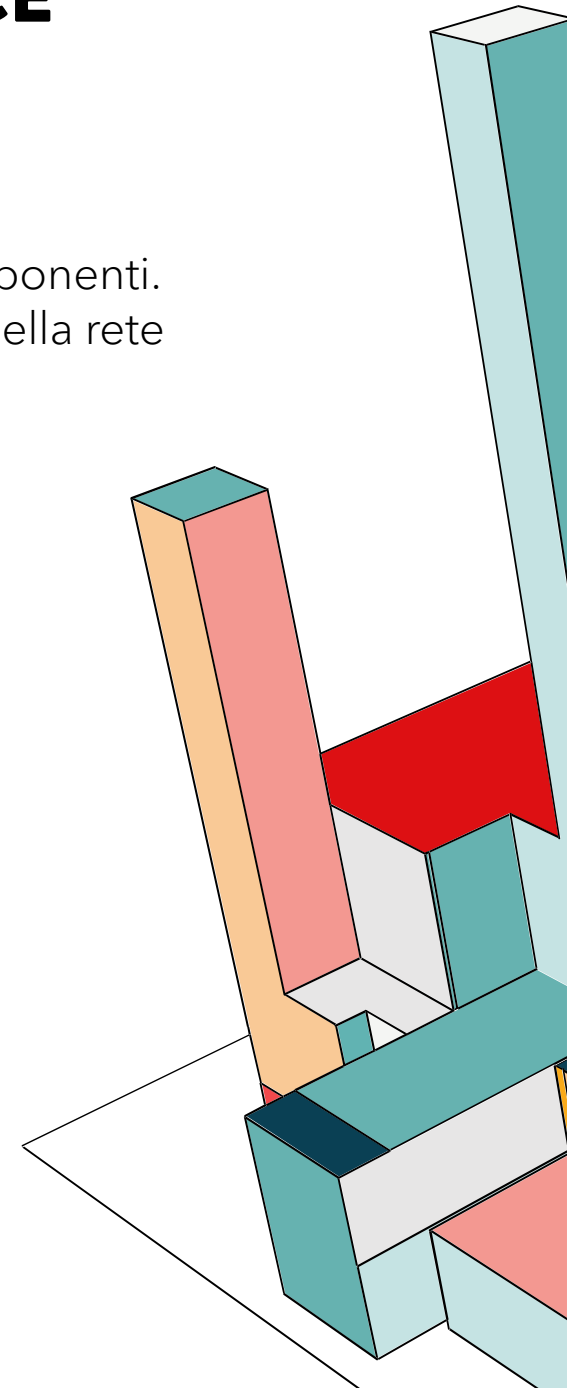
Non Applicabile (N/A)

Il sistema non effettua query DNS verso server esterni o pubblici per far comunicare i propri componenti. La risoluzione dei nomi è gestita interamente e in modo sicuro dal motore di **Docker** all'interno della rete privata locale.

SC-22: ARCHITECTURE AND PROVISIONING FOR NAME/ADDRESS RESOLUTION SERVICE

Non Applicabile (N/A)

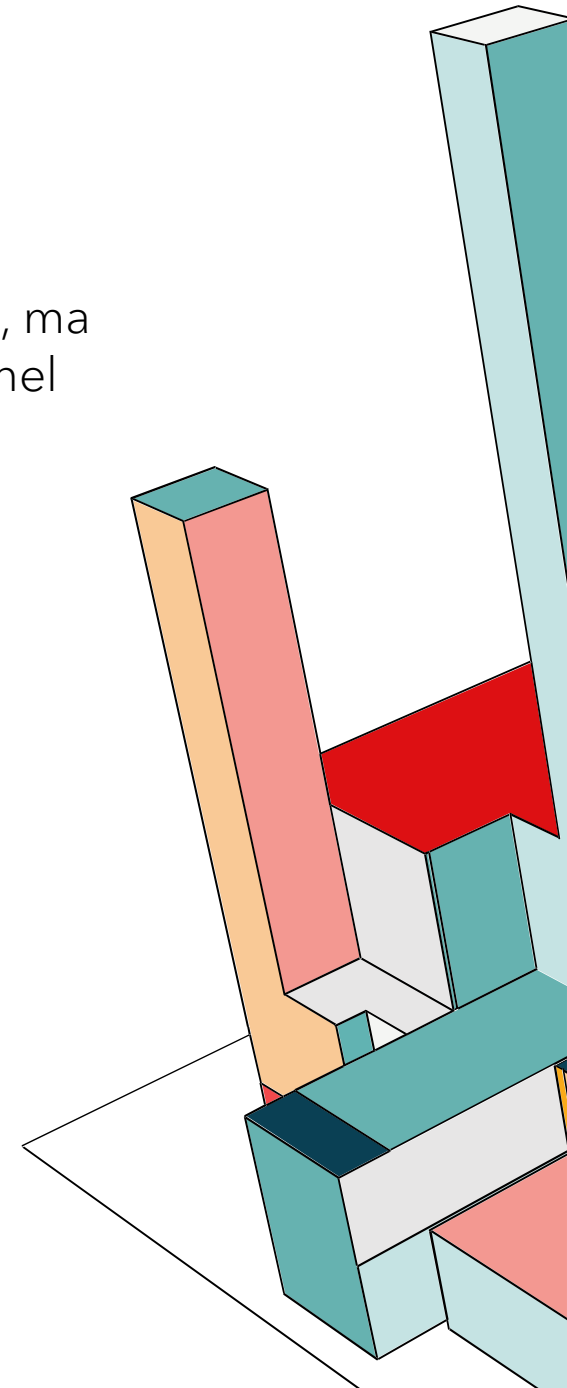
Il sistema non fornisce né gestisce servizi DNS per l'organizzazione.



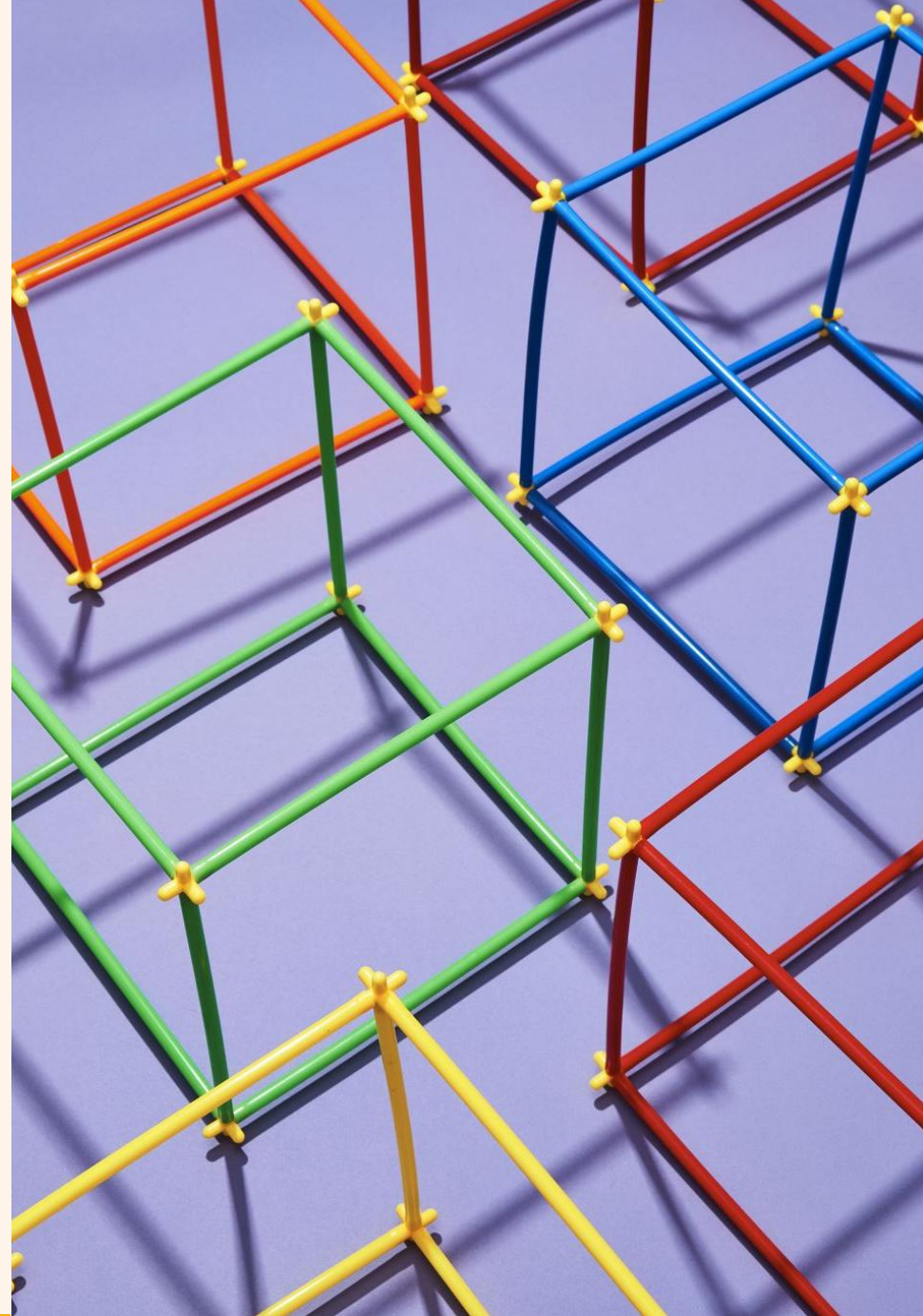
SC-39: PROCESS ISOLATION

Il sistema garantisce l'isolamento dei processi attraverso l'utilizzo dei **container Docker**:

- **Isolamento fisico:** Ogni componente dell'architettura gira in un ambiente di esecuzione separato ed indipendente, ognuno con un suo spazio di memoria e file system.
- **Comunicazioni controllate:** Non possono interferire tra loro a livello di codice o memoria, ma possono comunicare solo attraverso la rete interna virtuale e solo sulle porte specificate nel docker-compose.

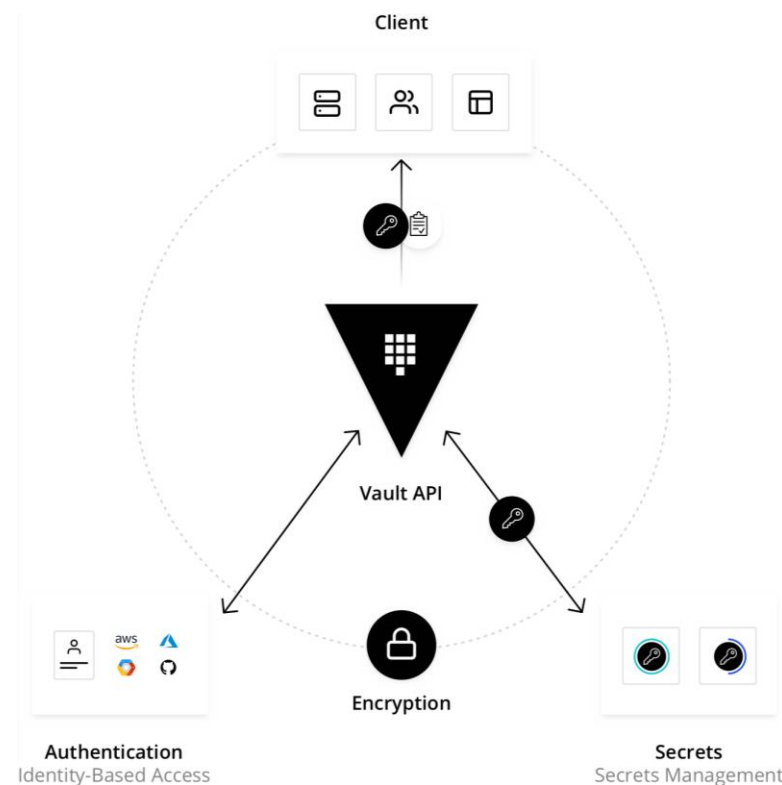


VAULT



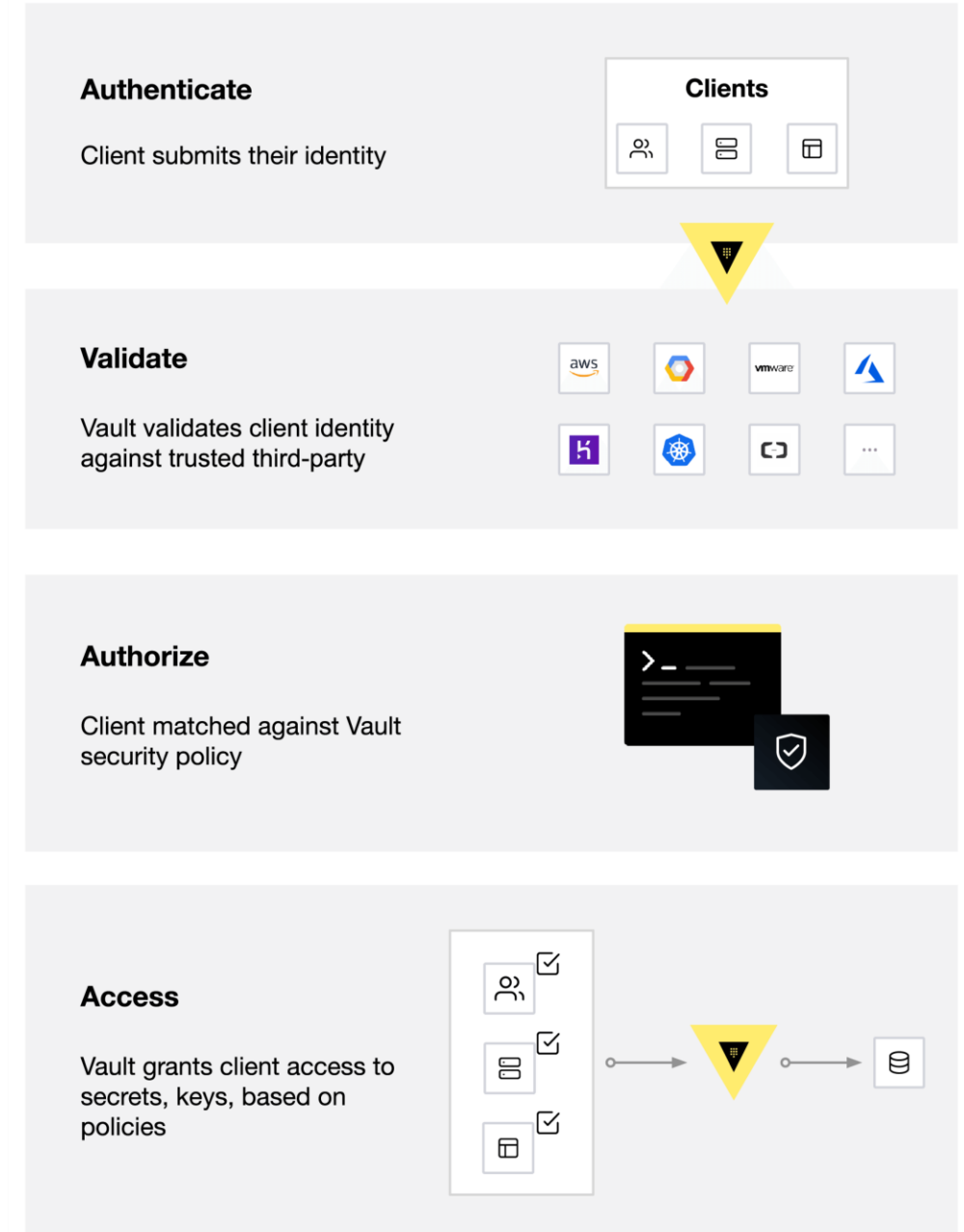
CHE COS'È VAULT?

Vault è un sistema centralizzato di cifratura e gestione dei dati segreti, basato sulle identità. Utilizzando l'interfaccia unificata di Vault, è possibile verificare in ogni momento, attraverso registri di controllo dettagliati, chi, dove, come e quando ha eseguito date operazioni con le credenziali segrete. Oltre a fornire un repository sicuro per lo storage di password, token API, chiavi di cifratura, certificati e quant'altro, Vault permette di governare il ciclo di vita completo di tali dati, automatizzando, ad esempio, i processi di creazione, erogazione, scadenza, revoca, rinnovo delle password.



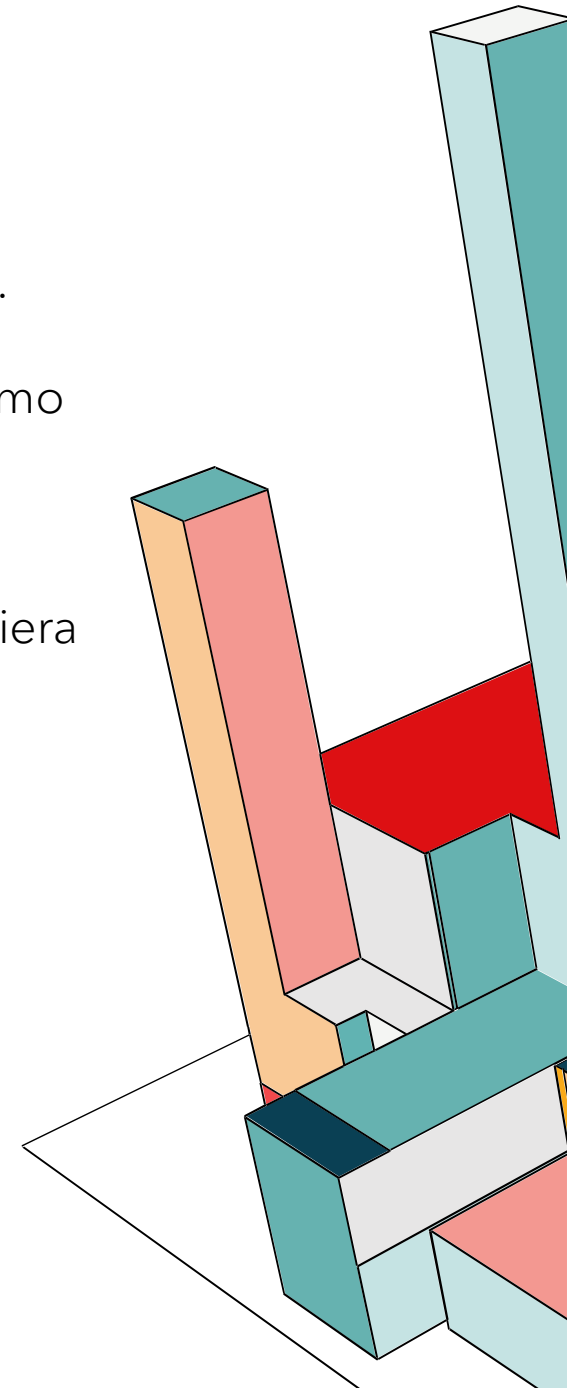
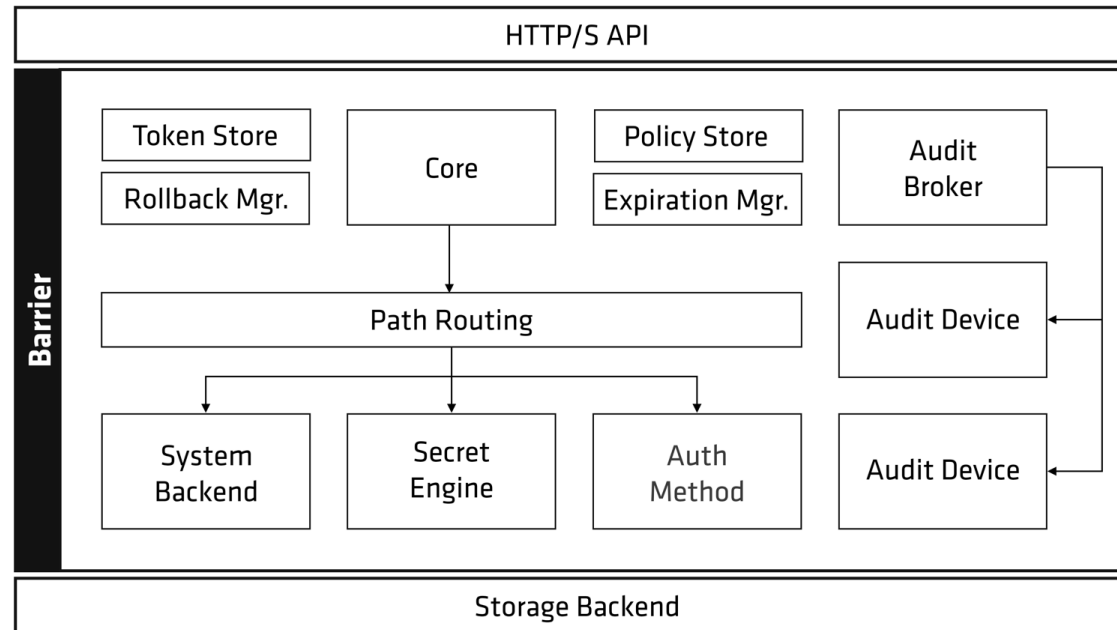
WORKFLOW

1. **Authenticate**: il presenta la sua identità; una volta autenticato, gli viene assegnata una policy e viene generato un token associato a tale policy.
2. **Validate**: il sistema convalida l'identità del client rispetto a servizi di terze-parti affidabili
3. **Authorize**: il client ha corrisposto alla politica di sicurezza di Vault; il confronto avviene tra il token del client e la security policy di Vault
4. **Access**: Vault concede al client l'accesso a segreti e chiavi sulla base delle policies; il token emesso può essere riutilizzato per operazioni future.



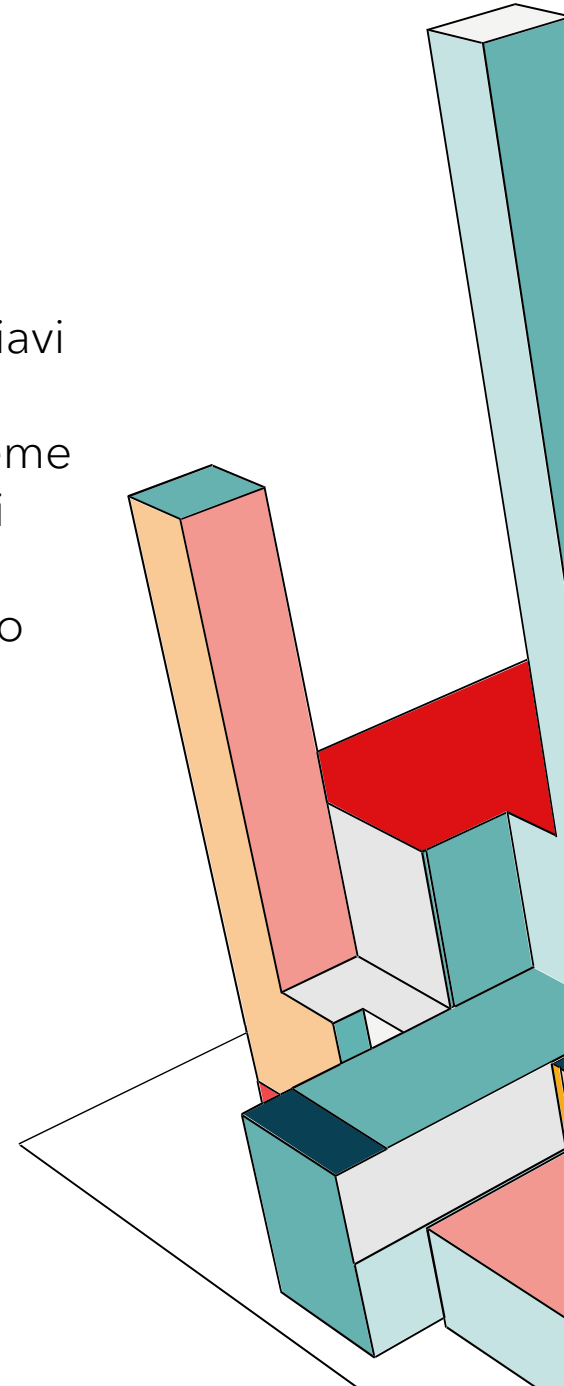
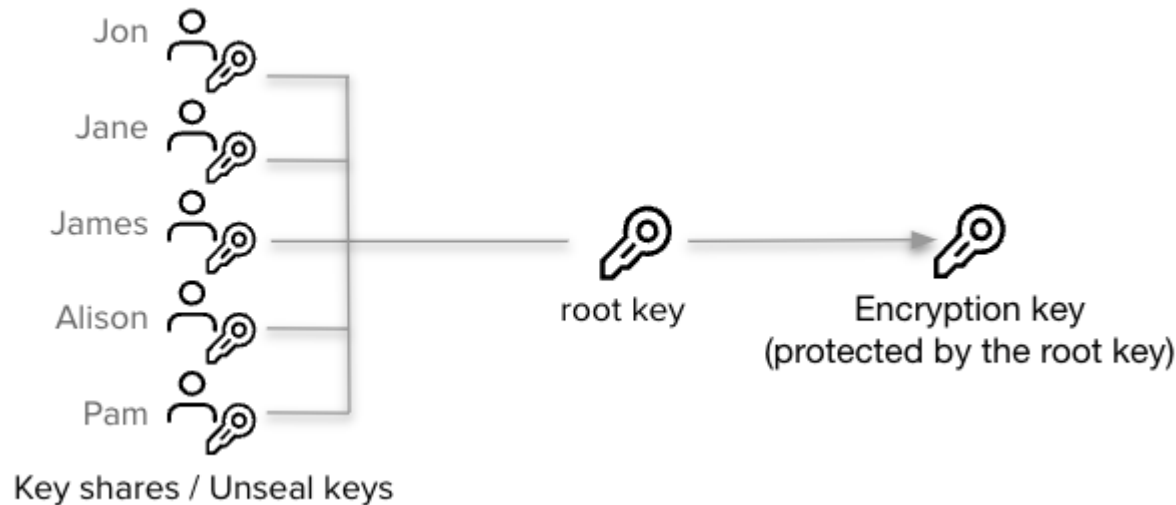
THE ENCRYPTION BARRIER

Vault opera secondo il principio "Zero Trust" verso il suo stesso backend di storage. Questo perché lo storage risiede all'esterno della barriera, e quindi è considerato untrusted; pertanto, Vault cifra i dati prima di inviarli allo storage. Questo meccanismo garantisce che se un attaccante tentasse di accedervi, i dati non potrebbero essere compromessi poiché rimangono cifrati finché Vault non li decifra. Il Core è il componente fidato in cui risiedono i dati in chiaro. La Barriera è il confine tra il core e il backend storage. Tutto ciò che attraversa la barriera uscendo dal Core viene cifrato.



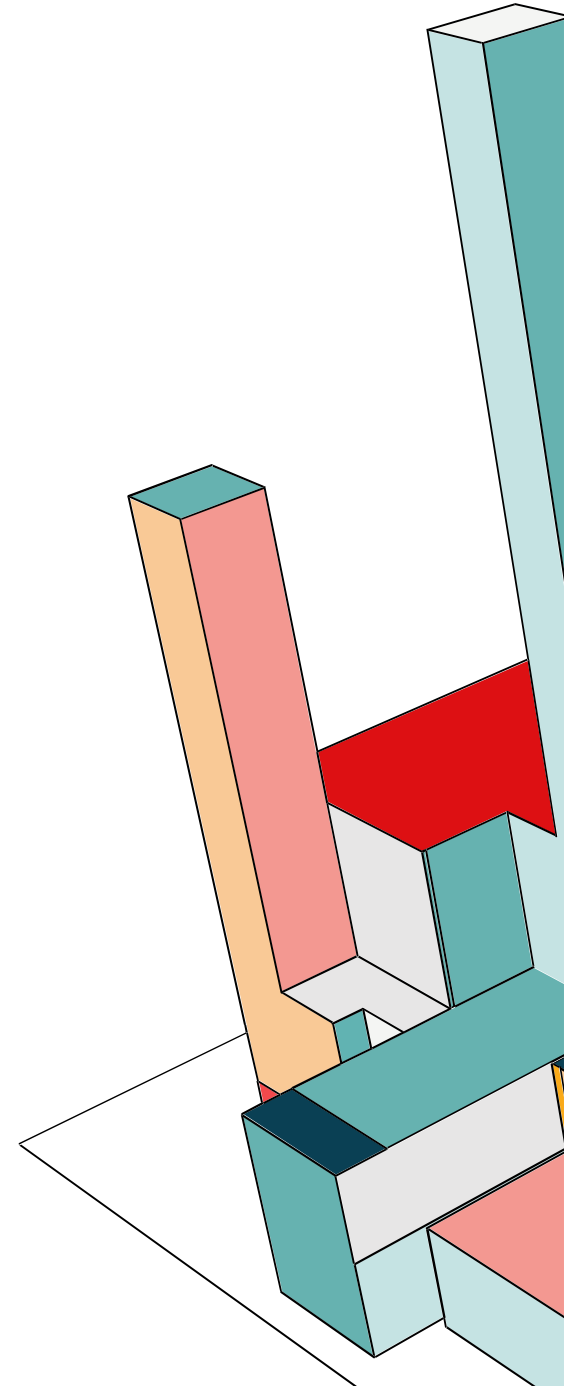
SEALE/UNSEAL

Quando un server Vault viene avviato, inizia in uno stato sigillato (sealed). Prima di poter eseguire qualsiasi operazione, deve essere aperto (unsealed) fornendo le chiavi di sblocco. Durante l'inizializzazione, Vault genera una Encryption Key, usata per proteggere tutti i dati. Questa chiave è protetta da una Root Key, che è salvata insieme ai dati ma cifrata da un altro meccanismo: Unseal Key. Per esempio nel nostro caso i dati contenuti nel database Mongo sono crittati con l'encryption key, la quale a sua volta è protetta dalla root key, che è cifrata dalla unseal key. In pratica quando siamo nello stato "Sealed" significa che Vault non può leggere il proprio storage.



VAULT SECRET ENGINE KEY-VALUE

In questo esempio didattico usiamo Vault come Static Secret Management. Cioè usiamo Vault semplicemente come un database cifrato per le password. Quindi noi abbiamo caricato fisicamente le password nel database di Vault. All'avvio, il backend chiede a Vault le password e Vault risponde con la password statica che abbiamo salvato. Il limite è la password che è statica. Se un attaccante la scopre, può usarla finché non ci ricordiamo di cambiarla.



VAULT SECRET ENGINE KEY-VALUE

Inizializziamo Vault e ci salviamo 3 chiavi unseal e la root key.

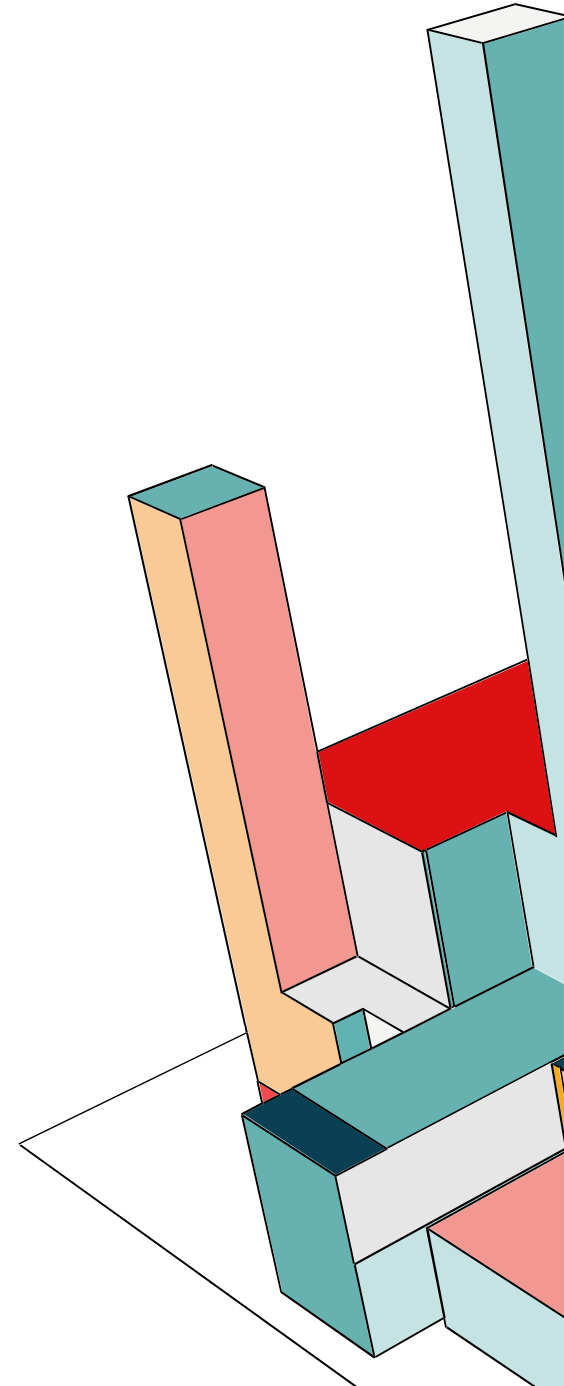
```
/ $ vault operator init
Unseal Key 1: SeArYTkQBmW6j2oN1PKpTI3rgUrCr6cxh0HrFt4wHSMm
Unseal Key 2: 1rRFMg3VITY566H76HVn0CPM08wGngdMnwNULq7LGfzB
Unseal Key 3: p/b7Xkf487czl1djpvC98VzTsbeBxvCu/9eqY//kQb0Y
Unseal Key 4: H3A1WMisjn1bfSC+hx02wAZoFsicmkJk1aEuqh/Z7Qcn
Unseal Key 5: s6oxhPce6DL2oSMzS01/59XrAjIbXsFexaV1gML7ekkN

Initial Root Token: hvs.KwZZ78ZTeT3MzCG5vK8E69Tq

Vault initialized with 5 key shares and a key threshold of 3. Please securely
distribute the key shares printed above. When the Vault is re-sealed,
restarted, or stopped, you must supply at least 3 of these keys to unseal it
before it can start servicing requests.

Vault does not store the generated root key. Without at least 3 keys to
reconstruct the root key, Vault will remain permanently sealed!

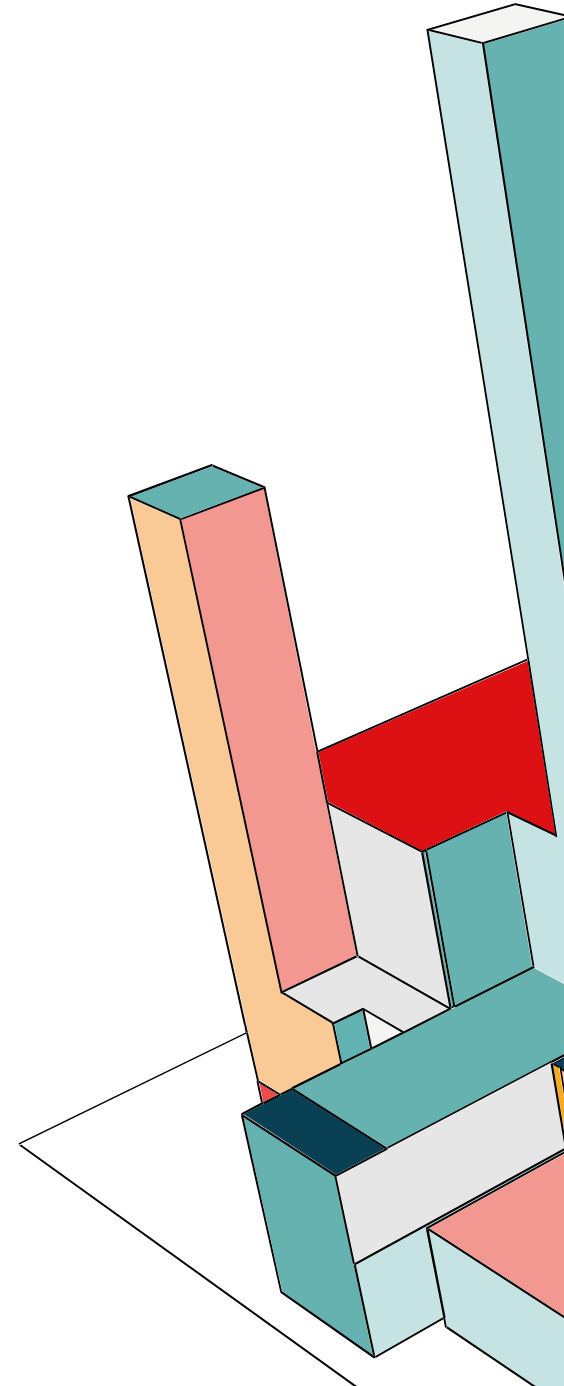
It is possible to generate new unseal keys, provided you have a quorum of
existing unseal keys shares. See "vault operator rekey" for more information
.
```



VAULT SECRET ENGINE KEY-VALUE

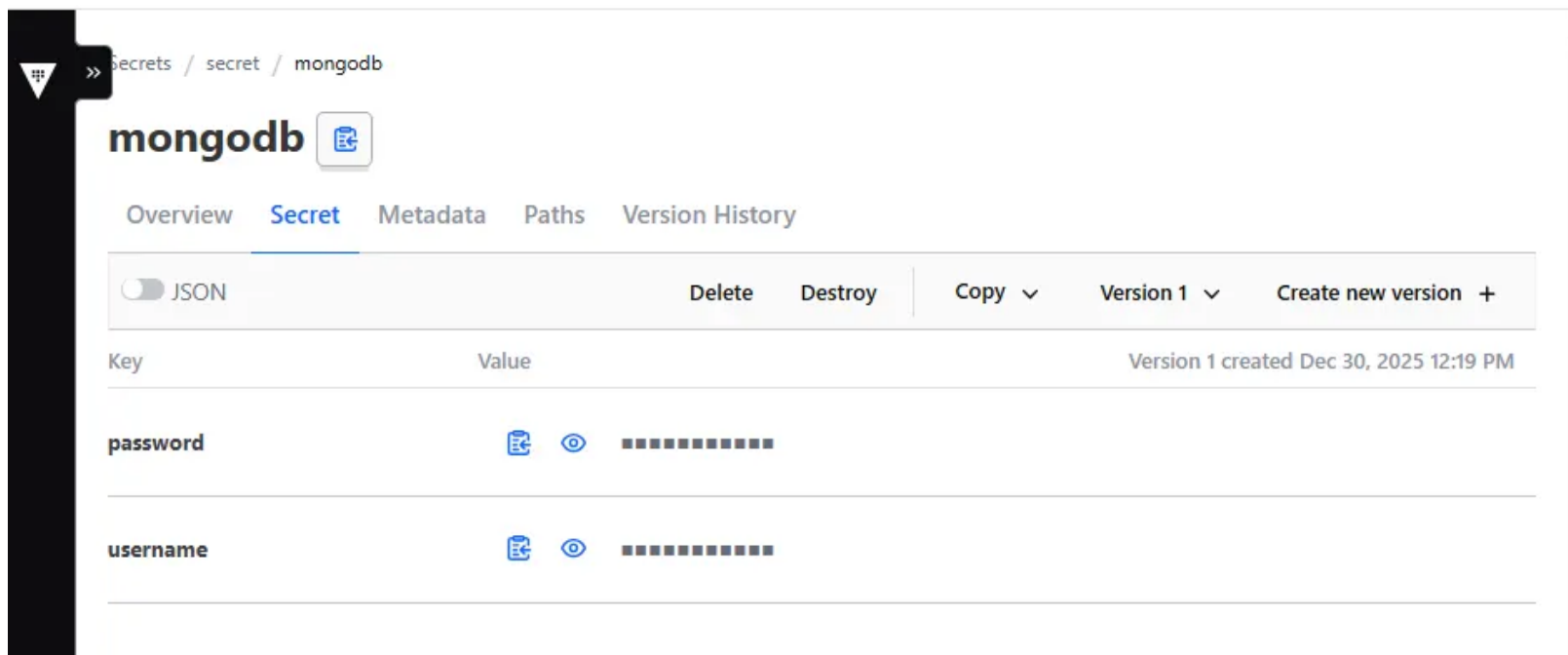
Dopo aver effettuato il login in Vault tramite il root token abilitiamo il motore dei segreti Key-Value Store che permette di salvare coppie chiave-valore, nel nostro caso username e password.

```
/ $ vault kv put secret/mongodb \  
> username="username" \  
> password="password"  
=== Secret Path ===  
secret/data/mongodb  
  
===== Metadata =====  
Key                Value  
---                -  
created_time       2025-12-30T11:19:00.722653873Z  
custom_metadata    <nil>  
deletion_time      n/a  
destroyed          false  
version            1  
/ $ |
```



VAULT SECRET ENGINE KEY-VALUE

Possiamo vedere i segreti da Vault interfaccia grafica:



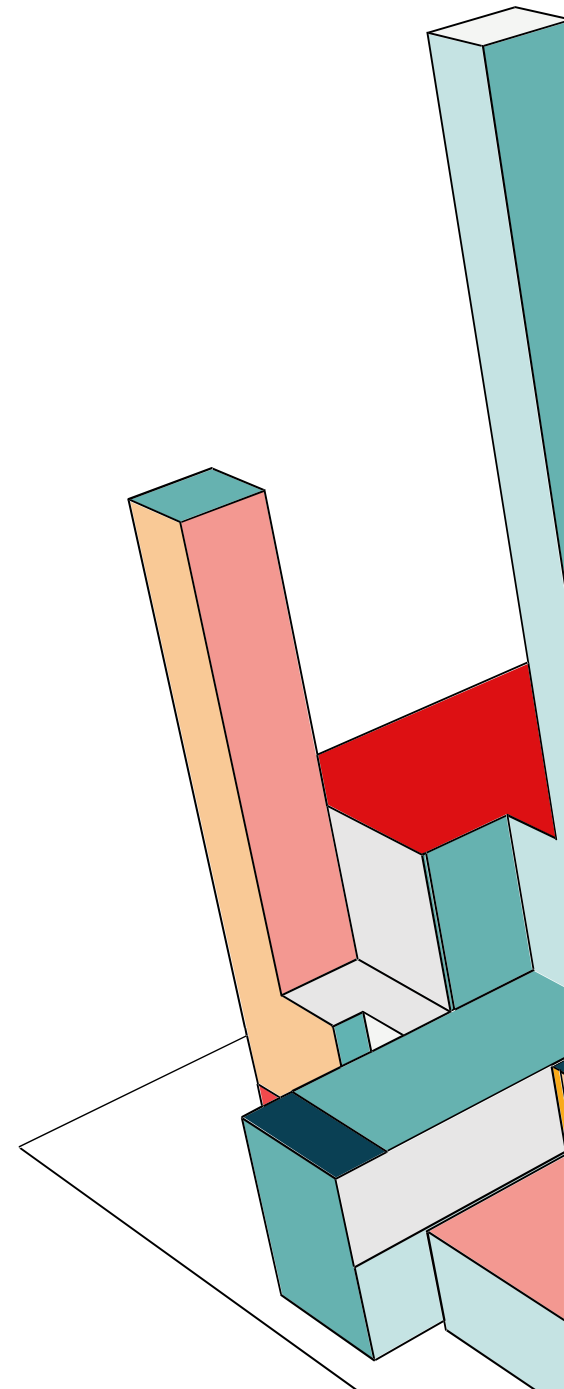
Secrets / secret / mongodb

mongodb

Overview **Secret** Metadata Paths Version History

☐ JSON Delete Destroy Copy ▾ Version 1 ▾ Create new version +

Key	Value	Version 1 created Dec 30, 2025 12:19 PM
password	 	
username	 	

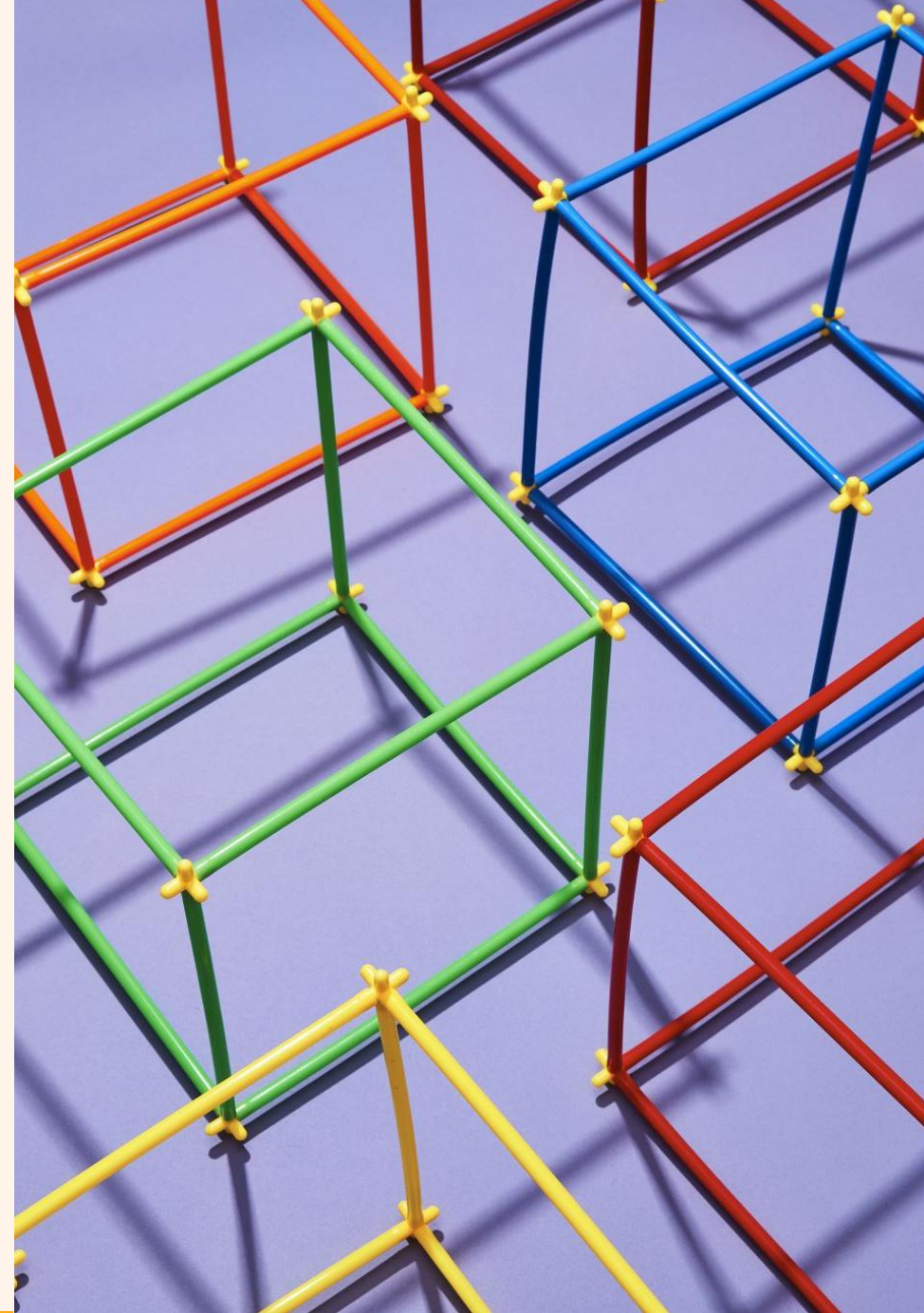


VAULT DYNAMIC SECRETS & PKI

Un approccio di utilizzo più sicuro. Infatti la prima differenza è che, invece di custodire un certificato statico creato, Vault lo crea ogni volta che lo chiediamo e valido per un certo tempo, nel nostro caso un mese. Inoltre ora Vault funge da CA interna. Infatti prima dovevamo creare noi un certificato con OpenSSL e salvarlo su un file, quindi se la chiave privata veniva compromessa dovevamo revocare tutto. Invece ora abbiamo creato uno script `genera_cert.ps1` che chiede a Vault un'identità per Mongo, e Vault crea una coppia di chiavi privata e pubblica apposta. Inoltre firma il certificato con la sua Root CA, lo consegna nel file che gli abbiamo indicato e quando il certificato scadrà, si eliminerà da solo. In questo modo riduciamo la superficie di attacco temporale.

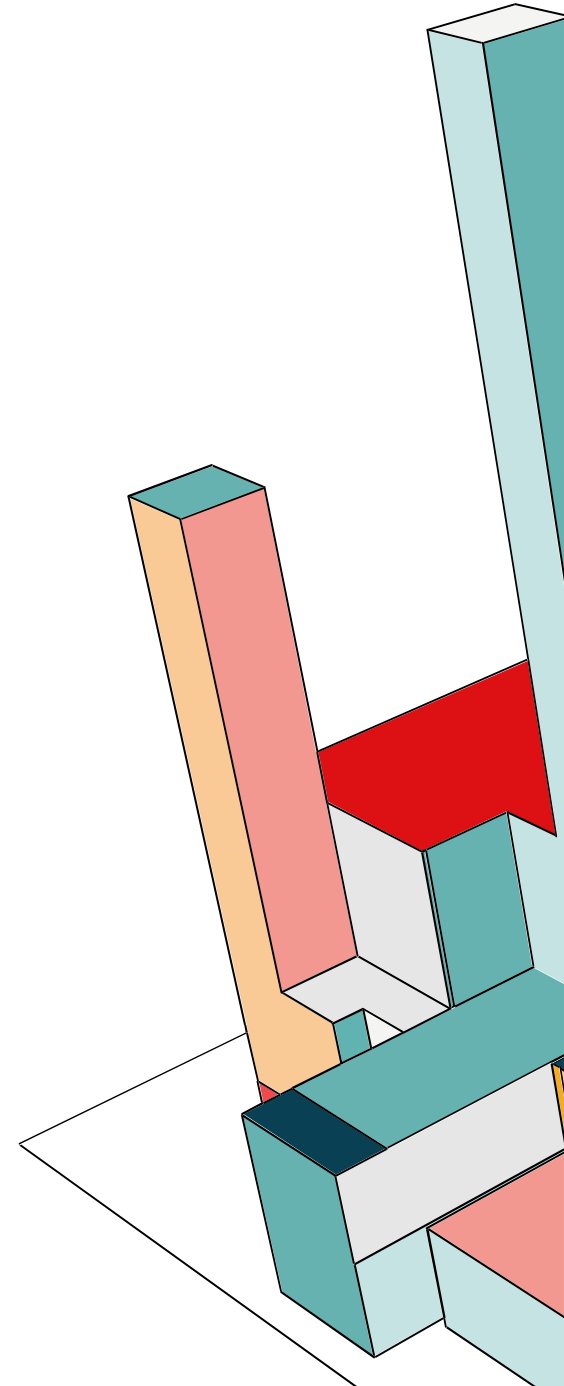
```
> genera_cert.ps1
1  .\vault.exe write -address="http://127.0.0.1:8200"
2  -format=json pki/issue/mongo-role common_name="mongo" ttl="24h" > cert.json
3
4  $json = Get-Content cert.json | ConvertFrom-Json
5
6  if ($null -eq $json.data) {
7      Write-Host "ERRORE CRITICO: Vault non ha restituito dati validi." -ForegroundColor Red
8      Write-Host "Contenuto del file cert.json:"
9      Get-Content cert.json
10     exit
11 }
12
13 $certContent = $json.data.private_key + "`n" + $json.data.certificate + "`n" + $json.data.issuing_ca
14
15 $outputFile = "./mongodb-certs/mongodb.pem"
16 Set-Content -Path $outputFile -Value $certContent -NoNewline
17
18 Write-Host "Certificato generato con successo: $outputFile" -ForegroundColor Green
19
```

VULNERABILITY ASSESSMENT



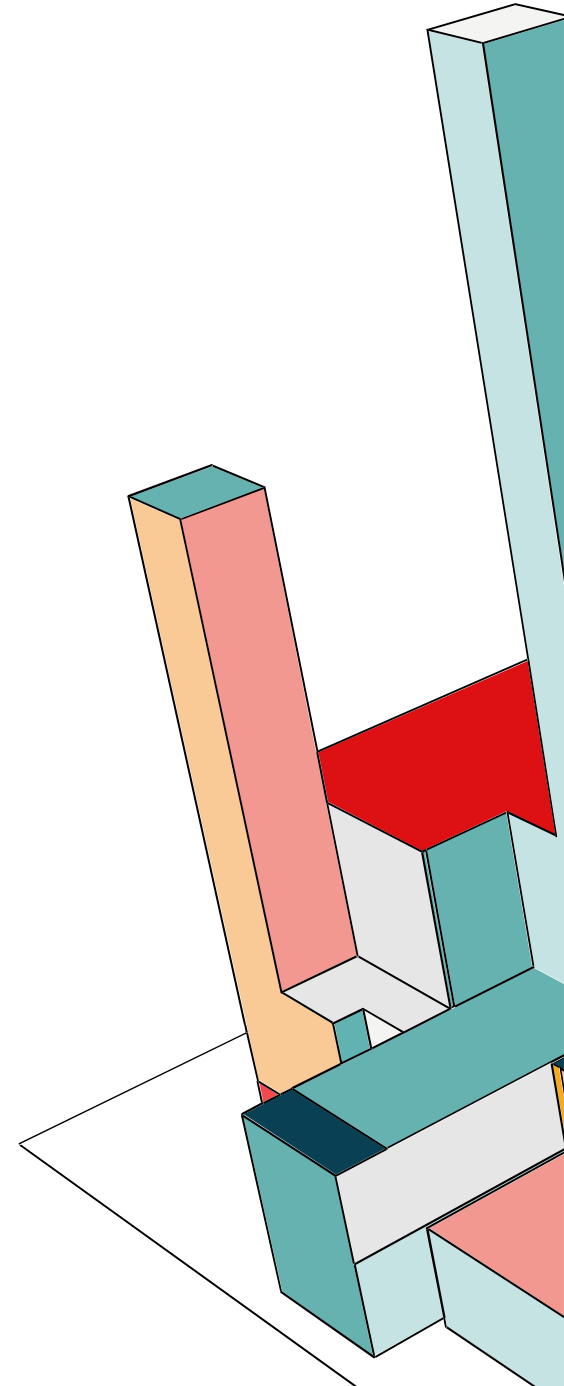
NESSUS TENABLE

Tenable Web App Scanning fornisce una scansione completa delle vulnerabilità per le moderne applicazioni web. L'accurata copertura delle vulnerabilità di Tenable Web App Scanning riduce al minimo i falsi positivi e i falsi negativi, assicurando che i team di sicurezza comprendano i veri rischi di sicurezza nelle loro applicazioni web.



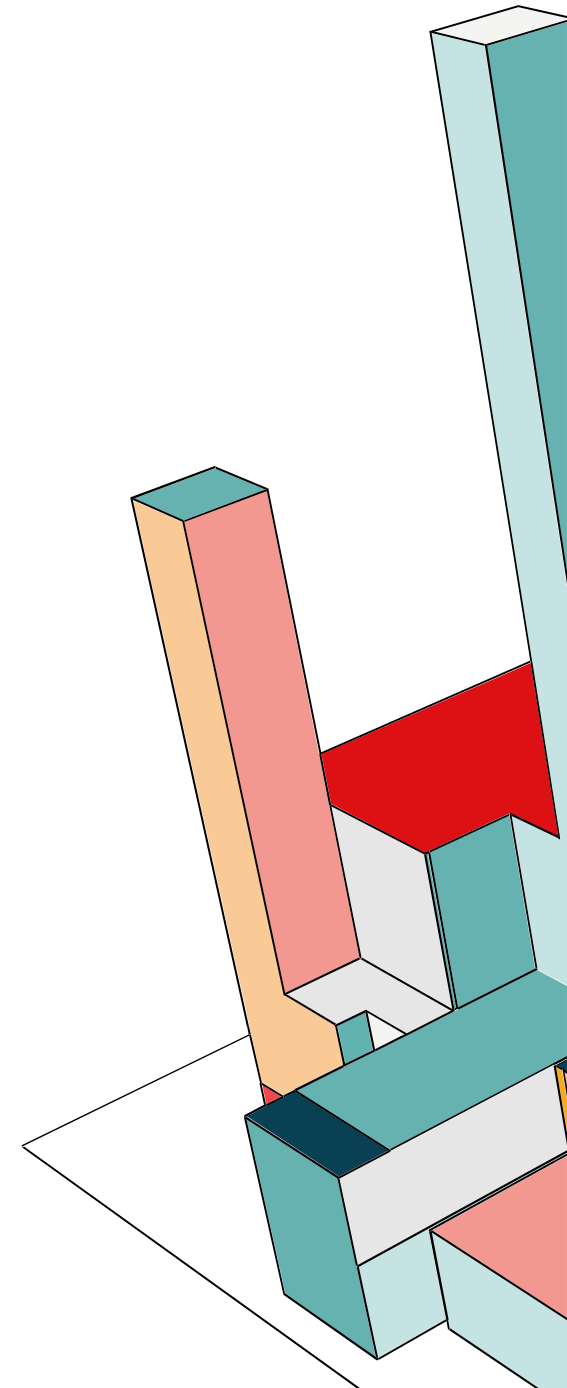
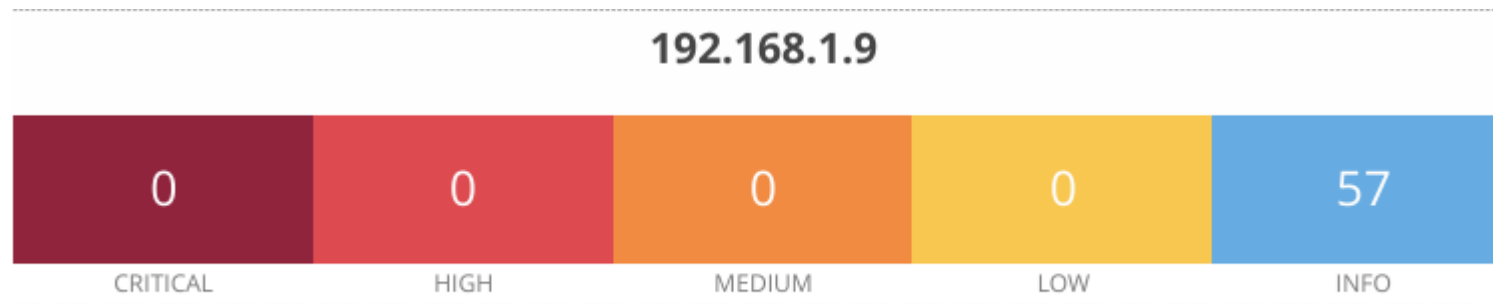
NESSUS TENABLE

Essendo uno scan fatto dall'esterno, Nessus non ha potuto testare le API del backend, i container e le eventuali vulnerabilità applicative come SQL Injection, Privilege Escalation e altri. Infatti per avere una scansione completa si dovrebbe aggiungere un'analisi dall'interno per esempio utilizzando tool come Trivy o Docker Scout.



REPORT

Anche se le vulnerabilità riscontrate sono tutte di tipo informativo, questa scansione ci dà comunque una panoramica su come abbiamo configurato il Web Server.

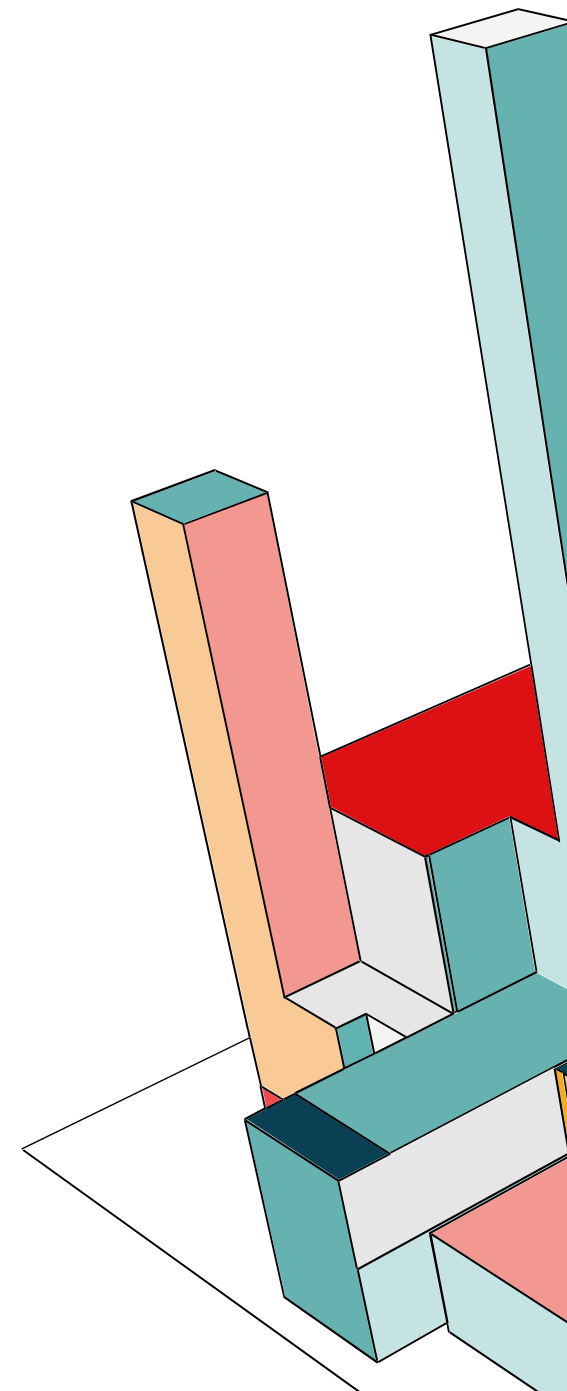


Lo strumento di scansione ha verificato la configurazione del server web per identificare i metodi HTTP abilitati. L'analisi viene effettuata principalmente tramite il metodo `OPTIONS`, che interroga il server sulle funzionalità supportate per una specifica risorsa.

L'attenzione è posta sull'identificazione di metodi potenzialmente rischiosi se non necessari, come `PUT` e `DELETE` (che permettono la modifica dei file sul server) o `TRACE` (vulnerabile al Cross-Site Tracing). Inoltre, viene analizzata la gestione del metodo `HEAD`: in alcune configurazioni errate, le restrizioni di autenticazione applicate alle richieste `GET` potrebbero non essere estese automaticamente alle richieste `HEAD`, permettendo potenzialmente a un attaccante di bypassare i controlli di accesso.

È necessario verificare manualmente se i metodi abilitati sono effettivamente richiesti per il funzionamento dell'applicazione.

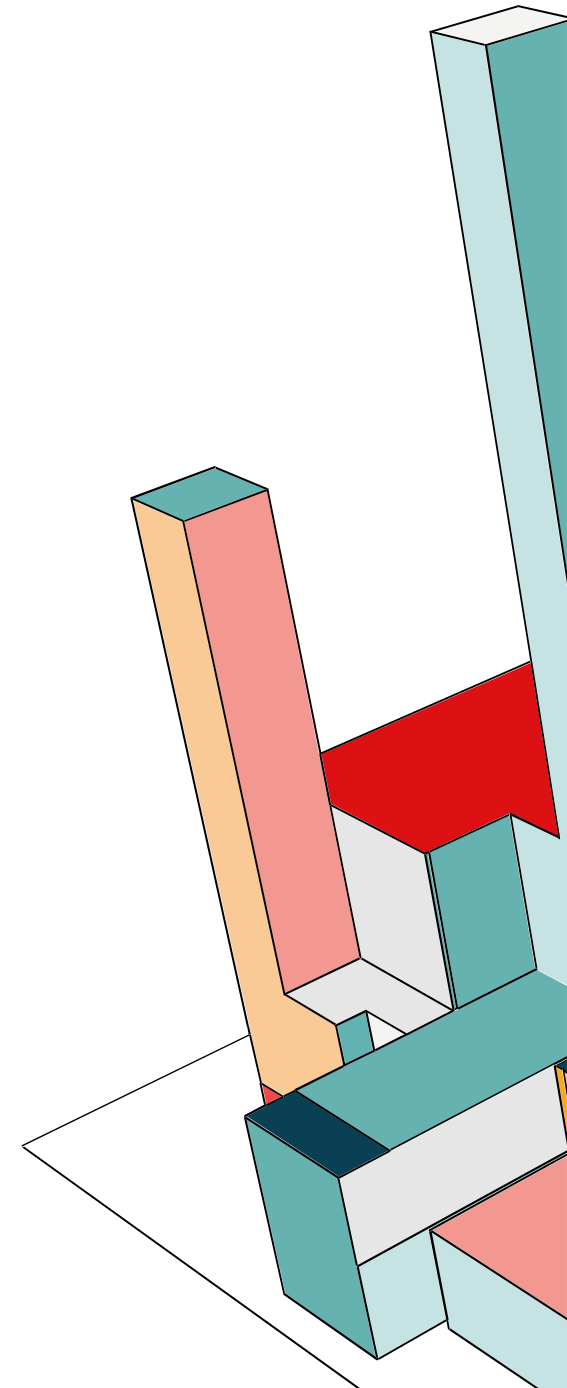
Lo scan ha rilevato che manca l'header HSTS. Anche se siamo in container, il Reverse Proxy Apache deve forzare HTTPS per evitare downgrade attacks.



Il server web remoto è configurato in modo tale da non restituire il codice di errore '404 Not Found' quando viene richiesto un file inesistente; potrebbe invece restituire una mappa del sito, una pagina di ricerca o una pagina di autenticazione. Nessus ha attivato alcune contromisure per gestire questa situazione. In realtà questa vulnerabilità è un falso positivo in quanto React, essendo una SPA, è programmata per rispondere con una regola di Fallback, cioè che se viene richiesto un file inesistente viene fornita una pagina index.html con status 200 OK, però ci mostra a video «Pagina non trovata».

91815 - Web Application Sitemap

Durante la fase di ricognizione, lo strumento di scansione ha eseguito un processo di crawling per identificare la struttura dell'applicazione web. Attraverso l'analisi dei link presenti nelle risposte del server, è stato possibile ricostruire una mappa dei contenuti accessibili pubblicamente. Questa attività ha permesso di enumerare le risorse statiche (come immagini, file CSS e script) e i percorsi di navigazione esposti dal server web, confermando che l'applicazione è raggiungibile e correttamente indicizzabile.



VAULT SECRET ENGINE KEY-VALUE

In questo esempio didattico usiamo Vault come Static Secret Management. Cioè usiamo Vault semplicemente come un database cifrato per le password. Quindi noi abbiamo caricato fisicamente le password nel database di Vault. All'avvio, il backend chiede a Vault le password e Vault risponde con la password statica che abbiamo salvato. Il limite è la password che è statica. Se un attaccante la scopre, può usarla finché non ci ricordiamo di cambiarla.

